

ПРИЛОЖЕНИЕ В. ТЕКСТ ПРОГРАММЫ АННОТАЦИЯ

В данном программном документе «Текст программы» приведён исходный текст web-приложения JobFlex.

В документе представлен раздел «Текст программы» со следующими подразделами:

1. Наименование программы
2. Область применения программы
3. Модули web -приложения
4. Исходный код web-приложения

В подразделе «Наименование программы» содержится наименование программы.

В подразделе «Область применения программы» содержится определение предназначения программы.

В подразделе «Модули web-приложения» содержится набор данных, отображающий характеристики каждого модуля, в том числе: название модуля в программе, описание модуля, количество строк кода в данном модуле, размер файла-модуля в КБ.

В подразделе «Исходный код web-приложения» содержится исходный код каждого модуля, представленного в разделе «Модули».

СОДЕРЖАНИЕ

1. ТЕКСТ ПРОГРАММЫ	3
1.1 Наименование программы	3
1.2 Область применения программы.....	3
1.3. Модули мобильного приложения.....	3
1.4. Исходный код web-приложения	4

1. ТЕКСТ ПРОГРАММЫ

1.1 Наименование программы

Наименование – web-приложение «JobFlex».

1.2 Область применения программы

Web-приложение JobFlex предназначено для автоматизации процесса подбора персонала и используется соискателями, работодателями и администраторами через браузер и REST API. Система обеспечивает ведение каталога вакансий с расширенной фильтрацией и сортировкой, управление откликами и статусами их рассмотрения, планирование и сопровождение собеседований с автоматической отправкой уведомлений через Telegram и email, встроенный чат между участниками, личный кабинет соискателя с профилем и резюме, личный кабинет работодателя с аналитикой и экспортом данных, а также административные функции – управление пользователями и ролями, просмотр журнала аудита и резервное копирование базы данных. Интеграции с внешними сервисами HH.ru, DreamJob, 2ГИС и Telegram Bot API расширяют возможности платформы. Область применения – компании, кадровые агентства и частные работодатели, которым необходима единая web-платформа для размещения вакансий, взаимодействия с кандидатами и контроля процесса найма.

1.3. Модули мобильного приложения

В Таблице 1 описание всех модулей программы, количество строк кода и размер файла.

Таблица 1 – Описание классов

№	Компонент проекта JobFlex	Краткое назначение	Кол-во файлов	Кол-во строк	Размер, файлов в КБ
1	2	3	4	5	6
1	accounts/ (рабочий код, без migrations/)	Пользователи, кабинеты, чат, модерация, Celery-задачи, management-команды	13	8 332	360,5

№	Компонент проекта JobFlex	Краткое назначение	Кол-во файлов	Кол-во строк	Размер, файлов в КБ
1	2	3	4	5	6
2	accounts/migrations/	Изменение схемы БД для приложения accounts	39	1 198	47,9
3	vacancies/ (рабочий код, без migrations/)	Вакансии, импорт с HH и «Труд всем», рейтинги, задачи Celery	24	5 282	204,1
4	vacancies/migrations/	Изменение схемы БД для приложения vacancies	25	924	35,1
5	config/	Настройки Django, Celery, URL проекта, OpenAPI и служебные точки входа	9	1 247	46,6
6	tools/	Вспомогательные утилиты для данных и диагностики	8	490	15,3
7	Корень каталога job_aggregator	manage.py, сценарий запуска, тестирование Locust и пр.	4	493	15,8
Итого	Весь перечень п. 1.4	Серверная часть на Python	122	17 966	725,3

1.4. Исходный код web-приложения

1. accounts/admin.py.

```

from django.contrib import admin
from .models import Applicant, Manager, Administrator, Moderator,
ApiActionLog, UserDocument, UserDocumentFile

admin.site.site_header = 'Администрирование JobFlex'
admin.site.site_title = 'Панель администратора JobFlex'
admin.site.index_title = 'Управление данными JobFlex'

@admin.register(Applicant)
class ApplicantAdmin(admin.ModelAdmin):
    list_display = (
        'user_label',
        'telegram_label',
        'phone',
        'gender',
        'city',
        'citizenship',
        'consent_email',
        'consent_telegram',
    )
    search_fields = ('user__email', 'user__first_name',
                    'user__last_name', 'telegram', 'phone', 'city')

    @admin.display(description='Пользователь')
    def user_label(self, obj):

```

```

        return obj.user

    @admin.display(description='Телеграм')
    def telegram_label(self, obj):
        return obj.telegram

@admin.register(Manager)
class ManagerAdmin(admin.ModelAdmin):
    list_display = ('user_label', 'telegram_label', 'company', 'phone')
    search_fields = ('user__email', 'user__first_name',
                    'user__last_name', 'telegram', 'company')

    @admin.display(description='Пользователь')
    def user_label(self, obj):
        return obj.user

    @admin.display(description='Телеграм')
    def telegram_label(self, obj):
        return obj.telegram

@admin.register(Administrator)
class AdministratorAdmin(admin.ModelAdmin):
    list_display = ('user_label', 'is_superuser_ok', 'created_at')
    search_fields = ('user__email', 'user__username', 'user__first_name',
                    'user__last_name')
    readonly_fields = ('created_at',)

    @admin.display(description='Пользователь')
    def user_label(self, obj):
        return obj.user

    @admin.display(boolean=True, description='Суперпользователь')
    def is_superuser_ok(self, obj):
        return obj.user.is_superuser

@admin.register(Moderator)
class ModeratorAdmin(admin.ModelAdmin):
    list_display = ('user_label', 'telegram_label', 'phone',
                    'created_at')
    search_fields = ('user__email', 'user__username', 'user__first_name',
                    'user__last_name', 'telegram', 'phone')
    readonly_fields = ('created_at',)

    @admin.display(description='Пользователь')
    def user_label(self, obj):
        return obj.user

    @admin.display(description='Телеграм')
    def telegram_label(self, obj):
        return obj.telegram

@admin.register(ApiActionLog)
class ApiActionLogAdmin(admin.ModelAdmin):
    list_display = (
        'created_at',
        'actor_role_label',
        'user_label',
        'method_label',
        'action',
        'endpoint',
        'success_label',
        'status_code_label',
    )

```

```

        list_filter = ('actor_role', 'method', 'success', 'endpoint',
'created_at')
        search_fields = ('user__username', 'user__email', 'action',
'endpoint', 'path')
        readonly_fields = ('created_at',)

@admin.display(description='Роль')
def actor_role_label(self, obj):
    return obj.get_actor_role_display()

@admin.display(description='Пользователь')
def user_label(self, obj):
    return obj.user

@admin.display(description='Метод')
def method_label(self, obj):
    return obj.method

@admin.display(boolean=True, description='Успешно')
def success_label(self, obj):
    return obj.success

@admin.display(description='Код ответа')
def status_code_label(self, obj):
    return obj.status_code

class UserDocumentFileInline(admin.TabularInline):
    model = UserDocumentFile
    extra = 0
    verbose_name = 'Файл'
    verbose_name_plural = 'Файлы'

@admin.register(UserDocument)
class UserDocumentAdmin(admin.ModelAdmin):
    list_display = ('id', 'user_label', 'doc_type', 'serial', 'number',
'created_at')
    list_filter = ('doc_type', 'created_at')
    search_fields = ('user__username', 'user__email', 'serial', 'number',
'issued_by')
    inlines = [UserDocumentFileInline]

@admin.display(description='Пользователь')
def user_label(self, obj):
    return obj.user

```

2. accounts/apps.py

```

from django.apps import AppConfig

class AccountsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'accounts'

```

3. accounts/management/commands/export_users.py

```

from django.core.management.base import BaseCommand
from django.contrib.auth.models import User
from accounts.models import Applicant, Manager

class Command(BaseCommand):
    help = "Export all users with applicant and manager data"

    def handle(self, *args, **kwargs):

```

```

users = User.objects.all()

for u in users:
    print("\n" + "=" * 80)
    print("USER ID:", u.id)

    for k, v in u.__dict__.items():
        if not k.startswith("_"):
            print(f"{k}: {v}")

    # Applicant
    if hasattr(u, "applicant"):
        print("\n--- APPLICANT ---")
        a = u.applicant
        for k, v in a.__dict__.items():
            if not k.startswith("_"):
                print(f"{k}: {v}")
    else:
        print("\n--- APPLICANT: NULL ---")

    # Manager
    if hasattr(u, "manager"):
        print("\n--- MANAGER ---")
        m = u.manager
        for k, v in m.__dict__.items():
            if not k.startswith("_"):
                print(f"{k}: {v}")
    else:
        print("\n--- MANAGER: NULL ---")

```

4. accounts/management/commands/flush.py

```

from django.contrib.auth import get_user_model
from django.core.management.commands.flush import Command as
DjangoFlushCommand

class Command(DjangoFlushCommand):

    def handle(self, *args, **options):
        super().handle(*args, **options)

        User = get_user_model()
        db_alias = options.get('database', 'default')

        user, created = User.objects.using(db_alias).get_or_create(
            username='aedgy',
            defaults={
                'is_staff': True,
                'is_superuser': True,
                'is_active': True,
                'email': '',
            },
        )

        user.is_staff = True
        user.is_superuser = True
        user.is_active = True
        user.set_password('aedgy123')
        user.save(using=db_alias)

        from accounts.models import Administrator

        Administrator.objects.using(db_alias).get_or_create(user=user)

```

```

        if created:
            self.stdout.write(self.style.SUCCESS('Default admin created:
aedygy'))
        else:
            self.stdout.write(self.style.SUCCESS('Default admin
refreshed: aedygy'))

```

5. accounts/management/commands/poll_telegram_updates.py

```

from django.core.management.base import BaseCommand
from django.conf import settings
import time

try:
    import requests
except Exception:
    requests = None

from accounts.models import Applicant, Manager
from accounts.telegram import send_hello_async

class Command(BaseCommand):
    help = 'Poll Telegram getUpdates and map /start tokens or usernames
to applicant.chat_id'

    def _process_start(self, chat_id, token_arg, username):
        if token_arg:
            try:
                applicant =
Applicant.objects.get(telegram_start_token=token_arg)
                applicant.telegram_chat_id = chat_id
                applicant.save(update_fields=['telegram_chat_id'])
                self.stdout.write(f'Mapped token {token_arg} -> chat_id
{chat_id}')
                if applicant.consent_telegram:
                    send_hello_async(chat_id)
                return True
            except Applicant.DoesNotExist:
                pass
        if username:
            applicant = Applicant.objects.filter(telegram__iexact='@' +
username).first()
            if applicant:
                applicant.telegram_chat_id = chat_id
                applicant.save(update_fields=['telegram_chat_id'])
                self.stdout.write(f'Mapped username @{username} ->
chat_id {chat_id}')
                if applicant.consent_telegram:
                    send_hello_async(chat_id)
                return True

            manager = Manager.objects.filter(telegram__iexact='@' +
username).first()
            if manager:
                manager.telegram_chat_id = chat_id
                manager.save(update_fields=['telegram_chat_id'])
                self.stdout.write(f'Mapped manager @{username} -> chat_id
{chat_id}')
                if manager.consent_telegram:
                    send_hello_async(chat_id, text='Телеграм подключён!
Теперь вы будете получать уведомления от JobFlex.')
                return True

```



```

        return False

    def handle(self, *args, **options):
        token = getattr(settings, 'TELEGRAM_BOT_TOKEN', None)
        if not token:
            self.stderr.write('TELEGRAM_BOT_TOKEN is not set in
settings')
            return
        if not requests:
            self.stderr.write('requests library is required. Install with
pip install requests')
            return

        offset = None
        self.stdout.write('Starting Telegram polling (press Ctrl+C to
stop)')
        try:
            while True:
                params = {'timeout': 30}
                if offset:
                    params['offset'] = offset
                url = f'https://api.telegram.org/bot{token}/getUpdates'
                try:
                    r = requests.get(url, params=params, timeout=40)
                    j = r.json()
                except Exception as e:
                    self.stderr.write(f'Error fetching updates: {e}')
                    time.sleep(5)
                    continue

                for upd in j.get('result', []):
                    update_id = upd.get('update_id')
                    offset = update_id + 1
                    msg = upd.get('message') or {}
                    text = msg.get('text', '')
                    chat = msg.get('chat') or {}
                    chat_id = chat.get('id')
                    username = chat.get('username')

                    if not text or not chat_id:
                        continue

                    if text.startswith('/start'):
                        parts = text.split(None, 1)
                        token_arg = parts[1].strip() if len(parts) > 1
                        self._process_start(chat_id, token_arg, username)

                time.sleep(1)
            except KeyboardInterrupt:
                self.stdout.write('Polling stopped')

```

6. accounts/management/commands/seed_demo_data.py

```

import base64
import random
from datetime import timedelta

from django.contrib.auth.models import User
from django.core.files.base import ContentFile
from django.core.management.base import BaseCommand
from django.db import transaction
from django.utils import timezone

```

```

from accounts.models import (
    Administrator,
    Applicant,
    Application,
    CalendarNote,
    Chat,
    Education,
    ExtraEducation,
    FilterPreset,
    Interview,
    Manager,
    Message,
    Moderator,
    UserFeedback,
    UserUiPreference,
    WorkExperience,
)
from vacancies.models import (
    Bookmark,
    Employer,
    ModeratorDeletionPhoto,
    ModeratorDeletionReport,
    Review,
    Vacancy,
    VacancyModerationState,
    VacancyReport,
    VacancyView,
)

class Command(BaseCommand):
    help = "Populate database with realistic demo data for UI testing."

    def add_arguments(self, parser):
        parser.add_argument(
            "--password",
            default="DemoPass123!",
            help="Password for all demo users (default: DemoPass123!)",
        )

    @transaction.atomic
    def handle(self, *args, **options):
        random.seed(42)
        password = options["password"]
        now = timezone.now()

        managers = self._create_managers(password=password)
        applicants = self._create_applicants(password=password)
        moderators = self._create_moderators(password=password)
        hh_employers = self._create_hh_employers()
        vacancies = self._create_vacancies(managers=managers,
        employers=hh_employers, now=now)

        self._create_reviews(vacancies=vacancies)
        applications = self._create_applications(applicants=applicants,
        vacancies=vacancies, now=now)
        self._create_bookmarks_and_views(applicants=applicants,
        vacancies=vacancies, now=now)
        self._create_vacancy_reports(applicants=applicants,
        vacancies=vacancies)
        self._create_moderation_states(moderators=moderators,
        vacancies=vacancies)
        self._create_moderator_deletion_reports(moderators=moderators,

```

```

vacancies=vacancies, now=now)
    self._create_user_feedback(users=[*managers, *applicants],
now=now)
    self._create_chats_and_messages(applications=applications,
now=now)
    self._create_interviews(applications=applications, now=now)
    self._create_calendar_notes(users=[*managers, *applicants,
*moderators], now=now)
    self._create_filter_presets(applicants=applicants)
    self._create_ui_preferences(users=[*managers, *applicants,
*moderators])

    self.stdout.write(self.style.SUCCESS("Demo data successfully
seeded."))
    self.stdout.write(
        self.style.WARNING(
            "Manager users: demo_manager_1..4 | Applicant users:
demo_applicant_1..10 | "
            "Moderator users: demo_moderator_1..3"
        )
    )
    self.stdout.write(self.style.WARNING(f"Password for all demo
users: {password}"))

def _create_user(self, username, password, first_name, last_name,
email):
    user, _ = User.objects.get_or_create(username=username)
    user.first_name = first_name
    user.last_name = last_name
    user.email = email
    user.is_active = True
    user.set_password(password)
    user.save()
    return user

def _create_managers(self, password):
    companies = [
        "ООО Альфа Тех",
        "ЗАО Север-Логистика",
        "ИП Соколова",
        "JobFlex Partners",
    ]
    cities = ["Москва", "Санкт-Петербург", "Казань", "Екатеринбург"]
    managers = []

    for i in range(1, 5):
        user = self._create_user(
            username=f"demo_manager_{i}",
            password=password,
            first_name=f"Менеджер{i}",
            last_name="Тестов",
            email=f"manager{i}@demo.local",
        )
        manager, _ = Manager.objects.get_or_create(user=user)
        manager.patronymic = "Иванович" if i % 2 else ""
        manager.company = companies[i - 1]
        manager.phone = f"+7 (9{i}1) 123-45-6{i}"
        manager.telegram = f"@demomgr{i}" if i != 4 else ""
        manager.consent_email = i % 2 == 1
        manager.consent_telegram = i in (1, 3)
        manager.save()
        managers.append(user)

```

```

        # Mirror manager city into applicant profile if it already
exists.
        # (Role switch UX in project relies on shared user and can
display either profile.)
        applicant = Applicant.objects.filter(user=user).first()
        if applicant:
            applicant.city = cities[i - 1]
            applicant.save(update_fields=["city"])

    return managers

    def _create_applicants(self, password):
        first_names = ["Алексей", "Мария", "Денис", "Ольга", "Руслан",
"Елена", "Игорь", "Светлана", "Павел", "Наталья"]
        last_names = ["Петров", "Смирнова", "Кузнецов", "Иванова",
"Соколов", "Волкова", "Лебедев", "Козлова", "Новиков", "Орлова"]
        cities = ["Москва", "Санкт-Петербург", "Новосибирск", "Казань",
"Самара", "Уфа", "Тула", "Краснодар", "Томск", "Воронеж"]
        citizenships = ["RU", "BY", "KZ", "KG", "AM", "TJ", "UZ", "UA",
"MD", "AZ"]
        applicants = []

        for i in range(1, 11):
            user = self._create_user(
                username=f"demo_applicant_{i}",
                password=password,
                first_name=first_names[i - 1],
                last_name=last_names[i - 1],
                email=f"applicant{i}@demo.local" if i % 3 != 0 else "",
            )
            applicant, _ = Applicant.objects.get_or_create(user=user)
            applicant.patronymic = "Сергеевич" if i % 2 else ""
            applicant.telegram = f"@demoapp{i}"
            applicant.phone = f"+7 (90{i}) 55{i}-2{i}-0{i}"
            applicant.gender = "M" if i % 2 else "F"
            applicant.city = cities[i - 1]
            applicant.birth_date = timezone.localtime() -
timedelta(days=365 * (20 + i))
            applicant.citizenship = citizenships[i - 1]
            applicant.skills = random.sample(
                ["Python", "Django", "SQL", "Docker", "Git", "React",
"Figma", "Excel", "1C", "Linux"],
                k=4,
            )
            applicant.consent_email = bool(user.email) and (i % 2 == 0)
            applicant.consent_telegram = i % 2 == 1
            applicant.salary_expectation_from = 70000 + i * 15000
            applicant.salary_expectation_to =
applicant.salary_expectation_from + 60000
            applicant.desired_position = random.choice(
                ["Python-разработчик", "Аналитик данных", "Менеджер
проектов", "QA инженер"]
            )
            applicant.about_me = "Ищу интересные проекты, где можно расти
и приносить пользу команде."
            if i % 2 == 0:
                applicant.location_type = "metro"
                applicant.metro_city_id = "1"
                applicant.metro_station_id = f"{10+i}"
                applicant.metro_station_name = random.choice(["Курская",
"Технопарк", "Площадь Ленина", "Горьковская"])
                applicant.metro_line_name =
random.choice(["Сокольническая", "Арбатско-Покровская", "Невско-
```

```

Василеостровская"])
        applicant.metro_line_color = random.choice(["#d32f2f",
"#388e3c", "#1976d2"])
        applicant.metro_stations = [
            {
                "cityId": "1",
                "stationId": applicant.metro_station_id,
                "stationName": applicant.metro_station_name,
                "lineName": applicant.metro_line_name,
                "lineColor": applicant.metro_line_color,
            }
        ]
        applicant.address = ""
    else:
        applicant.location_type = "address"
        applicant.address = f"г. {cities[i - 1]}, ул. Тестовая,
д. {i * 3}"
        applicant.metro_stations = []
        applicant.save()
        applicants.append(user)

    self._upsert_education_and_experience(applicant=applicant,
index=i)

    return applicants

def _create_moderators(self, password):
    specs = [
        ("demo_moderator_1", "Анастасия", "Фёдорова",
"mod1@demo.local"),
        ("demo_moderator_2", "Виктор", "Зайцев", "mod2@demo.local"),
        ("demo_moderator_3", "Полина", "Власова", "mod3@demo.local"),
    ]
    moderators = []
    for i, (username, first_name, last_name, email) in
enumerate(specs, start=1):
        user = self._create_user(
            username=username,
            password=password,
            first_name=first_name,
            last_name=last_name,
            email=email,
        )
        moderator, _ = Moderator.objects.get_or_create(user=user)
        moderator.patronymic = random.choice(["Александровна",
"Владимировна", "Сергеевна", ""])
        moderator.phone = f"+7 (925) 77{i}-1{i}-2{i}"
        moderator.telegram = ""
        moderator.notes = "Демо-профиль модератора для проверки
аналитики."
        moderator.save()
        moderators.append(user)
    return moderators

def _upsert_education_and_experience(self, applicant, index):
    Education.objects.filter(applicant=applicant).delete()
    WorkExperience.objects.filter(applicant=applicant).delete()
    ExtraEducation.objects.filter(applicant=applicant).delete()

    Education.objects.create(
        applicant=applicant,
        level=random.choice(["bachelor", "master", "higher",
"vocational"]),

```

```

        institution=random.choice(
            ["МГУ", "СПбГУ", "КФУ", "НГУ", "МГТУ им. Баумана",
"ИТМО"]
        ),
        graduation_year=2012 + index,
        faculty=random.choice(["Информатика", "Экономика",
"Менеджмент", "Прикладная математика"]),
        specialization=random.choice(["Разработка ПО", "Аналитика",
"Логистика", "Финансы"]),
        order=0,
    )
    if index % 3 == 0:
        Education.objects.create(
            applicant=applicant,
            level="candidate",
            institution="НИУ ВШЭ",
            graduation_year=2020 + (index % 4),
            faculty="Исследовательский факультет",
            specialization="Data Science",
            order=1,
        )

        WorkExperience.objects.create(
            applicant=applicant,
            company=random.choice(["Яндекс", "Сбер", "Т-Банк", "Ozon",
"VK"]),
            position=random.choice(["Инженер", "Аналитик", "Координатор
проектов"]),
            start_month=1 + (index % 12),
            start_year=2016 + index % 5,
            end_month=6 + (index % 6),
            end_year=2021 + (index % 3),
            is_current=False,
            responsibilities="Работа с клиентскими задачами,
автоматизация рутины, взаимодействие с командой.",
            order=0,
        )

        WorkExperience.objects.create(
            applicant=applicant,
            company=random.choice(["Wildberries", "Авито", "Ростелеком",
"Контур"]),
            position=random.choice(["Senior Специалист", "Team Lead",
"Продуктовый аналитик"]),
            start_month=2,
            start_year=2022,
            end_month=None,
            end_year=None,
            is_current=True,
            responsibilities="Веду ключевые процессы и помогаю развивать
продукт.",
            order=1,
        )

        ExtraEducation.objects.create(
            applicant=applicant,
            name=random.choice(["Курс по Docker", "SQL Advanced",
"Английский B2"]),
            description="Практический интенсив с проектной работой.",
            order=0,
        )

    def _create_hh_employers(self):
        employer_specs = [

```

```

        ("hh-demo-001", "ТехноСофт",
"https://logo.clearbit.com/yandex.ru"),
        ("hh-demo-002", "ФинГрупп",
"https://logo.clearbit.com/sber.ru"),
        ("hh-demo-003", "Логистик Про",
"https://logo.clearbit.com/ozon.ru"),
    ]
    employers = []
    for hh_id, name, logo in employer_specs:
        employer, _ = Employer.objects.get_or_create(hh_id=hh_id,
defaults={"name": name})
        employer.name = name
        employer.logo_url = logo
        employer.hh_rating = random.choice([3.9, 4.2, 4.5, 4.7])
        employer.raw = {"logo_urls": {"original": logo}}
        employer.save()
        employers.append(employer)
    return employers

def _create_vacancies(self, managers, employers, now):
    vacancies = []

    # External vacancies
    for i in range(1, 16):
        employer = employers[(i - 1) % len(employers)]
        external_id = f"demo_hh_{i:03d}"
        vacancy, _ = Vacancy.objects.get_or_create(
            external_id=external_id,
            defaults={"title": f"Внешняя вакансия {i}", "country":
"Россия", "published_at": now},
        )
        vacancy.title = random.choice(
            [
                "Backend Python Developer",
                "Data Analyst",
                "HR Manager",
                "DevOps Engineer",
                "Frontend Developer",
            ]
        )
        vacancy.company = employer.name
        vacancy.employer = employer
        vacancy.country = "Россия"
        vacancy.region = random.choice(["Москва", "Санкт-Петербург",
"Казань", "Новосибирск"])
        vacancy.experience_id = random.choice(["noExperience",
"between1And3", "between3And6"])
        vacancy.experience_name = random.choice(["Без опыта", "1-3
года", "3-6 лет"])
        vacancy.salary_from = random.choice([None, 80000, 120000,
180000])
        vacancy.salary_to = random.choice([None, 150000, 220000,
280000])
        vacancy.salary_currency = "RUR" if (vacancy.salary_from or
vacancy.salary_to) else ""
        vacancy.work_format =
random.choice([Vacancy.WorkFormat.REMOTE, Vacancy.WorkFormat.HYBRID,
Vacancy.WorkFormat.ONSITE])
        vacancy.is_remote = vacancy.work_format ==
Vacancy.WorkFormat.REMOTE
        vacancy.is_hybrid = vacancy.work_format ==
Vacancy.WorkFormat.HYBRID
        vacancy.is_onsite = vacancy.work_format ==

```

```

Vacancy.WorkFormat.ONSITE
        vacancy.schedule_id = random.choice(["fullDay", "shift",
"flexible", "remote"])
        vacancy.schedule_name = random.choice(["Полный день",
"Сменный график", "Гибкий график"])
        vacancy.employment_id = random.choice(["full", "part",
"project"])
        vacancy.employment_name = random.choice(["Полная занятость",
"Частичная занятость", "Проектная работа"])
        vacancy.employment_form_id = random.choice(["office",
"remote", "mixed"])
        vacancy.employment_form_name = random.choice(["Офис",
"Удалённо", "Смешанный"])
        vacancy.is_internship = i % 5 == 0
        vacancy.accept_temporary = i % 4 == 0
        vacancy.accept_incomplete_resumes = i % 3 == 0
        vacancy.accept_kids = i % 7 == 0
        vacancy.url = f"https://trudvsem.ru/vacancy/{900000 + i}"
        vacancy.published_at = now - timedelta(days=i)
        vacancy.raw_json = {"source": "hh", "employer": {"logo_urls":
{"original": employer.logo_url}}}
        vacancy.description = "Подробное описание вакансии с
обязанностями, требованиями и условиями."
        vacancy.branded_description = "<p>Брендированное описание
компании и команды.</p>"
        vacancy.key_skills_text = "Python, SQL, Git, Командная
работа"

        vacancy.created_by = None
        vacancy.is_active = True
        vacancy.save()
        vacancies.append(vacancy)

# Site-created vacancies by managers
for i in range(1, 13):
    manager_user = managers[(i - 1) % len(managers)]
    external_id = f"demo_site_{i:03d}"
    vacancy, _ = Vacancy.objects.get_or_create(
        external_id=external_id,
        defaults={"title": f"Site Vacancy {i}", "country":
"Россия", "published_at": now},
    )
    vacancy.title = random.choice(
        [
            "Менеджер по продажам",
            "Python-разработчик",
            "Оператор поддержки",
            "Системный аналитик",
            "Маркетолог",
        ]
    )
    vacancy.company = getattr(getattr(manager_user, "manager",
None), "company", "") or manager_user.get_full_name()
    vacancy.country = "Россия"
    vacancy.region = random.choice(["Москва", "Самара",
"Краснодар", "Пермь"])
    vacancy.experience_id = random.choice(["noExperience",
"between1And3", "between3And6"])
    vacancy.experience_name = random.choice(["Без опыта", "1-3
года", "3-6 лет"])
    vacancy.salary_from = random.choice([50000, 70000, 95000,
130000])
    vacancy.salary_to = vacancy.salary_from +
random.choice([30000, 50000, 80000])

```



```

        vacancy.salary_currency = "RUR"
        vacancy.work_format =
random.choice([Vacancy.WorkFormat.ONSITE, Vacancy.WorkFormat.HYBRID,
Vacancy.WorkFormat.REMOTE])
        vacancy.is_remote = vacancy.work_format ==
Vacancy.WorkFormat.REMOTE
        vacancy.is_hybrid = vacancy.work_format ==
Vacancy.WorkFormat.HYBRID
        vacancy.is_onsite = vacancy.work_format ==
Vacancy.WorkFormat.ONSITE
        vacancy.schedule_id = random.choice(["fullDay", "shift",
"flexible"])
        vacancy.schedule_name = random.choice(["Полный день",
"Сменный график", "Гибкий график"])
        vacancy.employment_id = random.choice(["full", "part",
"project", "volunteer"])
        vacancy.employment_name = random.choice(["Полная занятость",
"Частичная занятость", "Проектная работа"])
        vacancy.employment_form_id = random.choice(["office",
"remote", "mixed"])
        vacancy.employment_form_name = random.choice(["Офис",
"Удалённо", "Смешанный"])
        vacancy.is_internship = i % 6 == 0
        vacancy.accept_temporary = i % 3 == 0
        vacancy.accept_incomplete_resumes = i % 2 == 0
        vacancy.accept_kids = i % 5 == 0
        vacancy.url = ""
        vacancy.published_at = now - timedelta(days=(i + 2))
        vacancy.raw_json = {"source": "site"}
        vacancy.key_skills_text = random.choice(
            [
                "Коммуникация, CRM, B2B",
                "Python, Django, PostgreSQL",
                "Поддержка клиентов, HelpDesk",
                "SQL, BI, аналитика",
            ]
        )
        vacancy.description = "Требуется специалист в команду для
развития направления."
        vacancy.branded_description = ""
        vacancy.created_by = manager_user
        vacancy.is_active = i % 8 != 0
        vacancy.employee_type = random.choice(["permanent",
"temporary", ""])
        vacancy.contract_labor = i % 2 == 0
        vacancy.contract_gpc = i % 4 == 0
        vacancy.work_schedule = random.choice(["5/2", "2/2", "6/1",
"Свободный"])
        vacancy.hours_per_day = random.choice(["4", "6", "8", "10"])
        vacancy.has_night_shifts = i % 5 == 0
        vacancy.salary_gross = i % 2 == 1
        vacancy.salary_period = random.choice(["month", "project"])
        vacancy.payment_frequency = random.choice(["weekly",
"biweekly", "monthly"])
        vacancy.work_address = f"г. {vacancy.region}, ул. Рабочая, д.
{10 + i}"
        vacancy.hide_address = i % 7 == 0
        vacancy.address_comment = random.choice(["Бизнес-центр",
"Вход со двора", "Рядом с метро", ""])
        vacancy.lat = 55.75 + (i * 0.01)
        vacancy.lon = 37.61 + (i * 0.01)
        vacancy.contact_phone = f"+7 (901) 700-1{i:02d}"
        vacancy.metro_station_name = random.choice(["Курская",

```

```

"Таганская", "Белорусская", "Площадь Ленина"])
    vacancy.metro_line_color = random.choice(["#d32f2f",
"#1976d2", "#388e3c"])
    vacancy.metro_line_name = random.choice(["Красная", "Синяя",
"Зелёная"])
    vacancy.metro_city_id = "1"
    vacancy.benefits_text = "ДМС, обучение, гибкий график,
корпоративные мероприятия."
    vacancy.requirements_text = "Ответственность,
коммуникабельность, опыт работы с релевантными инструментами."
    vacancy.working_conditions_text = "Современный офис,
наставничество, понятные KPI."
    vacancy.save()
    vacancies.append(vacancy)

return vacancies

def _create_reviews(self, vacancies):
    texts = [
        "Отличный процесс собеседования, быстро дали обратную
связь.",
        "Интересные задачи и адекватная команда.",
        "Прозрачные условия, понятные требования.",
        "Хорошая коммуникация с HR и руководителем.",
    ]
    for i, vacancy in enumerate(vacancies[:20], start=1):
        Review.objects.get_or_create(
            vacancy=vacancy,
            author=f"Гость{i}",
            text=texts[i % len(texts)],
        )

def _create_applications(self, applicants, vacancies, now):
    site_vacancies = [v for v in vacancies if v.created_by_id and
v.is_active]
    statuses = [Application.STATUS_PENDING,
Application.STATUS_VIEWED, Application.STATUS_ACCEPTED,
Application.STATUS_REJECTED]
    applications = []

    for idx, applicant_user in enumerate(applicants, start=1):
        chosen = random.sample(site_vacancies, k=min(4,
len(site_vacancies)))
        for j, vacancy in enumerate(chosen, start=1):
            app, _ = Application.objects.get_or_create(
                vacancy=vacancy,
                applicant=applicant_user,
                defaults={"cover_letter": "Готов быстро включиться в
работу.", "status": statuses[(idx + j) % 4]},
            )
            app.cover_letter = random.choice(
                [
                    "Хочу применить опыт в вашей команде.",
                    "Готов к тестовому заданию и интервью в удобное
время.",
                    "Имею релевантный опыт и сильную мотивацию.",
                    "Ищу долгосрочный проект, где можно
развиваться.",
                ]
            )
            app.status = statuses[(idx + j) % 4]
            app.save(update_fields=["cover_letter", "status"])
            applications.append(app)

```

```

        return applications

    def _create_bookmarks_and_views(self, applicants, vacancies, now):
        active_vacancies = [v for v in vacancies if v.is_active]
        for i, user in enumerate(applicants, start=1):
            for vacancy in random.sample(active_vacancies, k=min(8,
len(active_vacancies))):
                Bookmark.objects.get_or_create(user=user,
vacancy=vacancy)
                for step, vacancy in
enumerate(random.sample(active_vacancies, k=min(10,
len(active_vacancies))), start=1):
                    view, _ = VacancyView.objects.get_or_create(user=user,
vacancy=vacancy)
                    view.viewed_at = now - timedelta(days=step, hours=i)
                    view.save(update_fields=["viewed_at"])

    def _create_chats_and_messages(self, applications, now):
        text_pairs = [
            ("Здравствуйте! Подскажите, пожалуйста, детали по вакансии.",
"Добрый день! Конечно, давайте обсудим."),
            ("Когда удобно пройти интервью?", "Завтра в 14:00 или в
пятницу в 11:00."),
            ("Есть ли удаленный формат?", "Да, гибридный формат возможен
после испытательного срока."),
        ]

        for app in applications[:30]:
            if not app.vacancy.created_by_id:
                continue
            manager_user = app.vacancy.created_by
            applicant_user = app.applicant
            chat, _ = Chat.objects.get_or_create(manager=manager_user,
applicant=applicant_user)
            for pidx, (q_text, a_text) in enumerate(text_pairs):
                q_msg, _ = Message.objects.get_or_create(
                    chat=chat,
                    sender=applicant_user,
                    text=q_text,
                )
                q_msg.created_at = now - timedelta(days=3 - pidx,
hours=2)

                q_msg.is_read = True
                q_msg.save(update_fields=["created_at", "is_read"])

                a_msg, _ = Message.objects.get_or_create(
                    chat=chat,
                    sender=manager_user,
                    text=a_text,
                )
                a_msg.created_at = now - timedelta(days=3 - pidx,
hours=1)

                a_msg.is_read = pidx % 2 == 0
                a_msg.save(update_fields=["created_at", "is_read"])

    def _create_interviews(self, applications, now):
        accepted_apps = [a for a in applications if a.status ==
Application.STATUS_ACCEPTED]
        for idx, app in enumerate(accepted_apps[:20], start=1):
            if not app.vacancy.created_by_id:
                continue
            status = random.choice([Interview.STATUS_SCHEDULED,
Interview.STATUS_CANCELLED, Interview.STATUS_DONE])

```

```

        interview, _ = Interview.objects.get_or_create(
            manager=app.vacancy.created_by,
            applicant=app.applicant,
            vacancy=app.vacancy,
            scheduled_at=now + timedelta(days=(idx % 10) + 1,
hours=idx % 5),
        )
        interview.location = random.choice(
            ["Google Meet", "Zoom", "Офис на Курской", "Telegram
call", "MS Teams"])
        )
        interview.notes = "Подготовить кейс и обсудить предыдущий
опыт."

        interview.status = status
        interview.reminded_1d = status != Interview.STATUS_SCHEDULED
        interview.reminded_1h = status == Interview.STATUS_DONE
        interview.reminded_now = status == Interview.STATUS_DONE
        interview.save()

    def _create_calendar_notes(self, users, now):
        colors = ["#c2a35a", "#f59e0b", "#22c55e", "#3b82f6", "#a855f7",
"#ef4444"]
        note_titles = [
            "Созвон с HR",
            "Тестовое задание",
            "Фоллоу-ап по отклику",
            "Собеседование",
            "Дедлайн портфолио",
            "Обновить резюме",
        ]
        for idx, user in enumerate(users, start=1):
            for d in range(-8, 12, 4):
                note_date = timezone.localtime() + timedelta(days=d +
(idx % 3))

                title = note_titles[(idx + d) % len(note_titles)]
                note_text = f"{title}: подготовить материалы и отметить
результат."

                note_time = None if (idx + d) % 3 == 0 else
timezone.datetime(2026, 1, 1, (9 + idx) % 24, (10 * idx) % 60).time()
                CalendarNote.objects.get_or_create(
                    user=user,
                    date=note_date,
                    title=title,
                    text=note_text,
                    defaults={
                        "color": colors[(idx + d) % len(colors)],
                        "note_time": note_time,
                        "reminded": (idx + d) % 4 == 0,
                    },
                )

    def _create_filter_presets(self, applicants):
        for idx, user in enumerate(applicants, start=1):
            FilterPreset.objects.update_or_create(
                user=user,
                name="Удаленка + высокий доход",
                defaults={
                    "filters": {
                        "q": "python",
                        "region": random.choice(["Москва", "Санкт-
Петербург", "Казань"]),
                        "salary_from": 120000 + idx * 5000,
                        "remote_only": True,

```

```

        "experience": ["between1And3", "between3And6"],
    }
    },
)
FilterPreset.objects.update_or_create(
    user=user,
    name="Стажировка / старт карьеры",
    defaults={
        "filters": {
            "q": "",
            "is_internship": True,
            "accept_incomplete_resumes": True,
            "employment": ["part", "project"],
            "schedule": ["flexible"],
        }
    },
)

def _create_ui_preferences(self, users):
    for idx, user in enumerate(users, start=1):
        UserUiPreference.objects.update_or_create(
            user=user,
            defaults={"theme": "dark" if idx % 2 == 0 else "light"},
        )

def _create_user_feedback(self, users, now):
    texts = [
        ("suggestion", "Добавьте отдельный фильтр по уровню зарплаты  
после налогов."),
        ("criticism", "Мобильная версия календаря иногда неудобна: не  
видно все заметки за день."),
        ("suggestion", "Хочется быстрый фильтр по типу занятости  
прямо в шапке страницы."),
        ("criticism", "В карточке вакансии не всегда сразу видно, кто  
работодатель."),
        ("suggestion", "Сделайте автосохранение черновика отклика при  
заполнении формы."),
        ("suggestion", "Нужна сортировка по дате обновления вакансии,  
а не только по релевантности."),
    ]
    for idx, user in enumerate(users[:10]):
        kind, msg = texts[idx % len(texts)]
        status = UserFeedback.STATUS_ACTIVE
        if idx in (7,):
            status = UserFeedback.STATUS_ARCHIVED
        elif idx in (8,):
            status = UserFeedback.STATUS_RESOLVED
        elif idx in (9,):
            status = UserFeedback.STATUS_REJECTED
        fb, _ = UserFeedback.objects.get_or_create(
            user=user,
            message=msg,
            defaults={'kind': kind, 'status': status},
        )
        fb.kind = kind
        fb.status = status
        fb.save(update_fields=['kind', 'status'])

def _create_vacancy_reports(self, applicants, vacancies):
    site_vacancies = [v for v in vacancies if v.created_by_id and not
v.is_moderator_deleted]
    if not site_vacancies:
        return

```

```

        reason_pool = [
            (VacancyReport.REASON_SCAM, "Подозрение на мошенничество:
просят оплату до трудоустройства."),
            (VacancyReport.REASON_SPAM, "Похоже на спам или рекламную
публикацию."),
            (VacancyReport.REASON_MISLEADING, "Описание вакансии не
совпадает с реальными требованиями."),
            (VacancyReport.REASON_SUSPICIOUS, "Подозрительные условия и
непрозрачные договоренности."),
            (VacancyReport.REASON_OTHER, "Требуется дополнительная
проверка модератором."),
        ]
        status_pool = [
            VacancyReport.SELF_STATUS_NEW,
            VacancyReport.SELF_STATUS_IN_WORK,
            VacancyReport.SELF_STATUS_DONE,
        ]
        for vac_idx, vacancy in enumerate(site_vacancies[:9]):
            report_count = min(3 + (vac_idx % 3), len(applicants))
            reporters = random.sample(applicants, k=report_count)
            for user_idx, user in enumerate(reporters):
                reason_code, reason_text = reason_pool[(vac_idx +
user_idx) % len(reason_pool)]
                self_status = status_pool[(vac_idx + user_idx) %
len(status_pool)]
                report, _ = VacancyReport.objects.get_or_create(
                    vacancy=vacancy,
                    user=user,
                    defaults={
                        "reason_code": reason_code,
                        "reason_text": reason_text,
                        "self_status": self_status,
                    },
                )
                report.reason_code = reason_code
                report.reason_text = reason_text
                report.self_status = self_status
                report.save(update_fields=["reason_code", "reason_text",
"self_status", "updated_at"])

    def _create_moderation_states(self, moderators, vacancies):
        site_vacancies = [v for v in vacancies if v.created_by_id and not
v.is_moderator_deleted][:8]
        if not site_vacancies:
            return
        statuses = [
            VacancyModerationState.STATUS_NEW,
            VacancyModerationState.STATUS_IN_WORK,
            VacancyModerationState.STATUS_WAITING,
        ]
        notes = [
            "Проверить историю жалоб менеджера.",
            "Запрошены дополнительные пояснения от работодателя.",
            "Ожидаю ответ от менеджера по текущему кейсу.",
            "Отмечены неоднозначные условия, нужна доп. проверка.",
            "",
        ]
        for m_idx, moderator in enumerate(moderators):
            for v_idx, vacancy in enumerate(site_vacancies):
                VacancyModerationState.objects.update_or_create(
                    vacancy=vacancy,
                    moderator=moderator,
                    defaults={

```

```

        "status": statuses[(m_idx + v_idx) %
len(statuses)],
        "note": notes[(m_idx * 2 + v_idx) % len(notes)],
    },
)

def _create_moderator_deletion_reports(self, moderators, vacancies,
now):
    site_vacancies = [v for v in vacancies if v.created_by_id]
    if len(site_vacancies) < 3 or not moderators:
        return

    admin_user = User.objects.filter(is_superuser=True).first()
    png_bytes = base64.b64decode(
        "iVBORw0KGgoAAAANSUgAAAAIAAAACCAIAAAD91JpzAAAADk1EQVQII12P4
z8BQDwADhQGAWjR9awAAAABJR5ErkJggg=="
    )

    # Active deletion report
    vac_active = site_vacancies[-1]
    mod_active = moderators[0]
    vac_active.is_moderator_deleted = True
    vac_active.is_active = False
    vac_active.save(update_fields=["is_moderator_deleted",
"is_active", "updated_at"])
    active_report, _ = ModeratorDeletionReport.objects.get_or_create(
        vacancy=vac_active,
        moderator=mod_active,
        defaults={
            "manager": vac_active.created_by,
            "reason": "Вакансия содержит признаки мошеннической схемы
и была скрыта до проверки.",
            "vacancy_title": vac_active.title,
            "vacancy_company": vac_active.company,
            "vacancy_description": vac_active.description[:800] if
vac_active.description else "",
            "vacancy_external_id": vac_active.external_id or "",
            "manager_full_name":
vac_active.created_by.get_full_name() if vac_active.created_by else "",
            "manager_email": vac_active.created_by.email if
vac_active.created_by else "",
            "moderator_full_name": mod_active.get_full_name(),
            "moderator_email": mod_active.email,
            "reports_count":
VacancyReport.objects.filter(vacancy=vac_active).count(),
            "dominant_reason_code": VacancyReport.REASON_SCAM,
            "dominant_reason_label": "Подозрение на мошенничество",
            "is_restored": False,
        },
    )
    if not active_report.photos.exists():
        ModeratorDeletionPhoto.objects.create(
            report=active_report,
            image=ContentFile(png_bytes, name="seed-evidence-
active.png"),
            order=0,
        )

    # Restored deletion report
    vac_restored = site_vacancies[-2]
    mod_restored = moderators[1] if len(moderators) > 1 else
moderators[0]
    vac_restored.is_moderator_deleted = False

```

```

        vac_restored.is_active = True
        vac_restored.save(update_fields=["is_moderator_deleted",
"is_active", "updated_at"])
        restored_report, _ =
ModeratorDeletionReport.objects.get_or_create(
            vacancy=vac_restored,
            moderator=mod_restored,
            defaults={
                "manager": vac_restored.created_by,
                "reason": "Описание вакансии было скорректировано, после
проверки восстановлена публикация.",
                "vacancy_title": vac_restored.title,
                "vacancy_company": vac_restored.company,
                "vacancy_description": vac_restored.description[:800] if
vac_restored.description else "",
                "vacancy_external_id": vac_restored.external_id or "",
                "manager_full_name":
vac_restored.created_by.get_full_name() if vac_restored.created_by else
"",
                "manager_email": vac_restored.created_by.email if
vac_restored.created_by else "",
                "moderator_full_name": mod_restored.get_full_name(),
                "moderator_email": mod_restored.email,
                "reports_count":
VacancyReport.objects.filter(vacancy=vac_restored).count(),
                "dominant_reason_code": VacancyReport.REASON_MISLEADING,
                "dominant_reason_label": "Некорректное описание
вакансии",
                "is_restored": True,
                "restored_by": admin_user if (admin_user and
Administrator.objects.filter(user=admin_user).exists()) else None,
                "restored_at": now - timedelta(days=1),
            },
        )
    if not restored_report.photos.exists():
        ModeratorDeletionPhoto.objects.create(
            report=restored_report,
            image=ContentFile(png_bytes, name="seed-evidence-
restored-1.png"),
            order=0,
        )
        ModeratorDeletionPhoto.objects.create(
            report=restored_report,
            image=ContentFile(png_bytes, name="seed-evidence-
restored-2.png"),
            order=1,
        )

```

7. accounts/management/commands/seed_moderator_data.py

"""Management command: populate demo data for the moderator role.

Run after seed_demo_data has already been executed:

```
python manage.py seed_moderator_data
python manage.py seed_moderator_data --password MyPass1!
```

Creates:

- 3 demo moderator accounts (demo_moderator_1..3)
- VacancyReport rows (user complaints) on site-created vacancies
- VacancyModerationState entries so moderators have a realistic complaint feed
- 2 ModeratorDeletionReport rows (one active, one already restored) with placeholder images so the admin reports page has content


```

All objects use get_or_create, so re-running is safe.
"""

import io
import random
from datetime import timedelta

from django.contrib.auth.models import User
from django.core.files.base import ContentFile
from django.core.management.base import BaseCommand
from django.db import transaction
from django.utils import timezone

from accounts.models import CalendarNote, Moderator
from vacancies.models import (
    ModeratorDeletionPhoto,
    ModeratorDeletionReport,
    Vacancy,
    VacancyModerationState,
    VacancyReport,
)

def _tiny_png() -> bytes:
    """Return a minimal valid 2x2 red PNG (no Pillow dependency)."""
    import base64
    # pre-encoded 2x2 solid-red PNG
    b64 = (
        "iVBORw0KGgoAAAANSUhEUgAAAAIAAAACCAIAAAD91JpzAAAADklEQVQI12P4z8BQ
"
        "DwADhQGAWjR9awAAAABJRU5ErkJggg=="
    )
    return base64.b64decode(b64)

class Command(BaseCommand):
    help = "Seed demo data for the Moderator role (complaints, states,
deletion reports)."
```

```

    def add_arguments(self, parser):
        parser.add_argument(
            "--password",
            default="DemoPass123!",
            help="Password for demo moderator accounts (default:
DemoPass123!)",
        )

    @transaction.atomic
    def handle(self, *args, **options):
        random.seed(7)
        password = options["password"]
        now = timezone.now()

        moderators = self._create_moderators(password=password)
        applicants = self._get_demo_applicants()
        site_vacancies = self._get_site_vacancies()

        if not site_vacancies:
            self.stdout.write(
                self.style.WARNING(
                    "No site-created vacancies found. Run seed_demo_data
first."
                )
            )

```

```

        return

    if not applicants:
        self.stdout.write(
            self.style.WARNING(
                "No demo applicants found. Run seed_demo_data first."
            )
        )
        return

    reports = self._create_vacancy_reports(applicants=applicants,
site_vacancies=site_vacancies, now=now)
    self._create_moderation_states(moderators=moderators,
site_vacancies=site_vacancies, now=now)
    self._create_deletion_reports(moderators=moderators,
site_vacancies=site_vacancies, now=now)
    self._create_calendar_notes(moderators=moderators, now=now)

    self.stdout.write(self.style.SUCCESS("Moderator demo data seeded
successfully."))
    self.stdout.write(self.style.WARNING(
        "Moderator accounts: demo_moderator_1, demo_moderator_2,
demo_moderator_3"
    ))
    self.stdout.write(self.style.WARNING(f"Password: {password}"))
    self.stdout.write(self.style.WARNING(
        f"Created {len(reports)} user complaint(s) on site
vacancies."
    ))
    # -----
    #  Helpers
    # -----

    def _create_user(self, username, password, first_name, last_name,
email):
        user, _ = User.objects.get_or_create(username=username)
        user.first_name = first_name
        user.last_name = last_name
        user.email = email
        user.is_active = True
        user.set_password(password)
        user.save()
        return user

    # -----
    #  Moderators
    # -----

    def _create_moderators(self, password):
        specs = [
            ("demo_moderator_1", "Анастасия", "Фёдорова",
"mod1@demo.local"),
            ("demo_moderator_2",
"Виктор", "Зайцев", "mod2@demo.local"),
            ("demo_moderator_3",
"Полина", "Власова", "mod3@demo.local"),
        ]
        moderators = []
        for username, first, last, email in specs:
            user = self._create_user(username, password, first, last,
email)
            mod, _ = Moderator.objects.get_or_create(user=user)
            mod.patronymic = random.choice(["Александровна",

```

```

"Владимировна", "Сергеевна",
                                "Иванович", "Петрович", "")
        mod.phone = f"+7 (915) {random.randint(100, 999)}-
{random.randint(10, 99)}-{random.randint(10, 99)}"
        mod.notes = "Демо-модератор для тестирования интерфейса."
        mod.save()
        moderators.append(user)
    return moderators

# -----
# Existing demo data lookups
# -----

def _get_demo_applicants(self):
    return list(
        User.objects.filter(username__startswith="demo_applicant_")
        .order_by("username")
    )

def _get_site_vacancies(self):
    """Return site-created (non-HH) active vacancies, excluding
already-deleted ones."""
    return list(
        Vacancy.objects.filter(
            external_id__startswith="demo_site_",
            is_moderator_deleted=False,
        ).order_by("external_id")
    )

# -----
# Vacancy reports (user complaints)
# -----

REASONS = [
    (VacancyReport.REASON_SCAM, "Просят переводить деньги или
купить что-то до работы."),
    (VacancyReport.REASON_SPAM, "Вакансия — скрытая реклама
курсов."),
    (VacancyReport.REASON_MISLEADING, "Реальные условия отличаются
от описания."),
    (VacancyReport.REASON_SUSPICIOUS, "Работодатель не отвечает на
вопросы, требует NDA сразу."),
    (VacancyReport.REASON_OTHER, "Вакансия выглядит
подозрительно."),
]

def _create_vacancy_reports(self, applicants, site_vacancies, now):
    created = []
    # Pick at most 8 vacancies to receive complaints
    target_vacancies = site_vacancies[:8]
    reason_pool = self.REASONS

    for vac_idx, vacancy in enumerate(target_vacancies):
        # Each vacancy gets 2-5 reporters (different applicants)
        num_reporters = min(2 + (vac_idx % 4), len(applicants))
        reporters = random.sample(applicants, k=num_reporters)
        for user_idx, user in enumerate(reporters):
            reason_code, reason_text = reason_pool[(vac_idx +
user_idx) % len(reason_pool)]
            self_status = random.choice([
                VacancyReport.SELF_STATUS_NEW,
                VacancyReport.SELF_STATUS_IN_WORK,
                VacancyReport.SELF_STATUS_NEW, # weight NEW higher

```

```

    ])
    report, created_now =
VacancyReport.objects.get_or_create(
    vacancy=vacancy,
    user=user,
    defaults={
        "reason_code": reason_code,
        "reason_text": reason_text,
        "self_status": self_status,
    },
)
    if not created_now:
        # Update existing to keep data fresh
        report.reason_code = reason_code
        report.reason_text = reason_text
        report.self_status = self_status
        report.save(update_fields=["reason_code",
"reason_text", "self_status"])
        created.append(report)
    return created

# -----
# Moderation states (per moderator, per vacancy card)
# -----

def _create_moderation_states(self, moderators, site_vacancies, now):
    statuses = [
        VacancyModerationState.STATUS_NEW,
        VacancyModerationState.STATUS_IN_WORK,
        VacancyModerationState.STATUS_WAITING,
    ]
    notes = [
        "Проверить историю менеджера — были жалобы ранее.",
        "Запросить дополнительные документы у работодателя.",
        "Похоже на паттерн мошенничества, нужна второе мнение.",
        "Ожидаю ответа от менеджера по почте.",
        "Вакансия выглядит нормально, оставил под наблюдением.",
        "",
    ]
    # Give each moderator states for the first 6 reported vacancies
    target = site_vacancies[:6]
    for mod_idx, mod_user in enumerate(moderators):
        for vac_idx, vacancy in enumerate(target):
            status = statuses[(mod_idx + vac_idx) % len(statuses)]
            note = notes[(mod_idx * 2 + vac_idx) % len(notes)]
            VacancyModerationState.objects.update_or_create(
                vacancy=vacancy,
                moderator=mod_user,
                defaults={"status": status, "note": note},
            )

# -----
# Deletion reports
# -----

def _create_deletion_reports(self, moderators, site_vacancies, now):
    if len(site_vacancies) < 3 or not moderators:
        return

    png = _tiny_png()

    # --- Report 1: active (not restored) ---
    vac1 = site_vacancies[-1]      # last site vacancy → soft-deleted

```

```

mod_user = moderators[0]
manager_user = vac1.created_by

# soft-delete the vacancy if not already deleted
if not vac1.is_moderator_deleted:
    vac1.is_moderator_deleted = True
    vac1.is_active = False
    vac1.save(update_fields=["is_moderator_deleted",
"is_active"])

    repl, _ = ModeratorDeletionReport.objects.get_or_create(
        vacancy=vac1,
        moderator=mod_user,
        defaults={
            "manager": manager_user,
            "reason": (
                "Вакансия содержит признаки мошенничества: от
соискателя требуют "
                "приобрести оборудование за собственный счёт, возврат
якобы гарантирован "
                "после испытательного срока. Несколько пользователей
подтвердили схему."
            ),
            "vacancy_title": vac1.title,
            "vacancy_company": vac1.company,
            "vacancy_description": vac1.description[:500] if
vac1.description else "",
            "vacancy_external_id": vac1.external_id or "",
            "manager_full_name": manager_user.get_full_name() if
manager_user else "",
            "manager_email": manager_user.email if manager_user else
"",
            "moderator_full_name": mod_user.get_full_name(),
            "moderator_email": mod_user.email,
            "reports_count":
VacancyReport.objects.filter(vacancy=vac1).count(),
            "dominant_reason_code": VacancyReport.REASON_SCAM,
            "dominant_reason_label": "Подозрение на мошенничество",
            "is_restored": False,
        },
    )
    # Attach a demo photo if none yet
    if not repl.photos.exists():
        ModeratorDeletionPhoto.objects.create(
            report=repl,
            image=ContentFile(png, name="screenshot_evidence.png"),
            order=0,
        )

    # --- Report 2: restored by admin ---
    vac2 = site_vacancies[-2]
    mod_user2 = moderators[1] if len(moderators) > 1 else
moderators[0]
    manager_user2 = vac2.created_by

    admin_user = User.objects.filter(is_superuser=True).first()

    rep2, _ = ModeratorDeletionReport.objects.get_or_create(
        vacancy=vac2,
        moderator=mod_user2,
        defaults={
            "manager": manager_user2,
            "reason": (

```

```

занятость, "
        "Описание вводило в заблуждение: указана полная
        "по факту предлагается работа по ГПХ без социальных
гарантий. "
        "Менеджер согласился скорректировать объявление."
    ),
    "vacancy_title": vac2.title,
    "vacancy_company": vac2.company,
    "vacancy_description": vac2.description[:500] if
vac2.description else "",
    "vacancy_external_id": vac2.external_id or "",
    "manager_full_name": manager_user2.get_full_name() if
manager_user2 else "",
    "manager_email": manager_user2.email if manager_user2
else "",
    "moderator_full_name": mod_user2.get_full_name(),
    "moderator_email": mod_user2.email,
    "reports_count":
VacancyReport.objects.filter(vacancy=vac2).count(),
    "dominant_reason_code": VacancyReport.REASON_MISLEADING,
    "dominant_reason_label": "Некорректное описание
вакансии",
    "is_restored": True,
    "restored_by": admin_user,
    "restored_at": now - timedelta(days=1),
    },
    )
    # Ensure vacancy itself is visible again (restored)
    if vac2.is_moderator_deleted:
        vac2.is_moderator_deleted = False
        vac2.is_active = True
        vac2.save(update_fields=["is_moderator_deleted",
"is_active"])

    if not rep2.photos.exists():
        ModeratorDeletionPhoto.objects.create(
            report=rep2,
            image=ContentFile(png, name="evidence_before.png"),
            order=0,
        )
        ModeratorDeletionPhoto.objects.create(
            report=rep2,
            image=ContentFile(png, name="evidence_after.png"),
            order=1,
        )

# -----
# Calendar notes for moderators
# -----

def _create_calendar_notes(self, moderators, now):
    colors = ["#3b82f6", "#22c55e", "#f59e0b", "#a855f7", "#ef4444"]
    titles = [
        "Разобрать новые жалобы",
        "Созвон с командой модерации",
        "Проверить статус вакансий «В работе»",
        "Написать отчёт за неделю",
        "Обновить критерии модерации",
        "Плановая проверка старых жалоб",
    ]
    for idx, user in enumerate(moderators, start=1):
        for d in range(-4, 10, 3):
            note_date = timezone.localdate() + timedelta(days=d +

```

```

idx)

        title = titles[(idx + d) % len(titles)]
        CalendarNote.objects.get_or_create(
            user=user,
            date=note_date,
            title=title,
            defaults={
                "text": f"{title}: проверить очередь и
зафиксировать результат.",
                "color": colors[(idx + d) % len(colors)],
                "note_time": None if (idx + d) % 3 == 0 else
                    timezone.datetime(2026, 1, 1, (9 +
idx) % 24, 0).time(),
                "reminded": False,
            },
        )
    )

```

8. accounts/models.py

```

from django.db import models
from django.contrib.auth.models import User

class Applicant(models.Model):
    GENDER_CHOICES = [('M', 'Мужской'), ('F', 'Женский')]
    CITIZENSHIP_CHOICES = [
        ('RU', 'Россия'), ('BY', 'Беларусь'), ('KZ', 'Казахстан'),
        ('KG', 'Киргизия'), ('AM', 'Армения'), ('TJ', 'Таджикистан'),
        ('UZ', 'Узбекистан'), ('UA', 'Украина'), ('MD', 'Молдова'),
        ('AZ', 'Азербайджан'), ('TM', 'Туркменистан'),
    ]

    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='applicant', verbose_name='Пользователь')
    patronymic = models.CharField('Отчество', max_length=150, blank=True)
    telegram = models.CharField('Телеграм', max_length=100)
    telegram_chat_id = models.BigIntegerField('ID чата Telegram',
null=True, blank=True)
    telegram_start_token = models.CharField('Стартовый токен Telegram',
max_length=64, null=True, blank=True, unique=True)
    consent_email = models.BooleanField('Согласие на e-mail',
default=False)
    consent_telegram = models.BooleanField('Согласие на Telegram',
default=False)
    phone = models.CharField('Телефон', max_length=30, blank=True)
    gender = models.CharField('Пол', max_length=1,
choices=GENDER_CHOICES, blank=True)
    city = models.CharField('Город', max_length=150, blank=True)
    birth_date = models.DateField('Дата рождения', null=True, blank=True)
    citizenship = models.CharField('Гражданство', max_length=2,
choices=CITIZENSHIP_CHOICES, blank=True)
    skills = models.JSONField('Навыки', default=list, blank=True)
    # Location preference: nearest metro station -OR- free-text address
    LOCATION_TYPE_CHOICES = [('metro', 'Ст. метро'), ('address',
'Адрес'), ('', 'Не указано')]
    location_type = models.CharField('Тип местоположения',
max_length=10, blank=True, default='')
    metro_city_id = models.CharField('ID города метро (HH)',
max_length=10, blank=True)
    metro_station_id = models.CharField('ID станции метро (HH)',
max_length=20, blank=True)
    metro_station_name = models.CharField('Название станции',
max_length=150, blank=True)
    metro_line_name = models.CharField('Название линии',

```

```

max_length=150, blank=True)
    metro_line_color = models.CharField('Цвет линии HEX',
max_length=7, blank=True)
    metro_stations = models.JSONField('Список станций метро',
default=list, blank=True)
    address = models.CharField('Адрес', max_length=500,
blank=True)
    # Salary expectations
    salary_expectation_from = models.IntegerField('Зарплата от
(ожидаемая)', null=True, blank=True)
    salary_expectation_to = models.IntegerField('Зарплата до
(ожидаемая)', null=True, blank=True)
    avatar = models.ImageField('Аватар', upload_to='avatars/%Y/',
null=True, blank=True)
    # Resume extras
    about_me = models.TextField('О себе', blank=True)
    desired_position = models.CharField('Желаемая должность',
max_length=255, blank=True)
    github_url = models.URLField('GitHub', max_length=300,
blank=True)
    portfolio_url = models.URLField('Портфолио / сайт',
max_length=300, blank=True)
    # Role history
    was_manager = models.BooleanField('Был менеджером', default=False)
    # Preserved company logo path when this applicant temporarily
switches back from manager role
    manager_company_logo = models.CharField('Путь к логотипу компании',
max_length=500, blank=True)
    # Uploaded Word resume file
    resume_file = models.FileField('Файл резюме (Word)',
upload_to='resumes/%Y/', null=True, blank=True)

    def __str__(self):
        return f"{self.user.get_full_name()} <{self.user.email}>"

    class Meta:
        verbose_name = 'Соискатель'
        verbose_name_plural = 'Соискатели'

class Education(models.Model):
    LEVEL_CHOICES = [
        ('secondary', 'Среднее'),
        ('vocational', 'Среднее специальное'),
        ('incomplete_higher', 'Неоконченное высшее'),
        ('higher', 'Высшее'),
        ('bachelor', 'Бакалавр'),
        ('master', 'Магистр'),
        ('candidate', 'Кандидат наук'),
        ('doctor', 'Доктор наук'),
    ]

    applicant = models.ForeignKey(Applicant,
on_delete=models.CASCADE, related_name='educations')
    level = models.CharField('Уровень образования',
max_length=30, choices=LEVEL_CHOICES)
    institution = models.CharField('Учебное заведение',
max_length=255)
    graduation_year = models.IntegerField('Год выпуска/окончания',
null=True, blank=True)
    faculty = models.CharField('Факультет', max_length=255,
blank=True)
    specialization = models.CharField('Специализация', max_length=255,
blank=True)

```



```

        order = models.PositiveSmallIntegerField('Порядок',
default=0)

    class Meta:
        verbose_name = 'Образование'
        verbose_name_plural = 'Образование'
        ordering = ['order']

    def __str__(self):
        return f"{self.get_level_display()} - {self.institution}"

class ExtraEducation(models.Model):
    applicant = models.ForeignKey(Applicant, on_delete=models.CASCADE,
related_name='extra_educations')
    name = models.CharField('Название', max_length=255)
    document_serial = models.CharField('Серийный номер документа',
max_length=120, blank=True)
    description = models.TextField('Описание', blank=True)
    order = models.PositiveSmallIntegerField('Порядок', default=0)

    class Meta:
        verbose_name = 'Доп. образование'
        verbose_name_plural = 'Доп. образование'
        ordering = ['order']

    def __str__(self):
        return self.name

class WorkExperience(models.Model):
    MONTH_CHOICES = [
        (1, 'Январь'), (2, 'Февраль'), (3, 'Март'), (4, 'Апрель'),
        (5, 'Май'), (6, 'Июнь'), (7, 'Июль'), (8, 'Август'),
        (9, 'Сентябрь'), (10, 'Октябрь'), (11, 'Ноябрь'), (12,
'Декабрь'),
    ]

    applicant = models.ForeignKey(Applicant,
on_delete=models.CASCADE, related_name='work_experiences')
    company = models.CharField('Компания', max_length=255)
    position = models.CharField('Должность', max_length=255)
    start_month = models.IntegerField('Месяц начала', null=True,
blank=True, choices=MONTH_CHOICES)
    start_year = models.IntegerField('Год начала', null=True,
blank=True)
    end_month = models.IntegerField('Месяц окончания', null=True,
blank=True, choices=MONTH_CHOICES)
    end_year = models.IntegerField('Год окончания', null=True,
blank=True)
    is_current = models.BooleanField('Работаю сейчас',
default=False)
    responsibilities = models.TextField('Обязанности и достижения',
blank=True)
    order = models.PositiveSmallIntegerField('Порядок',
default=0)

    class Meta:
        verbose_name = 'Опыт работы'
        verbose_name_plural = 'Опыт работы'
        ordering = ['order']

    def __str__(self):
        return f"{self.position} - {self.company}"

```

```

class Manager(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='manager', verbose_name='Пользователь')
    patronymic = models.CharField('Отчество', max_length=150, blank=True)
    company = models.CharField('Компания', max_length=255, blank=True)
    phone = models.CharField('Телефон', max_length=30, blank=True)
    telegram = models.CharField('Телеграм', max_length=100, blank=True)
    telegram_chat_id = models.BigIntegerField('ID чата Telegram',
null=True, blank=True)
    consent_email = models.BooleanField('Согласие на e-mail',
default=False)
    consent_telegram = models.BooleanField('Согласие на Telegram',
default=False)
    avatar = models.ImageField('Аватар', upload_to='avatars/%Y/',
null=True, blank=True)
    company_logo = models.ImageField('Логотип компании',
upload_to='company_logos/%Y/', null=True, blank=True)

    class Meta:
        verbose_name = 'Менеджер'
        verbose_name_plural = 'Менеджеры'

    def __str__(self):
        return f"{self.user.get_full_name()} <{self.user.email}>"

class Administrator(models.Model):
    """
    Профиль администратора системы.
    Пользователь считается администратором только если:
    • он является superuser (user.is_superuser == True)
    • у него есть этот профиль (Administrator)
    """
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name='admin_profile',
        verbose_name='Пользователь',
    )
    created_at = models.DateTimeField('Дата назначения',
auto_now_add=True)
    notes = models.TextField('Заметки', blank=True)

    class Meta:
        verbose_name = 'Администратор'
        verbose_name_plural = 'Администраторы'

    def clean(self):
        from django.core.exceptions import ValidationError
        if not self.user.is_superuser:
            raise ValidationError(
                'Администратором может быть только суперпользователь
(is_superuser=True).'
            )

    def save(self, *args, **kwargs):
        self.full_clean()
        super().save(*args, **kwargs)

    def __str__(self):
        return f"Admin: {self.user.get_full_name() or
self.user.username}"

class Moderator(models.Model):

```

```

"""
Профиль модератора системы.
Модератор не является superuser по умолчанию и имеет отдельный
кабинет.
"""
user = models.OneToOneField(
    User,
    on_delete=models.CASCADE,
    related_name='moderator_profile',
    verbose_name='Пользователь',
)
patronymic = models.CharField('Отчество', max_length=150, blank=True)
phone = models.CharField('Телефон', max_length=30, blank=True)
telegram = models.CharField('Telegram', max_length=100, blank=True)
notes = models.TextField('Заметки', blank=True)
created_at = models.DateTimeField('Дата назначения',
auto_now_add=True)

class Meta:
    verbose_name = 'Модератор'
    verbose_name_plural = 'Модераторы'

    def __str__(self):
        return f'Moderator: {self.user.get_full_name() or
self.user.username}'

class Chat(models.Model):
    """Прямой чат между одним менеджером и одним соискателем."""
    manager = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='manager_chats')
    applicant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='applicant_chats')
    created_at = models.DateTimeField(auto_now_add=True)
    deleted_by_manager = models.BooleanField('Удалён
менеджером', default=False)
    deleted_by_applicant = models.BooleanField('Удалён соискателем',
default=False)

    class Meta:
        unique_together = [('manager', 'applicant')]
        ordering = ['-created_at']
        verbose_name = 'Чат'
        verbose_name_plural = 'Чаты'

    def __str__(self):
        return f'Чат {self.manager.username} ↔ {self.applicant.username}'

    def other_user(self, me):
        return self.applicant if me == self.manager else self.manager

    def unread_count(self, me):
        return
self.messages.filter(is_read=False).exclude(sender=me).count()

class Message(models.Model):
    chat = models.ForeignKey(Chat, on_delete=models.CASCADE,
related_name='messages')
    sender = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='sent_messages')
    text = models.TextField('Текст', blank=True)
    file = models.FileField('Файл', upload_to='chat_files/%Y/%m/',
null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

```

```

is_read = models.BooleanField('Прочитано', default=False)

class Meta:
    ordering = ['created_at']
    verbose_name = 'Сообщение'
    verbose_name_plural = 'Сообщения'

@property
def file_basename(self):
    """Возвращает только имя файла без префикса пути загрузки."""
    if not self.file:
        return ''
    import os
    return os.path.basename(self.file.name)

@property
def file_is_image(self):
    import os
    if not self.file:
        return False
    ext = os.path.splitext(self.file.name)[1].lower()
    return ext in ('.jpg', '.jpeg', '.png', '.gif', '.bmp', '.webp',
'.svg')

class Application(models.Model):
    """A job application: one applicant → one vacancy."""
    STATUS_PENDING = 'pending'
    STATUS_VIEWED = 'viewed'
    STATUS_ACCEPTED = 'accepted'
    STATUS_REJECTED = 'rejected'
    STATUS_CHOICES = [
        ('pending', 'Ожидает'),
        ('viewed', 'Просмотрено'),
        ('accepted', 'Приглашение'),
        ('rejected', 'Отклонено'),
    ]

    vacancy = models.ForeignKey('vacancies.Vacancy',
on_delete=models.CASCADE,
related_name='applications')
    applicant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='applications')
    cover_letter = models.TextField('Сопроводительное письмо',
blank=True)
    status = models.CharField('Статус', max_length=16,
choices=STATUS_CHOICES, default='pending')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = [('vacancy', 'applicant')]
        ordering = ['-created_at']
        verbose_name = 'Отклик'
        verbose_name_plural = 'Отклики'

    def __str__(self):
        return f"{self.applicant.get_full_name()} → {self.vacancy.title}"

    def __str__(self):
        return f"{self.sender.username}: {self.text[:40]}"

class FilterPreset(models.Model):
    """A saved set of vacancy search filters belonging to one user."""
    user = models.ForeignKey(User, on_delete=models.CASCADE,

```

```

related_name='filter_presets')
    name = models.CharField('Название', max_length=100)
    # All filter params stored as a plain dict, mirrors GET params used
    by VacancyListView.
    # Multi-value params (experience, schedule, etc.) are stored as
    lists.
    filters = models.JSONField('Фильтры', default=dict)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['name']
        verbose_name = 'Пресет фильтров'
        verbose_name_plural = 'Пресеты фильтров'

    def __str__(self):
        return f"{self.user.username}: {self.name}"

class UserUiPreference(models.Model):
    """Persistent UI preferences (theme, etc.) for a user."""
    THEME_CHOICES = [('light', 'Light'), ('dark', 'Dark')]

    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='ui_preference')
    theme = models.CharField('Тема сайта', max_length=10,
choices=THEME_CHOICES, default='light')
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = 'Настройка интерфейса'
        verbose_name_plural = 'Настройки интерфейса'

    def __str__(self):
        return f"{self.user.username}: {self.theme}"

class CalendarNote(models.Model):
    """A personal note added by a user to a specific calendar day."""
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='calendar_notes')
    date = models.DateField('Дата')
    title = models.CharField('Название', max_length=200, blank=True,
default='')
    text = models.TextField('Текст заметки', blank=True,
default='')
    color = models.CharField('Цвет', max_length=20, blank=True,
default='#c2a35a')
    note_time = models.TimeField('Время', null=True, blank=True)
    reminded = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['created_at']
        verbose_name = 'Заметка календаря'
        verbose_name_plural = 'Заметки календаря'

    def __str__(self):
        return f"{self.user.username} [{self.date}]: {self.text[:40]}"

class Interview(models.Model):
    """A scheduled interview between a manager and an applicant."""
    STATUS_SCHEDULED = 'scheduled'
    STATUS_CANCELLED = 'cancelled'
    STATUS_DONE = 'done'

```

```

STATUS_CHOICES = [
    ('scheduled', 'Запланировано'),
    ('cancelled', 'Отменено'),
    ('done', 'Завершено'),
]

manager = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='interviews_as_manager')
applicant = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='interviews_as_applicant')
vacancy = models.ForeignKey('vacancies.Vacancy',
on_delete=models.CASCADE,
related_name='interviews',
null=True, blank=True)
scheduled_at = models.DateTimeField('Дата и время')
location = models.CharField('Место / ссылка', max_length=500,
blank=True)
notes = models.TextField('Заметки', blank=True)
status = models.CharField('Статус', max_length=16,
choices=STATUS_CHOICES, default='scheduled')
created_at = models.DateTimeField(auto_now_add=True)

# Reminder flags - set to True once the reminder has been sent
reminded_1d = models.BooleanField(default=False)
reminded_1h = models.BooleanField(default=False)
reminded_now = models.BooleanField(default=False)

class Meta:
    ordering = ['scheduled_at']
    verbose_name = 'Собеседование'
    verbose_name_plural = 'Собеседования'

def __str__(self):
    return f"{self.manager.get_full_name()} → {self.applicant.get_full_name()} [{self.scheduled_at:%d.%m.%Y %H:%M}]"

class ApiActionLog(models.Model):
    """Audit log for selected non-frequent API actions."""
    ACTOR_CHOICES = [
        ('system', 'Система'),
        ('admin', 'Администратор'),
        ('moderator', 'Модератор'),
        ('manager', 'Менеджер'),
        ('applicant', 'Соискатель'),
        ('user', 'Пользователь'),
    ]

    user = models.ForeignKey(
        User,
        on_delete=models.SET_NULL,
        related_name='api_action_logs',
        null=True,
        blank=True,
    )
    actor_role = models.CharField('Роль', max_length=20,
choices=ACTOR_CHOICES, default='system')
method = models.CharField('HTTP-метод', max_length=10)
path = models.CharField('Путь', max_length=255)
endpoint = models.CharField('Эндпоинт', max_length=120, blank=True)
action = models.CharField('Действие', max_length=120)
success = models.BooleanField('Успешно', default=True)
status_code = models.PositiveSmallIntegerField('Код ответа',
null=True, blank=True)

```

```

        before_data = models.JSONField('До изменения', default=dict,
blank=True)
        after_data = models.JSONField('После изменения', default=dict,
blank=True)
        meta = models.JSONField('Метаданные', default=dict, blank=True)
        created_at = models.DateTimeField('Создано', auto_now_add=True)

class Meta:
    ordering = ['-created_at']
    verbose_name = 'Лог API-действия'
    verbose_name_plural = 'Логи API-действий'

    def __str__(self):
        return f"{self.created_at:%d.%m.%Y %H:%M:%S} {self.method} {self.action} ({self.actor_role})"

class UserFeedback(models.Model):
    """Suggestion or criticism sent by a non-admin/non-moderator user."""
    KIND_SUGGESTION = 'suggestion'
    KIND_CRITICISM = 'criticism'
    KIND_CHOICES = [
        (KIND_SUGGESTION, 'Предложение'),
        (KIND_CRITICISM, 'Критика'),
    ]

    user = models.ForeignKey(
        User,
        on_delete=models.CASCADE,
        related_name='feedback_items',
    )
    kind = models.CharField(max_length=16, choices=KIND_CHOICES,
default=KIND_SUGGESTION)
    message = models.TextField('Сообщение')
    STATUS_ACTIVE = 'active'
    STATUS_ARCHIVED = 'archived'
    STATUS_RESOLVED = 'resolved'
    STATUS_REJECTED = 'rejected'
    STATUS_CHOICES = [
        (STATUS_ACTIVE, 'В ленте'),
        (STATUS_ARCHIVED, 'В архиве'),
        (STATUS_RESOLVED, 'Реализовано'),
        (STATUS_REJECTED, 'Отклонено'),
    ]
    status = models.CharField(max_length=16, choices=STATUS_CHOICES,
default=STATUS_ACTIVE, db_index=True)
    processed_at = models.DateTimeField(null=True, blank=True)
    processed_by = models.ForeignKey(
        User,
        null=True,
        blank=True,
        on_delete=models.SET_NULL,
        related_name='processed_feedback_items',
    )
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)

class Meta:
    ordering = ['-created_at']
    verbose_name = 'Предложение/критика пользователя'
    verbose_name_plural = 'Предложения и критика пользователей'

    def __str__(self):
        return f"{self.user.username}: {self.get_kind_display()} ({self.created_at:%d.%m.%Y})"

```

```

class UserDocument(models.Model):
    """Personal identity/tax/social document attached in user profile."""
    DOC_PASSPORT_RF = 'passport_rf'
    DOC_SNILS = 'snils'
    DOC_INN = 'inn'
    DOC_FOREIGN_PASSPORT = 'foreign_passport'
    DOC_DRIVER_LICENSE = 'driver_license'
    DOC_MILITARY_ID = 'military_id'
    DOC_TYPE_CHOICES = [
        (DOC_PASSPORT_RF, 'Паспорт РФ'),
        (DOC_SNILS, 'СНИЛС'),
        (DOC_INN, 'ИНН'),
        (DOC_FOREIGN_PASSPORT, 'Загранпаспорт'),
        (DOC_DRIVER_LICENSE, 'Водительское удостоверение'),
        (DOC_MILITARY_ID, 'Военный билет'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='documents')
    doc_type = models.CharField('Тип документа', max_length=32,
choices=DOC_TYPE_CHOICES, db_index=True)
    serial = models.CharField('Серия', max_length=32, blank=True)
    number = models.CharField('Номер', max_length=32)
    issued_date = models.DateField('Дата выдачи', null=True, blank=True)
    issued_by = models.CharField('Кем выдан', max_length=255, blank=True)
    division_code = models.CharField('Код подразделения', max_length=16,
blank=True)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        verbose_name = 'Документ пользователя'
        verbose_name_plural = 'Документы пользователей'

    def __str__(self):
        base = self.get_doc_type_display()
        if self.serial:
            return f"{self.user.username}: {base} {self.serial}"
        return f"{self.user.username}: {base} {self.number}"

class UserDocumentFile(models.Model):
    document = models.ForeignKey(UserDocument, on_delete=models.CASCADE,
related_name='files')
    file = models.FileField('Файл документа',
upload_to='user_documents/%Y/%m/')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['created_at']
        verbose_name = 'Файл документа пользователя'
        verbose_name_plural = 'Файлы документов пользователей'

    def __str__(self):
        return f"Файл документа #{self.document_id}"

```

9. accounts/tasks.py

```

"""Celery tasks for the accounts app (interview reminders, etc.)."""
import logging

```



```

from celery import shared_task
from django.conf import settings
from django.utils import timezone

logger = logging.getLogger(__name__)

# — Helpers


---



def _interview_reminder_text(interview, reminder_type, recipient_user):
    """Return the notification message body for an interview reminder."""
    scheduled = timezone.localtime(interview.scheduled_at)
    date_str = scheduled.strftime('%d.%m.%Y')
    time_str = scheduled.strftime('%H:%M')
    vacancy_title = interview.vacancy.title if interview.vacancy else '-'
    location = interview.location or 'не указано'

    when_label = {
        '1d': 'Завтра',
        '1h': 'Через 1 час',
        'now': 'Через 5 минут',
    }.get(reminder_type, '')

    # Indicate the other party depending on who receives the reminder
    if recipient_user.pk == interview.manager_id:
        other_name = interview.applicant.get_full_name() or
interview.applicant.username
        role_line = f'Соискатель: {other_name}\n'
    else:
        role_line = ''

    return (
        f"📧 Напоминание о собеседовании\n\n"
        f"{when_label}: {date_str} в {time_str}\n"
        f"Вакансия: {vacancy_title}\n"
        f"{role_line}"
        f"Место/ссылка: {location}"
    )

def _send_interview_reminder(user, interview, reminder_type):
    """Send Telegram and/or email reminder to *user* about
    *interview*."""
    from django.core.mail import send_mail

    # Resolve consent flags + contact info from the correct role profile.
    # Use the interview relationship (manager / applicant) to determine
    which
    # profile to read — this avoids reading the wrong profile when a user
    has
    # both an Applicant and a Manager record (e.g. after a role switch).
    consent_email = False
    consent_telegram = False
    tg_chat_id = None
    email = user.email

    if user.pk == interview.manager_id:
        # This user is the manager for this interview
        try:
            profile = user.manager
            consent_email = profile.consent_email
            consent_telegram = profile.consent_telegram
            tg_chat_id = profile.telegram_chat_id

```

```

        except Exception:
            pass
    else:
        # This user is the applicant for this interview
        try:
            profile = user.applicant
            consent_email = profile.consent_email
            consent_telegram = profile.consent_telegram
            tg_chat_id = profile.telegram_chat_id
        except Exception:
            pass

    text = _interview_reminder_text(interview, reminder_type, user)

    # — Telegram —————
    if consent_telegram and tg_chat_id:
        try:
            from accounts.telegram import send_hello
            send_hello(tg_chat_id, text=text)
        except Exception as exc:
            logger.warning('interview_reminder: telegram failed for user
%s: %s', user.pk, exc)

    # — Email —————
    if consent_email and email:
        when_label = {
            '1d': 'завтра',
            '1h': 'через 1 час',
            'now': 'сейчас',
        }.get(reminder_type, '')
        subject = f"Напоминание: собеседование {when_label} – JobFlex"
        from_email = (
            getattr(settings, 'EMAIL_HOST_USER', None)
            or getattr(settings, 'DEFAULT_FROM_EMAIL',
'noreply@jobflex.ru')
        )
        try:
            send_mail(
                subject,
                text + '\n\n– JobFlex',
                from_email,
                [email],
                fail_silently=False,
            )
        except Exception as exc:
            logger.warning('interview_reminder: email failed for user %s:
%s', user.pk, exc)

    # — Periodic task —————

@shared_task(name='accounts.tasks.notify_interview_reminders_task')
def notify_interview_reminders_task():
    """Check all scheduled interviews and send reminders at 1 day, 1
hour, and 5 min before.

    Runs every 60 seconds via Celery Beat. Each reminder is sent at most
once
(guarded by the reminded_1d / reminded_1h / reminded_now flags).
"""
    from accounts.models import Interview

    now = timezone.now()

```

```

qs = (
    Interview.objects
    .filter(status='scheduled')
    .select_related(
        'manager', 'manager__applicant', 'manager__manager',
        'applicant', 'applicant__applicant', 'applicant__manager',
        'vacancy',
    )
)

sent_1d = sent_1h = sent_now = 0

for iv in qs:
    secs = (iv.scheduled_at - now).total_seconds()

    # — 1-day window: 23h50m - 24h10m (85800-87000 s) ———
    if 85800 <= secs <= 87000 and not iv.reminded_1d:
        _send_interview_reminder(iv.manager, iv, '1d')
        _send_interview_reminder(iv.applicant, iv, '1d')
        Interview.objects.filter(pk=iv.pk).update(reminded_1d=True)
        sent_1d += 1

    # — 1-hour window: 50min - 70min (3000-4200 s) —————
    elif 3000 <= secs <= 4200 and not iv.reminded_1h:
        _send_interview_reminder(iv.manager, iv, '1h')
        _send_interview_reminder(iv.applicant, iv, '1h')
        Interview.objects.filter(pk=iv.pk).update(reminded_1h=True)
        sent_1h += 1

    # — 5-min window: 0 - 10min (0 - 600 s) - runs every 60s so
    cannot be missed ———
    elif 0 <= secs <= 600 and not iv.reminded_now:
        _send_interview_reminder(iv.manager, iv, 'now')
        _send_interview_reminder(iv.applicant, iv, 'now')
        Interview.objects.filter(pk=iv.pk).update(reminded_now=True)
        sent_now += 1

    logger.info(
        'notify_interview_reminders: sent 1d=%d, 1h=%d, now=%d',
        sent_1d, sent_1h, sent_now,
    )
    return {'sent_1d': sent_1d, 'sent_1h': sent_1h, 'sent_now': sent_now}

@shared_task(name='accounts.tasks.send_interview_notification_task')
def send_interview_notification_task(interview_id, reminder_type):
    """Send Telegram + email notifications to manager and applicant at
    exact scheduled time.

    Scheduled via apply_async(eta=...) when an interview is created.
    Also updates the reminded_* flag so the periodic task does not
    double-send.
    """
    from accounts.models import Interview

    try:
        iv = Interview.objects.select_related('manager', 'applicant',
        'vacancy').get(pk=interview_id)
    except Interview.DoesNotExist:
        return # interview was deleted

    if iv.status != Interview.STATUS_SCHEDULED:
        return # interview was cancelled, nothing to send

```

```

    # Guard against double-send from the periodic polling task
    flag_field = {'ld': 'reminded_ld', 'lh': 'reminded_lh', 'now':
'reminded_now'}.get(reminder_type)
    if flag_field:
        if getattr(iv, flag_field):
            return # already sent by periodic task
        Interview.objects.filter(pk=iv.pk).update(**{flag_field: True})

    _send_interview_reminder(iv.manager, iv, reminder_type)
    _send_interview_reminder(iv.applicant, iv, reminder_type)
    logger.info('send_interview_notification_task: sent %s for interview
%d', reminder_type, interview_id)

# — Calendar-note reminders


---



@shared_task(name='accounts.tasks.notify_calendar_note_reminders_task')
def notify_calendar_note_reminders_task():
    """Send email/Telegram reminders for calendar notes whose time is
now.

Runs every 60 seconds via Celery Beat. Each note is reminded at most
once
(guarded by the CalendarNote.reminded flag).
"""
    from django.core.mail import send_mail
    from accounts.models import CalendarNote
    import datetime

    now_local = timezone.localtime(timezone.now())
    today = now_local.date()

    # Tight reminder window: from 60 seconds ago up to now.
    # This keeps delivery close to the exact note_time while tolerating
minor
    # task scheduling jitter.
    now_dt = datetime.datetime.combine(today, now_local.time())
    window_start_dt = now_dt - datetime.timedelta(seconds=60)
    window_start = window_start_dt.time()
    window_end = now_dt.time()

    qs = (
        CalendarNote.objects
        .filter(
            date=today,
            reminded=False,
            note_time__isnull=False,
            note_time__gte=window_start,
            note_time__lte=window_end,
        )
        .select_related('user', 'user__applicant', 'user__manager')
    )

    sent = 0
    for note in qs:
        user = note.user

        # Resolve consent flags. Prefer manager profile when present
because
        # manager users can also have an Applicant profile record.
        consent_email = False
        consent_telegram = False
        tg_chat_id = None

```

```

try:
    prof = user.manager
    consent_email = prof.consent_email
    consent_telegram = prof.consent_telegram
    tg_chat_id = prof.telegram_chat_id
except Exception:
    pass

if not consent_email and not consent_telegram:
    try:
        prof = user.applicant
        consent_email = prof.consent_email
        consent_telegram = prof.consent_telegram
        tg_chat_id = prof.telegram_chat_id
    except Exception:
        pass

text = (
    f"📅 Напоминание о заметке\n\n"
    f"Сегодня в {note.note_time.strftime('%H:%M')} \n"
    f"{note.text}"
)

# Telegram
if consent_telegram and tg_chat_id:
    try:
        from accounts.telegram import send_hello
        send_hello(tg_chat_id, text=text)
    except Exception as exc:
        logger.warning('note_reminder: telegram failed for user
%s: %s', user.pk, exc)

# Email
if consent_email and user.email:
    from_email = (
        getattr(settings, 'EMAIL_HOST_USER', None)
        or getattr(settings, 'DEFAULT_FROM_EMAIL',
'noreply@jobflex.ru')
    )
    try:
        send_mail(
            f"Напоминание: заметка в
{note.note_time.strftime('%H:%M')} - JobFlex",
            text + '\n\n- JobFlex',
            from_email,
            [user.email],
            fail_silently=False,
        )
    except Exception as exc:
        logger.warning('note_reminder: email failed for user %s:
%s', user.pk, exc)

CalendarNote.objects.filter(pk=note.pk).update(reminded=True)
sent += 1

logger.info('notify_calendar_note_reminders: sent=%d', sent)
return {'sent': sent}

```

10. accounts/telegram.py

```

from django.conf import settings
import threading

```

```

import json

try:
    import requests
except Exception:
    requests = None
    import urllib.request
    import urllib.parse

def _send_via_requests(token, chat_id, text):
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    resp = requests.post(url, json={'chat_id': chat_id, 'text': text})
    return resp.status_code, resp.text

def _send_via_urllib(token, chat_id, text):
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    data = urllib.parse.urlencode({'chat_id': chat_id, 'text':
text}).encode('utf-8')
    req = urllib.request.Request(url, data=data, method='POST')
    with urllib.request.urlopen(req, timeout=10) as r:
        return r.getcode(), r.read().decode('utf-8')

def send_hello(telegram_handle, text=None):
    """Send a welcome message to `telegram_handle` (e.g. '@username' or
chat id).

    This is best-effort: Telegram bots can only message users who already
started
a conversation with the bot. Failures are logged but do not raise.
"""
    token = getattr(settings, 'TELEGRAM_BOT_TOKEN', None)
    if not token:
        return False

    if not text:
        text = 'Привет! Спасибо за регистрацию. Это приветственное
сообщение от JobFlex.'

    try:
        if requests:
            status, body = _send_via_requests(token, telegram_handle,
text)
        else:
            status, body = _send_via_urllib(token, telegram_handle, text)
        # Basic success check
        if 200 <= int(status) < 300:
            return True
    except Exception as e:
        body = str(e)

    # log to console for now
    try:
        print('telegram_send_failed', {'to': telegram_handle, 'resp':
body})
    except Exception:
        pass
    return False

def send_hello_async(telegram_handle, text=None):
    t = threading.Thread(target=send_hello, args=(telegram_handle, text),
daemon=True)
    t.start()
    return True

```

```

# Cache for bot info
# None = not yet fetched; False = fetched but failed; str = username
_BOT_USERNAME = None

def get_bot_username():
    """Return bot username (without @), querying Telegram `getMe` API
    once and caching result.

    Returns None on failure. Failures are cached to avoid blocking every
    request.
    """
    global _BOT_USERNAME
    if _BOT_USERNAME is not None: # False (failed) or str (success) -
        don't retry
        return _BOT_USERNAME or None

    token = getattr(settings, 'TELEGRAM_BOT_TOKEN', None)
    if not token:
        _BOT_USERNAME = False
        return None

    try:
        if requests:
            url = f"https://api.telegram.org/bot{token}/getMe"
            r = requests.get(url, timeout=3)
            j = r.json()
            if j.get('ok') and isinstance(j.get('result'), dict):
                username = j['result'].get('username')
                if username:
                    _BOT_USERNAME = username
                    return username
            else:
                url = f"https://api.telegram.org/bot{token}/getMe"
                with urllib.request.urlopen(url, timeout=3) as r:
                    b = r.read().decode('utf-8')
                    j = json.loads(b)
                    if j.get('ok') and isinstance(j.get('result'), dict):
                        username = j['result'].get('username')
                        if username:
                            _BOT_USERNAME = username
                            return username
        except Exception:
            pass

        _BOT_USERNAME = False # cache failure - don't retry on next request
        return None

def resolve_chat_id_by_token(start_token, limit=100):
    """Scan recent Telegram updates looking for a /start message
    containing start_token.

    Returns the chat_id integer if found, otherwise None.
    """
    token = getattr(settings, 'TELEGRAM_BOT_TOKEN', None)
    if not token or not start_token:
        return None

    try:
        params = {'limit': limit, 'timeout': 0, 'allowed_updates':
['message']}
        if requests:
            url = f"https://api.telegram.org/bot{token}/getUpdates"

```

```

        r = requests.get(url, params=params, timeout=10)
        updates = r.json().get('result', [])
    else:
        import urllib.parse as _up
        qs = _up.urlencode(params)
        url = f"https://api.telegram.org/bot{token}/getUpdates?{qs}"
        with urllib.request.urlopen(url, timeout=10) as r:
            updates = json.loads(r.read().decode('utf-
8')).get('result', [])

        for upd in reversed(updates):
            msg = upd.get('message') or {}
            text = (msg.get('text') or '').strip()
            chat_id = (msg.get('chat') or {}).get('id')
            if chat_id and text.startswith('/start'):
                parts = text.split(None, 1)
                if len(parts) > 1 and parts[1].strip() == start_token:
                    return chat_id
    except Exception:
        pass
    return None

def get_bot_link(start_token=None):
    username = get_bot_username()
    if not username:
        return None
    url = f"https://t.me/{username}"
    if start_token:
        url += f"?start={start_token}"
    return url

```

— Chat message notifications

```

def _notify_task(recipient_user, sender_name, text_preview, chat_pk):
    """Run in a background thread: send email and/or Telegram
    notification."""
    from django.conf import settings as _s
    from django.core.mail import send_mail as _send_mail

    site_url = getattr(_s, 'SITE_URL',
'http://localhost:8000').rstrip('/')
    chat_url = f"{site_url}/accounts/chats/{chat_pk}/"

    # Resolve the recipient's consent profile (Applicant or Manager)
    consent_email = False
    consent_telegram = False
    tg_chat_id = None
    email = recipient_user.email

    try:
        profile = recipient_user.applicant
        consent_email = profile.consent_email
        consent_telegram = profile.consent_telegram
        tg_chat_id = profile.telegram_chat_id
    except Exception:
        pass

    if not consent_email and not consent_telegram:
        try:
            profile = recipient_user.manager
            consent_email = profile.consent_email
            consent_telegram = profile.consent_telegram

```



```

        tg_chat_id = profile.telegram_chat_id
    except Exception:
        pass

    preview = text_preview[:120] + ('...' if len(text_preview) > 120 else
    '')

    # — Telegram
    

---


    if consent_telegram and tg_chat_id:
        tg_text = (
            f"🗨 Новое сообщение от {sender_name}:\n\n"
            f"«{preview}»\n\n"
            f"Открыть чат: {chat_url}"
        )
        try:
            send_hello(tg_chat_id, text=tg_text)
        except Exception:
            pass

    # — Email
    

---


    if consent_email and email:
        subject = f"Новое сообщение от {sender_name} – JobFlex"
        body = (
            f"Здравствуйте, {recipient_user.get_full_name() or"
            f"recipient_user.username}!\n\n"
            f"{sender_name} написал(а) вам в чате:\n\n"
            f"«{preview}»\n\n"
            f"Перейти в чат: {chat_url}\n\n"
            f"— JobFlex"
        )
        # Mail.ru (and most SMTP servers) require From == authenticated
        user
        from_email = getattr(_s, 'EMAIL_HOST_USER', None) or getattr(_s,
        'DEFAULT_FROM_EMAIL', 'noreply@jobflex.ru')
        try:
            _send_mail(
                subject,
                body,
                from_email,
                [email],
                fail_silently=False,
            )
        except Exception as exc:
            print(f'[chat_notify] email failed to {email}: {exc}')

    def notify_new_chat_message(recipient_user, sender_name, text_preview,
    chat_pk):
        """Fire-and-forget: notify recipient about a new chat message (email
        + Telegram)."""
        t = threading.Thread(
            target=_notify_task,
            args=(recipient_user, sender_name, text_preview, chat_pk),
            daemon=True,
        )
        t.start()

```

11. accounts/tests_moderator.py

```

from datetime import datetime, timezone

from django.contrib.auth.models import User

```

```

from django.test import TestCase
from django.urls import reverse

from accounts.models import Administrator, Moderator, Manager
from vacancies.models import Vacancy, VacancyReport

def _make_site_vacancy(external_id='site-test-1'):
    return Vacancy.objects.create(
        external_id=external_id,
        title='Site vacancy',
        company='Site company',
        country='Россия',
        url='http://localhost/site-vacancy',
        published_at=datetime(2026, 1, 1, tzinfo=timezone.utc),
        raw_json={},
    )

class ModeratorRoleTests(TestCase):
    def setUp(self):
        self.admin = User.objects.create_user(
            username='admin',
            email='admin@example.com',
            password='admin-pass-123',
            is_superuser=True,
            is_staff=True,
        )
        Administrator.objects.create(user=self.admin)

    def test_admin_can_create_moderator_from_admin_users_page(self):
        self.client.force_login(self.admin)
        resp = self.client.post(
            reverse('accounts:admin_users'),
            data={
                'action': 'create_moderator',
                'new_moderator_username': 'mod1',
                'new_moderator_email': 'mod1@example.com',
                'new_moderator_password': 'mod-pass-123',
                'new_moderator_first_name': 'Mod',
                'new_moderator_last_name': 'Erator',
            },
        )
        self.assertEqual(resp.status_code, 302)
        created_user = User.objects.filter(username='mod1').first()
        self.assertIsNotNone(created_user)
        self.assertTrue(Moderator.objects.filter(user=created_user).exists())

    def test_moderator_profile_redirects_from_profile_to_workspace(self):
        user = User.objects.create_user(
            username='moderator-user',
            email='moderator@example.com',
            password='mod-pass-123',
        )
        Moderator.objects.create(user=user)
        self.client.force_login(user)

        resp = self.client.get(reverse('accounts:profile'))
        self.assertEqual(resp.status_code, 302)
        self.assertEqual(resp.url,
            reverse('accounts:moderator_analytics'))

    def test_non_moderator_cannot_update_report_status(self):
        reporter = User.objects.create_user('reporter',

```

```

'reporter@example.com', 'test-pass-123')
    vac = _make_site_vacancy()
    report = VacancyReport.objects.create(
        vacancy=vac,
        user=reporter,
        reason_code=VacancyReport.REASON_SPAM,
        reason_text='spam',
    )
    self.client.force_login(reporter)
    resp = self.client.post(
        reverse('report-self-status', args=[report.pk]),
        data='{"self_status":"done","moderator_note":"ok"}',
        content_type='application/json',
    )
    self.assertEqual(resp.status_code, 403)

def test_manager_cannot_open_moderator_workspace(self):
    manager_user = User.objects.create_user('manager1',
'manager1@example.com', 'test-pass-123')
    Manager.objects.create(user=manager_user, telegram='@manager1')
    self.client.force_login(manager_user)
    resp = self.client.get(reverse('accounts:moderator_analytics'))
    self.assertEqual(resp.status_code, 403)

```

12. accounts/urls.py

```

from django.urls import path
from . import views

```

```

app_name = 'accounts'

```

```

urlpatterns = [
    path('login/', views.login_page, name='login'),
    path('api/login/', views.api_login, name='api_login'),
    path('api/logout/', views.api_logout, name='api_logout'),
    path('api/ui/theme/', views.api_user_theme, name='api_user_theme'),
    path('api/set-consent/', views.api_set_consent,
name='api_set_consent'),
    path('api/test-message/', views.api_test_message,
name='api_test_message'),
    path('api/unlink-telegram/', views.api_unlink_telegram,
name='api_unlink_telegram'),
    path('api/set-email-consent/', views.api_set_email_consent,
name='api_set_email_consent'),
    path('api/test-email/', views.api_test_email, name='api_test_email'),
    path('api/unlink-email/', views.api_unlink_email,
name='api_unlink_email'),
    path('register/', views.register_page, name='register'),
    path('profile/', views.profile_page, name='profile'),
    path('api/profile-data/', views.api_profile_data,
name='api_profile_data'),
    path('api/profile/documents/', views.api_profile_documents,
name='api_profile_documents'),
    path('api/profile/documents/<int:pk>/delete/',
views.api_profile_document_delete, name='api_profile_document_delete'),
    path('api/feedback/', views.api_feedback, name='api_feedback'),
    path('api/metro-data/', views.api_metro_data, name='api_metro_data'),
    path('api/profile/update/', views.api_profile_update,
name='api_profile_update'),
    path('api/profile/education/', views.api_education_create,
name='api_education_create'),
    path('api/profile/education/<int:pk>/', views.api_education_crud,
name='api_education_crud'),

```

```

        path('api/profile/extra-education/', views.api_extra_edu_create,
name='api_extra_edu_create'),
        path('api/profile/extra-education/<int:pk>/',
views.api_extra_edu_crud, name='api_extra_edu_crud'),
        path('api/profile/work/', views.api_work_create,
name='api_work_create'),
        path('api/profile/work/<int:pk>/', views.api_work_crud,
name='api_work_crud'),
        path('api/profile/skills/', views.api_skills_update,
name='api_skills_update'),
        path('applicants/<int:pk>/resume/', views.resume_pdf,
name='resume_pdf'),
        path('applicants/<int:pk>/resume-word/', views.resume_word_view,
name='resume_word_view'),
        path('api/apply/', views.api_apply, name='api_apply'),
        path('api/applications/<int:pk>/status/',
views.api_application_status, name='api_application_status'),
        path('api/upload-avatar/', views.api_upload_avatar,
name='api_upload_avatar'),
        path('api/upload-company-logo/', views.api_upload_company_logo,
name='api_upload_company_logo'),
        path('api/upload-resume/', views.api_upload_resume,
name='api_upload_resume'),
        path('api/delete-resume/', views.api_delete_resume,
name='api_delete_resume'),
        path('api/bookmark/<int:pk>/', views.api_toggle_bookmark,
name='api_toggle_bookmark'),
        path('api/analytics/', views.api_applicant_analytics,
name='api_applicant_analytics'),
        path('api/resume/analyze/', views.api_resume_analyze,
name='api_resume_analyze'),
        path('vacancies/<int:pk>/applicants/', views.vacancy_applications,
name='vacancy_applications'),
        path('vacancies/<int:pk>/applicants/export/',
views.export_applications_csv, name='export_applications_csv'),
        path('manager/analytics/', views.manager_analytics,
name='manager_analytics'),
        path('moderator/analytics/', views.moderator_analytics,
name='moderator_analytics'),
        path('chats/', views.chat_list, name='chat_list'),
        path('chats/<int:pk>/', views.chat_detail, name='chat_detail'),
        path('api/chats/start/', views.api_chat_start,
name='api_chat_start'),
        path('api/chats/<int:pk>/send/', views.api_chat_send,
name='api_chat_send'),
        path('api/chats/<int:pk>/messages/', views.api_chat_messages,
name='api_chat_messages'),
        path('api/chats/<int:pk>/delete/', views.api_chat_delete,
name='api_chat_delete'),
        path('api/switch-role/', views.api_switch_role,
name='api_switch_role'),
        path('delete/', views.delete_account, name='delete_account'),
        path('logout/', views.logout_view, name='logout'),
        path('api/register/', views.api_register, name='api_register'),
        path('register-manager/', views.manager_register_page,
name='manager_register'),
        path('api/register-manager/', views.api_register_manager,
name='api_register_manager'),
        path('api/send-telegram-welcome/', views.api_send_telegram_welcome,
name='api_send_telegram_welcome'),
        path('telegram-webhook/', views.telegram_webhook,
name='telegram_webhook'),
        path('message-rules/', views.message_rules, name='message_rules'),

```

```

        path('terms/', views.terms, name='terms'),
        path('api/presets/', views.api_presets, name='api_presets'),
        path('api/presets/<int:pk>/', views.api_preset_detail,
name='api_preset_detail'),
        path('admin-panel/', views.admin_panel, name='admin_panel'),
        path('admin-users/', views.admin_users, name='admin_users'),
        path('admin-moderator-reports/', views.admin_moderator_reports,
name='admin_moderator_reports'),
        path('admin-moderator-reports/<int:report_id>/pdf/',
views.admin_moderator_report_pdf, name='admin_moderator_report_pdf'),
        path('admin-profile/', views.admin_profile_page,
name='admin_profile_page'),
        path('api/admin/backup/create/', views.api_admin_backup_create,
name='api_admin_backup_create'),
        path('api/admin/backup/restore/', views.api_admin_backup_restore,
name='api_admin_backup_restore'),
        path('api/admin/backup/delete/', views.api_admin_backup_delete,
name='api_admin_backup_delete'),
        path('api/admin/feedback/<int:feedback_id>/action/',
views.api_admin_feedback_action, name='api_admin_feedback_action'),
        path('api/calendar/events/', views.api_calendar_events,
name='api_calendar_events'),
        path('api/calendar/note/', views.api_calendar_note_save,
name='api_calendar_note_save'),
        path('api/calendar/month/', views.api_calendar_month_export,
name='api_calendar_month_export'),
        path('api/calendar/notes-index/', views.api_calendar_notes_index,
name='api_calendar_notes_index'),
        path('api/interviews/schedule/', views.api_interview_schedule,
name='api_interview_schedule'),
        path('api/interviews/cancel/', views.api_interview_cancel,
name='api_interview_cancel'),
        path('api/interviews/reschedule/', views.api_interview_reschedule,
name='api_interview_reschedule'),
        path('api/manager/calendar/events/',
views.api_manager_calendar_events, name='api_manager_calendar_events'),
]

```

13. accounts/views.py

```

import csv
import json
import os
import re
import uuid
from datetime import date, timedelta
from django.db.models import Q
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.models import User
from django.http import JsonResponse, HttpResponse
from django.core.paginator import Paginator
from django.urls import reverse
from urllib.parse import urlencode
from django.views.decorators.csrf import ensure_csrf_cookie, csrf_exempt
from django.contrib.auth.decorators import login_required
from django.contrib.auth import authenticate, login as auth_login, logout
as auth_logout
from django.core.mail import send_mail
from django.conf import settings as django_settings

from rest_framework.decorators import api_view, authentication_classes,
permission_classes
from rest_framework.permissions import AllowAny

```

```

from rest_framework.permissions import IsAuthenticated
from drf_yasg.utils import swagger_auto_schema
from drf_yasg import openapi
from django.http.request import RawPostDataException

from .models import Applicant, Manager, Administrator, Moderator,
Education, ExtraEducation, WorkExperience, Chat, Message, Application,
FilterPreset, CalendarNote, Interview, UserUiPreference, ApiActionLog,
UserFeedback, UserDocument, UserDocumentFile
from .telegram import send_hello_async, get_bot_username,
resolve_chat_id_by_token, notify_new_chat_message

# _____ Admin helpers
_____

def is_admin_user(user):
    """Returns True only if the user is a superuser AND has an
    Administrator profile."""
    return (
        user.is_authenticated
        and user.is_superuser
        and hasattr(user, 'admin_profile')
    )

def admin_required(view_func):
    """Decorator: allows access only to verified administrators."""
    from functools import wraps
    from django.http import HttpResponseForbidden

    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            from django.contrib.auth.views import redirect_to_login
            return redirect_to_login(request.get_full_path(),
login_url='/accounts/login/')
        if not is_admin_user(request.user):
            return HttpResponseForbidden(
                '<h2>403 – Доступ запрещён</h2>'
                '<p>Эта страница доступна только администраторам
системы.</p>'
            )
        return view_func(request, *args, **kwargs)
    return wrapper

def is_moderator_user(user):
    """Returns True only if authenticated user has a Moderator
    profile."""
    return (
        user.is_authenticated
        and hasattr(user, 'moderator_profile')
    )

def moderator_required(view_func):
    """Decorator: allows access only to moderators."""
    from functools import wraps
    from django.http import HttpResponseForbidden

    @wraps(view_func)
    def wrapper(request, *args, **kwargs):

```

```

        if not request.user.is_authenticated:
            from django.contrib.auth.views import redirect_to_login
            return redirect_to_login(request.get_full_path(),
login_url='/accounts/login/')
        if not is_moderator_user(request.user):
            return HttpResponseRedirect(
                '<h2>403 – Доступ запрещён</h2>'
                '<p>Эта страница доступна только модераторам
системы.</p>'
            )
        return view_func(request, *args, **kwargs)
    return wrapper

def _resolve_actor_role(user):
    """Map authenticated user to a stable actor role for API audit
logs."""
    if not user or not getattr(user, 'is_authenticated', False):
        return 'system'
    if is_admin_user(user):
        return 'admin'
    if is_moderator_user(user):
        return 'moderator'
    if hasattr(user, 'manager'):
        return 'manager'
    if hasattr(user, 'applicant'):
        return 'applicant'
    return 'user'

def _json_safe(data):
    """Ensure value is JSON-serializable for JSONField storage."""
    if data is None:
        return {}
    try:
        return json.loads(json.dumps(data, default=str,
ensure_ascii=False))
    except Exception:
        return {'raw': str(data)}

def _log_api_action(request, action, before=None, after=None, *,
success=True, status_code=None, meta=None, endpoint=None):
    """Write one API action log row. Logging failures must not break
endpoint flow."""
    try:
        ApiActionLog.objects.create(
            user=request.user if getattr(request.user,
'is_authenticated', False) else None,
            actor_role=_resolve_actor_role(getattr(request, 'user',
None)),
            method=(getattr(request, 'method', '') or '').upper()[:10],
            path=(getattr(request, 'path', '') or '')[:255],
            endpoint=(endpoint or '')[:120],
            action=(action or 'api_action')[:120],
            success=bool(success),
            status_code=status_code,
            before_data=_json_safe(before),
            after_data=_json_safe(after),
            meta=_json_safe(meta),
        )
    except Exception:
        pass

```

```
def _humanize_payload(data):
    """Render compact human-readable payload text for before/after
    columns."""
    if not data:
        return '-'
    if isinstance(data, dict):
        chunks = []
        for key, value in data.items():
            if isinstance(value, (dict, list)):
                value_txt = json.dumps(value, ensure_ascii=False)
            else:
                value_txt = str(value)
            chunks.append(f'{key}: {value_txt}')
        return '\n'.join(chunks) if chunks else '-'
    if isinstance(data, list):
        return ', '.join(str(v) for v in data) if data else '-'
    return str(data)
```

```
# _____ Page views (HTML)
```

```
@ensure_csrf_cookie
def register_page(request):
    bot_username = get_bot_username()
    return render(request, 'accounts/register.html', {
        'telegram_bot_username': bot_username,
        'today': date.today().isoformat(),
    })

@ensure_csrf_cookie
def manager_register_page(request):
    return render(request, 'accounts/register_manager.html')

def message_rules(request):
    return render(request, 'accounts/message_rules.html')

def terms(request):
    return render(request, 'accounts/terms.html')

@ensure_csrf_cookie
def login_page(request):
    return render(request, 'accounts/login.html')

@login_required
@ensure_csrf_cookie
def profile_page(request):
    if is_admin_user(request.user):
        return redirect('accounts:admin_panel')
    if is_moderator_user(request.user):
        return redirect('accounts:moderator_analytics')
    return render(request, 'accounts/profile.html')

@login_required
def logout_view(request):
```



```

    auth_logout(request)
    return redirect('/')

# ----- API views (JSON)
# -----

@swagger_auto_schema(method='post', operation_summary="Регистрация нового
пользователя", tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
def api_register(request):
    def _register_fail(code, *, status=400, extra=None):
        payload = {'error': code}
        if extra:
            payload.update(extra)
        _log_api_action(
            request,
            action='register_applicant',
            before={'username': username, 'email': email, 'telegram':
telegram},
            after=payload,
            success=False,
            status_code=status,
            endpoint='api_register',
        )
        return JsonResponse({'error': code}, status=status)

    data = None
    # Prefer DRF-parsed data (handles JSON and form-data)
    try:
        data = getattr(request, 'data', None)
    except Exception:
        data = None

    # If no parsed data, try to read raw JSON body safely
    if not data:
        try:
            raw = None
            try:
                raw = request.body
            except RawPostDataException:
                raw = None
            if raw:
                try:
                    data = json.loads(raw.decode('utf-8'))
                except Exception:
                    data = None
        except Exception:
            data = None

    # Finally, fallback to form-encoded POST
    if not data:
        try:
            if hasattr(request, 'POST') and request.POST:
                data = dict(request.POST)
                for k, v in list(data.items()):
                    if isinstance(v, list) and len(v) == 1:
                        data[k] = v[0]
            else:
                data = {}
        except Exception:
            data = {}

```

```

last_name = (data.get('last_name') or '').strip()
first_name = (data.get('first_name') or '').strip()
patronymic = (data.get('patronymic') or '').strip()
username = (data.get('username') or '').strip()
email = (data.get('email') or '').strip().lower()
telegram = (data.get('telegram') or '').strip()
phone = (data.get('phone') or '').strip()
gender = (data.get('gender') or '').strip().upper()
city = (data.get('city') or '').strip()
birth_date_raw = (data.get('birth_date') or '').strip()
citizenship = (data.get('citizenship') or '').strip().upper()
consent_email = bool(data.get('consent_email'))
consent_telegram = bool(data.get('consent_telegram'))
password = data.get('password')

if not last_name or not first_name or not username \
    or not phone or not gender or not city or not birth_date_raw \
    or not citizenship or not password:
    return _register_fail('missing_fields')

if telegram and not telegram.startswith('@'):
    return _register_fail('invalid_telegram_format')

# Phone: must be Russian format – 11 digits starting with 7 or 8
phone_digits = re.sub(r'\D', '', phone)
if phone_digits.startswith('8'):
    phone_digits = '7' + phone_digits[1:]
if not re.match(r'^7[0-9]{10}$', phone_digits):
    return _register_fail('invalid_phone')

# Gender
if gender not in ('M', 'F'):
    return _register_fail('invalid_gender')

# City
if len(city) < 2 or len(city) > 100:
    return _register_fail('invalid_city')

# Birth date
try:
    birth_date = date.fromisoformat(birth_date_raw) # expects YYYY-MM-DD
except ValueError:
    return _register_fail('invalid_birth_date')
today = date.today()
age = today.year - birth_date.year - ((today.month, today.day) <
(birth_date.month, birth_date.day))
if age < 14:
    return _register_fail('too_young')
if age > 100:
    return _register_fail('invalid_birth_date')

# Citizenship
VALID_CITIZENSHIPS = {'RU', 'BY', 'KZ', 'KG', 'AM', 'TJ', 'UZ', 'UA',
'MD', 'AZ', 'TM'}
if citizenship not in VALID_CITIZENSHIPS:
    return _register_fail('invalid_citizenship')

if telegram:
    if Applicant.objects.filter(telegram__iexact=telegram).exists()
or \

```

```

        Manager.objects.filter(telegram__iexact=telegram).exists():
            return _register_fail('telegram_exists')

# username uniqueness and format check
if not username:
    return _register_fail('missing_fields')
if len(username) < 3 or len(username) > 30:
    return _register_fail('invalid_username')
if not re.match(r'^[A-Za-z0-9_.-]+$', username):
    return _register_fail('invalid_username_format')
if User.objects.filter(username__iexact=username).exists():
    return _register_fail('username_exists')

if email and User.objects.filter(email__iexact=email).exists():
    return _register_fail('email_exists')

# Contact consents are allowed only when the corresponding contact is
present.
consent_email = bool(email) and consent_email
consent_telegram = bool(telegram) and consent_telegram

user = User.objects.create_user(username=username, email=email,
password=password,
                                first_name=first_name,
last_name=last_name)
token = uuid.uuid4().hex

applicant = Applicant.objects.create(
    user=user, patronymic=patronymic, telegram=telegram,
    consent_email=consent_email, consent_telegram=consent_telegram,
    telegram_start_token=token,
    phone=phone, gender=gender, city=city,
    birth_date=birth_date, citizenship=citizenship,
    skills=data.get('skills') if isinstance(data.get('skills'), list)
else [],
)

# Education (optional list of dicts)
edu_list = data.get('education') or []
if isinstance(edu_list, list):
    VALID_EDU_LEVELS = {c[0] for c in Education.LEVEL_CHOICES}
    for i, entry in enumerate(edu_list):
        if not isinstance(entry, dict):
            continue
        level = (entry.get('level') or '').strip()
        institution = (entry.get('institution') or '').strip()
        if not level or level not in VALID_EDU_LEVELS or not
institution:
            continue
        try:
            grad_year = int(entry['graduation_year']) if
entry.get('graduation_year') else None
        except (ValueError, TypeError):
            grad_year = None
        Education.objects.create(
            applicant=applicant,
            level=level,
            institution=institution,
            graduation_year=grad_year,
            faculty=(entry.get('faculty') or '').strip(),
            specialization=(entry.get('specialization') or
''.strip(),
            order=i,

```

```

    )

    # Work experience (optional list of dicts)
    work_list = data.get('work_experience') or []
    if isinstance(work_list, list):
        for i, entry in enumerate(work_list):
            if not isinstance(entry, dict):
                continue
            company = (entry.get('company') or '').strip()
            position = (entry.get('position') or '').strip()
            if not company or not position:
                continue
            def _safe_int(val):
                try:
                    return int(val) if val else None
                except (ValueError, TypeError):
                    return None
            WorkExperience.objects.create(
                applicant=applicant,
                company=company,
                position=position,
                start_month=_safe_int(entry.get('start_month')),
                start_year=_safe_int(entry.get('start_year')),
                end_month=_safe_int(entry.get('end_month')),
                end_year=_safe_int(entry.get('end_year')), # Can be null
                is_current=bool(entry.get('is_current')),
                responsibilities=(entry.get('responsibilities') or
                                '').strip(),
                order=i,
            )

        _log_api_action(
            request,
            action='register_applicant',
            before={'username': username, 'email': email, 'telegram':
telegram},
            after={'created_user_id': user.pk, 'created_applicant_id':
applicant.pk},
            success=True,
            status_code=200,
            endpoint='api_register',
        )
    return JsonResponse({'ok': True, 'start_token': token})

@swagger_auto_schema(method='post', operation_summary="Регистрация
менеджера", tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
def api_register_manager(request):

    def _register_manager_fail(code, *, status=400):
        _log_api_action(
            request,
            action='register_manager',
            before={'username': username, 'email': email, 'telegram':
telegram},
            after={'error': code},
            success=False,
            status_code=status,
            endpoint='api_register_manager',
        )
    return JsonResponse({'error': code}, status=status)

```

```

try:
    data = getattr(request, 'data', None) or {}
except Exception:
    data = {}

last_name = (data.get('last_name') or '').strip()
first_name = (data.get('first_name') or '').strip()
patronymic = (data.get('patronymic') or '').strip()
username = (data.get('username') or '').strip()
email = (data.get('email') or '').strip().lower()
telegram = (data.get('telegram') or '').strip()
company = (data.get('company') or '').strip()
phone = (data.get('phone') or '').strip()
password = data.get('password')

if not last_name or not first_name or not username or not password:
    return _register_manager_fail('missing_fields')

if telegram and not telegram.startswith('@'):
    return _register_manager_fail('invalid_telegram_format')

# Check telegram uniqueness across both Applicant and Manager
if telegram:
    if Applicant.objects.filter(telegram__iexact=telegram).exists()
or \
        Manager.objects.filter(telegram__iexact=telegram).exists():
        return _register_manager_fail('telegram_exists')

if len(username) < 3 or len(username) > 30:
    return _register_manager_fail('invalid_username')
if not re.match(r'^[A-Za-z0-9_.\-]+$ ', username):
    return _register_manager_fail('invalid_username_format')
if User.objects.filter(username__iexact=username).exists():
    return _register_manager_fail('username_exists')
if email and User.objects.filter(email__iexact=email).exists():
    return _register_manager_fail('email_exists')

user = User.objects.create_user(
    username=username, email=email, password=password,
    first_name=first_name, last_name=last_name,
)
manager = Manager.objects.create(user=user, patronymic=patronymic,
telegram=telegram,
                                company=company, phone=phone)

_log_api_action(
    request,
    action='register_manager',
    before={'username': username, 'email': email, 'telegram':
telegram},
    after={'created_user_id': user.pk, 'created_manager_id':
manager.pk},
    success=True,
    status_code=200,
    endpoint='api_register_manager',
)

return JsonResponse({'ok': True})

@swagger_auto_schema(method='post', operation_summary="Отправить
приветственное сообщение в Telegram", tags=['accounts'])

```

```

@api_view(['POST'])
@permission_classes([AllowAny])
def api_send_telegram_welcome(request):
    try:
        data = json.loads(request.body.decode('utf-8'))
    except Exception:
        return JsonResponse({'error': 'invalid_json'}, status=400)

    telegram = (data.get('telegram') or '').strip()
    consent_telegram = bool(data.get('consent_telegram'))

    if not telegram:
        return JsonResponse({'error': 'missing_telegram'}, status=400)
    if not telegram.startswith('@'):
        return JsonResponse({'error': 'invalid_telegram_format'},
status=400)
    if not consent_telegram:
        return JsonResponse({'error': 'no_consent'}, status=400)

    applicant =
Applicant.objects.filter(telegram__iexact=telegram).first()
    if not applicant:
        return JsonResponse({'error': 'unknown_telegram'}, status=400)

    if not applicant.telegram_chat_id:
        return JsonResponse({'error': 'chat_id_missing',
                            'message': 'Пожалуйста, откройте чат с ботом
и нажмите /start.'}, status=400)

    try:
        send_hello_async(applicant.telegram_chat_id)
    except Exception:
        pass

    return JsonResponse({'ok': True})

@swagger_auto_schema(method='post', operation_summary="Вебхук Telegram",
tags=['accounts'])
@api_view(['POST'])
@authentication_classes([])
@permission_classes([AllowAny])
@csrf_exempt
def telegram_webhook(request):
    try:
        data = json.loads(request.body.decode('utf-8'))
    except Exception:
        return HttpResponse('ok')

    message = data.get('message') or {}
    text = message.get('text', '')
    chat = message.get('chat') or {}
    chat_id = chat.get('id')
    username = chat.get('username')

    if text and text.startswith('/start') and chat_id:
        parts = text.split(None, 1)
        token = None
        if len(parts) > 1:
            token = parts[1].strip()

        if token:
            try:

```

```

        applicant =
Applicant.objects.get(telegram_start_token=token)
        applicant.telegram_chat_id = chat_id
        applicant.save(update_fields=['telegram_chat_id'])
        if applicant.consent_telegram:
            send_hello_async(chat_id)
        return HttpResponse('ok')
    except Applicant.DoesNotExist:
        pass

    if username:
        applicant = Applicant.objects.filter(telegram__iexact='@' +
username).first()
        if applicant:
            applicant.telegram_chat_id = chat_id
            applicant.save(update_fields=['telegram_chat_id'])
            if applicant.consent_telegram:
                send_hello_async(chat_id)
            return HttpResponse('ok')

        # Also try to match a manager by Telegram username
        manager = Manager.objects.filter(telegram__iexact='@' +
username).first()
        if manager:
            manager.telegram_chat_id = chat_id
            manager.save(update_fields=['telegram_chat_id'])
            if manager.consent_telegram:
                send_hello_async(chat_id, text='Телеграм подключён!
Теперь вы будете получать уведомления от JobFlex.')
            return HttpResponse('ok')

    return HttpResponse('ok')

@swagger_auto_schema(method='post', operation_summary="Вход в систему",
tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
def api_login(request):
    try:
        data = json.loads(request.body.decode('utf-8'))
    except Exception:
        return JsonResponse({'error': 'invalid_json'}, status=400)

    identifier = (data.get('username') or data.get('email') or
'').strip()
    password = data.get('password') or ''

    if not identifier or not password:
        return JsonResponse({'error': 'missing_fields'}, status=400)

    # Try authenticate by username first
    user = authenticate(request, username=identifier, password=password)
    # If failed and identifier looks like email, try resolving email-
>username
    if user is None and '@' in identifier:
        try:
            u = User.objects.filter(email__iexact=identifier).first()
            if u:
                user = authenticate(request, username=u.username,
password=password)
        except Exception:
            user = None

```

```

if user is None:
    return JsonResponse({'error': 'invalid_credentials'}, status=400)

auth_login(request, user)
if is_admin_user(user):
    redirect_to = '/accounts/admin-panel/'
elif is_moderator_user(user):
    redirect_to = '/accounts/moderator/analytics/'
else:
    redirect_to = '/'
return JsonResponse({'ok': True, 'redirect_to': redirect_to})

@swagger_auto_schema(method='post', operation_summary="Выход из системы",
tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
def api_logout(request):
    try:
        auth_logout(request)
    except Exception:
        pass
    return JsonResponse({'ok': True})

@swagger_auto_schema(method='get', operation_summary="Получить тему
интерфейса", tags=['accounts'])
@swagger_auto_schema(method='post', operation_summary="Сохранить тему
интерфейса", tags=['accounts'])
@api_view(['GET', 'POST'])
@permission_classes([AllowAny])
def api_user_theme(request):
    """Get/save persistent UI theme for the authenticated user."""
    if request.method == 'GET':
        if not request.user.is_authenticated:
            return JsonResponse({'ok': True, 'theme': None})
        pref = UserUiPreference.objects.filter(user=request.user).first()
        return JsonResponse({'ok': True, 'theme': pref.theme if pref else
None})

    # POST
    if not request.user.is_authenticated:
        return JsonResponse({'error': 'not_authenticated'}, status=401)

    try:
        data = request.data if hasattr(request, 'data') else {}
        if not data:
            data = json.loads(request.body.decode('utf-8'))
    except Exception:
        return JsonResponse({'error': 'invalid_json'}, status=400)

    theme = str((data or {}).get('theme') or '').strip().lower()
    if theme not in ('light', 'dark'):
        return JsonResponse({'error': 'invalid_theme'}, status=400)

    UserUiPreference.objects.update_or_create(
        user=request.user,
        defaults={'theme': theme},
    )
    return JsonResponse({'ok': True, 'theme': theme})

```



```

@swagger_auto_schema(method='get', operation_summary="Данные профиля
текущего пользователя", tags=['accounts'])
@api_view(['GET'])
@permission_classes([AllowAny])
@login_required
def api_profile_data(request):
    user = request.user
    # Administrators have their own separate profile page
    if is_admin_user(user):
        return JsonResponse({'error': 'admin_has_no_profile'},
status=403)
    applicant = Applicant.objects.filter(user=user).first()
    manager = Manager.objects.filter(user=user).first()
    data = {
        'first_name': user.first_name,
        'last_name': user.last_name,
        'email': user.email,
        'username': user.username,
        'role': 'manager' if manager else 'applicant',
    }
    bot_start_url = None
    if applicant:
        bot_username = get_bot_username()
        if bot_username and applicant.telegram_start_token:
            bot_start_url =
f'https://t.me/{bot_username}?start={applicant.telegram_start_token}'
            data.update({
                'telegram': applicant.telegram,
                'patronymic': getattr(applicant, 'patronymic', ''),
                'consent_telegram': applicant.consent_telegram,
                'consent_email': applicant.consent_email,
                'tg_linked': bool(applicant.telegram_chat_id),
                'phone': applicant.phone,
                'gender': applicant.gender,
                'city': applicant.city,
                'birth_date': applicant.birth_date.isoformat() if
applicant.birth_date else '',
                'citizenship': applicant.citizenship,
                'skills': applicant.skills or [],
                'location_type': applicant.location_type,
                'metro_city_id': applicant.metro_city_id,
                'metro_station_id': applicant.metro_station_id,
                'metro_station_name': applicant.metro_station_name,
                'metro_line_name': applicant.metro_line_name,
                'metro_line_color': applicant.metro_line_color,
                'metro_stations': applicant.metro_stations or [],
                'address': applicant.address,
                'avatar_url': applicant.avatar.url if
applicant.avatar else '',
                'resume_file_url': applicant.resume_file.url if
applicant.resume_file else '',
                'resume_file_name':
__import__('os').path.basename(applicant.resume_file.name) if
applicant.resume_file else '',
                'education': [
                    {
                        'id': e.pk,
                        'level': e.level,
                        'level_display': e.get_level_display(),
                        'institution': e.institution,
                        'graduation_year': e.graduation_year,
                        'faculty': e.faculty,
                        'specialization': e.specialization,

```

```

        }
        for e in applicant.educations.all()
    ],
    'extra_education': [
        {
            'id': x.pk,
            'name': x.name,
            'description': x.description,
            'document_serial': x.document_serial,
        }
        for x in applicant.extra_educations.all()
    ],
    'work_experience': [
        {
            'id': w.pk,
            'company': w.company,
            'position': w.position,
            'start_month': w.start_month,
            'start_year': w.start_year,
            'end_month': w.end_month,
            'end_year': w.end_year,
            'is_current': w.is_current,
            'responsibilities': w.responsibilities,
        }
        for w in applicant.work_experiences.all()
    ],
    })
if manager:
    data.update({
        'patronymic': manager.patronymic,
        'telegram': manager.telegram,
        'company': manager.company,
        'phone': manager.phone,
        'company_logo_url': manager.company_logo.url if
manager.company_logo else '',
        'consent_telegram': manager.consent_telegram,
        'consent_email': manager.consent_email,
        'tg_linked': bool(manager.telegram_chat_id),
    })
    if not bot_start_url:
        bot_username = get_bot_username()
        if bot_username and manager.telegram:
            bot_start_url = f'https://t.me/{bot_username}'
    # Avatar: prefer applicant.avatar (persists through role
switches); fall back to manager.avatar
    if not data.get('avatar_url'):
        data['avatar_url'] = manager.avatar.url if manager.avatar
    else ''
    return JsonResponse({'ok': True, 'user': data, 'bot_start_url':
bot_start_url})

def _feedback_submission_state(user):
    """Return feedback submission state for one user with a 7-day
cooldown."""
    from django.utils import timezone

    if not user.is_authenticated:
        return {
            'can_submit': False,
            'reason': 'not_authenticated',
            'last_created_at': None,
            'next_allowed_at': None,

```

```

        'remaining_days': None,
    }
    if is_admin_user(user) or is_moderator_user(user):
        return {
            'can_submit': False,
            'reason': 'forbidden_role',
            'last_created_at': None,
            'next_allowed_at': None,
            'remaining_days': None,
        }

    last_item = UserFeedback.objects.filter(user=user).order_by('-
created_at').first()
    if not last_item:
        return {
            'can_submit': True,
            'reason': 'ok',
            'last_created_at': None,
            'next_allowed_at': None,
            'remaining_days': 0,
        }

    next_allowed_at = last_item.created_at + timedelta(days=7)
    now = timezone.now()
    can_submit = now >= next_allowed_at
    remaining_days = 0 if can_submit else max(1, (next_allowed_at -
now).days + 1)
    return {
        'can_submit': can_submit,
        'reason': 'ok' if can_submit else 'cooldown',
        'last_created_at': last_item.created_at.isoformat(),
        'next_allowed_at': next_allowed_at.isoformat(),
        'remaining_days': remaining_days,
    }

```

```

@swagger_auto_schema(method='get', operation_summary="Статус отправки
предложений/критики", tags=['accounts'])
@swagger_auto_schema(method='post', operation_summary="Отправить
предложение/критику по сайту", tags=['accounts'])
@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_feedback(request):
    """
    Users except administrators and moderators can submit feedback.
    Cooldown: one message per 7 days.
    """
    state = _feedback_submission_state(request.user)

    if request.method == 'GET':
        return JsonResponse({'ok': True, **state})

    if state['reason'] == 'forbidden_role':
        return JsonResponse({'error': 'forbidden', 'detail':
'Администратор и модератор не могут отправлять предложения.'},
status=403)
    if not state['can_submit']:
        return JsonResponse({
            'error': 'cooldown',
            'detail': f"Следующее сообщение можно отправить через
{state['remaining_days']} дн.",
            **state,

```

```

        }, status=429)

        data = request.data if hasattr(request, 'data') else {}
        kind = str((data or {}).get('kind') or
UserFeedback.KIND_SUGGESTION).strip()
        if kind not in (UserFeedback.KIND_SUGGESTION,
UserFeedback.KIND_CRITICISM):
            kind = UserFeedback.KIND_SUGGESTION

        message = str((data or {}).get('message') or '').strip()
        if len(message) < 10:
            return JsonResponse({'error': 'message_too_short', 'detail':
'Опишите предложение подробнее (минимум 10 символов).'}, status=400)
        if len(message) > 2000:
            return JsonResponse({'error': 'message_too_long', 'detail':
'Слишком длинное сообщение (максимум 2000 символов).'}, status=400)

        item = UserFeedback.objects.create(
            user=request.user,
            kind=kind,
            message=message,
        )
        _log_api_action(
            request,
            action='user_feedback_create',
            before={},
            after={'feedback_id': item.pk, 'kind': kind},
            endpoint='accounts.api_feedback',
            status_code=201,
        )
        new_state = _feedback_submission_state(request.user)
        return JsonResponse({'ok': True, 'id': item.pk, **new_state},
status=201)

DOCUMENT_MAX_BYTES = 10 * 1024 * 1024 # 10 MB
DOCUMENT_ALLOWED_TYPES = {
    'image/jpeg',
    'image/png',
    'image/webp',
    'application/pdf',
}
DOCUMENT_ALLOWED_EXTS = {'.jpg', '.jpeg', '.png', '.webp', '.pdf'}
DOCUMENT_MAX_FILES = 6

def _normalize_doc_value(raw):
    return re.sub(r'[\s\~]', '', str(raw or '').strip())

def _mask_doc_number(number):
    value = str(number or '').strip()
    if not value:
        return ''
    if len(value) <= 4:
        return '*' * len(value)
    return ('*' * (len(value) - 4)) + value[-4:]

def _serialize_user_document(doc):
    return {
        'id': doc.pk,
        'doc_type': doc.doc_type,
    }

```

```

        'doc_type_label': doc.get_doc_type_display(),
        'serial': doc.serial,
        'number': doc.number,
        'number_masked': _mask_doc_number(doc.number),
        'issued_date': doc.issued_date.isoformat() if doc.issued_date
else '',
        'issued_by': doc.issued_by,
        'division_code': doc.division_code,
        'created_at': doc.created_at.isoformat(),
        'files': [
            {
                'id': f.pk,
                'name': os.path.basename(f.file.name),
                'url': f.file.url,
            }
            for f in doc.files.all()
        ],
    }

def _validate_document_payload(doc_type, serial, number, division_code):
    normalized_serial = _normalize_doc_value(serial)
    normalized_number = _normalize_doc_value(number)
    normalized_division = _normalize_doc_value(division_code)

    if doc_type == UserDocument.DOC_PASSPORT_RF:
        if not re.fullmatch(r'\d{4}', normalized_serial):
            return False, 'invalid_passport_series', 'Серия паспорта РФ
должна содержать 4 цифры.', '', '', ''
        if not re.fullmatch(r'\d{6}', normalized_number):
            return False, 'invalid_passport_number', 'Номер паспорта РФ
должен содержать 6 цифр.', '', '', ''
        if normalized_division and not re.fullmatch(r'\d{3}-?\d{3}',
normalized_division):
            return False, 'invalid_division_code', 'Код подразделения
должен быть в формате XXX-XXX.', '', '', ''
        normalized_division = normalized_division if not
normalized_division else f"{normalized_division[:3]}-
{normalized_division[-3:]}"
        elif doc_type == UserDocument.DOC_SNILS:
            if not re.fullmatch(r'\d{11}', normalized_number):
                return False, 'invalid_snils', 'СНИЛС должен содержать 11
цифр.', '', '', ''
            normalized_serial = ''
        elif doc_type == UserDocument.DOC_INN:
            if not re.fullmatch(r'\d{10}|\d{12}', normalized_number):
                return False, 'invalid_inn', 'ИНН должен содержать 10 или 12
цифр.', '', '', ''
            normalized_serial = ''
        elif doc_type == UserDocument.DOC_FOREIGN_PASSPORT:
            if not re.fullmatch(r'\d{2}', normalized_serial):
                return False, 'invalid_foreign_series', 'Серия загранпаспорта
должна содержать 2 цифры.', '', '', ''
            if not re.fullmatch(r'\d{7}', normalized_number):
                return False, 'invalid_foreign_number', 'Номер загранпаспорта
должен содержать 7 цифр.', '', '', ''
        elif doc_type == UserDocument.DOC_DRIVER_LICENSE:
            if not re.fullmatch(r'\d{4}', normalized_serial):
                return False, 'invalid_driver_series', 'Серия водительского
удостоверения должна содержать 4 цифры.', '', '', ''
            if not re.fullmatch(r'\d{6}', normalized_number):
                return False, 'invalid_driver_number', 'Номер водительского
удостоверения должен содержать 6 цифр.', '', '', ''

```

```

        elif doc_type == UserDocument.DOC_MILITARY_ID:
            if len(normalized_number) < 4:
                return False, 'invalid_military_number', 'Номер военного
билета слишком короткий.', '', '', ''
            if len(normalized_serial) > 16 or len(normalized_number) > 32:
                return False, 'invalid_military_fields', 'Серия/номер
военного билета указаны некорректно.', '', '', ''
            else:
                return False, 'invalid_doc_type', 'Неизвестный тип документа.',
'', '', ''

        return True, '', '', normalized_serial, normalized_number,
normalized_division

@swagger_auto_schema(methods=['get'], operation_summary="Список
документов пользователя", tags=['documents'])
@swagger_auto_schema(methods=['post'], operation_summary="Добавить
документ пользователя", tags=['documents'])
@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_profile_documents(request):
    """Создайте/перечислите документы, удостоверяющие личность
пользователя, прикрепленные к профилю."""
    if is_admin_user(request.user) or is_moderator_user(request.user):
        return JsonResponse({'error': 'forbidden', 'detail': 'Для этой
роли раздел документов недоступен.'}, status=403)
    if not Applicant.objects.filter(user=request.user).exists() and not
Manager.objects.filter(user=request.user).exists():
        return JsonResponse({'error': 'profile_not_found', 'detail':
'Профиль пользователя не найден.'}, status=404)

    if request.method == 'GET':
        docs =
UserDocument.objects.filter(user=request.user).prefetch_related('files')
        return JsonResponse({'ok': True, 'items':
[_serialize_user_document(d) for d in docs]})

    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)

    doc_type = str(request.POST.get('doc_type') or '').strip()
    serial = str(request.POST.get('serial') or '').strip()
    number = str(request.POST.get('number') or '').strip()
    issued_by = str(request.POST.get('issued_by') or '').strip()
    issued_date_raw = str(request.POST.get('issued_date') or '').strip()
    division_code = str(request.POST.get('division_code') or '').strip()

    ok, err_code, err_detail, normalized_serial, normalized_number,
normalized_division = _validate_document_payload(
        doc_type,
        serial,
        number,
        division_code,
    )
    if not ok:
        return JsonResponse({'error': err_code, 'detail': err_detail},
status=400)

    files = request.FILES.getlist('files')
    if not files:
        return JsonResponse({'error': 'files_required', 'detail':

```

```

'Прикрепите хотя бы один файл документа.'}, status=400)
    if len(files) > DOCUMENT_MAX_FILES:
        return JsonResponse({'error': 'too_many_files', 'detail': f'Можно
прикрепить не более {DOCUMENT_MAX_FILES} файлов.'}, status=400)
    for f in files:
        ext = os.path.splitext(f.name or '')[1].lower()
        if f.size > DOCUMENT_MAX_BYTES:
            return JsonResponse({'error': 'file_too_large', 'detail':
'Один из файлов превышает 10 МБ.'}, status=400)
        if (f.content_type not in DOCUMENT_ALLOWED_TYPES) and (ext not in
DOCUMENT_ALLOWED_EXTS):
            return JsonResponse({'error': 'invalid_file_type', 'detail':
'Допустимы только JPG, PNG, WEBP и PDF.'}, status=400)

        issued_date_val = None
        if issued_date_raw:
            try:
                issued_date_val = date.fromisoformat(issued_date_raw)
            except ValueError:
                return JsonResponse({'error': 'invalid_issued_date',
'detail': 'Некорректная дата выдачи.'}, status=400)

        doc = UserDocument.objects.create(
            user=request.user,
            doc_type=doc_type,
            serial=normalized_serial,
            number=normalized_number,
            issued_date=issued_date_val,
            issued_by=issued_by,
            division_code=normalized_division,
        )
        for f in files:
            UserDocumentFile.objects.create(document=doc, file=f)

        _log_api_action(
            request,
            action='user_document_create',
            before={},
            after={'document_id': doc.pk, 'doc_type': doc.doc_type, 'files':
len(files)}),
            endpoint='accounts.api_profile_documents',
            status_code=201,
        )
        doc =
UserDocument.objects.filter(pk=doc.pk).prefetch_related('files').first()
        return JsonResponse({'ok': True, 'item':
_serialize_user_document(doc)}, status=201)

@swagger_auto_schema(method='post', operation_summary="Удалить документ
пользователя", tags=['documents'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_profile_document_delete(request, pk):
    """Delete one user document and all attached files."""
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    doc = UserDocument.objects.filter(pk=pk,
user=request.user).prefetch_related('files').first()
    if not doc:
        return JsonResponse({'error': 'not_found', 'detail': 'Документ не
найден.'}, status=404)

```

```

doc_type = doc.doc_type
doc_id = doc.pk
for f in doc.files.all():
    f.file.delete(save=False)
doc.delete()
_log_api_action(
    request,
    action='user_document_delete',
    before={'document_id': doc_id, 'doc_type': doc_type},
    after={'deleted': True},
    endpoint='accounts.api_profile_document_delete',
    status_code=200,
)
return JsonResponse({'ok': True})

@swagger_auto_schema(method='get', operation_summary="Данные о станциях метро", tags=['accounts'])
@api_view(['GET'])
@permission_classes([AllowAny])
def api_metro_data(request):
    """Return full metro station data from the locally cached HH JSON."""
    metro_path = os.path.join(os.path.dirname(__file__), '..', 'tools', 'metro_hh.json')
    metro_path = os.path.normpath(metro_path)
    try:
        with open(metro_path, encoding='utf-8') as f:
            data = json.load(f)
            return JsonResponse({'ok': True, 'cities': data})
    except FileNotFoundError:
        return JsonResponse({'ok': False, 'cities': []})

@swagger_auto_schema(method='patch', operation_summary="Обновить профиль", tags=['accounts'])
@api_view(['PATCH'])
@permission_classes([IsAuthenticated])
def api_profile_update(request):
    from datetime import date as _date

    user = request.user
    data = (request.data or {})

    # — User-level fields —————
    user_changed = []
    for field in ('first_name', 'last_name', 'email', 'username'):
        if field not in data:
            continue
        val = str(data[field]).strip()
        if field == 'username':
            if len(val) < 3 or len(val) > 30:
                return JsonResponse({'error': 'invalid_username'},
status=400)
            if not re.match(r'^[A-Za-z0-9_.\-]+$ ', val):
                return JsonResponse({'error': 'invalid_username_format'},
status=400)
            if
User.objects.filter(username__iexact=val).exclude(pk=user.pk).exists():
                return JsonResponse({'error': 'username_exists'},
status=400)
            elif field == 'email':
                val = val.lower()
                if

```



```

User.objects.filter(email=val).exclude(pk=user.pk).exists():
    return JsonResponse({'error': 'email_exists'},
status=400)
    setattr(user, field, val)
    user_changed.append(field)
if user_changed:
    user.save(update_fields=user_changed)

# — Applicant fields —————
applicant = Applicant.objects.filter(user=user).first()
manager = Manager.objects.filter(user=user).first()
if applicant and not manager:
    # Validate phone format if provided
    if 'phone' in data and data['phone']:
        import re as _re2
        _pd = _re2.sub(r'\D', '', str(data['phone']))
        if _pd.startswith('8'): _pd = '7' + _pd[1:]
        if not _re2.match(r'^7[0-9]{10}$', _pd):
            return JsonResponse({'error': 'invalid_phone'},
status=400)
    a_changed = []
    for f in ('patronymic', 'telegram', 'phone', 'gender', 'city',
'citizenship',
        'desired_position', 'github_url', 'portfolio_url'):
        if f in data:
            setattr(applicant, f, str(data[f]).strip())
            a_changed.append(f)
    if 'about_me' in data:
        applicant.about_me = str(data['about_me']).strip()
        a_changed.append('about_me')
    if 'birth_date' in data and data['birth_date']:
        try:
            applicant.birth_date =
_date.fromisoformat(str(data['birth_date']))
            a_changed.append('birth_date')
        except ValueError:
            return JsonResponse({'error': 'invalid_birth_date'},
status=400)
    # Location fields
    for f in ('location_type', 'metro_city_id', 'metro_station_id',
        'metro_station_name', 'metro_line_name',
'metro_line_color', 'address'):
        if f in data:
            setattr(applicant, f, str(data[f]).strip())
            a_changed.append(f)
    if 'metro_stations' in data:
        stations = data['metro_stations']
        applicant.metro_stations = stations if isinstance(stations,
list) else []
    # Keep legacy single-station fields in sync with first entry
    if applicant.metro_stations:
        first = applicant.metro_stations[0]
        applicant.metro_station_id = first.get('stationId', '')
        applicant.metro_station_name = first.get('stationName',
'')

        applicant.metro_line_name = first.get('lineName', '')
        applicant.metro_line_color = first.get('lineColor', '')
        applicant.metro_city_id = first.get('cityId', '')
        for f in ('metro_station_id', 'metro_station_name',
'metro_line_name',
            'metro_line_color', 'metro_city_id'):
            if f not in a_changed:
                a_changed.append(f)

```

```

        else:
            applicant.metro_station_id = ''
            applicant.metro_station_name = ''
            applicant.metro_line_name = ''
            applicant.metro_line_color = ''
            for f in ('metro_station_id', 'metro_station_name',
'metro_line_name', 'metro_line_color'):
                if f not in a_changed:
                    a_changed.append(f)
            a_changed.append('metro_stations')
        if a_changed:
            applicant.save(update_fields=a_changed)

        # Salary expectations (stored separately; allow clearing with
null)
        salary_changed = []
        for f in ('salary_expectation_from', 'salary_expectation_to'):
            if f in data:
                raw = data[f]
                if raw is None or str(raw).strip() == '':
                    setattr(applicant, f, None)
                else:
                    try:
                        setattr(applicant, f, int(str(raw).strip()))
                    except (ValueError, TypeError):
                        return JsonResponse({'error': 'invalid_salary'},
status=400)
                    salary_changed.append(f)
            if salary_changed:
                applicant.save(update_fields=salary_changed)

        # — Manager fields —————
        if manager:
            # Uniqueness checks
            if 'telegram' in data:
                tg_val = str(data.get('telegram', '')).strip()
                if tg_val and
Manager.objects.filter(telegram=tg_val).exclude(user=user).exists():
                    return JsonResponse({'error': 'telegram_exists'},
status=400)
            if 'phone' in data:
                ph_val = str(data.get('phone', '')).strip()
                if ph_val and
Manager.objects.filter(phone=ph_val).exclude(user=user).exists():
                    return JsonResponse({'error': 'phone_exists'},
status=400)
            m_changed = []
            for f in ('patronymic', 'telegram', 'phone', 'company'):
                if f in data:
                    setattr(manager, f, str(data.get(f, '')).strip())
                    m_changed.append(f)
            if m_changed:
                manager.save(update_fields=m_changed)

        return JsonResponse({'ok': True})

# —————
# Application (отклик) views
# —————

@swagger_auto_schema(method='post', operation_summary="Откликнуться на
вакансию", tags=['accounts'])

```

```

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_apply(request):
    """Соискатель подает заявку на вакансию."""
    if not hasattr(request.user, 'applicant'):
        return JsonResponse({'error': 'not_applicant'}, status=403)
    data = request.data or {}
    vacancy_id = data.get('vacancy_id')
    cover_letter = (data.get('cover_letter') or '').strip()
    if not vacancy_id:
        return JsonResponse({'error': 'vacancy_id required'}, status=400)
    from vacancies.models import Vacancy
    from django.shortcuts import get_object_or_404
    vacancy = get_object_or_404(Vacancy, pk=vacancy_id)
    if vacancy.is_hh:
        return JsonResponse({'error': 'hh_vacancy'}, status=400)
    if not vacancy.is_active:
        return JsonResponse({'error': 'vacancy_archived'}, status=400)
    # Заблокировать подачу заявки на собственную вакансию (как
    работодателю, так и любому пользователю, который ее создал)
    if vacancy.created_by_id == request.user.pk:
        return JsonResponse({'error': 'own_vacancy'}, status=403)
    app, created = Application.objects.get_or_create(
        vacancy=vacancy, applicant=request.user,
        defaults={'cover_letter': cover_letter},
    )
    if not created and cover_letter:
        # allow updating cover letter if still pending
        if app.status == 'pending':
            app.cover_letter = cover_letter
            app.save(update_fields=['cover_letter'])

    # Notify the vacancy manager about the new application (fire-and-
    forget)
    if created and vacancy.created_by_id:
        try:
            mgr_user = vacancy.created_by
            mgr = getattr(mgr_user, 'manager', None)
            if mgr:
                applicant_name = request.user.get_full_name() or
                request.user.username
                msg = (
                    f"📧 Новый отклик на «{vacancy.title}»\n"
                    f"Соискатель: {applicant_name}"
                )
                if mgr.consent_telegram and mgr.telegram_chat_id:
                    send_hello_async(mgr.telegram_chat_id, msg)
                if mgr.consent_email and mgr_user.email:
                    send_mail(
                        subject=f"Новый отклик на «{vacancy.title}»",
                        message=msg,
                        from_email=django_settings.DEFAULT_FROM_EMAIL,
                        recipient_list=[mgr_user.email],
                        fail_silently=True,
                    )
        except Exception:
            pass # notifications are best-effort

    return JsonResponse({'ok': True, 'created': created, 'status':
    app.status})

```

```

@swagger_auto_schema(method='post', operation_summary="Изменить статус отклика", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_application_status(request, pk):
    """Manager updates the status of an application."""
    from django.shortcuts import get_object_or_404
    if not hasattr(request.user, 'manager'):
        _log_api_action(
            request,
            action='application_status_change',
            before={'application_id': pk},
            after={'error': 'not_manager'},
            success=False,
            status_code=403,
            endpoint='api_application_status',
        )
        return JsonResponse({'error': 'not_manager'}, status=403)
    app = get_object_or_404(Application, pk=pk)
    new_status = (request.data or {}).get('status', '')
    valid = {s for s, _ in Application.STATUS_CHOICES}
    if new_status not in valid:
        _log_api_action(
            request,
            action='application_status_change',
            before={'application_id': app.pk, 'status': app.status},
            after={'error': 'invalid_status', 'requested_status':
new_status},
            success=False,
            status_code=400,
            endpoint='api_application_status',
        )
        return JsonResponse({'error': 'invalid_status'}, status=400)
    old_status = app.status
    app.status = new_status
    app.save(update_fields=['status'])
    _log_api_action(
        request,
        action='application_status_change',
        before={'application_id': app.pk, 'status': old_status},
        after={
            'application_id': app.pk,
            'status': app.status,
            'vacancy_id': app.vacancy_id,
            'applicant_id': app.applicant_id,
        },
        success=True,
        status_code=200,
        endpoint='api_application_status',
    )
    return JsonResponse({'ok': True, 'status': app.status})

```

— Avatar / logo upload

```

UPLOAD_MAX_BYTES = 5 * 1024 * 1024 # 5 MB
UPLOAD_ALLOWED_TYPES = {'image/jpeg', 'image/png', 'image/webp',
'image/gif'}

```

```

def _check_upload(f):
    """Return error string or None."""

```

```

    if not f:
        return 'no_file'
    if f.size > UPLOAD_MAX_BYTES:
        return 'file_too_large'
    if f.content_type not in UPLOAD_ALLOWED_TYPES:
        return 'invalid_type'
    return None

@swagger_auto_schema(method='post', operation_summary="Загрузить аватар
пользователя", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_upload_avatar(request):
    """Upload / replace user avatar.
    Saves the same file to BOTH applicant and manager records so the
photo
is shared between roles and survives role switches.
    """
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    f = request.FILES.get('avatar')
    err = _check_upload(f)
    if err:
        return JsonResponse({'error': err}, status=400)

    applicant = Applicant.objects.filter(user=request.user).first()
    manager = Manager.objects.filter(user=request.user).first()

    if not applicant and not manager:
        return JsonResponse({'error': 'profile_not_found'}, status=404)

    saved_url = None

    # Applicant is the primary storage (record never gets deleted on role
switch)
    if applicant:
        if applicant.avatar:
            applicant.avatar.delete(save=False)
        applicant.avatar = f
        applicant.save(update_fields=['avatar'])
        saved_url = applicant.avatar.url

    # Sync the same file path to manager record (no extra disk copy
needed)
    if manager:
        if manager.avatar:
            manager.avatar.delete(save=False)
        if applicant and applicant.avatar:
            manager.avatar = applicant.avatar.name # string assignment,
points to same file
            manager.save(update_fields=['avatar'])
        else:
            # manager-only user: save file normally
            f.seek(0)
            manager.avatar = f
            manager.save(update_fields=['avatar'])
        if saved_url is None:
            saved_url = manager.avatar.url

    return JsonResponse({'ok': True, 'url': saved_url})

```

```

@swagger_auto_schema(method='post', operation_summary="Загрузить логотип
компании", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_upload_company_logo(request):
    """Upload / replace company logo (managers only)."""
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    if not hasattr(request.user, 'manager'):
        return JsonResponse({'error': 'not_manager'}, status=403)
    f = request.FILES.get('logo')
    err = _check_upload(f)
    if err:
        return JsonResponse({'error': err}, status=400)
    manager = request.user.manager
    if manager.company_logo:
        manager.company_logo.delete(save=False)
    manager.company_logo = f
    manager.save(update_fields=['company_logo'])
    return JsonResponse({'ok': True, 'url': manager.company_logo.url})

```

— Resume file upload

```

RESUME_MAX_BYTES = 10 * 1024 * 1024 # 10 MB
RESUME_ALLOWED_TYPES = {
    'application/vnd.openxmlformats-
officedocument.wordprocessingml.document', # .docx
    'application/msword', # .doc
}
RESUME_ALLOWED_EXTS = {'.docx', '.doc'}

```

```

@swagger_auto_schema(method='post', operation_summary="Загрузить резюме
(Word)", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_upload_resume(request):
    """Upload a Word resume file (.docx / .doc) for the logged-in
applicant."""
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    applicant = Applicant.objects.filter(user=request.user).first()
    if not applicant:
        return JsonResponse({'error': 'profile_not_found'}, status=404)
    f = request.FILES.get('resume')
    if not f:
        return JsonResponse({'error': 'no_file'}, status=400)
    if f.size > RESUME_MAX_BYTES:
        return JsonResponse({'error': 'file_too_large'}, status=400)
    ext = os.path.splitext(f.name)[1].lower()
    if ext not in RESUME_ALLOWED_EXTS:
        return JsonResponse({'error': 'invalid_type'}, status=400)
    # Delete the old file if it exists
    if applicant.resume_file:
        applicant.resume_file.delete(save=False)
    applicant.resume_file = f
    applicant.save(update_fields=['resume_file'])
    return JsonResponse({

```

```

        'ok': True,
        'url': applicant.resume_file.url,
        'name': os.path.basename(applicant.resume_file.name),
    })

@swagger_auto_schema(method='post', operation_summary="Удалить
загруженное резюме", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_delete_resume(request):
    """DELETE the uploaded Word resume file for the logged-in
applicant."""
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    applicant = Applicant.objects.filter(user=request.user).first()
    if not applicant:
        return JsonResponse({'error': 'profile_not_found'}, status=404)
    if applicant.resume_file:
        applicant.resume_file.delete(save=False)
        applicant.resume_file = None
        applicant.save(update_fields=['resume_file'])
    return JsonResponse({'ok': True})

```

— Bookmarks

```

@swagger_auto_schema(method='post', operation_summary="Добавить/убрать
вакансию из закладок", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_toggle_bookmark(request, pk):
    """POST - toggle bookmark on a vacancy. Returns {'ok',
'bookmarked'}."""
    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    from vacancies.models import Vacancy, Bookmark
    try:
        vacancy = Vacancy.objects.get(pk=pk)
    except Vacancy.DoesNotExist:
        return JsonResponse({'error': 'not_found'}, status=404)
    obj, created = Bookmark.objects.get_or_create(user=request.user,
vacancy=vacancy)
    if not created:
        obj.delete()
        return JsonResponse({'ok': True, 'bookmarked': False})
    return JsonResponse({'ok': True, 'bookmarked': True})

```

— Applicant analytics

```

@swagger_auto_schema(method='get', operation_summary="Персональная
аналитика соискателя", tags=['accounts'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
@login_required
def api_applicant_analytics(request):
    """Return personal analytics: bookmarks, viewed vacancies, profile
data for charts."""

```

```

from vacancies.models import Bookmark, VacancyView

user = request.user

# — All bookmarks (loaded once; slice for display list)


---


all_bookmarks_qs = (
    Bookmark.objects.filter(user=user)
    .select_related('vacancy__employer')
    .order_by('-created_at')
)

bookmark_list = []
for b in all_bookmarks_qs[:20]:
    v = b.vacancy
    bookmark_list.append({
        'id': v.pk, 'external_id': v.external_id, 'title': v.title,
'company': v.company,
        'region': v.region,
        'salary_from': v.salary_from, 'salary_to': v.salary_to,
        'salary_currency': v.salary_currency,
        'logo_url': v.employer_logo_url or '',
        'saved_at': b.created_at.strftime('%d.%m.%Y'),
        'experience_name': v.experience_name or '',
    })

# — Recently viewed


---


viewed_qs = (
    VacancyView.objects.filter(user=user)
    .select_related('vacancy__employer')
    .order_by('-viewed_at')[:200]
)
viewed_list = []
for vv in viewed_qs:
    v = vv.vacancy
    viewed_list.append({
        'id': v.pk, 'external_id': v.external_id, 'title': v.title,
'company': v.company,
        'region': v.region,
        'salary_from': v.salary_from, 'salary_to': v.salary_to,
        'salary_currency': v.salary_currency,
        'logo_url': v.employer_logo_url or '',
        'viewed_at': vv.viewed_at.strftime('%d.%m.%Y %H:%M'),
    })

# — Profile completeness + skills + salary


---


applicant = getattr(user, 'applicant', None)
profile_completeness = {'filled': 0, 'total': 0, 'fields': {}}
skills = []
salary_data = {'expectation_from': None, 'expectation_to': None,
'market': []}
charts_data = {'experience': {}, 'work_format': {}}

if applicant:
    fields_map = {
        'Фото': bool(applicant.avatar),
        'Имя': bool(user.first_name and user.first_name.strip()),
        'Телефон': bool(applicant.phone),
        'Город': bool(applicant.city),
        'Дата рождения': bool(applicant.birth_date),
        'Пол': bool(applicant.gender),
    }

```



```

        'Гражданство': bool(applicant.citizenship),
        'Навыки': bool(applicant.skills),
        'Образование': applicant.educations.exists(),
        'Опыт работы': applicant.work_experiences.exists(),
        'Зарплатные ожидания':
bool(applicant.salary_expectation_from),
    }
    profile_completeness = {
        'filled': sum(fields_map.values()),
        'total': len(fields_map),
        'fields': fields_map,
    }

    skills = applicant.skills or []

    # Salary data + chart distributions — single pass over all
bookmarks
    from collections import Counter
    market_salaries = []
    exp_counter = Counter()
    fmt_counter = Counter()
    for b in all_bookmarks_qs:
        v = b.vacancy
        if v.salary_from or v.salary_to:
            market_salaries.append({
                'from': v.salary_from,
                'to': v.salary_to,
                'currency': v.salary_currency or 'RUB',
            })
        exp_counter[v.experience_name or 'Не указан'] += 1
        has_fmt = False
        if v.is_remote: fmt_counter['Удалённо'] += 1; has_fmt = True
        if v.is_hybrid: fmt_counter['Гибрид'] += 1; has_fmt = True
        if v.is_onsite and not v.is_remote and not v.is_hybrid:
            fmt_counter['Офис'] += 1; has_fmt = True
        if not has_fmt:
            fmt_counter['Не указан'] += 1

    salary_data = {
        'expectation_from': applicant.salary_expectation_from,
        'expectation_to': applicant.salary_expectation_to,
        'market': market_salaries,
    }
    charts_data = {
        'experience': dict(exp_counter.most_common()),
        'work_format': dict(fmt_counter.most_common()),
    }

    # — Activity data (heatmap + weekly line chart)


---


    from datetime import date as _date, timedelta
    from django.db.models import Count as _Count
    from django.db.models.functions import (
        TruncDate as _TruncDate, TruncWeek as _TruncWeek,
        TruncMonth as _TruncMonth, TruncYear as _TruncYear,
        ExtractHour as _ExtractHour,
    )

    today_d = _date.today()
    since_365 = today_d - timedelta(days=364)
    since_16w = today_d - timedelta(weeks=16)
    since_30d = today_d - timedelta(days=30)
    since_24m = today_d - timedelta(days=730)

```

```

daily_qs = (
    VacancyView.objects
    .filter(user=user, viewed_at__date__gte=since_365)
    .annotate(day=_TruncDate('viewed_at'))
    .values('day')
    .annotate(count=_Count('id'))
    .order_by('day')
)
daily_counts = {str(row['day']): row['count'] for row in daily_qs}

weekly_qs = (
    VacancyView.objects
    .filter(user=user, viewed_at__date__gte=since_16w)
    .annotate(week=_TruncWeek('viewed_at'))
    .values('week')
    .annotate(count=_Count('id'))
    .order_by('week')
)
weekly_counts = [
    {'week': row['week'].strftime('%d.%m'), 'count': row['count']}
    for row in weekly_qs
]

hourly_qs = (
    VacancyView.objects
    .filter(user=user, viewed_at__date__gte=since_30d)
    .annotate(hour=_ExtractHour('viewed_at'))
    .values('hour')
    .annotate(count=_Count('id'))
    .order_by('hour')
)
hourly_counts = {str(row['hour']): row['count'] for row in hourly_qs}

monthly_qs = (
    VacancyView.objects
    .filter(user=user, viewed_at__date__gte=since_24m)
    .annotate(month=_TruncMonth('viewed_at'))
    .values('month')
    .annotate(count=_Count('id'))
    .order_by('month')
)
monthly_counts = [
    {'month': row['month'].strftime('%m.%Y'), 'count': row['count']}
    for row in monthly_qs
]

yearly_qs = (
    VacancyView.objects
    .filter(user=user)
    .annotate(year=_TruncYear('viewed_at'))
    .values('year')
    .annotate(count=_Count('id'))
    .order_by('year')
)
yearly_counts = [
    {'year': row['year'].strftime('%Y'), 'count': row['count']}
    for row in yearly_qs
]

activity_data = {
    'daily': daily_counts,
    'weekly': weekly_counts,

```

```

        'hourly': hourly_counts,
        'monthly': monthly_counts,
        'yearly': yearly_counts,
        'today': str(today_d),
    }

    return JsonResponse({
        'ok': True,
        'bookmarks': bookmark_list,
        'viewed_vacancies': viewed_list,
        'profile_completeness': profile_completeness,
        'skills': skills,
        'salary_data': salary_data,
        'activity': activity_data,
        'charts_data': charts_data,
    })

@login_required
def vacancy_applications(request, pk):
    """Manager sees all applicants who applied to a specific vacancy."""
    from django.shortcuts import get_object_or_404
    from django.db.models import Prefetch
    if not hasattr(request.user, 'manager'):
        from django.shortcuts import redirect
        return redirect('vacancy-list')
    from vacancies.models import Vacancy
    vacancy = get_object_or_404(Vacancy, pk=pk)
    apps = (
        Application.objects
        .filter(vacancy=vacancy)
        .select_related('applicant', 'applicant__applicant')
        .prefetch_related(
            Prefetch('applicant__applicant__educations',
                    queryset=Education.objects.order_by('-order')),
            Prefetch('applicant__applicant__work_experiences',
                    queryset=WorkExperience.objects.order_by('-order')),
        )
        .order_by('-created_at')
    )
    return render(request, 'accounts/vacancy_applications.html', {
        'vacancy': vacancy,
        'applications': apps,
        'status_choices': Application.STATUS_CHOICES,
    })

@swagger_auto_schema(method='get', operation_summary="Анализ резюме",
tags=['accounts'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def api_resume_analyze(request):
    """Analyze the current user's resume for completeness,
inconsistencies, and spelling."""
    import requests as _req
    from datetime import date as _date

    applicant = getattr(request.user, 'applicant', None)
    if not applicant:
        return JsonResponse({'error': 'not_applicant'}, status=403)

```

```

educations = list(applicant.educations.all())
experiences = list(applicant.work_experiences.all())
today_year = _date.today().year
birth_year = applicant.birth_date.year if applicant.birth_date else
None

issues = []

# — Completeness —————
if not applicant.about_me:
    issues.append({'type': 'tip', 'section': 'О себе',
                  'text': 'Добавьте краткое описание «О себе» – это
первое, что читает рекрутер'})
if not applicant.desired_position:
    issues.append({'type': 'tip', 'section': 'Желаемая должность',
                  'text': 'Укажите желаемую должность, чтобы
рекрутер сразу понял, на что вы претендуете'})
if not applicant.skills:
    issues.append({'type': 'warning', 'section': 'Навыки',
                  'text': 'Навыки не указаны – это снижает шансы
попасть в результаты поиска'})
if not applicant.phone:
    issues.append({'type': 'tip', 'section': 'Контакты',
                  'text': 'Укажите номер телефона для связи'})
if not applicant.city:
    issues.append({'type': 'tip', 'section': 'Город',
                  'text': 'Укажите город проживания'})
if not applicant.birth_date:
    issues.append({'type': 'tip', 'section': 'Личные данные',
                  'text': 'Укажите дату рождения'})
if not experiences:
    issues.append({'type': 'warning', 'section': 'Опыт работы',
                  'text': 'Не добавлен опыт работы'})
if not educations:
    issues.append({'type': 'warning', 'section': 'Образование',
                  'text': 'Не добавлено образование'})
if not applicant.salary_expectation_from:
    issues.append({'type': 'tip', 'section': 'Зарплата',
                  'text': 'Укажите ожидаемую зарплату – работодатели
ориентируются на неё'})
if not applicant.github_url and not applicant.portfolio_url:
    issues.append({'type': 'tip', 'section': 'Ссылки',
                  'text': 'Добавьте ссылку на GitHub или портфолио –
это выделяет вас среди кандидатов'})

# — Temporal / logical inconsistencies —————
for exp in experiences:
    label = f'«{exp.company}»'
    if exp.start_year:
        if birth_year and exp.start_year < birth_year + 14:
            issues.append({'type': 'suspicious', 'section': 'Опыт
работы',
                          'text': f'{label}: начало в
{exp.start_year}, но по дате рождения вам было бы < 14 лет'})
        if exp.start_year > today_year + 1:
            issues.append({'type': 'error', 'section': 'Опыт работы',
                          'text': f'{label}: дата начала в будущем
({exp.start_year})'})
    if exp.end_year:
        if exp.end_year < exp.start_year:
            issues.append({'type': 'error', 'section': 'Опыт
работы',

```

```

        'text': f'{label}: год окончания
({exp.end_year}) раньше года начала ({exp.start_year})'))
        if not exp.is_current and exp.end_year > today_year:
            issues.append({'type': 'suspicious', 'section': 'Опыт
работы',
                           'text': f'{label}: год окончания
{exp.end_year} в будущем, но не отмечено «Работаю сейчас»'})
        if exp.responsibilities and len(exp.responsibilities.strip()) <
30:
            issues.append({'type': 'tip', 'section': 'Опыт работы',
                           'text': f'{label}: описание обязанностей очень
короткое — расскажите подробнее'})

# Overlapping work periods
periods = []
for exp in experiences:
    if exp.start_year:
        end = today_year if exp.is_current else (exp.end_year or
today_year)
        periods.append((exp.start_year, end, exp.company))
for i in range(len(periods)):
    for j in range(i + 1, len(periods)):
        a_s, a_e, a_c = periods[i]
        b_s, b_e, b_c = periods[j]
        overlap = min(a_e, b_e) - max(a_s, b_s)
        if overlap >= 1:
            issues.append({'type': 'suspicious', 'section': 'Опыт
работы',
                           'text': f'Пересекающиеся периоды: «{a_c}»
и «{b_c}» ({max(a_s,b_s)}-{min(a_e,b_e)} г.)'})

for edu in educations:
    if edu.graduation_year:
        if birth_year and edu.graduation_year < birth_year + 15:
            issues.append({'type': 'suspicious', 'section':
'Образование',
                           'text': f'«{edu.institution}»: год
окончания {edu.graduation_year} — вам было бы < 15 лет'})
        if edu.graduation_year > today_year + 8:
            issues.append({'type': 'suspicious', 'section':
'Образование',
                           'text': f'«{edu.institution}»: год
окончания {edu.graduation_year} — слишком далеко в будущем'})

# High salary with no experience
if applicant.salary_expectation_from and not experiences:
    if applicant.salary_expectation_from > 200_000:
        issues.append({'type': 'suspicious', 'section': 'Зарплата',
                        'text': f'Ожидаемая зарплата
{applicant.salary_expectation_from:,} ₽ при отсутствии опыта работы —
возможно, завышена'})

# — Spell check via Yandex Speller (free, no key needed) —
spell_errors = []
texts_to_check = []
if applicant.about_me:
    texts_to_check.append(('О себе', applicant.about_me))
if applicant.desired_position:
    texts_to_check.append(('Желаемая должность',
applicant.desired_position))
for exp in experiences:
    if exp.responsibilities:
        texts_to_check.append((f'Опыт: {exp.company}',

```

```

exp.responsibilities))

    try:
        for label, text in texts_to_check[:6]:
            resp = _req.post(

'https://speller.yandex.net/services/spellservice.json/checkText',
            data={'text': text[:3000], 'lang': 'ru,en', 'options':
4},
                timeout=4,
            )
            if resp.status_code == 200:
                for err in resp.json()[:8]:
                    spell_errors.append({
                        'section': label,
                        'word': err.get('word', ''),
                        'suggestions': err.get('s', [])[:3],
                    })
    except Exception:
        pass # spell check is best-effort

    return JsonResponse({'ok': True, 'issues': issues, 'spell_errors':
spell_errors})

@login_required
def export_applications_csv(request, pk):
    """Manager downloads a CSV with all applicants for a specific
vacancy."""
    from django.shortcuts import get_object_or_404
    from vacancies.models import Vacancy
    if not hasattr(request.user, 'manager'):
        return redirect('vacancy-list')
    vacancy = get_object_or_404(Vacancy, pk=pk)
    apps = (
        Application.objects
        .filter(vacancy=vacancy)
        .select_related('applicant', 'applicant__applicant')
        .order_by('-created_at')
    )
    response = HttpResponse(content_type='text/csv; charset=utf-8-sig')
    filename = f'applicants_{vacancy.pk}.csv'
    response['Content-Disposition'] = f'attachment;
filename="{filename}"'
    writer = csv.writer(response)
    writer.writerow(['ФИО', 'Email', 'Телефон', 'Город', 'Навыки',
'Статус', 'Дата отклика', 'Сопроводительное письмо'])
    status_labels = dict(Application.STATUS_CHOICES)
    for app in apps:
        u = app.applicant
        ap = getattr(u, 'applicant', None)
        writer.writerow([
            u.get_full_name() or u.username,
            u.email,
            ap.phone if ap else '',
            ap.city if ap else '',
            ', '.join(ap.skills) if ap and ap.skills else '',
            status_labels.get(app.status, app.status),
            app.created_at.strftime('%d.%m.%Y'),
            app.cover_letter,
        ])
    return response

```

```

@login_required
def resume_pdf(request, pk):
    """Print-friendly resume page for an applicant (accessible to the
    applicant and managers)."""
    from django.shortcuts import get_object_or_404
    applicant = get_object_or_404(Applicant, pk=pk)
    is_own = (request.user == applicant.user)
    is_mgr = hasattr(request.user, 'manager')
    is_admin = request.user.is_superuser
    if not (is_own or is_mgr or is_admin):
        from django.http import Http404
        raise Http404

    # Manager viewing resume → mark pending applications as "viewed"
    if is_mgr and not is_own:
        from vacancies.models import Vacancy
        mgr_vacancies = Vacancy.objects.filter(created_by=request.user)
        Application.objects.filter(
            applicant=applicant.user,
            vacancy__in=mgr_vacancies,
            status='pending',
        ).update(status='viewed')

    educations = applicant.educations.order_by('order')
    experiences = applicant.work_experiences.order_by('order')
    extra_educations = applicant.extra_educations.order_by('order')
    return render(request, 'accounts/resume_pdf.html', {
        'applicant': applicant,
        'educations': educations,
        'experiences': experiences,
        'extra_educations': extra_educations,
    })

@login_required
def resume_word_view(request, pk):
    """Render a Word resume (.docx) as styled HTML (manager or owner
    only)."""
    from django.shortcuts import get_object_or_404, redirect
    from django.http import Http404
    from django.utils.html import escape

    applicant = get_object_or_404(Applicant, pk=pk)
    is_own = (request.user == applicant.user)
    is_mgr = hasattr(request.user, 'manager')
    if not (is_own or is_mgr):
        raise Http404

    # Manager viewing resume → mark pending applications as "viewed"
    if is_mgr and not is_own:
        from vacancies.models import Vacancy
        mgr_vacancies = Vacancy.objects.filter(created_by=request.user)
        Application.objects.filter(
            applicant=applicant.user,
            vacancy__in=mgr_vacancies,
            status='pending',
        ).update(status='viewed')

    if not applicant.resume_file:
        return redirect('accounts:resume_pdf', pk=pk)

    file_path = applicant.resume_file.path

```

```

ext = os.path.splitext(file_path)[1].lower()

html_parts = []

try:
    if ext == '.docx':
        from docx import Document
        from docx.oxml.ns import qn as _qn
        from docx.text.paragraph import Paragraph as _Paragraph
        from docx.table import Table as _Table
        from docx.enum.text import WD_ALIGN_PARAGRAPH

        ALIGN_MAP = {
            WD_ALIGN_PARAGRAPH.LEFT: 'left',
            WD_ALIGN_PARAGRAPH.CENTER: 'center',
            WD_ALIGN_PARAGRAPH.RIGHT: 'right',
            WD_ALIGN_PARAGRAPH.JUSTIFY: 'justify',
            WD_ALIGN_PARAGRAPH.DISTRIBUTE: 'justify',
        }

        def _emu_to_px(emu):
            if emu is None:
                return None
            return round(emu / 914400 * 96)

        def _process_para(para):
            fmt = para.paragraph_format
            style_name = (para.style.name or '').lower()
            level = None
            if 'heading 1' in style_name:
                level = 1
            elif 'heading 2' in style_name:
                level = 2
            elif 'heading 3' in style_name:
                level = 3

            para_styles = []
            align = ALIGN_MAP.get(para.alignment)
            if align:
                para_styles.append(f'text-align:{align}')
            li = fmt.left_indent
            if li is not None:
                px = _emu_to_px(li)
                if px and px > 0:
                    para_styles.append(f'margin-left:{px}px')
            sb = fmt.space_before
            if sb is not None:
                px = _emu_to_px(sb)
                if px and px > 0:
                    para_styles.append(f'margin-top:{px}px')
            sa = fmt.space_after
            if sa is not None:
                px = _emu_to_px(sa)
                if px and px > 0:
                    para_styles.append(f'margin-bottom:{px}px')

            runs_html = ''
            for run in para.runs:
                text = escape(run.text)
                if not text:
                    continue
                rs = []
                if run.font.size:

```



```

        pt = run.font.size / 12700
        if pt > 0:
            rs.append(f'font-size:{pt:.0f}pt')
    try:
        rgb = run.font.color.rgb
        if rgb is not None:
            rs.append(f'color:#{str(rgb)}')
    except Exception:
        pass
    decorations = []
    if run.underline:
        decorations.append('underline')
    if run.font.strike:
        decorations.append('line-through')
    if decorations:
        rs.append(f'text-decoration:{"
".join(decorations)}')
    if rs:
        text = f'<span
style="{";".join(rs)}">{text}</span>'
        if run.bold and run.italic:
            text = f'<strong><em>{text}</em></strong>'
        elif run.bold:
            text = f'<strong>{text}</strong>'
        elif run.italic:
            text = f'<em>{text}</em>'
        runs_html += text

    if not runs_html.strip():
        return '<p class="dp-empty">&nbsp;</p>'
    tag = f'h{level}' if level else 'p'
    css = f'dh{level}' if level else 'dp'
    style_attr = f' style="{";".join(para_styles)}"' if
para_styles else ''
    return f'<{tag}
class="{css}"{style_attr}>{runs_html}</{tag}>'

def _process_table(table):
    rows_html = ''
    for row in table.rows:
        cells_html = ''
        for cell in row.cells:
            cell_inner = ''.join(_process_para(p) for p in
cell.paragraphs)
            cells_html += f'<td>{cell_inner}</td>'
        rows_html += f'<tr>{cells_html}</tr>'
    return f'<table class="dt">{rows_html}</table>'

doc = Document(file_path)
for child in doc.element.body:
    if child.tag == _qn('w:p'):
        html_parts.append(_process_para(_Paragraph(child,
doc)))
    elif child.tag == _qn('w:tbl'):
        html_parts.append(_process_table(_Table(child, doc)))
    else:
        html_parts.append('<p class="dp">Предпросмотр .doc не
поддерживается. Загрузите файл в формате .docx</p>')
    except Exception:
        html_parts.append('<p class="dp">Не удалось прочитать файл.
Убедитесь, что файл не повреждён.</p>')

return render(request, 'accounts/resume_word.html', {

```

```

        'applicant':          applicant,
        'educations':         applicant.educations.order_by('order'),
        'extra_educations':   applicant.extra_educations.order_by('order'),
        'experiences':         applicant.work_experiences.order_by('order'),
        'content_html':        '\n'.join(html_parts),
        'file_name':           os.path.basename(applicant.resume_file.name),
    })

# -----
#  Manager analytics
# -----

@login_required
def manager_analytics(request):
    """Aggregated analytics page for a manager: vacancy performance +
    application stats."""
    from django.db.models import Count, Q as DQ
    from vacancies.models import Vacancy, VacancyView
    if not hasattr(request.user, 'manager'):
        return redirect('vacancy-list')

    user = request.user

    vacancies = (
        Vacancy.objects
        .filter(created_by=user)
        .annotate(
            total_views=Count('views', distinct=True),
            total_apps=Count('applications', distinct=True),
            accepted=Count('applications',
filter=DQ(applications__status='accepted'), distinct=True),
            rejected=Count('applications',
filter=DQ(applications__status='rejected'), distinct=True),
            pending=Count('applications',
filter=DQ(applications__status='pending'), distinct=True),
        )
        .order_by('-created_at')
    )

    total_vacancies = vacancies.count()
    total_active     = sum(1 for v in vacancies if v.is_active)
    total_apps       = sum(v.total_apps   for v in vacancies)
    total_accepted   = sum(v.accepted     for v in vacancies)
    total_rejected    = sum(v.rejected     for v in vacancies)
    total_pending    = sum(v.pending      for v in vacancies)
    total_views      = sum(v.total_views  for v in vacancies)

    # Recent applications (past 30 days)
    from datetime import timedelta
    from django.utils import timezone
    since_30d = timezone.now() - timedelta(days=30)
    recent_apps = (
        Application.objects
        .filter(vacancy__created_by=user, created_at__gte=since_30d)
        .select_related('applicant', 'vacancy')
        .order_by('-created_at')[:20]
    )

    manager_obj = request.user.manager
    return render(request, 'accounts/manager_analytics.html', {
        'vacancies':          vacancies,
        'total_vacancies':    total_vacancies,
    })

```

```

        'total_active':    total_active,
        'total_apps':     total_apps,
        'total_accepted': total_accepted,
        'total_rejected': total_rejected,
        'total_pending':  total_pending,
        'total_views':    total_views,
        'recent_apps':    recent_apps,
        'status_choices': Application.STATUS_CHOICES,
        # Notification settings (for the Notifications tab)
        'mgr_consent_telegram': manager_obj.consent_telegram,
        'mgr_consent_email':    manager_obj.consent_email,
        'mgr_tg_linked':        bool(manager_obj.telegram_chat_id),
        'mgr_telegram':         manager_obj.telegram,
    })

# -----
# Moderator analytics
# -----

@moderator_required
def moderator_analytics(request):
    """Moderator workspace: reported vacancies queue + moderator
    productivity metrics."""
    from django.db.models import Count, Avg, F, Max, ExpressionWrapper,
    DurationField, Q
    from django.db.models.functions import TruncMonth
    from django.utils import timezone
    from vacancies.models import VacancyReport, VacancyModerationState

    report_qs = (
        VacancyReport.objects
        .select_related('vacancy', 'user', 'reviewed_by')
        .order_by('-created_at')
    )
    my_reports_qs = report_qs.filter(reviewed_by=request.user)

    pending_count =
    report_qs.filter(self_status=VacancyReport.SELF_STATUS_NEW).count()
    in_work_count =
    report_qs.filter(self_status=VacancyReport.SELF_STATUS_IN_WORK).count()
    done_count =
    my_reports_qs.filter(self_status=VacancyReport.SELF_STATUS_DONE).count()
    handled_count = my_reports_qs.count()

    avg_reaction =
    my_reports_qs.filter(reviewed_at__isnull=False).annotate(
        reaction_time=ExpressionWrapper(
            F('reviewed_at') - F('created_at'),
            output_field=DurationField(),
        )
    ).aggregate(value=Avg('reaction_time'))['value']

    try:
        avg_reaction_minutes = round(avg_reaction.total_seconds() / 60,
1) if avg_reaction else None
    except Exception:
        avg_reaction_minutes = None

    # --- Charts and benchmarks for the moderator workspace ---
    my_new_count =
    my_reports_qs.filter(self_status=VacancyReport.SELF_STATUS_NEW).count()
    my_in_work_count =

```

```

my_reports_qs.filter(self_status=VacancyReport.SELF_STATUS_IN_WORK).count
()

my_done_count = done_count

reason_labels_map = dict(VacancyReport.REASON_CHOICES)
reason_rows =
report_qs.values('reason_code').annotate(total=Count('id')).order_by('-
total')
reasons_chart = {
    'labels': [reason_labels_map.get(row['reason_code'],
row['reason_code']) for row in reason_rows],
    'values': [row['total'] for row in reason_rows],
}

month_rows = (
    my_reports_qs.filter(reviewed_at__isnull=False)
        .annotate(month=TruncMonth('reviewed_at'))
        .values('month')
        .annotate(total=Count('id'))
        .order_by('month')
)
month_map = {}
for row in month_rows:
    month_val = row.get('month')
    if not month_val:
        continue
    key = month_val.strftime('%Y-%m')
    month_map[key] = row['total']
month_series = []
today = timezone.now().date().replace(day=1)
for idx in range(5, -1, -1):
    m = (today.month - idx - 1) % 12 + 1
    y = today.year + ((today.month - idx - 1) // 12)
    key = f'{y:04d}-{m:02d}'
    month_series.append({
        'label': f'{m:02d}.{str(y)[2:]}',
        'value': month_map.get(key, 0),
    })

per_moderator = (
    report_qs.filter(reviewed_by__moderator_profile__isnull=False)
        .values('reviewed_by_id', 'reviewed_by__username',
'reviewed_by__first_name', 'reviewed_by__last_name')
        .annotate(
            handled_total=Count('id'),
            done_total=Count('id',
filter=Q(self_status=VacancyReport.SELF_STATUS_DONE)),
            avg_reaction=Avg(
                ExpressionWrapper(F('reviewed_at') - F('created_at'),
output_field=DurationField()),
                filter=Q(reviewed_at__isnull=False),
            ),
        )
        .order_by('-handled_total', 'reviewed_by__username')
)
moderators_total = max(1, Moderator.objects.count())
peers = []
for row in per_moderator:
    full_name = f"{{(row.get('reviewed_by__first_name') or
'').strip()}} {{(row.get('reviewed_by__last_name') or '').strip()}}".strip()
    label = full_name or row.get('reviewed_by__username') or
f"moderator-{{row.get('reviewed_by_id')}}"
    avg_delta = row.get('avg_reaction')

```

```

        peers.append({
            'id': row.get('reviewed_by_id'),
            'label': label,
            'handled_total': row.get('handled_total') or 0,
            'done_total': row.get('done_total') or 0,
            'avg_reaction_minutes': round(avg_delta.total_seconds() / 60,
1) if avg_delta else None,
        })

    team_avg_handled = round(sum(p['handled_total'] for p in peers) /
len(peers), 2) if peers else 0
    peer_reaction_values = [p['avg_reaction_minutes'] for p in peers if
p['avg_reaction_minutes'] is not None]
    team_avg_reaction = round(sum(peer_reaction_values) /
len(peer_reaction_values), 1) if peer_reaction_values else None

    rank_by_handled = 1
    for idx, peer in enumerate(peers, start=1):
        if peer['id'] == request.user.id:
            rank_by_handled = idx
            break
    percentile = round((moderators_total - rank_by_handled + 1) /
moderators_total * 100)

    compare = {
        'team_avg_handled': team_avg_handled,
        'my_handled': handled_count,
        'handled_delta_pct': round(((handled_count - team_avg_handled) /
team_avg_handled) * 100, 1) if team_avg_handled else 0.0,
        'team_avg_reaction_minutes': team_avg_reaction,
        'my_avg_reaction_minutes': avg_reaction_minutes,
        'reaction_delta_pct': (
            round(((team_avg_reaction - avg_reaction_minutes) /
team_avg_reaction) * 100, 1)
            if (team_avg_reaction and avg_reaction_minutes is not None)
            else 0.0
        ),
        'rank': rank_by_handled,
        'total_moderators': moderators_total,
        'percentile': percentile,
    }

    top_peers = peers[:6]
    peer_chart = {
        'labels': [p['label'] for p in top_peers],
        'values': [p['handled_total'] for p in top_peers],
        'current_user_id': request.user.id,
        'ids': [p['id'] for p in top_peers],
    }
    metrics_payload = {
        'status': {
            'labels': ['Новые', 'В работе', 'Обработано'],
            'values': [my_new_count, my_in_work_count, my_done_count],
        },
        'reasons': reasons_chart,
        'history': {
            'labels': [item['label'] for item in month_series],
            'values': [item['value'] for item in month_series],
        },
        'peer': peer_chart,
        'compare': compare,
    }

```

```

        selected_review_status = (request.GET.get('review_status') or
        '').strip()
        allowed_review_status = {s[0] for s in
VacancyModerationState.STATUS_CHOICES}
        if selected_review_status and selected_review_status not in
allowed_review_status:
            selected_review_status = ''

    grouped_rows = (
        report_qs.values('vacancy_id')
        .annotate(
            reports_total=Count('id'),
            last_report_at=Max('created_at'),
        )
        .order_by('-last_report_at')
    )
    all_vacancy_ids = [row['vacancy_id'] for row in grouped_rows]
    state_qs = VacancyModerationState.objects.filter(
        moderator=request.user,
        vacancy_id__in=all_vacancy_ids,
    )
    state_map = {state.vacancy_id: state for state in state_qs}

    filtered_rows = []
    for row in grouped_rows:
        state_obj = state_map.get(row['vacancy_id'])
        effective_status = (state_obj.status if state_obj else
VacancyModerationState.STATUS_NEW)
        if selected_review_status and effective_status !=
selected_review_status:
            continue
        filtered_rows.append(row)

    vacancy_paginator = Paginator(filtered_rows, 20)
    vacancy_page_obj =
vacancy_paginator.get_page(request.GET.get('vacancy_page') or '1')

    page_vacancy_ids = [row['vacancy_id'] for row in
vacancy_page_obj.object_list]
    page_reports = (
        VacancyReport.objects
        .select_related('vacancy', 'user', 'reviewed_by')
        .filter(vacancy_id__in=page_vacancy_ids)
        .order_by('-created_at')
    )
    reports_by_vacancy = {}
    for rep in page_reports:
        reports_by_vacancy.setdefault(rep.vacancy_id, []).append(rep)

    reason_display_map = dict(VacancyReport.REASON_CHOICES)
    moderation_status_map = dict(VacancyModerationState.STATUS_CHOICES)
    vacancy_cards = []
    for row in vacancy_page_obj.object_list:
        vacancy_id = row['vacancy_id']
        vacancy_reports = reports_by_vacancy.get(vacancy_id, [])
        if not vacancy_reports:
            continue
        vacancy = vacancy_reports[0].vacancy

        reason_counts = {}
        for rep in vacancy_reports:
            reason_counts[rep.reason_code] =
reason_counts.get(rep.reason_code, 0) + 1

```

```

        dominant_reason = 'Неизвестно'
        if reason_counts:
            sorted_reasons = sorted(reason_counts.items(), key=lambda x:
x[1], reverse=True)
            if len(sorted_reasons) == 1 or sorted_reasons[0][1] >
sorted_reasons[1][1]:
                dominant_reason =
reason_display_map.get(sorted_reasons[0][0], sorted_reasons[0][0])

        latest_report = vacancy_reports[0]
        state_obj = state_map.get(vacancy_id)
        effective_status = state_obj.status if state_obj else
VacancyModerationState.STATUS_NEW
        summary_status_label =
moderation_status_map.get(effective_status, effective_status)
        summary_note = (state_obj.note or '').strip() if state_obj else
''

        compact_reports = []
        for rep in vacancy_reports:
            reporter = rep.user
            full_name = (reporter.get_full_name() or '').strip()
            reporter_name = full_name or reporter.username
            avatar_url = ''
            if hasattr(reporter, 'applicant') and
reporter.applicant.avatar:
                avatar_url = reporter.applicant.avatar.url
            elif hasattr(reporter, 'manager') and
reporter.manager.avatar:
                avatar_url = reporter.manager.avatar.url
            compact_reports.append({
                'id': rep.pk,
                'reporter_name': reporter_name,
                'created_at': rep.created_at,
                'reason_label': rep.get_reason_code_display(),
                'reason_text': rep.reason_text,
                'avatar_url': avatar_url,
                'avatar_letter': (reporter_name[:1] or '?').upper(),
            })

        vacancy_cards.append({
            'vacancy_id': vacancy.id,
            'vacancy_external_id': vacancy.external_id,
            'vacancy_title': vacancy.title,
            'vacancy_company': vacancy.company,
            'reports_total': row['reports_total'],
            'last_report_at': row['last_report_at'],
            'summary_status': effective_status,
            'summary_status_label': summary_status_label,
            'summary_note': summary_note,
            'dominant_reason': dominant_reason,
            'reports': compact_reports,
        })

    return render(request, 'accounts/moderator_analytics.html', {
        'vacancy_cards': vacancy_cards,
        'vacancy_page_obj': vacancy_page_obj,
        'pending_count': pending_count,
        'in_work_count': in_work_count,
        'done_count': done_count,
        'handled_count': handled_count,
        'avg_reaction_minutes': avg_reaction_minutes,
        'moderator_metrics_json': json.dumps(metrics_payload,

```

```

ensure_ascii=False),
    'peer_compare': compare,
    'status_choices': VacancyReport.SELF_STATUS_CHOICES,
    'review_status_choices': VacancyModerationState.STATUS_CHOICES,
    'selected_review_status': selected_review_status,
})

# -----
# Admin panel
# -----

@admin_required
def admin_panel(request):
    """Main page for system administrators."""
    from django.contrib.auth.models import User as _User
    from django.utils import timezone as _tz
    from vacancies.models import Vacancy, VacancyView, Bookmark,
    VacancyReport, VacancyModerationState

    from django.db.models import Q as _Q
    # Count users who currently have a Manager profile
    # OR applicants who have ever switched to manager (was_manager=True)
    total_managers = _User.objects.filter(
        _Q(manager__isnull=False) | _Q(applicant__was_manager=True)
    ).distinct().count()

    stats = {
        'total_users': _User.objects.count(),
        'total_applicants': Applicant.objects.count(),
        'total_managers': total_managers,
        'total_moderators': Moderator.objects.count(),
        'total_vacancies': Vacancy.objects.count(),
        'total_admins': Administrator.objects.count(),
    }

    # Build backup list for the admin panel.
    from pathlib import Path as _Path
    from datetime import datetime as _dt
    from django.conf import settings as _cfg
    _backup_dir = _Path(getattr(_cfg, 'BACKUP_DIR',
    _Path(_cfg.DATABASES['default']['NAME']).parent / 'backups'))
    backups = []
    if _backup_dir.exists():
        for _bf in sorted(_backup_dir.glob('db_backup_*.sqlite3'),
reverse=True):
            _ts = _dt.fromtimestamp(_bf.stat().st_mtime)
            backups.append({
                'filename': _bf.name,
                'size_kb': round(_bf.stat().st_size / 1024, 1),
                'created_str': _ts.strftime('%d.%m.%Y %H:%M:%S'),
            })

    try:
        log_page = int(request.GET.get('log_page', '1') or '1')
    except ValueError:
        log_page = 1

    action_label_map = {
        'backup_create': 'Создание бэкапа',
        'backup_restore': 'Восстановление бэкапа',
        'backup_delete': 'Удаление бэкапа',

```



```

        'switch_role': 'Смена роли',
        'register_applicant': 'Регистрация соискателя',
        'register_manager': 'Регистрация менеджера',
        'application_status_change': 'Изменение статуса отклика',
        'preset_create': 'Создание пресета',
        'preset_update': 'Обновление пресета',
        'preset_delete': 'Удаление пресета',
        'interview_schedule': 'Назначение собеседования',
        'interview_cancel': 'Отмена собеседования',
        'interview_reschedule': 'Перенос собеседования',
        'delete_account': 'Удаление аккаунта',
        'admin_user_delete': 'Удаление пользователя администратором',
        'admin_user_contacts_update': 'Обновление контактов
пользователя',
        'admin_user_role_change': 'Смена роли пользователя
администратором',
        'user_feedback_create': 'Новое предложение/критика от
пользователя',
        'admin_feedback_action': 'Действие администратора по
предложению',
        'user_document_create': 'Пользователь добавил документ',
        'user_document_delete': 'Пользователь удалил документ',
    }
    actor_label_map = {
        'system': 'Система',
        'admin': 'Администратор',
        'moderator': 'Модератор',
        'manager': 'Менеджер',
        'applicant': 'Соискатель',
        'user': 'Пользователь',
    }

    logs_qs = ApiActionLog.objects.select_related('user').order_by('-
created_at')
    logs_paginator = Paginator(logs_qs, 20)
    logs_page_obj = logs_paginator.get_page(log_page)
    audit_logs = []
    for item in logs_page_obj.object_list:
        username = 'system'
        if item.user:
            username = item.user.get_full_name() or item.user.username
        audit_logs.append({
            'id': item.pk,
            'created_at': item.created_at,
            'actor_role': item.actor_role,
            'actor_label': actor_label_map.get(item.actor_role,
'Пользователь'),
            'username': username,
            'method': item.method,
            'endpoint': item.endpoint,
            'action': item.action,
            'action_label': action_label_map.get(item.action,
item.action.replace('_', ' ')),
            'success': item.success,
            'status_code': item.status_code,
            'before_human': _humanize_payload(item.before_data),
            'after_human': _humanize_payload(item.after_data),
            'before_json': json.dumps(item.before_data or {}),
            ensure_ascii=False, indent=2),
            'after_json': json.dumps(item.after_data or {}),
            ensure_ascii=False, indent=2),
        })

```

```

# — User feedback + product metrics for "Предложения" tab

now = _tz.now()
week_ago = now - timedelta(days=7)
month_ago = now - timedelta(days=30)

feedback_qs = UserFeedback.objects.select_related('user',
'processed_by').order_by('-created_at')

def _feedback_payload(item):
    author = item.user.get_full_name() or item.user.username
    role = _admin_user_role(item.user)
    role_label = {
        'manager': 'Менеджер',
        'applicant': 'Соискатель',
        'user': 'Пользователь',
        'moderator': 'Модератор',
        'admin': 'Администратор',
    }.get(role, 'Пользователь')
    avatar_url = ''
    applicant_profile =
Applicant.objects.filter(user=item.user).first()
    manager_profile = Manager.objects.filter(user=item.user).first()
    if applicant_profile and applicant_profile.avatar:
        avatar_url = applicant_profile.avatar.url
    elif manager_profile and manager_profile.avatar:
        avatar_url = manager_profile.avatar.url
    fallback_letter = (author[:1] if author else
item.user.username[:1]).upper()
    return {
        'id': item.pk,
        'author': author,
        'username': item.user.username,
        'role_code': role,
        'role_label': role_label,
        'kind_code': item.kind,
        'kind': item.get_kind_display(),
        'status': item.status,
        'status_label': item.get_status_display(),
        'message': item.message,
        'created_at': item.created_at,
        'processed_at': item.processed_at,
        'processed_by': (item.processed_by.get_full_name() or
item.processed_by.username) if item.processed_by else '',
        'avatar_url': avatar_url,
        'fallback_letter': fallback_letter,
    }

feedback_active_items = [_feedback_payload(x) for x in
feedback_qs.filter(status=UserFeedback.STATUS_ACTIVE)[:160]]
feedback_archived_items = [_feedback_payload(x) for x in
feedback_qs.filter(status=UserFeedback.STATUS_ARCHIVED)[:160]]

views_total = VacancyView.objects.count()
views_week =
VacancyView.objects.filter(viewed_at__gte=week_ago).count()
views_month =
VacancyView.objects.filter(viewed_at__gte=month_ago).count()
visitors_unique =
VacancyView.objects.values('user_id').distinct().count()
bookmarks_total = Bookmark.objects.count()
applications_total = Application.objects.count()
reports_total = VacancyReport.objects.count()

```

```

feedback_total = feedback_qs.count()
feedback_week = feedback_qs.filter(created_at__gte=week_ago).count()
feedback_month =
feedback_qs.filter(created_at__gte=month_ago).count()
feedback_active_count =
feedback_qs.filter(status=UserFeedback.STATUS_ACTIVE).count()
feedback_archived_count =
feedback_qs.filter(status=UserFeedback.STATUS_ARCHIVED).count()
feedback_resolved_count =
feedback_qs.filter(status=UserFeedback.STATUS_RESOLVED).count()
feedback_rejected_count =
feedback_qs.filter(status=UserFeedback.STATUS_REJECTED).count()
local_vacancies_total =
Vacancy.objects.filter(created_by__isnull=False).count()
local_vacancies_active =
Vacancy.objects.filter(created_by__isnull=False,
is_moderator_deleted=False, is_active=True).count()
moderation_cards_total = VacancyModerationState.objects.count()
avg_views_per_visitor = round(views_total / max(visitors_unique, 1),
1)

presets = list(FilterPreset.objects.values_list('filters',
flat=True))
keyword_counter = {}
region_counter = {}
experience_counter = {}
schedule_counter = {}
employment_counter = {}
remote_only_count = 0
for filters in presets:
    if not isinstance(filters, dict):
        continue
    q_raw = str(filters.get('q') or '').strip().lower()
    if q_raw:
        for token in re.split(r'^a-zA-Za-яA-Я0-9+', q_raw):
            token = token.strip()
            if len(token) < 2:
                continue
            keyword_counter[token] = keyword_counter.get(token, 0) +
1
    region = str(filters.get('region') or '').strip()
    if region:
        region_counter[region] = region_counter.get(region, 0) + 1
    for exp in (filters.get('experience') or []):
        exp = str(exp).strip()
        if exp:
            experience_counter[exp] = experience_counter.get(exp, 0)
+ 1
    for sch in (filters.get('schedule') or []):
        sch = str(sch).strip()
        if sch:
            schedule_counter[sch] = schedule_counter.get(sch, 0) + 1
    for emp in (filters.get('employment') or []):
        emp = str(emp).strip()
        if emp:
            employment_counter[emp] = employment_counter.get(emp, 0)
+ 1
    if bool(filters.get('remote_only')):
        remote_only_count += 1

def _top_label(counter_dict):
    if not counter_dict:
        return '-', 0

```

```

        label, count = max(counter_dict.items(), key=lambda kv: kv[1])
        return label, count

    top_keyword, top_keyword_count = _top_label(keyword_counter)
    top_region, top_region_count = _top_label(region_counter)
    top_experience, top_experience_count = _top_label(experience_counter)
    top_schedule, top_schedule_count = _top_label(schedule_counter)
    top_employment, top_employment_count = _top_label(employment_counter)

    experience_label_map = {
        'noExperience': 'Без опыта',
        'between1And3': '1-3 года',
        'between3And6': '3-6 лет',
        'moreThan6': 'Более 6 лет',
    }
    schedule_label_map = {
        'fullDay': 'Полный день',
        'shift': 'Сменный график',
        'flexible': 'Гибкий график',
        'remote': 'Удаленная работа',
        'flyInFlyOut': 'Вахтовый метод',
    }
    employment_label_map = {
        'full': 'Полная занятость',
        'part': 'Частичная занятость',
        'project': 'Проектная работа',
        'volunteer': 'Волонтерство',
        'probation': 'Стажировка',
    }

    top_experience = experience_label_map.get(top_experience,
top_experience)
    top_schedule = schedule_label_map.get(top_schedule, top_schedule)
    top_employment = employment_label_map.get(top_employment,
top_employment)
    total_presets = max(len(presets), 1)
    remote_only_share = round((remote_only_count / total_presets) * 100)

    suggestion_metric_groups = [
        {
            'title': 'Поведение пользователей',
            'metrics': [
                {'title': 'Заходов на сайт', 'value': views_total,
'hint': 'Все просмотры вакансий'},
                {'title': 'Уникальных посетителей', 'value':
visitors_unique, 'hint': 'Пользователи с хотя бы 1 просмотром'},
                {'title': 'Заходов за 7 дней', 'value': views_week,
'hint': 'Текущая недельная активность'},
                {'title': 'Заходов за 30 дней', 'value': views_month,
'hint': 'Месячная динамика посещений'},
                {'title': 'Среднее просмотров / посетитель', 'value':
avg_views_per_visitor, 'hint': 'Глубина взаимодействия'},
                {'title': 'Закладок', 'value': bookmarks_total, 'hint':
'Сохранённые вакансии'},
                {'title': 'Откликов', 'value': applications_total,
'hint': 'Поданные заявки'},
            ],
        },
        {
            'title': 'Поиск и фильтры',
            'metrics': [
                {'title': 'Топ поисковое слово', 'value': top_keyword,
'hint': f'Использований: {top_keyword_count}'},
                {'title': 'Топ регион фильтра', 'value': top_region,

```

```

'hint': f'Выбран: {top_region_count} раз'},
    {'title': 'Часто выбираемый опыт', 'value':
top_experience, 'hint': f'Выбран: {top_experience_count} раз'},
    {'title': 'Частый график', 'value': top_schedule, 'hint':
f'Выбран: {top_schedule_count} раз'},
    {'title': 'Частая занятость', 'value': top_employment,
'hint': f'Выбрана: {top_employment_count} раз'},
    {'title': 'Доля remote-only фильтров', 'value':
f'{remote_only_share}%', 'hint': f'{remote_only_count} из {len(presets)}
пресетов'},
    ],
    },
    {
        'title': 'Модерация и обратная связь',
        'metrics': [
            {'title': 'Жалоб на вакансии', 'value': reports_total,
'hint': 'Все пользовательские жалобы'},
            {'title': 'Карточек модерации', 'value':
moderation_cards_total, 'hint': 'Состояния работы модератора'},
            {'title': 'Локальных вакансий', 'value':
local_vacancies_total, 'hint': f'Активных: {local_vacancies_active}'},
            {'title': 'Предложений за 7 дней', 'value':
feedback_week, 'hint': f'За 30 дней: {feedback_month}'},
            {'title': 'Всего предложений', 'value': feedback_total,
'hint': 'Сообщения из новой вкладки'},
            {'title': 'Реализовано', 'value':
feedback_resolved_count, 'hint': 'Сообщения, отмеченные как
реализованные'},
            {'title': 'В архиве', 'value': feedback_archived_count,
'hint': 'Отложенные сообщения администратора'},
            {'title': 'Отклонено', 'value': feedback_rejected_count,
'hint': 'Сообщения, удаленные из ленты'},
            {'title': 'Активная лента', 'value':
feedback_active_count, 'hint': 'Сейчас в работе'},
        ],
    },
]

def _build_conic(segments):
    total = sum(seg['value'] for seg in segments) or 1
    cursor = 0.0
    parts = []
    for seg in segments:
        pct = round((seg['value'] / total) * 100, 1)
        seg['pct'] = pct
        seg['legend_bg'] = seg['color']
        start = cursor
        end = cursor + (seg['value'] / total) * 100
        parts.append(f"{seg['color']} {start:.2f}% {end:.2f}%")
        cursor = end
    conic = ", ".join(parts) if parts else "#d4d4d4 0% 100%"
    return conic, total

def _build_gradient_conic(segments):
    total = sum(seg['value'] for seg in segments) or 1
    cursor = 0.0
    parts = []
    for seg in segments:
        pct = round((seg['value'] / total) * 100, 1)
        seg['pct'] = pct
        seg['legend_bg'] = f"linear-gradient(135deg,
{seg['color_from']} 0%, {seg['color_to']} 100%)"
        start = cursor

```

```

        end = cursor + (seg['value'] / total) * 100
        # Two stops per segment -> smooth gradient inside one sector,
no split wedges.
        parts.append(f"{seg['color_from']} {start:.2f}%")
        parts.append(f"{seg['color_to']} {end:.2f}%")
        cursor = end
    conic = ", ".join(parts) if parts else "#d4d4d4 0% 100%"
    return conic, total

    pie_segments = [
        {'label': 'Просмотры', 'value': views_total, 'color_from':
'#8ab6f5', 'color_to': '#4f8ed8'},
        {'label': 'Закладки', 'value': bookmarks_total, 'color_from':
'#88d598', 'color_to': '#4fa966'},
        {'label': 'Отклики', 'value': applications_total, 'color_from':
'#f9c25d', 'color_to': '#e59410'},
        {'label': 'Жалобы', 'value': reports_total, 'color_from':
'#ef7a7a', 'color_to': '#c93e3e'},
        {'label': 'Предложения', 'value': feedback_total, 'color_from':
'#b586d3', 'color_to': '#8654ac'},
    ]
    pie_conic, suggestion_pie_total = _build_gradient_conic(pie_segments)

    feedback_status_segments = [
        {'label': 'В ленте', 'value': feedback_active_count,
'color_from': '#7cb4f4', 'color_to': '#4f8ed8'},
        {'label': 'В архиве', 'value': feedback_archived_count,
'color_from': '#f9c467', 'color_to': '#e89814'},
        {'label': 'Реализовано', 'value': feedback_resolved_count,
'color_from': '#67bd7f', 'color_to': '#2f7a45'},
        {'label': 'Отклонено', 'value': feedback_rejected_count,
'color_from': '#f07f7f', 'color_to': '#cb4343'},
    ]
    feedback_status_conic, feedback_status_total =
_build_gradient_conic(feedback_status_segments)

    return render(request, 'accounts/admin_panel.html', {
        'admin_profile': request.user.admin_profile,
        'stats': stats,
        'backups': backups,
        'audit_logs': audit_logs,
        'logs_page_obj': logs_page_obj,
        'feedback_active_items': feedback_active_items,
        'feedback_archived_items': feedback_archived_items,
        'feedback_resolved_count': feedback_resolved_count,
        'suggestion_metric_groups': suggestion_metric_groups,
        'suggestion_pie_segments': pie_segments,
        'suggestion_pie_conic': pie_conic,
        'suggestion_pie_total': suggestion_pie_total,
        'feedback_status_segments': feedback_status_segments,
        'feedback_status_conic': feedback_status_conic,
        'feedback_status_total': feedback_status_total,
    })

@swagger_auto_schema(method='post', operation_summary="Действие над
предложением/критикой (архив/отклонить/реализовать/вернуть)",
tags=['admin'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@admin_required
def api_admin_feedback_action(request, feedback_id):
    """Admin workflow for feedback messages:

```

```

archive/reject/resolve/restore."""
    from django.utils import timezone as _tz

    if request.method != 'POST':
        return JsonResponse({'error': 'method_not_allowed', 'detail':
            'Неверный метод запроса.'}, status=405)

    item = get_object_or_404(UserFeedback, pk=feedback_id)
    action = str((request.POST.get('action') or request.GET.get('action')
    or '')).strip().lower()

    action_map = {
        'archive': UserFeedback.STATUS_ARCHIVED, # реализовать потом
        'reject': UserFeedback.STATUS_REJECTED, # отклонить
        'resolve': UserFeedback.STATUS_RESOLVED, # реализовано
        'restore': UserFeedback.STATUS_ACTIVE, # вернуть в ленту из
архива
    }
    if action not in action_map:
        return JsonResponse({'error': 'invalid_action', 'detail':
            'Неизвестное действие.'}, status=400)

    old_status = item.status
    new_status = action_map[action]
    if old_status == new_status:
        return JsonResponse({'ok': True, 'status': new_status,
            'status_label': item.get_status_display()})

    item.status = new_status
    item.processed_by = request.user
    item.processed_at = _tz.now()
    item.save(update_fields=['status', 'processed_by', 'processed_at'])

    _log_api_action(
        request,
        action='admin_feedback_action',
        before={'feedback_id': item.pk, 'status': old_status},
        after={'feedback_id': item.pk, 'status': new_status, 'admin':
request.user.username},
        endpoint='accounts.api_admin_feedback_action',
        status_code=200,
    )

    return JsonResponse({
        'ok': True,
        'status': new_status,
        'status_label': item.get_status_display(),
    })

def _admin_user_role(user_obj):
    if is_admin_user(user_obj):
        return 'admin'
    if is_moderator_user(user_obj):
        return 'moderator'
    if hasattr(user_obj, 'manager'):
        return 'manager'
    if hasattr(user_obj, 'applicant'):
        return 'applicant'
    return 'user'

def _admin_set_user_role(target_user, new_role):

```

```

    """Set user role from admin panel while preserving contact data where
possible."""
    if new_role not in ('admin', 'moderator', 'manager', 'applicant'):
        return False, 'invalid_role'

    applicant = Applicant.objects.filter(user=target_user).first()
    manager = Manager.objects.filter(user=target_user).first()
    moderator = Moderator.objects.filter(user=target_user).first()

    if new_role == 'admin':
        if not target_user.is_superuser:
            target_user.is_superuser = True
            target_user.save(update_fields=['is_superuser'])
            Administrator.objects.get_or_create(user=target_user)
            Moderator.objects.filter(user=target_user).delete()
            Manager.objects.filter(user=target_user).delete()
            Applicant.objects.filter(user=target_user).delete()
            return True, 'role_updated'

    # Demote from admin when switching to business roles.
    if target_user.is_superuser:
        target_user.is_superuser = False
        target_user.save(update_fields=['is_superuser'])
        Administrator.objects.filter(user=target_user).delete()

    if new_role == 'moderator':
        if moderator:
            return True, 'role_updated'
        moderator = Moderator(user=target_user)
        if manager:
            moderator.patronymic = manager.patronymic
            moderator.phone = manager.phone
            moderator.telegram = manager.telegram
        elif applicant:
            moderator.patronymic = applicant.patronymic
            moderator.phone = applicant.phone
            moderator.telegram = applicant.telegram
        moderator.save()
        Manager.objects.filter(user=target_user).delete()
        Applicant.objects.filter(user=target_user).delete()
        return True, 'role_updated'

    Moderator.objects.filter(user=target_user).delete()

    if new_role == 'manager':
        if manager:
            return True, 'role_updated'
        manager = Manager(user=target_user)
        if applicant:
            manager.patronymic = applicant.patronymic
            manager.phone = applicant.phone
            manager.telegram = applicant.telegram
            if applicant.avatar:
                manager.avatar = applicant.avatar.name
            if applicant.manager_company_logo:
                manager.company_logo = applicant.manager_company_logo
            if not applicant.was_manager:
                applicant.was_manager = True
                applicant.save(update_fields=['was_manager'])
        elif moderator:
            manager.patronymic = moderator.patronymic
            manager.phone = moderator.phone
            manager.telegram = moderator.telegram

```



```

        manager.save()
        return True, 'role_updated'

# new_role == 'applicant'
applicant, _ = Applicant.objects.get_or_create(
    user=target_user,
    defaults={
        'telegram': (
            manager.telegram if manager and manager.telegram else (
                moderator.telegram if moderator and
moderator.telegram else f'@{target_user.username}'
            )
        ),
        'phone': (
            manager.phone if manager else (moderator.phone if
moderator else '')
        ),
        'patronymic': (
            manager.patronymic if manager else (moderator.patronymic
if moderator else '')
        ),
        'city': '',
    },
)
if manager:
    changed_fields = []
    if manager.avatar and not applicant.avatar:
        applicant.avatar = manager.avatar.name
        changed_fields.append('avatar')
    if manager.company_logo:
        applicant.manager_company_logo = manager.company_logo.name
        changed_fields.append('manager_company_logo')
    if changed_fields:
        applicant.save(update_fields=changed_fields)
    manager.delete()
return True, 'role_updated'

```

```

@admin_required
def admin_moderator_reports(request):
    """Admin tab: list of moderator vacancy-deletion reports."""
    from vacancies.models import ModeratorDeletionReport

    status_filter = (request.GET.get('status') or 'all').strip()
    qs = ModeratorDeletionReport.objects.select_related(
        'moderator', 'manager', 'vacancy', 'restored_by'
    ).prefetch_related('photos').order_by('-created_at')
    if status_filter == 'active':
        qs = qs.filter(is_restored=False)
    elif status_filter == 'restored':
        qs = qs.filter(is_restored=True)

    paginator = Paginator(qs, 20)
    page_obj = paginator.get_page(request.GET.get('page') or '1')

    return render(request, 'accounts/admin_moderator_reports.html', {
        'page_obj': page_obj,
        'status_filter': status_filter,
        'total_reports': ModeratorDeletionReport.objects.count(),
        'total_active':
ModeratorDeletionReport.objects.filter(is_restored=False).count(),
        'total_restored':
ModeratorDeletionReport.objects.filter(is_restored=True).count(),
    })

```

```

    })

@admin_required
def admin_moderator_report_pdf(request, report_id):
    """Generate a PDF document for a moderator deletion report."""
    from io import BytesIO
    from vacancies.models import ModeratorDeletionReport
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.units import mm
    from reportlab.lib.enums import TA_LEFT
    from reportlab.platypus import (
        SimpleDocTemplate, Paragraph, Spacer, Image, Table, TableStyle,
    )
    from reportlab.lib import colors
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont

    report = get_object_or_404(
        ModeratorDeletionReport.objects.prefetch_related('photos'),
        pk=report_id,
    )

    # Register a Unicode TTF font so the PDF can render Cyrillic
    properly.
    # Windows ships DejaVuSans via matplotlib, but safe-fallback: try a
    system font.
    font_name = 'DejaVuSans'
    try:
        candidates = [
            os.path.join(os.environ.get('WINDIR', 'C:/Windows'), 'Fonts',
            'arial.ttf'),
            os.path.join(os.environ.get('WINDIR', 'C:/Windows'), 'Fonts',
            'ARIAL.TTF'),
            '/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf',
        ]
        font_path = next((p for p in candidates if os.path.exists(p)),
        None)
        if font_path:
            pdfmetrics.registerFont(TTFont(font_name, font_path))
        else:
            font_name = 'Helvetica'
    except Exception:
        font_name = 'Helvetica'

    buf = BytesIO()
    doc = SimpleDocTemplate(
        buf, pagesize=A4,
        leftMargin=18*mm, rightMargin=18*mm,
        topMargin=16*mm, bottomMargin=16*mm,
        title=f'Moderator report #{report.pk}',
    )

    base = getSampleStyleSheet()['Normal']
    p_title = ParagraphStyle('t', parent=base, fontName=font_name,
    fontSize=16, leading=20, spaceAfter=8)
    p_h2 = ParagraphStyle('h2', parent=base, fontName=font_name,
    fontSize=13, leading=16, textColor=colors.HexColor('#1f3b70'),
    spaceBefore=10, spaceAfter=4)
    p_body = ParagraphStyle('b', parent=base, fontName=font_name,
    fontSize=10, leading=14, alignment=TA_LEFT)
    p_small = ParagraphStyle('s', parent=base, fontName=font_name,

```

```

fontSize=9, leading=12, textColor=colors.HexColor('#555555'))

    story = []
    story.append(Paragraph(f'Отчёт №{report.pk} – удаление вакансии',
p_title))
    story.append(Paragraph(
        f'Создан: {report.created_at.strftime("%d.%m.%Y %H:%M")}'
        + (f' · Восстановлен: {report.restored_at.strftime("%d.%m.%Y %H:%M")}' if report.is_restored and report.restored_at else ''),
        p_small,
    ))

    story.append(Paragraph('Вакансия', p_h2))
    vac_rows = [
        ['Название', report.vacancy_title or '-'],
        ['Компания', report.vacancy_company or '-'],
        ['ID', report.vacancy_external_id or '-'],
        ['Жалоб от пользователей', str(report.reports_count)],
        ['Доминирующий тег', report.dominant_reason_label or '-'],
    ]
    table_style = TableStyle([
        ('BACKGROUND', (0, 0), (0, -1), colors.HexColor('#f2f4f8')),
        ('BOX', (0, 0), (-1, -1), 0.5, colors.HexColor('#c9d1e2')),
        ('INNERGRID', (0, 0), (-1, -1), 0.25,
colors.HexColor('#dee3ed')),
        ('VALIGN', (0, 0), (-1, -1), 'TOP'),
        ('LEFTPADDING', (0, 0), (-1, -1), 6),
        ('RIGHTPADDING', (0, 0), (-1, -1), 6),
        ('TOPPADDING', (0, 0), (-1, -1), 4),
        ('BOTTOMPADDING', (0, 0), (-1, -1), 4),
    ])
    t = Table(
        [[Paragraph(k, p_body), Paragraph(str(v), p_body)] for k, v in
vac_rows],
        colWidths=[55*mm, 115*mm],
    )
    t.setStyle(table_style)
    story.append(t)

    if report.vacancy_description:
        story.append(Paragraph('Описание вакансии', p_h2))
        # strip HTML tags for safety
        text = re.sub(r'<[^>+>', ' ', report.vacancy_description or '')
        text = re.sub(r'\\s+', ' ', text).strip()
        if len(text) > 4000:
            text = text[:4000] + '...'
        story.append(Paragraph(text or '-', p_body))

    story.append(Paragraph('Менеджер вакансии', p_h2))
    mgr_rows = [
        ['ФИО', report.manager_full_name or '-'],
        ['Email', report.manager_email or '-'],
    ]
    t2 = Table([[Paragraph(k, p_body), Paragraph(str(v), p_body)] for k,
v in mgr_rows], colWidths=[55*mm, 115*mm])
    t2.setStyle(table_style)
    story.append(t2)

    story.append(Paragraph('Удаление', p_h2))
    del_rows = [
        ['Модератор', report.moderator_full_name or '-'],
        ['Email', report.moderator_email or '-'],
        ['Дата удаления', report.created_at.strftime('%d.%m.%Y %H:%M')],
    ]

```

```

        ['Статус',          'Восстановлено' if report.is_restored else
'Активно'],
        ['Причина',        report.reason or '-'],
    ]
    t3 = Table([[Paragraph(k, p_body), Paragraph(str(v).replace('\n',
'<br/>'), p_body)] for k, v in del_rows], colWidths=[55*mm, 115*mm])
    t3.setStyle(table_style)
    story.append(t3)

    photos = list(report.photos.all())
    if photos:
        story.append(Paragraph('Фото-доказательства', p_h2))
        row = []
        for idx, ph in enumerate(photos):
            try:
                img = Image(ph.image.path, width=80*mm, height=55*mm,
kind='proportional')
                row.append(img)
            except Exception:
                continue
            if len(row) == 2:
                story.append(Table([row], colWidths=[85*mm, 85*mm]))
                story.append(Spacer(1, 4*mm))
                row = []
        if row:
            story.append(Table([row + [''] * (2 - len(row))],
colWidths=[85*mm, 85*mm]))
        else:
            story.append(Paragraph('Фото-доказательства не предоставлены.',
p_small))

    doc.build(story)

    pdf_bytes = buf.getvalue()
    filename = f'moderator-report-{report.pk}.pdf'
    resp = HttpResponse(pdf_bytes, content_type='application/pdf')
    resp['Content-Disposition'] = f'inline; filename="{filename}"'
    return resp

@admin_required
def admin_users(request):
    """Admin page to manage users: list/paginate/edit
contacts/delete/switch role."""
    q = (request.GET.get('q') or '').strip()
    role_filter = (request.GET.get('role') or '').strip().lower()
    allowed_role_filters = {'', 'admin', 'moderator', 'manager',
'applicant', 'user'}
    if role_filter not in allowed_role_filters:
        role_filter = ''
    page_num = request.GET.get('page') or '1'
    status = (request.GET.get('status') or '').strip()
    status_text_map = {
        'saved': 'Данные пользователя сохранены.',
        'deleted': 'Пользователь удален.',
        'role_changed': 'Роль пользователя изменена.',
        'moderator_created': 'Модератор успешно создан.',
        'invalid_user': 'Некорректный идентификатор пользователя.',
        'not_found': 'Пользователь не найден.',
        'cannot_delete_self': 'Нельзя удалить собственный аккаунт.',
        'email_exists': 'Пользователь с таким email уже существует.',
        'username_exists': 'Пользователь с таким логином уже
существует.',

```

```

        'missing_fields': 'Заполните обязательные поля для создания
модератора.',
        'telegram_exists': 'Пользователь с таким Telegram уже
существует.',
        'invalid_telegram': 'Telegram должен начинаться с @.',
        'invalid_role': 'Недопустимая роль.',
        'unknown_action': 'Неизвестное действие.',
    }
    status_text = status_text_map.get(status, '')

    def _redirect_with_status(code):
        params = {'status': code}
        if q:
            params['q'] = q
        if role_filter:
            params['role'] = role_filter
        if page_num:
            params['page'] = page_num
        return redirect(reverse('accounts:admin_users') + '?' +
urlencode(params))

    if request.method == 'POST':
        action = (request.POST.get('action') or '').strip()

        if action == 'create_moderator':
            username = (request.POST.get('new_moderator_username') or
            '').strip()
            email = (request.POST.get('new_moderator_email') or
            '').strip().lower()
            password = (request.POST.get('new_moderator_password') or
            '').strip()
            first_name = (request.POST.get('new_moderator_first_name') or
            '').strip()
            last_name = (request.POST.get('new_moderator_last_name') or
            '').strip()
            patronymic = (request.POST.get('new_moderator_patronymic') or
            '').strip()
            phone = (request.POST.get('new_moderator_phone') or
            '').strip()
            telegram = ''

            if not username or not email or not password:
                return _redirect_with_status('missing_fields')
            if User.objects.filter(username__iexact=username).exists():
                return _redirect_with_status('username_exists')
            if User.objects.filter(email__iexact=email).exists():
                return _redirect_with_status('email_exists')
            if telegram and not telegram.startswith('@'):
                return _redirect_with_status('invalid_telegram')
            if telegram and (
Applicant.objects.filter(telegram__iexact=telegram).exists()
or
Manager.objects.filter(telegram__iexact=telegram).exists()
or
Moderator.objects.filter(telegram__iexact=telegram).exists()
):
                return _redirect_with_status('telegram_exists')

            new_user = User.objects.create_user(
                username=username,
                email=email,
                password=password,

```

```

        first_name=first_name,
        last_name=last_name,
    )
    Moderator.objects.create(
        user=new_user,
        patronymic=patronymic,
        phone=phone,
        telegram=telegram,
    )
    _log_api_action(
        request,
        action='admin_user_role_change',
        before={'create': 'moderator', 'username': username},
        after={'user_id': new_user.pk, 'role': 'moderator'},
        success=True,
        status_code=200,
        endpoint='admin_users',
    )
    return _redirect_with_status('moderator_created')

try:
    user_id = int(request.POST.get('user_id') or '0')
except ValueError:
    return _redirect_with_status('invalid_user')
target_user = User.objects.filter(pk=user_id).first()
if not target_user:
    return _redirect_with_status('not_found')

if action == 'delete':
    if target_user.pk == request.user.pk:
        return _redirect_with_status('cannot_delete_self')
    before = {
        'user_id': target_user.pk,
        'username': target_user.username,
        'role': _admin_user_role(target_user),
    }
    Applicant.objects.filter(user=target_user).delete()
    Manager.objects.filter(user=target_user).delete()
    Administrator.objects.filter(user=target_user).delete()
    Moderator.objects.filter(user=target_user).delete()
    target_user.delete()
    _log_api_action(
        request,
        action='admin_user_delete',
        before=before,
        after={'deleted': True},
        success=True,
        status_code=200,
        endpoint='admin_users',
    )
    return _redirect_with_status('deleted')

if action == 'save':
    first_name = (request.POST.get('first_name') or '').strip()
    last_name = (request.POST.get('last_name') or '').strip()
    email = (request.POST.get('email') or '').strip().lower()
    patronymic = (request.POST.get('patronymic') or '').strip()
    phone = (request.POST.get('phone') or '').strip()
    telegram = (request.POST.get('telegram') or '').strip()
    city = (request.POST.get('city') or '').strip()
    company = (request.POST.get('company') or '').strip()

    if email and

```

```

User.objects.filter(email__iexact=email).exclude(pk=target_user.pk).exists():
    return _redirect_with_status('email_exists')
if telegram and not telegram.startswith('@'):
    return _redirect_with_status('invalid_telegram')
if telegram and (

Applicant.objects.filter(telegram__iexact=telegram).exclude(user=target_user).exists()
    or
Manager.objects.filter(telegram__iexact=telegram).exclude(user=target_user).exists()
):
    return _redirect_with_status('telegram_exists')

before = {
    'user_id': target_user.pk,
    'first_name': target_user.first_name,
    'last_name': target_user.last_name,
    'email': target_user.email,
}

target_user.first_name = first_name
target_user.last_name = last_name
if email:
    target_user.email = email
target_user.save(update_fields=['first_name', 'last_name',
'email'])

applicant =
Applicant.objects.filter(user=target_user).first()
manager = Manager.objects.filter(user=target_user).first()
moderator =
Moderator.objects.filter(user=target_user).first()

if applicant:
    app_updates = []
    applicant.patronymic = patronymic
    app_updates.append('patronymic')
    applicant.phone = phone
    app_updates.append('phone')
    if telegram:
        applicant.telegram = telegram
        app_updates.append('telegram')
    applicant.city = city
    app_updates.append('city')
    applicant.save(update_fields=app_updates)

# For non-admin users, allow editing manager fields in the
same form
# even if they did not have a manager profile yet.
if (not is_admin_user(target_user)) and (not
is_moderator_user(target_user)) and not manager:
    manager = Manager.objects.create(
        user=target_user,
        patronymic=patronymic,
        phone=phone,
        telegram=telegram or '',
        company=company,
    )

if manager:
    mgr_updates = []

```

```

        manager.patronymic = patronymic
        mgr_updates.append('patronymic')
        manager.phone = phone
        mgr_updates.append('phone')
        manager.telegram = telegram or manager.telegram
        mgr_updates.append('telegram')
        manager.company = company
        mgr_updates.append('company')
        manager.save(update_fields=mgr_updates)

    if moderator:
        mod_updates = []
        moderator.patronymic = patronymic
        mod_updates.append('patronymic')
        moderator.phone = phone
        mod_updates.append('phone')
        moderator.telegram = telegram or moderator.telegram
        mod_updates.append('telegram')
        moderator.save(update_fields=mod_updates)

    _log_api_action(
        request,
        action='admin_user_contacts_update',
        before=before,
        after={
            'user_id': target_user.pk,
            'first_name': target_user.first_name,
            'last_name': target_user.last_name,
            'email': target_user.email,
            'phone': phone,
            'telegram': telegram,
            'city': city,
            'company': company,
        },
        success=True,
        status_code=200,
        endpoint='admin_users',
    )
    return _redirect_with_status('saved')

    if action == 'change_role':
        new_role = (request.POST.get('new_role') or '').strip()
        old_role = _admin_user_role(target_user)
        ok, code = _admin_set_user_role(target_user, new_role)
        _log_api_action(
            request,
            action='admin_user_role_change',
            before={'user_id': target_user.pk, 'from_role':
old_role},
            after={'to_role': new_role, 'result': code},
            success=ok,
            status_code=200 if ok else 400,
            endpoint='admin_users',
        )
        return _redirect_with_status('role_changed' if ok else code)

    return _redirect_with_status('unknown_action')

    users_qs = User.objects.select_related('applicant', 'manager',
'moderator_profile', 'admin_profile').order_by('-date_joined')
    if q:
        users_qs = users_qs.filter(
            Q(username__icontains=q)

```



```

        | Q(email__icontains=q)
        | Q(first_name__icontains=q)
        | Q(last_name__icontains=q)
        | Q(applicant__phone__icontains=q)
        | Q(manager__phone__icontains=q)
        | Q(applicant__telegram__icontains=q)
        | Q(manager__telegram__icontains=q)
        | Q(moderator_profile__telegram__icontains=q)
    ).distinct()
    if role_filter == 'admin':
        users_qs = users_qs.filter(Q(is_superuser=True) |
Q(admin_profile__isnull=False))
    elif role_filter == 'moderator':
        users_qs = users_qs.filter(moderator_profile__isnull=False,
admin_profile__isnull=True, is_superuser=False)
    elif role_filter == 'manager':
        users_qs = users_qs.filter(
            manager__isnull=False,
            admin_profile__isnull=True,
            moderator_profile__isnull=True,
            is_superuser=False,
        )
    elif role_filter == 'applicant':
        users_qs = users_qs.filter(
            applicant__isnull=False,
            manager__isnull=True,
            admin_profile__isnull=True,
            moderator_profile__isnull=True,
            is_superuser=False,
        )
    elif role_filter == 'user':
        users_qs = users_qs.filter(
            applicant__isnull=True,
            manager__isnull=True,
            admin_profile__isnull=True,
            moderator_profile__isnull=True,
            is_superuser=False,
        )

    paginator = Paginator(users_qs, 20)
    page_obj = paginator.get_page(page_num)

    rows = []
    for u in page_obj.object_list:
        applicant = getattr(u, 'applicant', None)
        manager = getattr(u, 'manager', None)
        moderator = getattr(u, 'moderator_profile', None)
        role = _admin_user_role(u)
        role_label = {
            'admin': 'Администратор',
            'moderator': 'Модератор',
            'manager': 'Менеджер',
            'applicant': 'Соискатель',
            'user': 'Пользователь',
        }.get(role, 'Пользователь')
        rows.append({
            'user': u,
            'role': role,
            'role_label': role_label,
            'first_name': u.first_name,
            'last_name': u.last_name,
            'email': u.email,
            'patronymic': (

```

```

        applicant.patronymic if applicant else (
            manager.patronymic if manager else
(moderator.patronymic if moderator else '')
        )
    ),
    'phone': (
        applicant.phone if applicant else (
            manager.phone if manager else (moderator.phone if
moderator else '')
        )
    ),
    'telegram': (
        applicant.telegram if applicant else (
            manager.telegram if manager else (moderator.telegram
if moderator else '')
        )
    ),
    'city': (applicant.city if applicant else ''),
    'company': (manager.company if manager else ''),
    'is_self': (u.pk == request.user.pk),
})

return render(request, 'accounts/admin_users.html', {
    'rows': rows,
    'page_obj': page_obj,
    'q': q,
    'role_filter': role_filter,
    'status': status,
    'status_text': status_text,
})

```

```

@admin_required
def admin_profile_page(request):
    """Admin profile page is intentionally disabled; keep admins in
control panel."""
    return redirect('accounts:admin_panel')

```

```

@swagger_auto_schema(method='post', operation_summary="Создать резервную
копию базы данных", tags=['admin'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@admin_required
def api_admin_backup_create(request):
    """POST – create an immediate database backup; return filename and
size."""
    if request.method != 'POST':
        _log_api_action(
            request,
            action='backup_create',
            after={'error': 'method_not_allowed'},
            success=False,
            status_code=405,
            endpoint='api_admin_backup_create',
        )
    return JsonResponse({'error': 'method_not_allowed'}, status=405)

import sqlite3 as _sqlite3
from contextlib import closing as _closing
from pathlib import Path as _Path
from datetime import datetime as _dt
from django.conf import settings as _cfg

```

```

        db_path = _Path(_cfg.DATABASES['default']['NAME'])
        backup_dir = _Path(getattr(_cfg, 'BACKUP_DIR', db_path.parent /
'backups'))
        backup_dir.mkdir(parents=True, exist_ok=True)

        timestamp = _dt.now().strftime('%Y%m%d_%H%M%S')
        backup_path = backup_dir / f'db_backup_{timestamp}.sqlite3'

        try:
            with _closing(_sqlite3.connect(str(db_path))) as src:
                with _closing(_sqlite3.connect(str(backup_path))) as dst:
                    src.backup(dst)
        except Exception as exc:
            _log_api_action(
                request,
                action='backup_create',
                after={'error': str(exc)},
                success=False,
                status_code=500,
                endpoint='api_admin_backup_create',
            )
            return JsonResponse({'error': str(exc)}, status=500)

    # Rotation
    max_count = getattr(_cfg, 'DB_BACKUP_MAX_COUNT', 3)
    existing = sorted(backup_dir.glob('db_backup_*.sqlite3'))
    while len(existing) > max_count:
        existing[0].unlink()
        existing = existing[1:]

    _log_api_action(
        request,
        action='backup_create',
        after={'filename': backup_path.name, 'size_kb':
round(backup_path.stat().st_size / 1024, 1)},
        success=True,
        status_code=200,
        endpoint='api_admin_backup_create',
    )
    return JsonResponse({
        'ok': True,
        'filename': backup_path.name,
        'size_kb': round(backup_path.stat().st_size / 1024, 1),
    })

@swagger_auto_schema(method='post', operation_summary="Восстановить базу
данных из резервной копии", tags=['admin'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@admin_required
def api_admin_backup_restore(request):
    """POST {filename} — restore the SQLite database from a named backup
file.

    Only plain filenames matching the expected pattern are accepted to
prevent
path-traversal attacks. A safety snapshot of the current database is
written to the backup directory before overwriting.
"""
    import sqlite3 as _sqlite3
    from contextlib import closing as _closing
    import shutil as _shutil

```

```

from pathlib import Path as _Path
from django.conf import settings as _cfg
from django.db import connections as _connections

if request.method != 'POST':
    _log_api_action(
        request,
        action='backup_restore',
        after={'error': 'method_not_allowed'},
        success=False,
        status_code=405,
        endpoint='api_admin_backup_restore',
    )
    return JsonResponse({'error': 'method_not_allowed'}, status=405)

try:
    data = json.loads(request.body)
except Exception:
    _log_api_action(
        request,
        action='backup_restore',
        after={'error': 'invalid_json'},
        success=False,
        status_code=400,
        endpoint='api_admin_backup_restore',
    )
    return JsonResponse({'error': 'invalid_json'}, status=400)

filename = (data.get('filename') or '').strip()
# Path-traversal guard: only allow the exact backup filename format.
if not filename or not re.fullmatch(r'db_backup_[0-9]{8}_[0-9]{6}\.sqlite3', filename):
    _log_api_action(
        request,
        action='backup_restore',
        after={'error': 'invalid_filename', 'filename': filename},
        success=False,
        status_code=400,
        endpoint='api_admin_backup_restore',
    )
    return JsonResponse({'error': 'invalid_filename'}, status=400)

db_path = _Path(_cfg.DATABASES['default']['NAME'])
backup_dir = _Path(getattr(_cfg, 'BACKUP_DIR', db_path.parent /
'backups'))
backup_path = backup_dir / filename

if not backup_path.exists():
    _log_api_action(
        request,
        action='backup_restore',
        after={'error': 'not_found', 'filename': filename},
        success=False,
        status_code=404,
        endpoint='api_admin_backup_restore',
    )
    return JsonResponse({'error': 'not_found'}, status=404)

# Close all active DB connections before replacing the underlying
file.
for _conn in _connections.all():
    try:
        _conn.close()

```

```

        except Exception:
            pass

    # Safety snapshot so the admin can undo if needed.
    safety = backup_dir / f'before_restore_{filename}'
    try:
        _shutil.copy2(str(db_path), str(safety))
    except Exception:
        pass

    try:
        with _closing(_sqlite3.connect(str(backup_path))) as src:
            with _closing(_sqlite3.connect(str(db_path))) as dst:
                src.backup(dst)
    except Exception as exc:
        try:
            _shutil.copy2(str(safety), str(db_path))
        except Exception:
            pass
        _log_api_action(
            request,
            action='backup_restore',
            before={'filename': filename},
            after={'error': f'restore_failed: {exc}'},
            success=False,
            status_code=500,
            endpoint='api_admin_backup_restore',
        )
        return JsonResponse({'error': f'restore_failed: {exc}'},
                            status=500)

    _log_api_action(
        request,
        action='backup_restore',
        before={'filename': filename},
        after={'restored_from': filename, 'safety_snapshot':
safety.name},
        success=True,
        status_code=200,
        endpoint='api_admin_backup_restore',
    )
    return JsonResponse({'ok': True, 'restored_from': filename})

@swagger_auto_schema(method='post', operation_summary="Удалить резервную
копию базы данных", tags=['admin'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@admin_required
def api_admin_backup_delete(request):
    """POST {filename} - delete a named backup file.

    Only filenames matching the expected pattern are accepted to prevent
    path-traversal attacks.
    """
    from pathlib import Path as _Path
    from django.conf import settings as _cfg

    if request.method != 'POST':
        _log_api_action(
            request,
            action='backup_delete',
            after={'error': 'method_not_allowed'},

```

```

        success=False,
        status_code=405,
        endpoint='api_admin_backup_delete',
    )
    return JsonResponse({'error': 'method_not_allowed'}, status=405)

try:
    data = json.loads(request.body)
except Exception:
    _log_api_action(
        request,
        action='backup_delete',
        after={'error': 'invalid_json'},
        success=False,
        status_code=400,
        endpoint='api_admin_backup_delete',
    )
    return JsonResponse({'error': 'invalid_json'}, status=400)

filename = (data.get('filename') or '').strip()
if not filename or not re.fullmatch(r'db_backup_[0-9]{8}_[0-9]{6}\.sqlite3', filename):
    _log_api_action(
        request,
        action='backup_delete',
        after={'error': 'invalid_filename', 'filename': filename},
        success=False,
        status_code=400,
        endpoint='api_admin_backup_delete',
    )
    return JsonResponse({'error': 'invalid_filename'}, status=400)

db_path = _Path(_cfg.DATABASES['default']['NAME'])
backup_dir = _Path(getattr(_cfg, 'BACKUP_DIR', db_path.parent /
'backups'))
backup_path = backup_dir / filename

if not backup_path.exists():
    _log_api_action(
        request,
        action='backup_delete',
        before={'filename': filename},
        after={'error': 'not_found'},
        success=False,
        status_code=404,
        endpoint='api_admin_backup_delete',
    )
    return JsonResponse({'error': 'not_found'}, status=404)

try:
    backup_path.unlink()
except Exception as exc:
    _log_api_action(
        request,
        action='backup_delete',
        before={'filename': filename},
        after={'error': str(exc)},
        success=False,
        status_code=500,
        endpoint='api_admin_backup_delete',
    )
    return JsonResponse({'error': str(exc)}, status=500)

```

```

        _log_api_action(
            request,
            action='backup_delete',
            before={'filename': filename},
            after={'deleted': filename},
            success=True,
            status_code=200,
            endpoint='api_admin_backup_delete',
        )
    return JsonResponse({'ok': True, 'deleted': filename})

# -----
# Chat views
# -----

@login_required
def chat_list(request):
    """All chats the current user participates in (excluding ones they
    deleted)."""
    user = request.user
    chats = (
        Chat.objects
        .filter(
            Q(manager=user, deleted_by_manager=False) |
            Q(applicant=user, deleted_by_applicant=False)
        )
        .select_related('manager', 'applicant',
                        'manager__manager', 'applicant__applicant')
        .prefetch_related('messages')
        .order_by('-created_at')
    )
    chat_data = []
    for chat in chats:
        is_manager = (user == chat.manager)
        other = chat.applicant if is_manager else chat.manager
        other_deleted = chat.deleted_by_applicant if is_manager else
chat.deleted_by_manager
        last_msg = chat.messages.last()
        unread =
chat.messages.filter(is_read=False).exclude(sender=user).count()
        profile = getattr(other, 'applicant', None) or getattr(other,
'manager', None)
        other_avatar_url = profile.avatar.url if profile and
profile.avatar else ''
        chat_data.append({
            'chat': chat,
            'other': other,
            'last_msg': last_msg,
            'unread': unread,
            'other_deleted': other_deleted,
            'other_avatar_url': other_avatar_url,
        })
    return render(request, 'accounts/chat_list.html', {'chat_data':
chat_data})

@login_required
def chat_detail(request, pk):
    """Single chat thread - only accessible by the participant who has
    NOT deleted it."""
    from django.shortcuts import get_object_or_404
    from django.http import Http404

```

```

        user = request.user
        chat = get_object_or_404(
            Chat.objects.select_related('manager', 'applicant',
                                       'manager__manager',
                                       'applicant__applicant'),
            Q(manager=user) | Q(applicant=user),
            pk=pk,
        )
        is_manager = (user == chat.manager)
        # If this user already soft-deleted the chat, they shouldn't see it
        if (is_manager and chat.deleted_by_manager) or (not is_manager and
            chat.deleted_by_applicant):
            raise Http404
        # Mark all incoming messages as read

    chat.messages.filter(is_read=False).exclude(sender=user).update(is_read=True)

    messages_qs =
    chat.messages.select_related('sender').order_by('created_at')
    other = chat.applicant if is_manager else chat.manager
    other_deleted = chat.deleted_by_applicant if is_manager else
    chat.deleted_by_manager
    profile = getattr(other, 'applicant', None) or getattr(other,
        'manager', None)
    other_avatar_url = profile.avatar.url if profile and profile.avatar
    else ''
    # Most recent application from this applicant on any of the manager's
    vacancies
    application = None
    if is_manager:
        application = (
            Application.objects
            .filter(applicant=other, vacancy__created_by=user)
            .order_by('-created_at')
            .first()
        )
    return render(request, 'accounts/chat_detail.html', {
        'chat': chat,
        'messages': messages_qs,
        'other': other,
        'other_deleted': other_deleted,
        'is_manager': is_manager,
        'other_avatar_url': other_avatar_url,
        'application': application,
    })

```

```

@swagger_auto_schema(method='post', operation_summary="Начать чат с
соискателем", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_chat_start(request):
    """Manager starts (or reopens) a chat with an applicant."""
    import django.db.models as _m
    user = request.user
    if not hasattr(user, 'manager'):
        return JsonResponse({'error': 'only_managers'}, status=403)
    applicant_user_id = (request.data or {}).get('applicant_user_id')
    if not applicant_user_id:
        return JsonResponse({'error': 'applicant_user_id_required'},
            status=400)
    try:
        applicant_user = User.objects.get(pk=applicant_user_id)

```



```

except User.DoesNotExist:
    return JsonResponse({'error': 'not_found'}, status=404)
if not hasattr(applicant_user, 'applicant'):
    return JsonResponse({'error': 'target_not_applicant'},
status=400)
chat, created = Chat.objects.get_or_create(manager=user,
applicant=applicant_user)
# If manager had previously deleted this chat, restore their view
if not created and chat.deleted_by_manager:
    chat.deleted_by_manager = False
    chat.save(update_fields=['deleted_by_manager'])
return JsonResponse({'ok': True, 'chat_id': chat.pk})

@swagger_auto_schema(method='post', operation_summary="Отправить
сообщение в чат", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_chat_send(request, pk):
    """Send a message (text and/or file attachment) in a chat."""
    from django.shortcuts import get_object_or_404
    user = request.user
    chat = get_object_or_404(
        Chat,
        Q(manager=user) | Q(applicant=user),
        pk=pk,
    )
    is_manager = (user == chat.manager)
    # Block if the *other* side deleted the chat
    other_deleted = chat.deleted_by_applicant if is_manager else
chat.deleted_by_manager
    if other_deleted:
        return JsonResponse({'error': 'other_deleted'}, status=403)

    # Support both JSON (text-only) and multipart/form-data (file +
optional text)
    data = request.data or {}
    text = (data.get('text') or '').strip()
    uploaded_file = request.FILES.get('file')

    if not text and not uploaded_file:
        return JsonResponse({'error': 'empty'}, status=400)
    if text and len(text) > 4000:
        return JsonResponse({'error': 'too_long'}, status=400)
    if uploaded_file and uploaded_file.size > 20 * 1024 * 1024:  # 20 MB
cap
        return JsonResponse({'error': 'file_too_large'}, status=400)

    msg = Message.objects.create(chat=chat, sender=user, text=text,
                                file=uploaded_file or None)

    # Build file info for the response
    file_url = msg.file.url if msg.file else None
    file_name = msg.file.name.split('/')[-1] if msg.file else None

    # Notify the recipient about the new message (fire-and-forget)
    recipient = chat.applicant if is_manager else chat.manager
    sender_name = user.get_full_name() or user.username
    notify_new_chat_message(recipient, sender_name, text or
f'[{file_name}]', chat.pk)

    return JsonResponse({
        'ok': True,

```

```

        'message': {
            'id': msg.pk,
            'text': msg.text,
            'file_url': file_url,
            'file_name': file_name,
            'sender_id': msg.sender_id,
            'created_at': msg.created_at.strftime('%d.%m.%Y %H:%M'),
            'is_mine': True,
        }
    })

@swagger_auto_schema(method='get', operation_summary="Получить новые сообщения чата", tags=['accounts'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def api_chat_messages(request, pk):
    """Poll new messages since a given id."""
    from django.shortcuts import get_object_or_404
    user = request.user
    chat = get_object_or_404(
        Chat,
        Q(manager=user) | Q(applicant=user),
        pk=pk,
    )
    since_id = int(request.GET.get('since', 0))
    msgs = chat.messages.filter(pk__gt=since_id).select_related('sender').order_by('created_at')
    # mark incoming as read
    msgs.filter(is_read=False).exclude(sender=user).update(is_read=True)
    return JsonResponse({
        'ok': True,
        'messages': [
            {
                'id': m.pk,
                'text': m.text,
                'file_url': m.file.url if m.file else None,
                'file_name': m.file.name.split('/')[-1] if m.file else None,
                'sender_id': m.sender_id,
                'sender_name': m.sender.get_full_name() or m.sender.username,
                'created_at': m.created_at.strftime('%d.%m.%Y %H:%M'),
                'is_mine': m.sender_id == user.pk,
            }
            for m in msgs
        ]
    })

@swagger_auto_schema(method='delete', operation_summary="Удалить чат", tags=['accounts'])
@api_view(['DELETE'])
@permission_classes([IsAuthenticated])
def api_chat_delete(request, pk):
    """Soft-delete a chat for the requesting user.
    If both sides have deleted it, the record is hard-deleted."""
    from django.shortcuts import get_object_or_404
    user = request.user
    chat = get_object_or_404(
        Chat,
        Q(manager=user) | Q(applicant=user),

```

```

        pk=pk,
    )
    if user == chat.manager:
        chat.deleted_by_manager = True
        chat.save(update_fields=['deleted_by_manager'])
    else:
        chat.deleted_by_applicant = True
        chat.save(update_fields=['deleted_by_applicant'])
    # Hard-delete only when both sides have dismissed it
    if chat.deleted_by_manager and chat.deleted_by_applicant:
        chat.delete()
    return JsonResponse({'ok': True})

@swagger_auto_schema(method='post', operation_summary="Сменить роль",
tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_switch_role(request):
    """Toggle current user between applicant and manager roles."""
    user = request.user
    before_role = 'manager' if Manager.objects.filter(user=user).exists()
    else 'applicant'
    manager = Manager.objects.filter(user=user).first()
    if manager:
        # Manager → Applicant: check for conflicts with existing
        Applicant records
        if manager.phone:
            if
            Applicant.objects.filter(phone=manager.phone).exclude(user=user).exists()
            :
                _log_api_action(
                    request,
                    action='switch_role',
                    before={'from_role': before_role},
                    after={'error': 'phone_conflict'},
                    success=False,
                    status_code=400,
                    endpoint='api_switch_role',
                )
                return JsonResponse({'error': 'phone_conflict'},
status=400)
            if manager.telegram:
                if
                Applicant.objects.filter(telegram=manager.telegram).exclude(user=user).ex
ists():
                    _log_api_action(
                        request,
                        action='switch_role',
                        before={'from_role': before_role},
                        after={'error': 'telegram_conflict'},
                        success=False,
                        status_code=400,
                        endpoint='api_switch_role',
                    )
                    return JsonResponse({'error': 'telegram_conflict'},
status=400)

        # Manager → Applicant: remove Manager record
        # Preserve avatar AND company logo path on the applicant record
    before deleting manager
    applicant, _ = Applicant.objects.get_or_create(user=user)
    applicant_changed = []

```

```

        if manager.avatar and not applicant.avatar:
            applicant.avatar = manager.avatar.name # string assignment
preserves file on disk
            applicant_changed.append('avatar')
        if manager.company_logo:
            applicant.manager_company_logo = manager.company_logo.name
            applicant_changed.append('manager_company_logo')
        if applicant_changed:
            applicant.save(update_fields=applicant_changed)
        manager.delete()
        new_role = 'applicant'
    else:
        # Applicant → Manager: check for conflicts with existing Manager
records
        applicant = Applicant.objects.filter(user=user).first()
        if applicant:
            if applicant.phone:
                if
Manager.objects.filter(phone=applicant.phone).exists():
                    _log_api_action(
                        request,
                        action='switch_role',
                        before={'from_role': before_role},
                        after={'error': 'phone_conflict'},
                        success=False,
                        status_code=400,
                        endpoint='api_switch_role',
                    )
                    return JsonResponse({'error': 'phone_conflict'},
status=400)
            if applicant.telegram:
                if
Manager.objects.filter(telegram=applicant.telegram).exists():
                    _log_api_action(
                        request,
                        action='switch_role',
                        before={'from_role': before_role},
                        after={'error': 'telegram_conflict'},
                        success=False,
                        status_code=400,
                        endpoint='api_switch_role',
                    )
                    return JsonResponse({'error': 'telegram_conflict'},
status=400)

        m = Manager(user=user)
        if applicant:
            m.patronymic = applicant.patronymic
            m.phone = applicant.phone
            m.telegram = applicant.telegram
            # Carry personal avatar path to the new manager record
            if applicant.avatar:
                m.avatar = applicant.avatar.name # string assignment
            # Restore previously saved company logo path (survives role
switches)
            if applicant.manager_company_logo:
                m.company_logo = applicant.manager_company_logo
            # Remember that this applicant has been a manager
            if not applicant.was_manager:
                applicant.was_manager = True
                applicant.save(update_fields=['was_manager'])
        m.save()
        new_role = 'manager'

```

```

_log_api_action(
    request,
    action='switch_role',
    before={'from_role': before_role},
    after={'to_role': new_role},
    success=True,
    status_code=200,
    endpoint='api_switch_role',
)
return JsonResponse({'ok': True, 'role': new_role})

@swagger_auto_schema(method='post', operation_summary="Добавить запись об образовании", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_education_create(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if not applicant:
        return JsonResponse({'error': 'no_applicant'}, status=403)
    data = request.data or {}
    institution = (data.get('institution') or '').strip()
    if not institution:
        return JsonResponse({'error': 'institution_required'},
status=400)
    level = (data.get('level') or '').strip()
    VALID = {c[0] for c in Education.LEVEL_CHOICES}
    if level not in VALID:
        return JsonResponse({'error': 'invalid_level'}, status=400)
    try:
        grad_year = int(data['graduation_year']) if
data.get('graduation_year') else None
    except (ValueError, TypeError):
        grad_year = None
    order = applicant.educations.count()
    edu = Education.objects.create(
        applicant=applicant, level=level, institution=institution,
        graduation_year=grad_year,
        faculty=(data.get('faculty') or '').strip(),
        specialization=(data.get('specialization') or '').strip(),
        order=order,
    )
    return JsonResponse({
        'ok': True, 'id': edu.pk, 'level': edu.level,
        'level_display': edu.get_level_display(),
        'institution': edu.institution, 'graduation_year':
edu.graduation_year,
        'faculty': edu.faculty, 'specialization': edu.specialization,
    })

@swagger_auto_schema(methods=['patch'], operation_summary="Обновить запись об образовании", tags=['accounts'])
@swagger_auto_schema(methods=['delete'], operation_summary="Удалить запись об образовании", tags=['accounts'])
@api_view(['PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
def api_education_crud(request, pk):
    edu = Education.objects.filter(pk=pk,
applicant__user=request.user).first()
    if not edu:
        return JsonResponse({'error': 'not_found'}, status=404)
    if request.method == 'DELETE':

```

```

        edu.delete()
        return JsonResponse({'ok': True})
    data = request.data or {}
    VALID = {c[0] for c in Education.LEVEL_CHOICES}
    if 'institution' in data:
        val = (data.get('institution') or '').strip()
        if not val:
            return JsonResponse({'error': 'institution_required'},
status=400)
        edu.institution = val
    if 'level' in data:
        lvl = (data.get('level') or '').strip()
        if lvl not in VALID:
            return JsonResponse({'error': 'invalid_level'}, status=400)
        edu.level = lvl
    if 'graduation_year' in data:
        try:
            edu.graduation_year = int(data['graduation_year']) if
data['graduation_year'] else None
        except (ValueError, TypeError):
            pass
    for f in ('faculty', 'specialization'):
        if f in data:
            setattr(edu, f, (data.get(f) or '').strip())
    edu.save()
    return JsonResponse({
        'ok': True, 'id': edu.pk, 'level': edu.level,
        'level_display': edu.get_level_display(),
        'institution': edu.institution, 'graduation_year':
edu.graduation_year,
        'faculty': edu.faculty, 'specialization': edu.specialization,
    })

```

```

@swagger_auto_schema(method='post', operation_summary="Добавить запись о
дополнительном образовании", tags=['accounts'])

```

```

@api_view(['POST'])

```

```

@permission_classes([IsAuthenticated])

```

```

def api_extra_edu_create(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if not applicant:
        return JsonResponse({'error': 'no_applicant'}, status=403)
    data = request.data or {}
    name = (data.get('name') or '').strip()
    if not name:
        return JsonResponse({'error': 'name_required'}, status=400)
    order = applicant.extra_educations.count()
    obj = ExtraEducation.objects.create(
        applicant=applicant,
        name=name,
        document_serial=(data.get('document_serial') or '').strip(),
        description=(data.get('description') or '').strip(),
        order=order,
    )
    return JsonResponse({
        'ok': True,
        'id': obj.pk,
        'name': obj.name,
        'description': obj.description,
        'document_serial': obj.document_serial,
    })

```

```

@swagger_auto_schema(methods=['patch'], operation_summary="Обновить запись о дополнительном образовании", tags=['accounts'])
@swagger_auto_schema(methods=['delete'], operation_summary="Удалить запись о дополнительном образовании", tags=['accounts'])
@api_view(['PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
def api_extra_edu_crud(request, pk):
    obj = ExtraEducation.objects.filter(pk=pk,
    applicant__user=request.user).first()
    if not obj:
        return JsonResponse({'error': 'not_found'}, status=404)
    if request.method == 'DELETE':
        obj.delete()
        return JsonResponse({'ok': True})
    data = request.data or {}
    if 'name' in data:
        val = (data.get('name') or '').strip()
        if not val:
            return JsonResponse({'error': 'name_required'}, status=400)
        obj.name = val
    if 'description' in data:
        obj.description = (data.get('description') or '').strip()
    if 'document_serial' in data:
        obj.document_serial = (data.get('document_serial') or '').strip()
    obj.save()
    return JsonResponse({
        'ok': True,
        'id': obj.pk,
        'name': obj.name,
        'description': obj.description,
        'document_serial': obj.document_serial,
    })

@swagger_auto_schema(method='post', operation_summary="Добавить место работы", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_work_create(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if not applicant:
        return JsonResponse({'error': 'no_applicant'}, status=403)
    data = request.data or {}
    company = (data.get('company') or '').strip()
    position = (data.get('position') or '').strip()
    if not company or not position:
        return JsonResponse({'error': 'company_position_required'},
status=400)

    def _si(v):
        try:
            return int(v) if v else None
        except (ValueError, TypeError):
            return None

    order = applicant.work_experiences.count()
    is_cur = bool(data.get('is_current'))
    w = WorkExperience.objects.create(
        applicant=applicant, company=company, position=position,
        start_month=_si(data.get('start_month')),
        start_year=_si(data.get('start_year')),
        end_month=None if is_cur else _si(data.get('end_month')),
        end_year=None if is_cur else _si(data.get('end_year')),

```

```

        is_current=is_cur,
        responsibilities=(data.get('responsibilities') or '').strip(),
        order=order,
    )
    return JsonResponse({
        'ok': True, 'id': w.pk, 'company': w.company, 'position':
w.position,
        'start_month': w.start_month, 'start_year': w.start_year,
        'end_month': w.end_month, 'end_year': w.end_year,
        'is_current': w.is_current, 'responsibilities':
w.responsibilities,
    })

@swagger_auto_schema(methods=['patch'], operation_summary="Обновить место
работы", tags=['accounts'])
@swagger_auto_schema(methods=['delete'], operation_summary="Удалить место
работы", tags=['accounts'])
@api_view(['PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
def api_work_crud(request, pk):
    w = WorkExperience.objects.filter(pk=pk,
applicant__user=request.user).first()
    if not w:
        return JsonResponse({'error': 'not_found'}, status=404)
    if request.method == 'DELETE':
        w.delete()
        return JsonResponse({'ok': True})
    data = request.data or {}

    def _si(v):
        try:
            return int(v) if v else None
        except (ValueError, TypeError):
            return None

    for f in ('company', 'position', 'responsibilities'):
        if f in data:
            setattr(w, f, (data.get(f) or '').strip())
    if 'is_current' in data:
        w.is_current = bool(data['is_current'])
    for f in ('start_month', 'start_year', 'end_month', 'end_year'):
        if f in data:
            setattr(w, f, _si(data[f]))
    if w.is_current:
        w.end_month = None
        w.end_year = None
    w.save()
    return JsonResponse({
        'ok': True, 'id': w.pk, 'company': w.company, 'position':
w.position,
        'start_month': w.start_month, 'start_year': w.start_year,
        'end_month': w.end_month, 'end_year': w.end_year,
        'is_current': w.is_current, 'responsibilities':
w.responsibilities,
    })

@swagger_auto_schema(method='put', operation_summary="Обновить список
навыков", tags=['accounts'])
@api_view(['PUT'])
@permission_classes([IsAuthenticated])
def api_skills_update(request):

```



```

applicant = Applicant.objects.filter(user=request.user).first()
if not applicant:
    return JsonResponse({'error': 'no_applicant'}, status=403)
skills = (request.data or {}).get('skills', [])
if not isinstance(skills, list):
    return JsonResponse({'error': 'invalid_skills'}, status=400)
applicant.skills = [s.strip() for s in skills if isinstance(s, str)
and s.strip()]
applicant.save(update_fields=['skills'])
return JsonResponse({'ok': True, 'skills': applicant.skills})

@swagger_auto_schema(method='post', operation_summary="Сохранить согласие
на уведомления в Telegram", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_set_consent(request):
    # Accept boolean-like values from JSON or form
    try:
        val = request.data.get('consent_telegram')
    except Exception:
        # fallback
        val = None

    if isinstance(val, bool):
        consent = val
    else:
        consent = str(val).lower() in ('1', 'true', 'yes', 'on')

    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        if consent and not (applicant.telegram or '').strip():
            return JsonResponse({'error': 'no_telegram'}, status=400)
        applicant.consent_telegram = consent
        if not applicant.telegram_start_token:
            applicant.telegram_start_token = uuid.uuid4().hex
            applicant.save(update_fields=['consent_telegram',
'telegram_start_token'])
        else:
            applicant.save(update_fields=['consent_telegram'])
        bot_username = get_bot_username()
        bot_start_url = None
        if bot_username and applicant.telegram_start_token:
            bot_start_url =
f'https://t.me/{bot_username}?start={applicant.telegram_start_token}'
        return JsonResponse({'ok': True, 'consent_telegram':
applicant.consent_telegram, 'bot_start_url': bot_start_url})

    manager = Manager.objects.filter(user=request.user).first()
    if manager:
        if consent and not (manager.telegram or '').strip():
            return JsonResponse({'error': 'no_telegram'}, status=400)
        manager.consent_telegram = consent
        manager.save(update_fields=['consent_telegram'])
        bot_start_url = None
        if consent:
            bot_username = get_bot_username()
            if bot_username:
                bot_start_url = f'https://t.me/{bot_username}'
            return JsonResponse({'ok': True, 'consent_telegram':
manager.consent_telegram, 'bot_start_url': bot_start_url})

    return JsonResponse({'error': 'no_profile'}, status=400)

```

```

@swagger_auto_schema(method='post', operation_summary="Отправить тестовое
уведомление в Telegram", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_test_message(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        if not applicant.consent_telegram:
            return JsonResponse({'error': 'no_consent'}, status=400)
        if not applicant.telegram_chat_id and
applicant.telegram_start_token:
            chat_id =
resolve_chat_id_by_token(applicant.telegram_start_token)
            if chat_id:
                applicant.telegram_chat_id = chat_id
                applicant.save(update_fields=['telegram_chat_id'])
        if not applicant.telegram_chat_id:
            return JsonResponse({'error': 'chat_id_missing'}, status=400)
        send_hello_async(applicant.telegram_chat_id,
                        text='Это тестовое сообщение от JobFlex.
Уведомления работают корректно.')
        return JsonResponse({'ok': True})

    manager = Manager.objects.filter(user=request.user).first()
    if manager:
        if not manager.consent_telegram:
            return JsonResponse({'error': 'no_consent'}, status=400)
        if not manager.telegram_chat_id:
            return JsonResponse({'error': 'chat_id_missing'}, status=400)
        send_hello_async(manager.telegram_chat_id,
                        text='Это тестовое сообщение от JobFlex.
Уведомления работают корректно.')
        return JsonResponse({'ok': True})

    return JsonResponse({'error': 'no_profile'}, status=400)

@swagger_auto_schema(method='post', operation_summary="Открепить
Telegram", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_unlink_telegram(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        applicant.telegram_chat_id = None
        applicant.consent_telegram = False
        applicant.save(update_fields=['telegram_chat_id',
'consent_telegram'])
        return JsonResponse({'ok': True})

    manager = Manager.objects.filter(user=request.user).first()
    if manager:
        manager.telegram_chat_id = None
        manager.consent_telegram = False
        manager.save(update_fields=['telegram_chat_id',
'consent_telegram'])
        return JsonResponse({'ok': True})

    return JsonResponse({'error': 'no_profile'}, status=400)

```

```

# ----- Email notification endpoints

@swagger_auto_schema(method='post', operation_summary="Сохранить согласие
на уведомления по электронной почте", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_set_email_consent(request):
    try:
        val = request.data.get('consent_email')
    except Exception:
        val = None

    if isinstance(val, bool):
        consent = val
    else:
        consent = str(val).lower() in ('1', 'true', 'yes', 'on')

    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        if consent and not (request.user.email or '').strip():
            return JsonResponse({'error': 'no_email'}, status=400)
        applicant.consent_email = consent
        applicant.save(update_fields=['consent_email'])
        return JsonResponse({'ok': True, 'consent_email':
applicant.consent_email})

    manager = Manager.objects.filter(user=request.user).first()
    if manager:
        if consent and not (request.user.email or '').strip():
            return JsonResponse({'error': 'no_email'}, status=400)
        manager.consent_email = consent
        manager.save(update_fields=['consent_email'])
        return JsonResponse({'ok': True, 'consent_email':
manager.consent_email})

    return JsonResponse({'error': 'no_profile'}, status=400)

@swagger_auto_schema(method='post', operation_summary="Отправить тестовое
письмо", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_test_email(request):
    # Resolve consent flag from whichever profile the caller has
    _has_consent = False
    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        _has_consent = applicant.consent_email
    else:
        manager = Manager.objects.filter(user=request.user).first()
        if manager:
            _has_consent = manager.consent_email
        else:
            return JsonResponse({'error': 'no_profile'}, status=400)
    if not _has_consent:
        return JsonResponse({'error': 'no_consent'}, status=400)

    email_address = request.user.email
    if not email_address:
        return JsonResponse({'error': 'no_email'}, status=400)

    # Guard: email backend must be configured with credentials

```

```

        host_user = django_settings.EMAIL_HOST_USER
        if not host_user and django_settings.EMAIL_BACKEND ==
'django.core.mail.backends.smtp.EmailBackend':
            return JsonResponse({'error': 'email_not_configured',
                                'detail': 'EMAIL_HOST_USER is not set in
.env'}, status=503)

        # Mail.ru (and most SMTP providers) require From == authenticated
user
        from_email = host_user if host_user else
django_settings.DEFAULT_FROM_EMAIL

        try:
            send_mail(
                subject='Тестовое уведомление от JobFlex',
                message='Это тестовое письмо от JobFlex. Email-уведомления
работают корректно.',
                from_email=from_email,
                recipient_list=[email_address],
                fail_silently=False,
            )
        except Exception as exc:
            return JsonResponse({'error': 'send_failed', 'detail': str(exc)},
status=500)

        return JsonResponse({'ok': True})

@swagger_auto_schema(method='post', operation_summary="Отключить
уведомления по электронной почте", tags=['accounts'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def api_unlink_email(request):
    applicant = Applicant.objects.filter(user=request.user).first()
    if applicant:
        applicant.consent_email = False
        applicant.save(update_fields=['consent_email'])
        return JsonResponse({'ok': True})

    manager = Manager.objects.filter(user=request.user).first()
    if manager:
        manager.consent_email = False
        manager.save(update_fields=['consent_email'])
        return JsonResponse({'ok': True})

    return JsonResponse({'error': 'no_profile'}, status=400)

@swagger_auto_schema(method='post', operation_summary="Удалить аккаунт",
tags=['accounts'])
@api_view(['POST'])
@permission_classes([AllowAny])
@login_required
def delete_account(request):
    user = request.user
    _log_api_action(
        request,
        action='delete_account',
        before={'user_id': user.pk, 'username': user.username},
        after={'deleted': True},
        success=True,
        status_code=200,
        endpoint='delete_account',

```

```

    )
    try:
        auth_logout(request)
    except Exception:
        pass
    Applicant.objects.filter(user=user).delete()
    User.objects.filter(pk=user.pk).delete()
    return JsonResponse({'ok': True, 'redirect': '/'})

# ----- Filter Presets endpoints -----

@swagger_auto_schema(methods=['get'], operation_summary="Список пресетов фильтров", tags=['presets'])
@swagger_auto_schema(methods=['post'], operation_summary="Создать пресет фильтров", tags=['presets'])
@api_view(['GET', 'POST'])
@permission_classes([IsAuthenticated])
def api_presets(request):
    if request.method == 'GET':
        presets =
list(FilterPreset.objects.filter(user=request.user).values('id', 'name',
'filters'))
        return JsonResponse({'ok': True, 'presets': presets})

    # POST — create new preset
    data = request.data or {}
    name = (data.get('name') or '').strip()
    if not name:
        _log_api_action(
            request,
            action='preset_create',
            after={'error': 'name_required'},
            success=False,
            status_code=400,
            endpoint='api_presets',
        )
        return JsonResponse({'error': 'name_required'}, status=400)
    filters = data.get('filters')
    if not isinstance(filters, dict):
        _log_api_action(
            request,
            action='preset_create',
            after={'error': 'filters_must_be_dict'},
            success=False,
            status_code=400,
            endpoint='api_presets',
        )
        return JsonResponse({'error': 'filters_must_be_dict'},
status=400)
    # Optionally back-fill skills from the applicant profile
    if data.get('from_profile_skills'):
        applicant = Applicant.objects.filter(user=request.user).first()
        if applicant and applicant.skills and not filters.get('skills'):
            filters = dict(filters)
            filters['skills'] = ', '.join(applicant.skills)
    preset = FilterPreset.objects.create(user=request.user, name=name,
filters=filters)
    _log_api_action(
        request,
        action='preset_create',
        after={'preset_id': preset.pk, 'name': preset.name, 'filters':
preset.filters},

```

```

        success=True,
        status_code=201,
        endpoint='api_presets',
    )
    return JsonResponse({'ok': True, 'id': preset.pk, 'name':
preset.name, 'filters': preset.filters}, status=201)

@swagger_auto_schema(methods=['patch'], operation_summary="Обновить
пресет фильтров", tags=['presets'])
@swagger_auto_schema(methods=['delete'], operation_summary="Удалить
пресет фильтров", tags=['presets'])
@api_view(['PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
def api_preset_detail(request, pk):
    preset = FilterPreset.objects.filter(pk=pk,
user=request.user).first()
    if not preset:
        _log_api_action(
            request,
            action='preset_update' if request.method == 'PATCH' else
'preset_delete',
            before={'preset_id': pk},
            after={'error': 'not_found'},
            success=False,
            status_code=404,
            endpoint='api_preset_detail',
        )
        return JsonResponse({'error': 'not_found'}, status=404)
    before_state = {'preset_id': preset.pk, 'name': preset.name,
'filters': preset.filters}
    if request.method == 'DELETE':
        preset.delete()
        _log_api_action(
            request,
            action='preset_delete',
            before=before_state,
            after={'deleted': True},
            success=True,
            status_code=200,
            endpoint='api_preset_detail',
        )
        return JsonResponse({'ok': True})
    # PATCH
    data = request.data or {}
    if 'name' in data:
        name = (data.get('name') or '').strip()
        if not name:
            _log_api_action(
                request,
                action='preset_update',
                before=before_state,
                after={'error': 'name_required'},
                success=False,
                status_code=400,
                endpoint='api_preset_detail',
            )
            return JsonResponse({'error': 'name_required'}, status=400)
        preset.name = name
    if 'filters' in data:
        filters = data.get('filters')
        if not isinstance(filters, dict):
            _log_api_action(

```

```

        request,
        action='preset_update',
        before=before_state,
        after={'error': 'filters_must_be_dict'},
        success=False,
        status_code=400,
        endpoint='api_preset_detail',
    )
    return JsonResponse({'error': 'filters_must_be_dict'},
status=400)
    preset.filters = filters
    preset.save()
    _log_api_action(
        request,
        action='preset_update',
        before=before_state,
        after={'preset_id': preset.pk, 'name': preset.name, 'filters':
preset.filters},
        success=True,
        status_code=200,
        endpoint='api_preset_detail',
    )
    return JsonResponse({'ok': True, 'id': preset.pk, 'name':
preset.name, 'filters': preset.filters})

# -----
#  Calendar API
# -----

@swagger_auto_schema(method='get', operation_summary="События календаря
(по date)", tags=['calendar'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
@login_required
def api_calendar_events(request):
    """GET ?date=YYYY-MM-DD - role-aware calendar events.
    Managers: applications on their vacancies + interviews + notes.
    Applicants: vacancy views + their own applications + interviews +
notes.
    Admins: notes + interviews only.
    """
    from datetime import date as _date
    from django.utils.timezone import localtime as _localtime
    from vacancies.models import VacancyView
    from django.db.models import Q as _Q

    raw = (request.GET.get('date') or '').strip()
    try:
        day = _date.fromisoformat(raw)
    except ValueError:
        return JsonResponse({'error': 'invalid_date'}, status=400)

    is_admin = is_admin_user(request.user)
    is_mod = is_moderator_user(request.user)
    is_mgr = (not is_admin and not is_mod) and hasattr(request.user,
'manager')
    events = []

    if not is_admin and not is_mod:
        if is_mgr:
            # - Applications on manager's vacancies -----
            apps_qs = (

```

```

        Application.objects
        .filter(vacancy__created_by=request.user,
created_at__date=day)
        .select_related('vacancy', 'applicant')
        .order_by('created_at')
    )
    for a in apps_qs:
        applicant_name = a.applicant.get_full_name() or
a.applicant.username
        events.append({
            'type': 'application',
            'icon': '✉',
            'title': 'Отклик: ' + applicant_name,
            'sub': a.vacancy.title,
            'time': _localtime(a.created_at).strftime('%H:%M'),
            'color': '#70b87e',
        })
    else:
        # - Vacancy views (applicant only) -----
        views_qs = (
            VacancyView.objects
            .filter(user=request.user, viewed_at__date=day)
            .select_related('vacancy')
            .order_by('viewed_at')
        )
        for v in views_qs:
            events.append({
                'type': 'view',
                'icon': '👁',
                'title': v.vacancy.title,
                'sub': v.vacancy.company or '',
                'time': _localtime(v.viewed_at).strftime('%H:%M'),
                'color': '#6c9bd2',
                'url': '/' + v.vacancy.external_id + '/',
            })

        # - Own applications (applicant only) -----
        apps_qs = (
            Application.objects
            .filter(applicant=request.user, created_at__date=day)
            .select_related('vacancy')
            .order_by('created_at')
        )
        STATUS_LABEL = {'pending': 'На рассмотрении', 'accepted':
'Принят', 'rejected': 'Отказ'}
        for a in apps_qs:
            events.append({
                'type': 'application',
                'icon': '✉',
                'title': 'Отклик: ' + a.vacancy.title,
                'sub': a.vacancy.company + ' · ' +
STATUS_LABEL.get(a.status, a.status),
                'time': _localtime(a.created_at).strftime('%H:%M'),
                'color': '#70b87e',
                'url': '/' + a.vacancy.external_id + '/',
            })

        # - Personal notes (both roles) -----
        notes_qs = CalendarNote.objects.filter(user=request.user, date=day)
        for n in notes_qs:
            note_time = n.note_time.strftime('%H:%M') if n.note_time else
_localtime(n.created_at).strftime('%H:%M')

```



```

        events.append({
            'type': 'note',
            'id': n.pk,
            'icon': '📝',
            'title': n.title or n.text[:60],
            'body': n.text,
            'sub': '',
            'time': note_time,
            'note_time_raw': n.note_time.strftime('%H:%M') if n.note_time
else '',
            'color': n.color or '#c2a35a',
        })

# - Interviews (both roles via Q filter) -----
itv_qs = (
    Interview.objects
        .filter(
            _Q(manager=request.user) | _Q(applicant=request.user),
            scheduled_at__date=day,
            status=Interview.STATUS_SCHEDULED,
        )
        .select_related('manager', 'applicant', 'vacancy')
        .order_by('scheduled_at')
)
for itv in itv_qs:
    if itv.manager_id == request.user.pk:
        title = 'Собеседование с ' + (itv.applicant.get_full_name()
or itv.applicant.username)
        sub = (itv.vacancy.title if itv.vacancy else '') + (' · ' +
itv.location if itv.location else '')
    else:
        title = 'Собеседование'
        if itv.vacancy:
            title += ': ' + itv.vacancy.title
        sub = (itv.manager.get_full_name() or itv.manager.username) +
(' · ' + itv.location if itv.location else '')
    events.append({
        'type': 'interview',
        'id': itv.pk,
        'icon': '🕒',
        'title': title,
        'sub': sub.strip(' · '),
        'time': _localtime(itv.scheduled_at).strftime('%H:%M'),
        'color': '#9b59b6',
        'can_cancel': itv.manager_id == request.user.pk,
        'url': ('/' + itv.vacancy.external_id + '/') if
itv.vacancy else None,
    })

events.sort(key=lambda e: e['time'])

# - month_marks: {date_str: [colors]} -----
month_marks = {}
first_day = day.replace(day=1)
if day.month == 12:
    last_day = day.replace(year=day.year + 1, month=1, day=1)
else:
    last_day = day.replace(month=day.month + 1, day=1)

def _add_mark(d_str, color):
    month_marks.setdefault(d_str, [])
    if color not in month_marks[d_str]:

```

```

        month_marks[d_str].append(color)

    if not is_admin and not is_mod:
        if is_mgr:
            for a in Application.objects.filter(
                vacancy__created_by=request.user,
                created_at__date__gte=first_day,
                created_at__date__lt=last_day,
            ).values_list('created_at', flat=True):
                _add_mark(str(a.date()), '#70b87e')
        else:
            for v in VacancyView.objects.filter(
                user=request.user,
                viewed_at__date__gte=first_day,
                viewed_at__date__lt=last_day,
            ).values_list('viewed_at', flat=True):
                _add_mark(str(v.date()), 'var(--accent)')
            for a in Application.objects.filter(
                applicant=request.user,
                created_at__date__gte=first_day,
                created_at__date__lt=last_day,
            ).values_list('created_at', flat=True):
                _add_mark(str(a.date()), '#70b87e')

    for n in CalendarNote.objects.filter(
        user=request.user, date__gte=first_day, date__lt=last_day
    ).values_list('date', 'color'):
        _add_mark(str(n[0]), n[1] or '#c2a35a')
    for itv in Interview.objects.filter(
        _Q(manager=request.user) | _Q(applicant=request.user),
        scheduled_at__date__gte=first_day,
        scheduled_at__date__lt=last_day,
        status=Interview.STATUS_SCHEDULED,
    ).values_list('scheduled_at', flat=True):
        _add_mark(str(itv.date()), '#9b59b6')

    return JsonResponse({'ok': True, 'events': events, 'month_marks':
month_marks, 'is_manager': is_mgr})

@swagger_auto_schema(methods=['post'], operation_summary="Создать заметку
в календаре", tags=['calendar'])
@swagger_auto_schema(methods=['patch'], operation_summary="Обновить
заметку в календаре", tags=['calendar'])
@swagger_auto_schema(methods=['delete'], operation_summary="Удалить
заметку из календаря", tags=['calendar'])
@api_view(['POST', 'PATCH', 'DELETE'])
@permission_classes([IsAuthenticated])
@login_required
def api_calendar_note_save(request):
    """POST {date, title?, text?, color?, time?} - create a note.
    DELETE {id} - remove a note.
    PATCH {id, date?, title?, text?, color?, time?} - update date /
content / color / time."""
    from datetime import date as _date, time as _time

    try:
        data = json.loads(request.body)
    except Exception:
        return JsonResponse({'error': 'invalid_json'}, status=400)

    def _parse_time(raw):
        raw = (raw or '').strip()

```

```

        if not raw:
            return None, False # (value, had_value)
        try:
            parts = raw.split(':')
            return _time(int(parts[0]), int(parts[1])), True
        except (ValueError, IndexError):
            return None, True # explicit clear

def _resolve_notify_settings(user):
    """Resolve notification channels for a calendar-note event."""
    consent_email = False
    consent_telegram = False
    tg_chat_id = None

    # Prefer manager profile if it exists (manager users can also
keep    # applicant records after role switches).
    try:
        prof = user.manager
        consent_email = bool(prof.consent_email)
        consent_telegram = bool(prof.consent_telegram)
        tg_chat_id = prof.telegram_chat_id
        return consent_email, consent_telegram, tg_chat_id
    except Exception:
        pass

    try:
        prof = user.applicant
        consent_email = bool(prof.consent_email)
        consent_telegram = bool(prof.consent_telegram)
        tg_chat_id = prof.telegram_chat_id
    except Exception:
        pass

    return consent_email, consent_telegram, tg_chat_id

def _notify_note_event(user, subject, text):
    # Admin calendar actions should be silent (no Telegram/email).
    if is_admin_user(user):
        return

    consent_email, consent_telegram, tg_chat_id =
_resolve_notify_settings(user)

    if consent_telegram and tg_chat_id:
        try:
            send_hello_async(tg_chat_id, text=text)
        except Exception:
            pass

    if consent_email and user.email:
        try:
            send_mail(
                subject=subject,
                message=text + '\n\n- JobFlex',
                from_email=django_settings.DEFAULT_FROM_EMAIL,
                recipient_list=[user.email],
                fail_silently=True,
            )
        except Exception:
            pass

if request.method == 'PATCH':

```

```

        note_id = data.get('id')
        try:
            note = CalendarNote.objects.get(pk=note_id,
user=request.user)
        except CalendarNote.DoesNotExist:
            return JsonResponse({'error': 'not_found'}, status=404)

        old_date = note.date
        old_time = note.note_time

        update_fields = []
        # Date
        new_date = (data.get('date') or '').strip()
        if new_date:
            try:
                note.date = _date.fromisoformat(new_date)
                update_fields.append('date')
            except (ValueError, AttributeError):
                return JsonResponse({'error': 'invalid_date'},
status=400)
        # Title
        if 'title' in data:
            note.title = (data['title'] or '').strip()
            update_fields.append('title')
        # Text / body
        if 'text' in data:
            note.text = (data['text'] or '').strip()
            update_fields.append('text')
        # Color
        if 'color' in data:
            raw_color = (data['color'] or '').strip()
            if raw_color:
                note.color = raw_color
                update_fields.append('color')
        # Time - if key present, update (even to null for "clear")
        if 'time' in data:
            note.note_time, _ = _parse_time(data['time'])
            update_fields.append('note_time')

        moved = (note.date != old_date) or (note.note_time != old_time)
        if moved:
            note.reminded = False
            if 'reminded' not in update_fields:
                update_fields.append('reminded')

        if update_fields:
            note.save(update_fields=update_fields)

        if moved:
            old_date_str = old_date.strftime('%d.%m.%Y')
            new_date_str = note.date.strftime('%d.%m.%Y')
            old_time_str = old_time.strftime('%H:%M') if old_time else
'без времени'
            new_time_str = note.note_time.strftime('%H:%M') if
note.note_time else 'без времени'
            note_title = note.title or (note.text[:60] if note.text else
'Без названия')
            msg = (
                '📅 Заметка перенесена\n\n'
                f'Заметка: {note_title}\n'
                f'Было: {old_date_str} в {old_time_str}\n'
                f'Стало: {new_date_str} в {new_time_str}'

```

```

        )
        _notify_note_event(
            request.user,
            subject='Заметка перенесена – JobFlex',
            text=msg,
        )

    return JsonResponse({'ok': True})

if request.method == 'POST':
    raw = (data.get('date') or '').strip()
    title = (data.get('title') or '').strip()
    text = (data.get('text') or '').strip()
    color = (data.get('color') or '#c2a35a').strip()
    time_raw = (data.get('time') or '').strip()
    if not title and not text:
        return JsonResponse({'error': 'empty_note'}, status=400)
    try:
        day = _date.fromisoformat(raw)
    except ValueError:
        return JsonResponse({'error': 'invalid_date'}, status=400)
    note_time, _ = _parse_time(time_raw)
    note = CalendarNote.objects.create(
        user=request.user, date=day,
        title=title, text=text, color=color, note_time=note_time,
    )
    return JsonResponse({'ok': True, 'id': note.pk})

if request.method == 'DELETE':
    note_id = data.get('id')
    note = CalendarNote.objects.filter(pk=note_id,
user=request.user).first()
    if not note:
        return JsonResponse({'ok': False})

    note_title = note.title or (note.text[:60] if note.text else 'Без
названия')
    date_str = note.date.strftime('%d.%m.%Y')
    time_str = note.note_time.strftime('%H:%M') if note.note_time
else 'без времени'

    note.delete()

    msg = (
        '🗑 Заметка удалена\n\n'
        f'Заметка: {note_title}\n'
        f'Дата: {date_str}\n'
        f'Время: {time_str}'
    )
    _notify_note_event(
        request.user,
        subject='Заметка удалена – JobFlex',
        text=msg,
    )
    return JsonResponse({'ok': True})

return JsonResponse({'error': 'method_not_allowed'}, status=405)

@swagger_auto_schema(method='get', operation_summary="Экспорт событий
календаря за месяц", tags=['calendar'])
@api_view(['GET'])

```

```

@permission_classes([IsAuthenticated])
@login_required
def api_calendar_month_export(request):
    """GET ?year=Y&month=M - all notes + interviews for the month (for
    .ics export / week-view preload)."""
    from datetime import date as _date
    try:
        year = int(request.GET.get('year', 0))
        month = int(request.GET.get('month', 0))
        if not (1 <= month <= 12):
            raise ValueError
    except (ValueError, TypeError):
        return JsonResponse({'error': 'invalid_params'}, status=400)

    events = []
    for note in CalendarNote.objects.filter(
        user=request.user, date__year=year, date__month=month
    ).order_by('date', 'note_time'):
        events.append({
            'type': 'note',
            'id': note.pk,
            'date': str(note.date),
            'title': note.title or note.text[:60],
            'text': note.text,
            'time': str(note.note_time)[:5] if note.note_time else None,
            'color': note.color or '#c2a35a',
        })

    try:
        ivs = Interview.objects.filter(
            applicant__user=request.user,
            scheduled_at__year=year,
            scheduled_at__month=month,
        ).exclude(status='cancelled').select_related('vacancy',
        'manager__user')
        for iv in ivs:
            dt = iv.scheduled_at
            events.append({
                'type': 'interview',
                'id': iv.pk,
                'date': dt.strftime('%Y-%m-%d'),
                'title': (iv.vacancy.title if (iv.vacancy and
        iv.vacancy.title) else 'Собеседование'),
                'company': (iv.vacancy.company_name if (iv.vacancy and
        iv.vacancy.company_name) else ''),
                'time': dt.strftime('%H:%M'),
                'location': iv.location or '',
            })
    except Exception:
        pass

    return JsonResponse({'ok': True, 'events': events})

@swagger_auto_schema(method='get', operation_summary="Индекс заметок
календаря (поиск)", tags=['calendar'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
@login_required
def api_calendar_notes_index(request):
    """GET - global note index for calendar search (all dates for current
    user)."""
    notes = []

```

```

        for note in
CalendarNote.objects.filter(user=request.user).order_by('date',
'note_time', 'created_at'):
            notes.append({
                'type': 'note',
                'id': note.pk,
                'date': str(note.date),
                'title': note.title or note.text[:60],
                'text': note.text or '',
                'time': str(note.note_time)[:5] if note.note_time else None,
                'color': note.color or '#c2a35a',
            })
        return JsonResponse({'ok': True, 'events': notes})

@swagger_auto_schema(method='post', operation_summary="Назначить
собеседование", tags=['interviews'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_interview_schedule(request):
    """POST {applicant_user_id, vacancy_id, date, time, location, notes}
    - schedule an interview."""
    from datetime import datetime as _dt, timezone as _tz

    if request.method != 'POST':
        _log_api_action(
            request,
            action='interview_schedule',
            after={'error': 'method_not_allowed'},
            success=False,
            status_code=405,
            endpoint='api_interview_schedule',
        )
        return JsonResponse({'error': 'method_not_allowed'}, status=405)
    if not hasattr(request.user, 'manager'):
        _log_api_action(
            request,
            action='interview_schedule',
            after={'error': 'forbidden'},
            success=False,
            status_code=403,
            endpoint='api_interview_schedule',
        )
        return JsonResponse({'error': 'forbidden'}, status=403)

    try:
        data = json.loads(request.body)
    except Exception:
        _log_api_action(
            request,
            action='interview_schedule',
            after={'error': 'invalid_json'},
            success=False,
            status_code=400,
            endpoint='api_interview_schedule',
        )
        return JsonResponse({'error': 'invalid_json'}, status=400)

    applicant_uid = data.get('applicant_user_id')
    date_str = (data.get('date') or '').strip()
    time_str = (data.get('time') or '00:00').strip()
    location = (data.get('location') or '').strip()

```

```

notes_text      = (data.get('notes') or '').strip()
vacancy_id      = data.get('vacancy_id')

if not applicant_uid or not date_str:
    _log_api_action(
        request,
        action='interview_schedule',
        after={'error': 'missing_fields', 'applicant_user_id':
applicant_uid, 'date': date_str},
        success=False,
        status_code=400,
        endpoint='api_interview_schedule',
    )
    return JsonResponse({'error': 'missing_fields'}, status=400)

try:
    applicant_user = User.objects.get(pk=applicant_uid,
applicant__isnull=False)
except User.DoesNotExist:
    _log_api_action(
        request,
        action='interview_schedule',
        after={'error': 'applicant_not_found', 'applicant_user_id':
applicant_uid},
        success=False,
        status_code=404,
        endpoint='api_interview_schedule',
    )
    return JsonResponse({'error': 'applicant_not_found'}, status=404)

try:
    naive = _dt.strptime(date_str + ' ' + time_str, '%Y-%m-%d %H:%M')
    from django.utils.timezone import make_aware as _make_aware
    scheduled_at = _make_aware(naive)
except ValueError:
    _log_api_action(
        request,
        action='interview_schedule',
        after={'error': 'invalid_datetime', 'date': date_str, 'time':
time_str},
        success=False,
        status_code=400,
        endpoint='api_interview_schedule',
    )
    return JsonResponse({'error': 'invalid_datetime'}, status=400)

vacancy = None
if vacancy_id:
    from vacancies.models import Vacancy
    vacancy = Vacancy.objects.filter(pk=vacancy_id).first()

itv = Interview.objects.create(
    manager=request.user,
    applicant=applicant_user,
    vacancy=vacancy,
    scheduled_at=scheduled_at,
    location=location,
    notes=notes_text,
)

# Update the application status to "accepted" (Приглашение)
if vacancy:
    Application.objects.filter(

```



```

        applicant=applicant_user, vacancy=vacancy
    ).exclude(status='accepted').update(status='accepted')

    # — Schedule exact-time Telegram/email notifications (24h, 1h, 5min
    before) —————
    try:
        from accounts.tasks import send_interview_notification_task
        from django.utils.timezone import now as _tz_now
        from datetime import timedelta as _td
        _now = _tz_now()
        for _label, _delta in [('1d', _td(hours=24)), ('1h',
        _td(hours=1)), ('now', _td(minutes=5))]:
            _eta = scheduled_at - _delta
            if _eta > _now:

send_interview_notification_task.apply_async(args=[itv.pk, _label],
eta=_eta)
    except Exception:
        pass # scheduling is best-effort

    # Post a system message in the chat between manager and applicant
    try:
        chat, _ = Chat.objects.get_or_create(
            manager=request.user, applicant=applicant_user
        )
        scheduled_str = scheduled_at.strftime('%d.%m.%Y %H:%M') + ' UTC'
        msg_lines = ['📅 Собеседование назначено']
        if vacancy:
            msg_lines[0] += ': ' + vacancy.title
        msg_lines.append('📅 ' + scheduled_str)
        if location:
            msg_lines.append('📍 ' + location)
        msg_text = '\n'.join(msg_lines)
        Message.objects.create(chat=chat, sender=request.user,
text=msg_text)
        sender_name = request.user.get_full_name() or
request.user.username
        notify_new_chat_message(applicant_user, sender_name, msg_text,
chat.pk)
    except Exception:
        pass # chat notification is best-effort

    # — Immediate Telegram / email notification to both parties
    —————
    try:
        from django.utils import timezone as _tz_util
        local_dt = _tz_util.localtime(scheduled_at)
        date_disp = local_dt.strftime('%d.%m.%Y')
        time_disp = local_dt.strftime('%H:%M')
        vacancy_title = vacancy.title if vacancy else '-'

        # Notification for the applicant
        applicant_profile = getattr(applicant_user, 'applicant', None)
        if applicant_profile:
            lines = [
                '📅 Вас приглашают на собеседование!',
                f'Вакансия: {vacancy_title}',
                f'Дата и время: {date_disp} в {time_disp}',
            ]
            if location:
                lines.append(f'Место/ссылка: {location}')
            msg_applicant = '\n'.join(lines)

```

```

        if applicant_profile.consent_telegram and
applicant_profile.telegram_chat_id:
            send_hello_async(applicant_profile.telegram_chat_id,
msg_applicant)
        if applicant_profile.consent_email and applicant_user.email:
            send_mail(
                subject=f'Приглашение на собеседование:
{vacancy_title}',
                message=msg_applicant + '\n\n- JobFlex',
                from_email=django_settings.DEFAULT_FROM_EMAIL,
                recipient_list=[applicant_user.email],
                fail_silently=True,
            )

        # Notification for the manager (confirmation)
manager_profile = getattr(request.user, 'manager', None)
if manager_profile:
    applicant_name = applicant_user.get_full_name() or
applicant_user.username
    lines = [
        '📅 Собеседование назначено',
        f'Соискатель: {applicant_name}',
        f'Вакансия: {vacancy_title}',
        f'Дата и время: {date_disp} в {time_disp}',
    ]
    if location:
        lines.append(f'Место/ссылка: {location}')
    msg_manager = '\n'.join(lines)
    if manager_profile.consent_telegram and
manager_profile.telegram_chat_id:
        send_hello_async(manager_profile.telegram_chat_id,
msg_manager)
    if manager_profile.consent_email and request.user.email:
        send_mail(
            subject=f'Собеседование назначено: {applicant_name}',
            message=msg_manager + '\n\n- JobFlex',
            from_email=django_settings.DEFAULT_FROM_EMAIL,
            recipient_list=[request.user.email],
            fail_silently=True,
        )
except Exception:
    pass # notifications are best-effort

_log_api_action(
    request,
    action='interview_schedule',
    after={
        'interview_id': itv.pk,
        'applicant_user_id': applicant_user.pk,
        'vacancy_id': vacancy.pk if vacancy else None,
        'scheduled_at': str(itv.scheduled_at),
        'location': location,
    },
    success=True,
    status_code=200,
    endpoint='api_interview_schedule',
)

return JsonResponse({'ok': True, 'id': itv.pk})

@swagger_auto_schema(method='delete', operation_summary="Отменить

```

```

    собеседование", tags=['interviews'])
@api_view(['DELETE'])
@permission_classes([IsAuthenticated])
@login_required
def api_interview_cancel(request):
    """DELETE {id} - cancel a scheduled interview (manager OR
    applicant)."""
    from django.db.models import Q as _Q

    if request.method != 'DELETE':
        _log_api_action(
            request,
            action='interview_cancel',
            after={'error': 'method_not_allowed'},
            success=False,
            status_code=405,
            endpoint='api_interview_cancel',
        )
        return JsonResponse({'error': 'method_not_allowed'}, status=405)

    try:
        data = json.loads(request.body)
    except Exception:
        _log_api_action(
            request,
            action='interview_cancel',
            after={'error': 'invalid_json'},
            success=False,
            status_code=400,
            endpoint='api_interview_cancel',
        )
        return JsonResponse({'error': 'invalid_json'}, status=400)

    try:
        itv = Interview.objects.select_related(
            'manager', 'applicant', 'vacancy'
        ).get(
            _Q(manager=request.user) | _Q(applicant=request.user),
            pk=data.get('id'),
            status=Interview.STATUS_SCHEDULED,
        )
    except Interview.DoesNotExist:
        _log_api_action(
            request,
            action='interview_cancel',
            before={'interview_id': data.get('id')},
            after={'error': 'not_found'},
            success=False,
            status_code=404,
            endpoint='api_interview_cancel',
        )
        return JsonResponse({'error': 'not_found'}, status=404)

    action = (data.get('action') or '').strip()
    before_state = {
        'interview_id': itv.pk,
        'status': itv.status,
        'scheduled_at': str(itv.scheduled_at),
        'vacancy_id': itv.vacancy_id,
    }
    itv.status = Interview.STATUS_CANCELLED
    itv.save(update_fields=['status'])

```

```

        # If manager explicitly rejects – update application status
        if action == 'reject' and itv.vacancy and request.user ==
itv.manager:
            Application.objects.filter(
                applicant=itv.applicant, vacancy=itv.vacancy
            ).update(status='rejected')

        canceller_name = request.user.get_full_name() or
request.user.username
        vacancy_title = itv.vacancy.title if itv.vacancy else '-'

        # Post a system message in the chat between manager and applicant
        try:
            chat, _ = Chat.objects.get_or_create(
                manager=itv.manager, applicant=itv.applicant
            )
            scheduled_str = itv.scheduled_at.strftime('%d.%m.%Y %H:%M') + '
UTC'

            msg_lines = ['✗ Собеседование отменено']
            if itv.vacancy:
                msg_lines[0] += ': ' + itv.vacancy.title
            msg_lines.append('📅 ' + scheduled_str)
            msg_lines.append('Отменил(а): ' + canceller_name)
            msg_text = '\n'.join(msg_lines)
            Message.objects.create(chat=chat, sender=request.user,
text=msg_text)
            other_user = itv.applicant if request.user == itv.manager else
itv.manager
            notify_new_chat_message(other_user, canceller_name, msg_text,
chat.pk)
        except Exception:
            pass # chat notification is best-effort

        # — Telegram / email cancellation notifications


---


        try:
            from django.utils import timezone as _tz_util
            from django.core.mail import send_mail as _send_mail

            local_dt = _tz_util.localtime(itv.scheduled_at)
            date_disp = local_dt.strftime('%d.%m.%Y')
            time_disp = local_dt.strftime('%H:%M')

            # Notify applicant
            applicant_profile = getattr(itv.applicant, 'applicant', None)
            if applicant_profile:
                lines = [
                    '✗ Собеседование отменено',
                    f'Вакансия: {vacancy_title}',
                    f'Дата и время: {date_disp} в {time_disp}',
                    f'Отменил(а): {canceller_name}',
                ]
                msg_a = '\n'.join(lines)
                if applicant_profile.consent_telegram and
applicant_profile.telegram_chat_id:
                    send_hello_async(applicant_profile.telegram_chat_id,
msg_a)
                if applicant_profile.consent_email and itv.applicant.email:
                    _send_mail(
                        subject=f'Собеседование отменено: {vacancy_title}',
                        message=msg_a + '\n\n– JobFlex',
                        from_email=django_settings.DEFAULT_FROM_EMAIL,

```

```

        recipient_list=[itv.applicant.email],
        fail_silently=True,
    )

    # Notify manager
    manager_profile = getattr(itv.manager, 'manager', None)
    if manager_profile:
        applicant_name = itv.applicant.get_full_name() or
itv.applicant.username
        lines = [
            '✗ Собеседование отменено',
            f'Соискатель: {applicant_name}',
            f'Вакансия: {vacancy_title}',
            f'Дата и время: {date_disp} в {time_disp}',
            f'Отменил(а): {canceller_name}',
        ]
        msg_m = '\n'.join(lines)
        if manager_profile.consent_telegram and
manager_profile.telegram_chat_id:
            send_hello_async(manager_profile.telegram_chat_id, msg_m)
        if manager_profile.consent_email and itv.manager.email:
            _send_mail(
                subject=f'Собеседование отменено: {applicant_name}',
                message=msg_m + '\n\n- JobFlex',
                from_email=django_settings.DEFAULT_FROM_EMAIL,
                recipient_list=[itv.manager.email],
                fail_silently=True,
            )
    except Exception:
        pass # notifications are best-effort

    _log_api_action(
        request,
        action='interview_cancel',
        before=before_state,
        after={
            'interview_id': itv.pk,
            'status': itv.status,
            'cancel_action': action,
        },
        success=True,
        status_code=200,
        endpoint='api_interview_cancel',
    )

    return JsonResponse({'ok': True})

@swagger_auto_schema(method='patch', operation_summary="Перенести
собеседование", tags=['interviews'])
@api_view(['PATCH'])
@permission_classes([IsAuthenticated])
@login_required
def api_interview_reschedule(request):
    """PATCH {id, date, time} - reschedule an interview (manager
only)."""
    from datetime import datetime as _dt

    if request.method != 'PATCH':
        _log_api_action(
            request,
            action='interview_reschedule',

```

```

        after={'error': 'method_not_allowed'},
        success=False,
        status_code=405,
        endpoint='api_interview_reschedule',
    )
    return JsonResponse({'error': 'method_not_allowed'}, status=405)
if not hasattr(request.user, 'manager'):
    _log_api_action(
        request,
        action='interview_reschedule',
        after={'error': 'forbidden'},
        success=False,
        status_code=403,
        endpoint='api_interview_reschedule',
    )
    return JsonResponse({'error': 'forbidden'}, status=403)

try:
    data = json.loads(request.body)
except Exception:
    _log_api_action(
        request,
        action='interview_reschedule',
        after={'error': 'invalid_json'},
        success=False,
        status_code=400,
        endpoint='api_interview_reschedule',
    )
    return JsonResponse({'error': 'invalid_json'}, status=400)

try:
    itv = Interview.objects.select_related('manager', 'applicant',
'vacancy').get(
        manager=request.user,
        pk=data.get('id'),
        status=Interview.STATUS_SCHEDULED,
    )
except Interview.DoesNotExist:
    _log_api_action(
        request,
        action='interview_reschedule',
        before={'interview_id': data.get('id')},
        after={'error': 'not_found'},
        success=False,
        status_code=404,
        endpoint='api_interview_reschedule',
    )
    return JsonResponse({'error': 'not_found'}, status=404)

date_str = (data.get('date') or '').strip()
time_str = (data.get('time') or '10:00').strip()
try:
    naive = _dt.strptime(date_str + ' ' + time_str, '%Y-%m-%d %H:%M')
    from django.utils.timezone import make_aware as _make_aware
    scheduled_at = _make_aware(naive)
except ValueError:
    _log_api_action(
        request,
        action='interview_reschedule',
        before={'interview_id': itv.pk},
        after={'error': 'invalid_datetime', 'date': date_str, 'time':
time_str},
        success=False,

```

```

        status_code=400,
        endpoint='api_interview_reschedule',
    )
    return JsonResponse({'error': 'invalid_datetime'}, status=400)

old_dt = itv.scheduled_at
itv.scheduled_at = scheduled_at
itv.reminded_ld = False
itv.reminded_lh = False
itv.reminded_now = False
itv.save(update_fields=['scheduled_at', 'reminded_ld', 'reminded_lh',
'reminded_now'])

# Notify applicant
try:
    from django.utils import timezone as _tz_util
    from django.core.mail import send_mail as _send_mail
    local_old = _tz_util.localtime(old_dt)
    local_new = _tz_util.localtime(scheduled_at)
    vacancy_title = itv.vacancy.title if itv.vacancy else '-'
    lines = [
        '📅 Собеседование перенесено',
        f'Вакансия: {vacancy_title}',
        f'Старое время: {local_old.strftime("%d.%m.%Y %H:%M")}',
        f'Новое время: {local_new.strftime("%d.%m.%Y %H:%M")}',
    ]
    if itv.location:
        lines.append(f'Место/ссылка: {itv.location}')
    msg = '\n'.join(lines)
    sender_name = request.user.get_full_name() or
request.user.username
    chat, _ = Chat.objects.get_or_create(manager=request.user,
applicant=itv.applicant)
    Message.objects.create(chat=chat, sender=request.user, text=msg)
    notify_new_chat_message(itv.applicant, sender_name, msg, chat.pk)
    applicant_profile = getattr(itv.applicant, 'applicant', None)
    if applicant_profile:
        if applicant_profile.consent_telegram and
applicant_profile.telegram_chat_id:
            send_hello_async(applicant_profile.telegram_chat_id, msg)
        if applicant_profile.consent_email and itv.applicant.email:
            _send_mail(
                subject=f'Собеседование перенесено: {vacancy_title}',
                message=msg + '\n\n- JobFlex',
                from_email=django_settings.DEFAULT_FROM_EMAIL,
                recipient_list=[itv.applicant.email],
                fail_silently=True,
            )
except Exception:
    pass

_log_api_action(
    request,
    action='interview_reschedule',
    before={'interview_id': itv.pk, 'scheduled_at': str(old_dt)},
    after={'interview_id': itv.pk, 'scheduled_at':
str(itv.scheduled_at)},
    success=True,
    status_code=200,
    endpoint='api_interview_reschedule',
)

```

```

return JsonResponse({'ok': True})

@swagger_auto_schema(method='get', operation_summary="События календаря менеджера", tags=['calendar'])
@api_view(['GET'])
@permission_classes([IsAuthenticated])
@login_required
def api_manager_calendar_events(request):
    """GET ?date=YYYY-MM-DD - return applications on manager's vacancies,
    interviews and notes."""
    from datetime import date as _date
    from django.utils.timezone import localtime as _localtime
    from django.db.models import Q as _Q

    if not hasattr(request.user, 'manager'):
        return JsonResponse({'error': 'forbidden'}, status=403)

    raw = (request.GET.get('date') or '').strip()
    try:
        day = _date.fromisoformat(raw)
    except ValueError:
        return JsonResponse({'error': 'invalid_date'}, status=400)

    events = []

    # - Applications on manager's vacancies -----
    apps_qs = (
        Application.objects
        .filter(vacancy__created_by=request.user, created_at__date=day)
        .select_related('vacancy', 'applicant')
        .order_by('created_at')
    )
    for a in apps_qs:
        applicant_name = a.applicant.get_full_name() or
a.applicant.username
        events.append({
            'type': 'application',
            'icon': '✉',
            'title': 'Отклик: ' + applicant_name,
            'sub': a.vacancy.title,
            'time': _localtime(a.created_at).strftime('%H:%M'),
            'color': '#70b87e',
        })

    # - Interviews (manager side) -----
    itv_qs = (
        Interview.objects
        .filter(manager=request.user, scheduled_at__date=day,
status=Interview.STATUS_SCHEDULED)
        .select_related('applicant', 'vacancy')
        .order_by('scheduled_at')
    )
    for itv in itv_qs:
        applicant_name = itv.applicant.get_full_name() or
itv.applicant.username
        sub = itv.vacancy.title if itv.vacancy else ''
        if itv.location:
            sub += (' · ' if sub else '') + itv.location
        events.append({
            'type': 'interview',
            'id': itv.pk,

```



```

        'icon':      '📝',
        'title':     'Собеседование с ' + applicant_name,
        'sub':       sub.strip(' '),
        'time':      _localtime(itv.scheduled_at).strftime('%H:%M'),
        'color':     '#9b59b6',
        'can_cancel': True,
    })

# - Personal notes
notes_qs = CalendarNote.objects.filter(user=request.user, date=day)
for n in notes_qs:
    note_time = n.note_time.strftime('%H:%M') if n.note_time else
    _localtime(n.created_at).strftime('%H:%M')
    events.append({
        'type': 'note',
        'id': n.pk,
        'icon': '📝',
        'title': n.title or n.text[:60],
        'body': n.text,
        'sub': '',
        'time': note_time,
        'note_time_raw': n.note_time.strftime('%H:%M') if n.note_time
else '',
        'color': n.color or '#c2a35a',
    })

events.sort(key=lambda e: e['time'])

# - month_marks: {date_str: [colors]}
month_marks = {}
first_day = day.replace(day=1)
if day.month == 12:
    last_day = day.replace(year=day.year + 1, month=1, day=1)
else:
    last_day = day.replace(month=day.month + 1, day=1)

def _add_mark(d_str, color):
    month_marks.setdefault(d_str, [])
    if color not in month_marks[d_str]:
        month_marks[d_str].append(color)

for a in Application.objects.filter(
    vacancy__created_by=request.user,
    created_at__date__gte=first_day,
    created_at__date__lt=last_day,
).values_list('created_at', flat=True):
    _add_mark(str(a.date()), '#70b87e')

for n in CalendarNote.objects.filter(
    user=request.user,
    date__gte=first_day,
    date__lt=last_day,
).values_list('date', 'color'):
    _add_mark(str(n[0]), n[1] or '#c2a35a')

for itv in Interview.objects.filter(
    manager=request.user,
    scheduled_at__date__gte=first_day,
    scheduled_at__date__lt=last_day,
    status=Interview.STATUS_SCHEDULED,
).values_list('scheduled_at', flat=True):
    _add_mark(str(itv.date()), '#9b59b6')

```

```

        return JsonResponse({'ok': True, 'events': events, 'month_marks':
month_marks})

```

14. vacancies/__init__.py

15. vacancies/admin.py

```

from django.contrib import admin

from vacancies.models import VacancyReport

@admin.register(VacancyReport)
class VacancyReportAdmin(admin.ModelAdmin):
    list_display = (
        'id',
        'vacancy_label',
        'user_label',
        'reason_label',
        'status_label',
        'reviewed_by_label',
        'created_at',
    )
    list_filter = ('reason_code', 'self_status', 'created_at')
    search_fields = ('vacancy__title', 'vacancy__company',
'user__username', 'user__email')

    @admin.display(description='Вакансия')
    def vacancy_label(self, obj):
        return obj.vacancy

    @admin.display(description='Пользователь')
    def user_label(self, obj):
        return obj.user

    @admin.display(description='Причина')
    def reason_label(self, obj):
        return obj.get_reason_code_display()

    @admin.display(description='Статус')
    def status_label(self, obj):
        return obj.get_self_status_display()

    @admin.display(description='Проверил')
    def reviewed_by_label(self, obj):
        return obj.reviewed_by

```

16. vacancies/apps.py

```

from django.apps import AppConfig

class VacanciesConfig(AppConfig):
    name = 'vacancies'
    # no app startup hooks required

```

17. vacancies/dreamjob.py

```

import json
import re
import html as _html_unescape
from urllib.request import Request, urlopen

USER_AGENT = 'job-aggregator-diploma/1.0'

def fetch_dreamjob_page(url: str) -> str:

```

```

req = Request(url, headers={'User-Agent': USER_AGENT})
with urlopen(req, timeout=20) as r:
    return r.read().decode('utf-8', errors='replace')

def normalize_company_name(name: str) -> str:
    if not name:
        return ''
    s = name.lower()
    # remove common legal forms and punctuation
    s =
re.sub(r'\b(ooo|ooo|zao|zao\.|nao|nao\.|nao|pao|ao|ao|pa|llc|inc|ltd|gmbh
|ooo\.|ooo\.|zao|ин)\b', '', s)
    s = re.sub(r'[\W_]+', ' ', s)
    s = re.sub(r'\s+', ' ', s).strip()
    return s

def search_employer_links_by_name(name: str) -> list:
    """Try several DreamJob search URL patterns to find employer links by
    name.

    Returns list of absolute URLs (strings).
    """
    base = 'https://dreamjob.ru'
    q = re.sub(r'\s+', '+', name.strip())
    candidates = [
        f'{base}/search?query={q}',
        f'{base}/search/?query={q}',
        f'{base}/search?q={q}',
        f'{base}/search/?q={q}',
        f'{base}/companies?query={q}',
        f'{base}/companies/search?query={q}',
    ]
    found = []
    for url in candidates:
        try:
            html = fetch_dreamjob_page(url)
        except Exception:
            continue
        # unescape and find /employers/<id> links
        t = _html_unescape.unescape(html)
        for m in re.finditer(r'href=["\'](/employers/\d+)["\']', t,
re.I):
            u = base + m.group(1)
            if u not in found:
                found.append(u)
        if found:
            break
    return found

def extract_company_name_from_html(text: str) -> str | None:
    t = _html_unescape.unescape(text)
    # try common patterns
    m = re.search(r'Отзывы сотрудников о компании\s*([^\n\r]+)', t,
re.I)
    if m:
        return m.group(1).strip()
    m = re.search(r'Работа в\s*([^\n\r]+)', t, re.I)
    if m:
        # trim trailing words like '►' or '—'
        name = m.group(1).strip()
        name = re.split(r'[\u2014-\u203A\u2039\u203A\u2022\u00B7\u00BB\u00AB\s]{1,}', name)[0]

```

```

        return name.strip()
    # fallback: title tag
    m = re.search(r'<title[^>]*>(.*?)</title>', t, re.I | re.S)
    if m:
        title = re.sub('<[^<]+?>', '', m.group(1)).strip()
        # try to extract last token after 'В ' or 'В КОМПАНИИ '
        m2 = re.search(r'В\s+(.)+', title)
        if m2:
            return m2.group(1).split(' ▶')[0].strip()
    return None

def extract_rating_from_html(text: str) -> str | None:
    t = _html_unescape.unescape(text)
    # JSON-LD
    for m in
re.finditer(r'<script[^>]+type=["\']application/ld\+json["\'][^>]*>(.*?)</script>', t, re.S | re.I):
    try:
        obj = json.loads(m.group(1))
        agg = obj.get('aggregateRating') or (obj if isinstance(obj,
dict) else None)
        if isinstance(agg, dict):
            v = agg.get('ratingValue') or agg.get('rating')
            if v:
                return str(v)
    except Exception:
        continue

    # specific DreamJob markup: rating-val">4,1 or data-rating="4.1"
    m = re.search(r'rating-val["\']>\s*>([0-5][\.,][0-9])', t, re.I)
    if m:
        return m.group(1)

    m = re.search(r'data-rating["\']?\s*[:]=?\s*["\']?([0-5][\.,][0-9])["\']?', t, re.I)
    if m:
        return m.group(1)

    # generic JS key like Rating:"3.5"
    m = re.search(r'Rating["\']?\s*[:]=?\s*["\']?([0-5](?:[\.,][0-9])?)["\']?', t, re.I)
    if m:
        return m.group(1)

    # visible text near word 'рейтинг'
    m = re.search(r'рейтинг[^\d]{0,40}([0-5](?:[\.,][0-9])?)', t, re.I)
    if m:
        return m.group(1)

    return None

def parse_rating_candidate(value: str) -> float | None:
    if value is None:
        return None
    try:
        s = str(value).strip().replace(',', '.', '')
        v = float(s)
        if 0 <= v <= 5:
            return v
    except Exception:
        return None
    return None

```

18. vacancies/hh_client.py

```
import requests
from django.conf import settings
from django.core.cache import cache

def _oauth_token():
    token = getattr(settings, "HH_API_TOKEN", "").strip()
    if token:
        return token

    cached = cache.get("hh:oauth:token")
    if cached:
        return cached

    client_id = getattr(settings, "HH_API_CLIENT_ID", "").strip()
    client_secret = getattr(settings, "HH_API_CLIENT_SECRET", "").strip()
    if not client_id or not client_secret:
        return ""

    try:
        r = requests.post(
            "https://hh.ru/oauth/token",
            data={
                "grant_type": "client_credentials",
                "client_id": client_id,
                "client_secret": client_secret,
            },
            timeout=15,
            headers={"User-Agent": getattr(settings, "HH_API_USER_AGENT",
"job-aggregator-diploma/1.0")},
        )
        r.raise_for_status()
        payload = r.json() if r.content else {}
        token = str(payload.get("access_token") or "").strip()
        if token:
            cache.set("hh:oauth:token", token, timeout=60 * 50)
            return token
    except Exception:
        return ""
    return ""

def hh_openapi_headers():
    headers = {
        "User-Agent": getattr(settings, "HH_API_USER_AGENT", "job-
aggregator-diploma/1.0"),
    }
    token = _oauth_token()
    if token:
        headers["Authorization"] = f"Bearer {token}"
    client_id = getattr(settings, "HH_API_CLIENT_ID", "")
    client_secret = getattr(settings, "HH_API_CLIENT_SECRET", "")
    if client_id:
        headers["HH-Client-Id"] = client_id
    if client_secret:
        headers["HH-Client-Secret"] = client_secret
    return headers
```

19. vacancies/management/__init__.py

20. vacancies/management/commands/__init__.py

21. vacancies/management/commands/backfill_descriptions.py

```

"""
Fetch full vacancy details from HH API (/vacancies/{id}) for all
vacancies
whose raw_json is missing a description, then update the stored raw_json.

Usage:
    python manage.py backfill_descriptions          # all vacancies
missing description
    python manage.py backfill_descriptions --id 539 # single vacancy by
local DB id
    python manage.py backfill_descriptions --limit 50 --delay 0.5
"""
import json
import time
from urllib.request import Request, urlopen
from urllib.error import HTTPError

from django.core.management.base import BaseCommand

from vacancies.models import Vacancy
from vacancies.hh_client import hh_openapi_headers

class Command(BaseCommand):
    help = "Backfill vacancy descriptions from HH detail API"

    def add_arguments(self, parser):
        parser.add_argument(
            "--id",
            type=int,
            default=None,
            help="Local DB id of a single vacancy to update",
        )
        parser.add_argument(
            "--limit",
            type=int,
            default=0,
            help="Max number of vacancies to process (0 = unlimited)",
        )
        parser.add_argument(
            "--delay",
            type=float,
            default=0.4,
            help="Seconds to sleep between API requests (default 0.4)",
        )
        parser.add_argument(
            "--force",
            action="store_true",
            help="Re-fetch even if description already exists",
        )

    def handle(self, *args, **options):
        single_id = options["id"]
        limit = options["limit"]
        delay = max(0.1, options["delay"])
        force = options["force"]

        if single_id:
            qs = Vacancy.objects.filter(id=single_id)
        else:
            qs = Vacancy.objects.all()
            if not force:
                # Only vacancies whose raw_json description is
                empty/missing

```

```

        qs = [v for v in qs if not (v.raw_json or
{}).get("description")]
        else:
            qs = list(qs)

    if not qs:
        self.stdout.write(self.style.WARNING("No vacancies to
process."))
        return

    if limit:
        qs = qs[:limit]

    total = len(qs)
    self.stdout.write(f"Processing {total} vacancies
(delay={delay}s)...")

    ok = 0
    failed = 0

    for i, vacancy in enumerate(qs, start=1):
        hh_id = (vacancy.raw_json or {}).get("id") or
vacancy.external_id
        if not hh_id:
            self.stdout.write(self.style.WARNING(f"  [{i}/{total}]
DB#{vacancy.id} - no HH id, skipping"))
            failed += 1
            continue

        url = f"https://api.hh.ru/vacancies/{hh_id}"
        try:
            req = Request(url, headers=hh_openapi_headers())
            with urlopen(req, timeout=15) as resp:
                data = json.loads(resp.read().decode("utf-8"))
        except HTTPError as e:
            self.stdout.write(self.style.ERROR(
                f"  [{i}/{total}] DB#{vacancy.id} HH#{hh_id} - HTTP
{e.code}"
            ))
            failed += 1
            if e.code == 429:
                self.stdout.write("  Rate-limited - sleeping 10 s...")
                time.sleep(10)
            continue
        except Exception as e:
            self.stdout.write(self.style.ERROR(
                f"  [{i}/{total}] DB#{vacancy.id} HH#{hh_id} - {e}"
            ))
            failed += 1
            continue

        desc = data.get("description", "")
        key_skills = data.get("key_skills", [])
        branded = data.get("branded_description", "")

        # Merge detail data into raw_json (detail is a superset of
list item)
        merged = {**(vacancy.raw_json or {}), **data}
        vacancy.raw_json = merged
        vacancy.description = desc
        vacancy.branded_description = branded
        vacancy.key_skills_text = ", ".join(
            s.get("name", "") for s in key_skills if isinstance(s,

```

```

dict)
    )
    vacancy.save(update_fields=["raw_json", "description",
                                "branded_description",
                                "key_skills_text"])

    desc_preview = desc[:60].replace("\n", " ") if desc else
"(empty) "
    branded_mark = f"  branded={len(branded)}ch" if branded else
""

    self.stdout.write(
        f"  [{i}/{total}] DB#{vacancy.id} HH#{hh_id} - "
        f"desc={len(desc)}ch{branded_mark}  '{desc_preview}...'")
    )
    ok += 1

    if i < total:
        time.sleep(delay)

    self.stdout.write(self.style.SUCCESS(
        f"\nDone. OK={ok}  Failed={failed}  Total={total}"
    ))

```

22. vacancies/management/commands/backfill_vacancy_fields.py

```

from django.core.management.base import BaseCommand

from vacancies.models import Vacancy

class Command(BaseCommand):
    help = "Backfill `salary_*`, `key_skills_text`,
schedule/employment/label fields from `raw_json`"

    def handle(self, *args, **options):
        updated = 0
        qs = Vacancy.objects.all()
        for v in qs:
            changed = False
            raw = v.raw_json if isinstance(v.raw_json, dict) else {}

            sal = raw.get("salary") or {}
            sf = sal.get("from")
            st = sal.get("to")
            cur = sal.get("currency") or ""

            if v.salary_from != sf:
                v.salary_from = sf
                changed = True
            if v.salary_to != st:
                v.salary_to = st
                changed = True
            if v.salary_currency != cur:
                v.salary_currency = cur
                changed = True

            ks = raw.get("key_skills") or []
            if isinstance(ks, list):
                text = ", ".join([str(s.get("name")) for s in ks if
isinstance(s, dict) and s.get("name")])
            else:
                text = ""

            if v.key_skills_text != text:

```



```

        v.key_skills_text = text
        changed = True

    # schedule / employment / label fields
    sched = raw.get("schedule") or {}
    new_schedule_id = str(sched.get("id") or "")
    new_schedule_name = str(sched.get("name") or "")
    emp = raw.get("employment") or {}
    new_employment_id = str(emp.get("id") or "")
    new_employment_name = str(emp.get("name") or "")
    ef = raw.get("employment_form") or {}
    new_ef_id = str(ef.get("id") or "")
    new_ef_name = str(ef.get("name") or "")
    new_intern = bool(raw.get("internship"))
    new_temp = bool(raw.get("accept_temporary"))
    new_incomplete = bool(raw.get("accept_incomplete_resumes"))
    new_kids = bool(raw.get("accept_kids"))

    for attr, new_val in [
        ("schedule_id", new_schedule_id),
        ("schedule_name", new_schedule_name),
        ("employment_id", new_employment_id),
        ("employment_name", new_employment_name),
        ("employment_form_id", new_ef_id),
        ("employment_form_name", new_ef_name),
        ("is_internship", new_intern),
        ("accept_temporary", new_temp),
        ("accept_incomplete_resumes", new_incomplete),
        ("accept_kids", new_kids),
    ]:
        if getattr(v, attr) != new_val:
            setattr(v, attr, new_val)
            changed = True

    if changed:
        v.save(update_fields=[
            "salary_from", "salary_to", "salary_currency",
            "key_skills_text",
            "schedule_id", "schedule_name",
            "employment_id", "employment_name",
            "employment_form_id", "employment_form_name",
            "is_internship", "accept_temporary",
            "accept_incomplete_resumes", "accept_kids",
        ])
        updated += 1

    self.stdout.write(self.style.SUCCESS(f"Updated={updated}"))

```

23. vacancies/management/commands/fetch_dreamjob_urls.py

```

import time
from django.core.management.base import BaseCommand
from django.utils import timezone

from vacancies.dreamjob import fetch_dreamjob_page,
extract_rating_from_html, extract_company_name_from_html,
parse_rating_candidate
from vacancies.models import Employer

class Command(BaseCommand):
    help = 'Fetch employer ratings from DreamJob public pages. Usage:
manage.py fetch_dreamjob_urls <url1> <url2> ...'

```

```

def add_arguments(self, parser):
    parser.add_argument('urls', nargs='+', help='DreamJob employer
URLs')
    parser.add_argument('--delay', type=float, default=0.7,
help='Seconds to sleep between requests')

def handle(self, *args, **options):
    urls = options['urls']
    delay = options.get('delay', 0.7) or 0.7

    from difflib import SequenceMatcher

    for u in urls:
        try:
            html = fetch_dreamjob_page(u)
        except Exception as e:
            self.stderr.write(f'Failed to fetch {u}: {e}')
            continue

        name = extract_company_name_from_html(html) or ''
        candidate = extract_rating_from_html(html)
        parsed = parse_rating_candidate(candidate)

        if not name:
            self.stdout.write(f'No company name detected on {u};
candidate={candidate}')
            time.sleep(delay)
            continue

        norm_name = normalize_company_name(name)
        # build candidate list from DB with normalized names
        best = None
        best_score = 0.0
        for emp in Employer.objects.all():
            en = normalize_company_name(emp.name)
            if not en:
                continue
            score = SequenceMatcher(a=norm_name, b=en).ratio()
            if score > best_score:
                best_score = score
                best = emp

        # threshold for fuzzy match
        if best and best_score >= 0.70:
            emp = best
            if parsed is not None:
                emp.dreamjob_rating = parsed
                emp.dreamjob_rating_raw = str(candidate)[:128]
                emp.dreamjob_rating_updated_at = timezone.now()
                emp.save(update_fields=['dreamjob_rating',
'dreamjob_rating_raw', 'dreamjob_rating_updated_at'])
                self.stdout.write(f'Updated Employer id={emp.id}
name="{emp.name}" -> {parsed} (score={best_score:.2f})')
            else:
                self.stdout.write(f'Candidate found for {emp.name}
but could not parse: {candidate} (score={best_score:.2f})')
            else:
                # fallback: try simple icontains
                qs = Employer.objects.filter(name__icontains=name)
                if qs.exists():
                    emp = qs.first()
                    if parsed is not None:
                        emp.dreamjob_rating = parsed

```

```

        emp.dreamjob_rating_raw = str(candidate)[:128]
        emp.dreamjob_rating_updated_at = timezone.now()
        emp.save(update_fields=['dreamjob_rating',
'dreamjob_rating_raw', 'dreamjob_rating_updated_at'])
        self.stdout.write(f'Updated Employer id={emp.id}
name="{emp.name}" -> {parsed} (icontains)')
    else:
        self.stdout.write(f'Candidate found for
{emp.name} but could not parse: {candidate} (icontains)')
    else:
        self.stdout.write(f'No Employer match for "{name}"
(from {u}); candidate={candidate} (best_score={best_score:.2f})')

        time.sleep(delay)

```

24. vacancies/management/commands/fetch_employer_details.py

```

import json
import re
import html as _html_unescape
import time
from datetime import timedelta
from urllib.request import Request, urlopen

from django.core.management.base import BaseCommand
from django.db.models import Q as models_Q
from django.utils import timezone

from vacancies.models import Employer
from vacancies.hh_client import hh_openapi_headers

_RATING_PAT = r'totalRating["\'"]?\s*:\s*["\'"]?([0-5](?:[\.,][0-9]{1,2})?)["\'"]?'
_DJID_PAT = r'employerDjId["\'"]?\s*:\s*["\'"]?(\d+)["\'"]?'

# — Defaults —
DEFAULT_LIMIT = 50          # employers per run (0 = all)
DEFAULT_COOLDOWN_H = 12     # skip employers updated within this many hours

def _parse_rating(raw) -> float | None:
    """Parse a rating value into a float in (0, 5], or None."""
    if raw is None:
        return None
    try:
        v = float(str(raw).strip().replace(',', ' '))
        return v if 0 < v <= 5 else None
    except (ValueError, TypeError):
        return None

def fetch_employer_page(url: str) -> str:
    req = Request(url, headers=hh_openapi_headers())
    with urlopen(req, timeout=20) as r:
        return r.read().decode('utf-8')

def _rating_from_ld(obj):
    """Extract ratingValue from a JSON-LD object."""
    if not isinstance(obj, dict):
        return None
    agg = obj.get('aggregateRating')
    if isinstance(agg, dict):
        return agg.get('ratingValue') or agg.get('rating')
    return obj.get('ratingValue') or obj.get('rating')

```

```

def extract_rating_from_html(html: str):
    """Extract employer rating from HH employer page HTML.

    Returns (rating_str | None, employer_dj_id | None).
    Targets the employer-aggregate totalRating (near employerDjId)
    to avoid picking up individual review scores.
    """
    html = _html_unescape.unescape(html)
    dj_id = None

    # Priority 1: employer-level totalRating near employerDjId (both
    orderings)
    for pat, id_grp, val_grp in [
        (_DJID_PAT + r'^{}}{0,300}' + _RATING_PAT, 1, 2),
        (_RATING_PAT + r'^{}}{0,300}' + _DJID_PAT, 2, 1),
    ]:
        m = re.search(pat, html, re.I)
        if m:
            dj_id = m.group(id_grp)
            v = _parse_rating(m.group(val_grp))
            if v:
                return str(v), dj_id
            break

    if not dj_id:
        m = re.search(_DJID_PAT, html, re.I)
        if m:
            dj_id = m.group(1)

    # Priority 2: JSON-LD aggregateRating
    for m in re.finditer(
        r'<script[^>]+type=["\']application/ld\+json["\'][^>]*>(.*?)</scr
ipt>',
        html, re.S | re.I,
    ):
        try:
            obj = json.loads(m.group(1))
            for o in (obj if isinstance(obj, list) else [obj]):
                v = _parse_rating(_rating_from_ld(o))
                if v:
                    return str(v), dj_id
        except Exception:
            continue

    # Priority 3: visible star symbols
    for m in re.finditer(r'([0-5](?:[\.,][0-9]{1,2})?)\s*(?:★|3Bc3)',
        html, re.I):
        v = _parse_rating(m.group(1))
        if v:
            return str(v), dj_id

    return None, dj_id

class Command(BaseCommand):
    help = 'Fetch employer details from HH API / page scraping; store
ratings.'

    def add_arguments(self, parser):
        parser.add_argument('--missing', action='store_true', help='Only
employers with missing hh_rating')
        parser.add_argument('--limit', type=int, default=DEFAULT_LIMIT,
            help=f'Max employers per run (0 = all,

```

```

default {DEFAULT_LIMIT}))')
    parser.add_argument('--delay', type=float, default=0.1,
help='Seconds between HTTP requests')
    parser.add_argument('--cooldown', type=int,
default=DEFAULT_COOLDOWN_H,
                        help=f'Skip employers updated within N hours
(default {DEFAULT_COOLDOWN_H}))')

    def handle(self, *args, **options):
        qs = Employer.objects.all()
        if options['missing']:
            qs = qs.filter(hh_rating__isnull=True)

        # Skip employers that were recently updated
        cooldown_hours = options.get('cooldown') or DEFAULT_COOLDOWN_H
        cutoff = timezone.now() - timedelta(hours=cooldown_hours)
        qs = qs.filter(
            models_Q(rating_updated_at__isnull=True) |
models_Q(rating_updated_at__lt=cutoff)
        )

        limit = options.get('limit') or 0
        delay = options.get('delay') or 0.1

        employers = list(qs[:limit] if limit else qs.all())
        total = len(employers)
        self.stdout.write(f'Processing {total} employers
(sequential)...')

        if not employers:
            self.stdout.write('Nothing to process.')
            return

        updated = 0
        skipped = 0
        start_time = time.monotonic()

        for emp in employers:
            try:
                candidate, source = self._fetch_rating(emp, delay)
                if self._save(emp, candidate, source):
                    updated += 1
            else:
                skipped += 1
            except Exception:
                skipped += 1

        elapsed = time.monotonic() - start_time
        self.stdout.write(self.style.SUCCESS(
            f'Done in {elapsed:.1f}s. Updated={updated},
Skipped={skipped}'
        ))

    # — private helpers —

    def _fetch_rating(self, emp, delay):
        """Try API → HH page → DreamJob. Return (candidate_str,
source)."""
        candidate = dj_id = None

        # 1) HH API (fast, no scraping)
        if emp.hh_id:
            try:

```

```

        req = Request(f'https://api.hh.ru/employers/{emp.hh_id}',
                      headers=hh_openapi_headers())
        with urlopen(req, timeout=10) as r:
            data = json.loads(r.read().decode('utf-8'))
            for k in ('hh_rating', 'rating', 'ratingValue',
'rating_raw'):
                if data.get(k):
                    candidate = data[k]
                    break
            if not candidate:
                agg = data.get('aggregateRating')
                if isinstance(agg, dict):
                    candidate = agg.get('ratingValue') or
agg.get('rating')
            except Exception:
                pass # will try page scrape

        # 2) HH page scrape (only if API didn't return a rating)
        if not candidate:
            for u in self._employer_urls(emp):
                try:
                    candidate, dj_id =
extract_rating_from_html(fetch_employer_page(u))
                    if candidate:
                        break
                except Exception:
                    pass
            if delay:
                time.sleep(delay)

        # Treat zero as missing
        if candidate is not None:
            try:
                if float(str(candidate).strip().replace(',', '.')) == 0:
                    candidate = None
            except Exception:
                pass

        # 3) DreamJob fallback (only if still nothing)
        if not candidate:
            candidate = self._try_dreamjob(emp, dj_id, delay)
            if candidate:
                return candidate, 'dreamjob'

        return candidate, 'hh'

def _employer_urls(self, emp):
    urls = []
    raw = emp.raw if isinstance(emp.raw, dict) else {}
    alt = raw.get('alternate_url') or raw.get('url')
    if alt:
        urls.append(alt)
    if emp.hh_id:
        urls.append(f'https://hh.ru/employer/{emp.hh_id}')
    return urls

def _try_dreamjob(self, emp, dj_id, delay):
    """Try DreamJob via dj_id, raw links, or name search. Return
candidate or None."""
    try:
        from vacancies.dreamjob import (
            fetch_dreamjob_page, extract_rating_from_html as
dj_extract,

```

```

        search_employer_links_by_name,
    )
except Exception:
    return None

urls = []
if dj_id:
    urls.append(f'https://dreamjob.ru/employers/{dj_id}')
raw = emp.raw if isinstance(emp.raw, dict) else {}
for k in ('alternate_url', 'url', 'site'):
    v = raw.get(k)
    if isinstance(v, str) and 'dreamjob.ru' in v:
        urls.append(v)

for u in urls:
    try:
        cand = dj_extract(fetch_dreamjob_page(u))
        if cand:
            return cand
    except Exception:
        pass
    if delay:
        time.sleep(delay)

try:
    for u in search_employer_links_by_name(emp.name):
        try:
            cand = dj_extract(fetch_dreamjob_page(u))
            if cand:
                return cand
        except Exception:
            pass
        if delay:
            time.sleep(delay)
except Exception:
    pass

return None

def _save(self, emp, candidate, source) -> bool:
    """Persist rating. Returns True if saved, False if skipped."""
    parsed = _parse_rating(candidate)
    if not parsed:
        return False

    now = timezone.now()
    if source == 'dreamjob':
        emp.dreamjob_rating = parsed
        emp.dreamjob_rating_raw = str(candidate)[:128]
        emp.dreamjob_rating_updated_at = now
        emp.save(update_fields=['dreamjob_rating',
'dreamjob_rating_raw', 'dreamjob_rating_updated_at'])
    else:
        emp.hh_rating = parsed
        emp.rating_raw = str(candidate)[:128]
        emp.rating_updated_at = now
        emp.save(update_fields=['hh_rating', 'rating_raw',
'rating_updated_at'])
    self.stdout.write(f'{emp.id} hh_id={emp.hh_id} ->
{source}:{parsed}')
    return True

```

25. vacancies/management/commands/fetch_hh.py

```

import json
import sys
from datetime import datetime
from urllib.parse import urlencode
from urllib.request import Request, urlopen

from django.core.management.base import BaseCommand
from django.core.cache import cache
from django.db import transaction
from django.utils import timezone

from vacancies.models import Vacancy, Employer
from vacancies.hh_client import hh_openapi_headers

class Command(BaseCommand):
    LAST_TS_CACHE_KEY = "hh_last_published_at_iso"
    def _console_safe(self, message: str) -> str:
        enc = getattr(sys.stdout, "encoding", None) or "utf-8"
        return message.encode(enc, errors="replace").decode(enc)

    help = "Fetch vacancies from HH API and save to database"
    DEFAULT_TEXTS = [
        "продавец",
        "кассир",
        "водитель",
        "бухгалтер",
        "менеджер",
        "администратор",
        "python",
    ]

    def add_arguments(self, parser):
        parser.add_argument("--text", default="", help="Search text (optional)")
        parser.add_argument(
            "--texts",
            default="",
            help="Comma-separated texts, e.g. 'продавец, бухгалтер, python'",
        )
        parser.add_argument(
            "--use-default-texts",
            action="store_true",
            help="Use built-in mix of IT and non-IT queries",
        )
        parser.add_argument("--pages", type=int, default=5, help="Max pages per query")
        parser.add_argument("--per-page", type=int, default=100, help="Items per page (max 100)")
        parser.add_argument("--area", type=int, default=None, help="HH area id (optional)")

    def handle(self, *args, **options):
        text = (options["text"] or "").strip()
        texts = (options["texts"] or "").strip()
        use_default_texts = options["use_default_texts"]
        pages = max(1, options["pages"])
        per_page = min(max(1, options["per_page"]), 100)
        area = options["area"]

        query_list = self._build_query_list(text, texts,

```



```

use_default_texts)
    incremental_mode = (not text and not texts and not
use_default_texts and area is None)
    since_dt = self._read_cursor() if incremental_mode else None

    created_total = 0
    updated_total = 0
    newest_seen = since_dt

    for query in query_list:
        query_title = query or "<all>"
        self.stdout.write(self._console_safe(f"Query:
{query_title}"))

        first_page_payload = self._fetch_page(query, area, per_page,
0, since_dt=since_dt)
        total_pages = min(first_page_payload.get("pages", 1), pages)

        first_items = first_page_payload.get("items", [])
        if incremental_mode and not self._has_new_ids(first_items):
            self.stdout.write(self._console_safe(" Page 1: no new
ids, stop incremental scan"))
            continue
        created_count, updated_count = self._save_items(first_items)
        newest_seen = self._max_dt(newest_seen,
self._max_published(first_items))
        created_total += created_count
        updated_total += updated_count
        self.stdout.write(self._console_safe(
            f" Page 1/{total_pages}: created={created_count},
updated={updated_count}"
        ))

        for page in range(1, total_pages):
            payload = self._fetch_page(query, area, per_page, page,
since_dt=since_dt)
            items = payload.get("items", [])
            if not items:
                break
            if since_dt and not self._has_newer(items, since_dt):
                break
            if incremental_mode and not self._has_new_ids(items):
                break

            created_count, updated_count = self._save_items(items)
            newest_seen = self._max_dt(newest_seen,
self._max_published(items))
            created_total += created_count
            updated_total += updated_count
            self.stdout.write(self._console_safe(
                f" Page {page + 1}/{total_pages}:
created={created_count}, updated={updated_count}"
            ))

        if incremental_mode and newest_seen:
            self._write_cursor(newest_seen)

        self.stdout.write(self.style.SUCCESS(
            f"Done. Created={created_total}, Updated={updated_total},
Total in DB={Vacancy.objects.count()}"))
    ))

def _build_query_list(self, text, texts, use_default_texts):

```

```

        if use_default_texts:
            return self.DEFAULT_TEXTS

        if texts:
            values = [value.strip() for value in texts.split(",") if
value.strip()]
            return values or [""]

        if text:
            return [text]

        return [""]

def _fetch_page(self, query, area, per_page, page, since_dt=None):
    params = {
        "page": page,
        "per_page": per_page,
        "order_by": "publication_time",
    }
    if query:
        params["text"] = query
    if area:
        params["area"] = area
    if since_dt:
        params["date_from"] =
since_dt.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S")
    return self._fetch("https://api.hh.ru/vacancies", params)

def _fetch(self, base_url, params):
    url = f"{base_url}?{urlencode(params)}"
    request = Request(url, headers=hh_openapi_headers())
    with urlopen(request, timeout=20) as response:
        return json.loads(response.read().decode("utf-8"))

@transaction.atomic
def _save_items(self, items):
    created_count = 0
    updated_count = 0

    for item in items:
        area = item.get("area") or {}
        country, region = self._extract_location(area)
        experience = item.get("experience") or {}
        flags = self._work_format_flags(item)
        work_format = self._primary_work_format(flags)
        # Prepare employer normalization: create or update Employer
when possible
        emp_payload = item.get('employer') or {}
        employer_obj = None
        hh_id = emp_payload.get('id') if isinstance(emp_payload,
dict) else None
        if hh_id:
            try:
                employer_obj, created =
Employer.objects.get_or_create(hh_id=str(hh_id), defaults={
                    'name': emp_payload.get('name', '') or '',
                    'raw': emp_payload or {},
                })
                # keep raw up-to-date (merge simple replacement)
            if not created:
                employer_obj.raw = emp_payload or {}
                employer_obj.name = emp_payload.get('name', '') or
employer_obj.name

```

```

except Exception:
    employer_obj = None

defaults = {
    "title": item.get("name", ""),
    "company": (item.get("employer") or {}).get("name", ""),
    "country": country,
    "region": region,
    "experience_id": str(experience.get("id") or ""),
    "experience_name": str(experience.get("name") or ""),
    "work_format": work_format,
    "is_remote": flags["remote"],
    "is_hybrid": flags["hybrid"],
    "is_onsite": flags["onsite"],
    "url": item.get("alternate_url", ""),
    "published_at": self._parse_dt(item.get("published_at")),
    "raw_json": item,
    "salary_from": (item.get("salary") or {}).get("from"),
    "salary_to": (item.get("salary") or {}).get("to"),
    "salary_currency": (item.get("salary") or
    {}).get("currency") or "",
    "salary_period": "month", # HH salaries are always per-
month
    "key_skills_text": "", ".join([
        str(s.get("name")) for s in (item.get("key_skills")
or []) if isinstance(s, dict)
    ]),
    # schedule / employment / label fields
    "schedule_id": str((item.get("schedule") or {}).get("id")
or ""),
    "schedule_name": str((item.get("schedule") or
    {}).get("name") or ""),
    "employment_id": str((item.get("employment") or
    {}).get("id") or ""),
    "employment_name": str((item.get("employment") or
    {}).get("name") or ""),
    "employment_form_id": str((item.get("employment_form") or
    {}).get("id") or ""),
    "employment_form_name": str((item.get("employment_form")
or {}).get("name") or ""),
    "is_internship": bool(item.get("internship")),
    "accept_temporary": bool(item.get("accept_temporary")),
    "accept_incomplete_resumes":
bool(item.get("accept_incomplete_resumes")),
    "accept_kids": bool(item.get("accept_kids")),
}

# attach employer instance if we found/created one
if employer_obj:
    defaults['employer'] = employer_obj

vacancy_obj, created = Vacancy.objects.update_or_create(
    external_id=str(item.get("id")),
    defaults=defaults,
)
created_count += int(created)
updated_count += int(not created)
# Description fetching is handled by
backfill_descriptions_task (Celery beat)
# which spaces requests at 0.5 s intervals to avoid HH rate-
limiting.

return created_count, updated_count

```

```

def _extract_location(self, area):
    name = (area or {}).get("name") or ""
    if name in {"Казахстан", "Беларусь", "Узбекистан", "Грузия",
"Армения", "Кыргызстан", "Азербайджан"}:
        return name, ""
    return "Россия", name

def _work_format_flags(self, item):
    formats = item.get("work_format") or []
    if isinstance(formats, dict):
        formats = [formats]

    format_ids = {
        str(format_item.get("id", "")).lower()
        for format_item in formats
        if isinstance(format_item, dict)
    }
    schedule_id = ((item.get("schedule") or {}).get("id") or
"" ).lower()
    label = (item.get("name") or "").lower()

    is_remote = "remote" in format_ids or schedule_id == "remote" or
"удален" in label
    is_hybrid = "hybrid" in format_ids or "гибрид" in label
    is_onsite = "on_site" in format_ids or "onsite" in format_ids or
"на месте" in label

    if not (is_remote or is_hybrid or is_onsite):
        is_onsite = True

    return {
        "remote": is_remote,
        "hybrid": is_hybrid,
        "onsite": is_onsite,
    }

def _primary_work_format(self, flags):
    if flags["remote"]:
        return Vacancy.WorkFormat.REMOTE
    if flags["hybrid"]:
        return Vacancy.WorkFormat.HYBRID
    return Vacancy.WorkFormat.ONSITE

def _parse_dt(self, value):
    if not value:
        return timezone.now()
    return datetime.fromisoformat(value.replace("Z", "+00:00"))

def _max_published(self, items):
    best = None
    for item in items or []:
        dt = self._parse_dt(item.get("published_at"))
        best = self._max_dt(best, dt)
    return best

def _max_dt(self, a, b):
    if a is None:
        return b
    if b is None:
        return a
    return a if a >= b else b

```

```

def _has_newer(self, items, since_dt):
    for item in items or []:
        if self._parse_dt(item.get("published_at")) > since_dt:
            return True
    return False

def _read_cursor(self):
    raw = (cache.get(self.LAST_TS_CACHE_KEY) or "").strip()
    if not raw:
        return None
    try:
        return datetime.fromisoformat(raw)
    except Exception:
        return None

def _write_cursor(self, dt):
    cache.set(self.LAST_TS_CACHE_KEY,
dt.astimezone(timezone.utc).isoformat(), timeout=60 * 60 * 24 * 30)

def _has_new_ids(self, items):
    ids = [str(item.get("id") or "") for item in (items or []) if
item.get("id")]
    if not ids:
        return False
    existing_ids = set(
        Vacancy.objects.filter(external_id__in=ids).values_list("external_id", flat=True)
    )
    return any(item_id not in existing_ids for item_id in ids)

```

26. vacancies/management/commands/fetch_trudvsem.py

```

from datetime import datetime, timedelta
from urllib.parse import urlparse, quote_plus, quote
import sys

import requests
import logging
from django.core.management.base import BaseCommand
from django.core.cache import cache
from django.conf import settings
from django.db import transaction
from django.utils import timezone

from vacancies.models import Vacancy

logger = logging.getLogger(__name__)

class Command(BaseCommand):
    help = "Fetch vacancies from trudvsem open API and save to database"
    LAST_TS_CACHE_KEY = "trudvsem_last_modify_at_iso"
    BOOTSTRAP_DONE_CACHE_KEY = "trudvsem_bootstrap_done"

    API_URL = "https://opendata.trudvsem.ru/api/v1/vacancies"

    def _console_safe(self, message: str) -> str:
        enc = getattr(sys.stdout, "encoding", None) or "utf-8"
        return message.encode(enc, errors="replace").decode(enc)

    def add_arguments(self, parser):
        parser.add_argument("--limit", type=int, default=200, help="How
many records to fetch")
        parser.add_argument("--offset", type=int, default=None,

```

```

help="Offset for trudvsem API")
    parser.add_argument("--text", default="", help="Optional text
filter")

    def handle(self, *args, **options):
        limit = max(1, int(options.get("limit") or 200))
        offset_opt = options.get("offset")
        offset = None if offset_opt is None else max(0, int(offset_opt))
        text = (options.get("text") or "").strip()
        recent_minutes = max(0, int(getattr(settings,
"FALLBACK_TRUDVSEM_RECENT_MINUTES", 10)))
        existing_fallback =
Vacancy.objects.filter(external_id__startswith="trudvsem-").count()
        bootstrap_done = bool(cache.get(self.BOOTSTRAP_DONE_CACHE_KEY))
        bootstrap_target = 10
        bootstrap_mode = (not bootstrap_done) and existing_fallback <
bootstrap_target and offset is None
        recent_mode = (offset is None and recent_minutes > 0 and not
bootstrap_mode)
        full_scan_mode = (offset is None and not bootstrap_mode and not
recent_mode)
        cutoff = timezone.now() - timedelta(minutes=recent_minutes) if
recent_mode else None
        cursor_dt = self._read_cursor() if recent_mode else None
        if cursor_dt:
            cutoff = cursor_dt
        max_pages_cap = max(1, int(getattr(settings,
"FALLBACK_TRUDVSEM_MAX_PAGES_PER_RUN", 1000)))
        if bootstrap_mode:
            # First run: ensure at least a small visible dataset.
            limit = max(limit, bootstrap_target - existing_fallback)
        if recent_mode:
            # Incremental mode: fetch ALL pages with new vacancies after
cursor/cutoff.
            # The loop stops naturally when a page has no newer rows.
            limit = max(limit, 50000)
        if full_scan_mode:
            # User-requested mode: walk all available pages from the
start each run.
            limit = max(limit, 50000)

        self._migrate_legacy_external_ids()
        self._refresh_legacy_logo_quality_once()
        fetched_total = 0
        created_total = 0
        updated_total = 0
        newest_seen = cursor_dt
        current_offset = self._resolve_offset(offset)
        remaining = limit
        max_pages = max_pages_cap if (recent_mode or full_scan_mode) else
max(1, (limit + 99) // 100)
        page = 0
        max_offset = 100
        empty_recent_streak = 0
        max_empty_recent_streak = 200
        while remaining > 0:
            page += 1
            page_limit = min(100, remaining) # trudvsem max page size
            payload = self._fetch(limit=page_limit,
offset=current_offset, text=text)
            raw_items = ((payload.get("results") or {}).get("vacancies")
or [])
            if page == 1:

```

```

        total = int((payload.get("meta") or {}).get("total") or
0)
        if total > 0:
            max_offset = max(1, min(1000, (total + 99) // 100))
            page_items = []
            for item in raw_items:
                v = item.get("vacancy") if isinstance(item, dict) else
None
                if isinstance(v, dict):
                    page_items.append(v)
            got_raw = len(page_items)
            if got_raw == 0:
                break

            if recent_mode:
                page_items = [v for v in page_items if self._is_recent(v,
cutoff)]

            if not page_items:
                empty_recent_streak += 1
                logger.info(
                    "Работа России: страница=%s offset=%s новых=0
streak=%s",
                    page, current_offset, empty_recent_streak,
                )
                current_offset = (current_offset + 1) % max_offset
                remaining -= got_raw
                if empty_recent_streak >= max_empty_recent_streak or
page >= max_pages:
                    break
                continue
            empty_recent_streak = 0
            logger.info(
                "Работа России: страница=%s offset=%s новых=%s",
                page, current_offset, len(page_items),
            )

            created_page, updated_page = self._save_items(page_items)
            fetched_total += len(page_items)
            created_total += created_page
            updated_total += updated_page
            newest_seen = self._max_dt(newest_seen,
self._max_modified(page_items))
            got = got_raw
            current_offset = (current_offset + 1) % max_offset
            remaining -= got
            if got < page_limit or page >= max_pages:
                break
            self._store_offset(current_offset)
            if recent_mode and newest_seen:
                self._write_cursor(newest_seen)

            self.stdout.write(self._console_safe(
                f"trudvsem done. mode={'bootstrap' if bootstrap_mode else
'recent' if recent_mode else 'full' if full_scan_mode else 'cursor'}
text={text or '<none>'} offset={offset} fetched={fetched_total}
created={created_total} updated={updated_total}"
            ))
            if bootstrap_mode and fetched_total > 0:
                cache.set(self.BOOTSTRAP_DONE_CACHE_KEY, 1, timeout=60 * 60 *
24 * 365)

def _next_fallback_text(self):
    raw = getattr(settings, "FALLBACK_TRUDVSEM_TEXTS", "")

```

```

        texts = [t.strip() for t in str(row).split(",") if t.strip()]
        if not texts:
            texts = [
                "продавец", "кассир", "водитель", "бухгалтер",
"менеджер",
                "администратор", "python", "аналитик", "разработчик",
            ]
        idx = int(cache.get("trudvsem_text_cursor", 0) or 0)
        text = texts[idx % len(texts)]
        cache.set("trudvsem_text_cursor", idx + 1, timeout=60 * 60 * 24 *
30)
        return text

    def _migrate_legacy_external_ids(self):
        """Convert legacy fallback ids from `trudvsem:<id>` to url-safe
`trudvsem-<id>`."""
        legacy =
list(Vacancy.objects.filter(external_id__startswith="trudvsem:").only("id
", "external_id"))
        if not legacy:
            return
        for row in legacy:
            new_ext = row.external_id.replace("trudvsem:", "trudvsem-",
1)
            Vacancy.objects.filter(pk=row.pk).update(external_id=new_ext)
        self.stdout.write(f"trudvsem legacy ids migrated: {len(legacy)}")

    def _resolve_offset(self, offset):
        if offset is not None:
            return max(0, int(offset))
        return int(cache.get("trudvsem_offset_cursor", 0) or 0)

    def _store_offset(self, offset):
        cache.set("trudvsem_offset_cursor", int(offset), timeout=60 * 60
* 24 * 7)

    def _refresh_legacy_logo_quality_once(self):
        cache_key = "trudvsem_logo_quality_refreshed_v2"
        if cache.get(cache_key):
            return
        rows = list(
            Vacancy.objects.filter(external_id__startswith="trudvsem-
").only("id", "company", "raw_json")[:2000]
        )
        if not rows:
            cache.set(cache_key, 1, timeout=60 * 60 * 24 * 30)
            return
        to_update = []
        for row in rows:
            raw = row.raw_json if isinstance(row.raw_json, dict) else {}
            emp = raw.get("employer") if isinstance(raw.get("employer"),
dict) else {}
            logos = emp.get("logo_urls") if
isinstance(emp.get("logo_urls"), dict) else {}
            original = str(logos.get("original") or "")
            if ("favicon.yandex.net/favicon/" in original and "?size=" in
original and logos.get("fallback_svg")):
                continue
            company_site = self._as_text(emp.get("alternate_url"))
            company_name = self._as_text(emp.get("name")) or
self._as_text(row.company)
            emp["logo_urls"] = self._logo_urls(company_site,
company_name)

```



```

        raw["employer"] = emp
        row.raw_json = raw
        to_update.append(row)
    if to_update:
        Vacancy.objects.bulk_update(to_update, ["raw_json"],
batch_size=200)
        cache.set(cache_key, 1, timeout=60 * 60 * 24 * 30)

    def _fetch(self, limit, offset, text):
        params = {"limit": limit, "offset": offset}
        if text:
            params["text"] = text
        try:
            r = requests.get(self.API_URL, params=params, timeout=25)
        except requests.exceptions.RequestException as exc:
            logger.warning("trudvsem request failed: offset=%s limit=%s
err=%s", offset, limit, exc)
            return {"results": {"vacancies": []}, "meta": {}}
        if r.status_code >= 500:
            # API can sporadically fail on some offset values; try
wrapped page zero.
            params["offset"] = 0
            try:
                r = requests.get(self.API_URL, params=params, timeout=25)
            except requests.exceptions.RequestException as exc:
                logger.warning("trudvsem fallback request failed:
offset=0 limit=%s err=%s", limit, exc)
                return {"results": {"vacancies": []}, "meta": {}}
            r.raise_for_status()
            return r.json()

    def _parse_dt(self, value):
        if not value:
            return timezone.now()
        try:
            return datetime.fromisoformat(str(value).replace("Z",
"+00:00"))
        except Exception:
            return timezone.now()

    def _is_recent(self, item, cutoff):
        if cutoff is None:
            return True
        dt = self._parse_dt(item.get("date_modify") or
item.get("creation-date"))
        if timezone.is_naive(dt):
            dt = timezone.make_aware(dt, timezone.get_current_timezone())
        return dt > cutoff

    def _max_modified(self, items):
        best = None
        for item in items or []:
            dt = self._parse_dt(item.get("date_modify") or
item.get("creation-date"))
            best = self._max_dt(best, dt)
        return best

    def _max_dt(self, a, b):
        if a is None:
            return b
        if b is None:
            return a
        return a if a >= b else b

```

```

def _read_cursor(self):
    raw = (cache.get(self.LAST_TS_CACHE_KEY) or "").strip()
    if not raw:
        return None
    try:
        return datetime.fromisoformat(raw)
    except Exception:
        return None

def _write_cursor(self, dt):
    cache.set(self.LAST_TS_CACHE_KEY, dt.isoformat(), timeout=60 * 60
* 24 * 30)

def _as_text(self, value):
    if value is None:
        return ""
    if isinstance(value, str):
        return value.strip()
    if isinstance(value, dict):
        parts = []
        for v in value.values():
            t = self._as_text(v)
            if t:
                parts.append(t)
        return " ".join(parts).strip()
    if isinstance(value, (list, tuple)):
        parts = []
        for v in value:
            t = self._as_text(v)
            if t:
                parts.append(t)
        return " ".join(parts).strip()
    return str(value).strip()

def _domain_from_url(self, url):
    u = self._as_text(url)
    if not u:
        return ""
    if "://" not in u:
        u = "https://" + u
    try:
        host = (urlparse(u).netloc or "").lower()
        if host.startswith("www."):
            host = host[4:]
        return host
    except Exception:
        return ""

def _logo_urls(self, company_site, company_name):
    # Higher quality logo strategy:
    # - yandex favicon with explicit size for better sharpness;
    # - inline SVG as guaranteed crisp fallback.
    name = (company_name or "Company").strip() or "Company"
    letters = "".join([w[:1] for w in name.split()[:2]]).upper() or
name[:1].upper()
    svg = (
        "<svg xmlns='http://www.w3.org/2000/svg' width='256'
height='256' viewBox='0 0 256 256'>"
        "<defs><linearGradient id='g' x1='0' y1='0' x2='1' y2='1'>"
        "<stop offset='0%' stop-color='#C2692A'>"
        "<stop offset='100%' stop-color='#7D3F1C'>"
        "</linearGradient></defs>"

```

```

        "<rect width='256' height='256' rx='36' fill='url(#g)' />"
        f"<text x='128' y='145' text-anchor='middle' font-
family='Inter,Arial,sans-serif' "
        "font-size='92' font-weight='700' fill='#ffffff'>"
        f"{letters}</text></svg>"
    )
    data_uri = "data:image/svg+xml;utf8," + quote(svg,
safe=":/#?&=,+-. _~!$'()*[]@")
    domain = self._domain_from_url(company_site)
    if domain:
        yandex_90 =
f"https://favicon.yandex.net/favicon/{domain}?size=90&stub=1"
        yandex_240 =
f"https://favicon.yandex.net/favicon/{domain}?size=240&stub=1"
        yandex_512 =
f"https://favicon.yandex.net/favicon/{domain}?size=512&stub=1"
        return {"90": yandex_90, "240": yandex_240, "original":
yandex_512, "fallback_svg": data_uri}
    return {"90": data_uri, "240": data_uri, "original": data_uri}

    def _map_item(self, item):
        ext_id = f"trudvsem-{item.get('id')}"
        title = self._as_text(item.get("job-name"))[:255]
        company = self._as_text((item.get("company") or
{ }).get("name"))[:255]
        region = self._as_text((item.get("region") or
{ }).get("name"))[:128]
        url = self._as_text(item.get("vac_url"))
        published_at = self._parse_dt(item.get("date_modify") or
item.get("creation-date"))

        salary_from = item.get("salary_min")
        salary_to = item.get("salary_max")
        if not isinstance(salary_from, int):
            salary_from = None
        if not isinstance(salary_to, int):
            salary_to = None

        schedule_name = self._as_text(item.get("schedule"))[:128]
        duty = self._as_text(item.get("duty"))
        requirement = self._as_text(item.get("requirement"))
        requirements = self._as_text(item.get("requirements"))
        description = "\n\n".join([p for p in [duty, requirement,
requirements] if p])
        company_site = self._as_text((item.get("company") or
{ }).get("site"))

        text_blob = " ".join(
            [
                title.lower(),
                schedule_name.lower(),
                duty.lower(),
                requirement.lower(),
                requirements.lower(),
            ]
        )
        is_remote = "удален" in text_blob or "дистан" in text_blob or
"remote" in text_blob
        is_hybrid = "гибрид" in text_blob
        is_onsite = not (is_remote or is_hybrid)

        return ext_id, {
            "title": title,

```

```

        "company": company,
        "country": "Россия",
        "region": region,
        "experience_id": "",
        "experience_name": "",
        "work_format": (
            Vacancy.WorkFormat.REMOTE
            if is_remote
            else Vacancy.WorkFormat.HYBRID
            if is_hybrid
            else Vacancy.WorkFormat.ONSITE
        ),
        "is_remote": is_remote,
        "is_hybrid": is_hybrid,
        "is_onsite": is_onsite,
        "url": url,
        "published_at": published_at,
        "raw_json": {
            "source": "trudvsem",
            "employer": {
                "id": self._as_text((item.get("company") or
{}).get("companycode")) or ext_id,
                "name": company,
                "alternate_url": company_site,
                "logo_urls": self._logo_urls(company_site, company),
            },
            **item,
        },
        "salary_from": salary_from,
        "salary_to": salary_to,
        "salary_currency": "RUR",
        "salary_period": "month",
        "key_skills_text": "",
        "description": description,
        "schedule_id": "",
        "schedule_name": schedule_name,
        "employment_id": "",
        "employment_name": "",
        "employment_form_id": "",
        "employment_form_name": "",
        "is_internship": False,
        "accept_temporary": False,
        "accept_incomplete_resumes": False,
        "accept_kids": False,
        "is_active": True,
    }

    @transaction.atomic
    def _save_items(self, items):
        if not items:
            return 0, 0

        now = timezone.now()
        mapped = {}
        for item in items:
            if not item.get("id"):
                continue
            ext_id, fields = self._map_item(item)
            mapped[ext_id] = fields

        ext_ids = list(mapped.keys())
        existing = {v.external_id: v for v in
Vacancy.objects.filter(external_id__in=ext_ids)}

```

```

to_create = []
to_update = []
for ext_id, fields in mapped.items():
    if ext_id in existing:
        obj = existing[ext_id]
        old_raw = obj.raw_json if isinstance(obj.raw_json, dict)
    else {}
        new_raw = fields.get("raw_json") if
isinstance(fields.get("raw_json"), dict) else {}
        if (
            old_raw.get("date_modify") ==
new_raw.get("date_modify")
            and old_raw.get("salary_min") ==
new_raw.get("salary_min")
            and old_raw.get("salary_max") ==
new_raw.get("salary_max")
            and obj.title == fields.get("title")
            and obj.url == fields.get("url")
        ):
            continue
        for k, v in fields.items():
            setattr(obj, k, v)
        obj.updated_at = now
        to_update.append(obj)
    else:
        to_create.append(Vacancy(external_id=ext_id,
created_at=now, updated_at=now, **fields))

if to_create:
    Vacancy.objects.bulk_create(to_create, batch_size=100)
if to_update:
    update_fields = list(mapped[next(iter(mapped))].keys()) +
["updated_at"]
    for obj in to_update:
        payload = {name: getattr(obj, name) for name in
update_fields}
        Vacancy.objects.filter(pk=obj.pk).update(**payload)

return len(to_create), len(to_update)

```

27. vacancies/management/commands/restore_db.py

"""Management command: restore the SQLite database from a backup file.

Usage:

```
python manage.py restore_db <backup_file>
```

<backup_file> can be:

- An absolute path to any .sqlite3 file.
- A bare filename (e.g. db_backup_20260308_120000.sqlite3) that will

be

resolved relative to settings.BACKUP_DIR.

A safety snapshot of the current database is written to BACKUP_DIR before overwriting so you can undo the restore manually if needed.

Example:

```
python manage.py restore_db db_backup_20260308_120000.sqlite3
"""
```

```
import shutil
import sqlite3
```

```

from contextlib import closing
from datetime import datetime
from pathlib import Path

from django.conf import settings
from django.core.management.base import BaseCommand, CommandError
from django.db import connections

class Command(BaseCommand):
    help = 'Restore the SQLite database from a backup file.'

    def add_arguments(self, parser):
        parser.add_argument(
            'backup_file',
            help=(
                'Path to the backup .sqlite3 file, or just the filename '
                '(resolved relative to settings.BACKUP_DIR).'
            ),
        )
        parser.add_argument(
            '--no-safety',
            action='store_true',
            default=False,
            help="Skip creating a safety snapshot of the current database before restoring.",
        )

    def handle(self, *args, **options):
        backup_arg = options['backup_file']
        no_safety = options['no_safety']

        db_path = Path(settings.DATABASES['default']['NAME'])
        backup_dir = Path(getattr(settings, 'BACKUP_DIR',
                                   db_path.parent / 'backups'))

        backup_path = Path(backup_arg)
        if not backup_path.is_absolute() or not backup_path.exists():
            # Try resolving as a bare filename inside BACKUP_DIR
            candidate = backup_dir / backup_arg
            if candidate.exists():
                backup_path = candidate
            else:
                raise CommandError(
                    f'Backup file not found: {backup_arg}\n'
                    f'(Searched in {backup_dir})'
                )

        if not backup_path.exists():
            raise CommandError(f'Backup file not found: {backup_path}')

        self.stdout.write(f'Restoring from: {backup_path}')
        self.stdout.write(f'Target DB: {db_path}')

        # Close all active database connections
        for conn in connections.all():
            try:
                conn.close()
            except Exception:
                pass

        # Safety snapshot
        if not no_safety:
            backup_dir.mkdir(parents=True, exist_ok=True)

```

```

        ts = datetime.now().strftime('%Y%m%d_%H%M%S')
        safety = backup_dir / f'before_restore_{ts}.sqlite3'
        try:
            shutil.copy2(str(db_path), str(safety))
            self.stdout.write(f'Safety copy saved: {safety.name}')
        except Exception as exc:
            self.stderr.write(f'Warning: could not create safety
copy: {exc}')

        # Restore using SQLite's online backup API (WAL-safe)
        try:
            with closing(sqlite3.connect(str(backup_path))) as src:
                with closing(sqlite3.connect(str(db_path))) as dst:
                    src.backup(dst)
        except Exception as exc:
            raise CommandError(f'Restore failed: {exc}')

        self.stdout.write(
            self.style.SUCCESS(
                f'\nDatabase successfully restored from
{backup_path.name}.\n'
                'Restart the Django server to ensure a fresh connection
pool.'
            )
        )
    )

```

28. vacancies/management/commands/sync_hh_dictionaries.py

```

import json
from urllib.request import Request, urlopen

from django.core.management.base import BaseCommand

from vacancies.models import HhArea, HhDictionaryItem
from vacancies.hh_client import hh_openapi_headers

class Command(BaseCommand):
    help = "Sync HH dictionaries and areas into local database"

    def handle(self, *args, **options):
        areas_payload = self._fetch_json("https://api.hh.ru/areas")
        dictionaries_payload =
self._fetch_json("https://api.hh.ru/dictionaries")

        areas_created, areas_updated = self._save_areas(areas_payload)
        dict_created, dict_updated =
self._save_dictionaries(dictionaries_payload)

        self.stdout.write(self.style.SUCCESS(
            "Done. "
            f"Areas created={areas_created}, updated={areas_updated}. "
            f"Dictionary items created={dict_created},
updated={dict_updated}."
        ))

    def _fetch_json(self, url):
        request = Request(url, headers=hh_openapi_headers())
        with urlopen(request, timeout=20) as response:
            return json.loads(response.read().decode("utf-8"))

    def _save_areas(self, payload):
        created_count = 0
        updated_count = 0

```

```

for area in self._iter_area_nodes(payload):
    defaults = {
        "name": str(area.get("name") or ""),
        "parent_id": str(area.get("_parent_id") or ""),
        "raw_json": area,
    }
    _, created = HhArea.objects.update_or_create(
        area_id=str(area.get("id") or ""),
        defaults=defaults,
    )
    created_count += int(created)
    updated_count += int(not created)

return created_count, updated_count

def _iter_area_nodes(self, nodes, parent_id=""):
    if not isinstance(nodes, list):
        return

    for node in nodes:
        if not isinstance(node, dict):
            continue

        item_id = node.get("id")
        if not item_id:
            continue

        flat_node = dict(node)
        flat_node["_parent_id"] = str(node.get("parent_id") or
parent_id or "")
        yield flat_node

        children = node.get("areas") or []
        yield from self._iter_area_nodes(children,
parent_id=str(item_id))

def _save_dictionaries(self, payload):
    created_count = 0
    updated_count = 0

    if not isinstance(payload, dict):
        return created_count, updated_count

    for dictionary_name, items in payload.items():
        for item in self._iter_dictionary_items(items):
            item_id = str(item.get("id") or "")
            name = str(item.get("name") or item.get("text") or "")
            if not item_id or not name:
                continue

            defaults = {
                "name": name,
                "parent_id": str(item.get("parent_id") or ""),
                "raw_json": item,
            }
            _, created = HhDictionaryItem.objects.update_or_create(
                dictionary=str(dictionary_name),
                item_id=item_id,
                defaults=defaults,
            )
            created_count += int(created)
            updated_count += int(not created)

```



```

        return created_count, updated_count

def _iter_dictionary_items(self, items):
    if not isinstance(items, list):
        return

    for item in items:
        if not isinstance(item, dict):
            continue

        yield item

        children = item.get("items") or []
        if isinstance(children, list):
            yield from self._iter_dictionary_items(children)

```

29. vacancies/models.py

```

from django.db import models
from urllib.parse import quote

class VacancyQuerySet(models.QuerySet):
    def visible_for_ru(self):
        return
self.filter(country="Россия").exclude(is_moderator_deleted=True)

class Vacancy(models.Model):
    class WorkFormat(models.TextChoices):
        REMOTE = "remote", "Удалённо"
        HYBRID = "hybrid", "Гибрид"
        ONSITE = "onsite", "На месте"

    external_id = models.CharField(max_length=64)
    # Источник выводится из "полезной" нагрузки external_id/raw.
    title = models.CharField(max_length=255)
    company = models.CharField(max_length=255, blank=True)

    # нормализованное отношение с работодателем (может быть обнулено для
    устаревших строк)
    employer = models.ForeignKey(
        'Employer',
        null=True,
        blank=True,
        on_delete=models.SET_NULL,
        related_name='vacancies'
    )

    country = models.CharField(max_length=128)
    region = models.CharField(max_length=128, blank=True)
    experience_id = models.CharField(max_length=32, blank=True)
    experience_name = models.CharField(max_length=128, blank=True)
    # Normalized salary fields parsed from `raw_json['salary']`
    salary_from = models.IntegerField(null=True, blank=True)
    salary_to = models.IntegerField(null=True, blank=True)
    salary_currency = models.CharField(max_length=16, blank=True)
    # Flattened key skills for efficient text search
    key_skills_text = models.TextField(blank=True)
    # Full-text description fields from external payload.
    # Stored separately so the list view never needs to read them.
    description = models.TextField(blank=True)
    branded_description = models.TextField(blank=True)
    work_format = models.CharField(
        max_length=16,

```

```

        choices=WorkFormat.choices,
        default=WorkFormat.ONSITE,
    )
    is_remote = models.BooleanField(default=False)
    is_hybrid = models.BooleanField(default=False)
    is_onsite = models.BooleanField(default=True)

    # Schedule / employment / label fields parsed from raw_json
    schedule_id = models.CharField(max_length=32, blank=True,
db_index=True)
    schedule_name = models.CharField(max_length=128, blank=True)
    employment_id = models.CharField(max_length=32, blank=True,
db_index=True)
    employment_name = models.CharField(max_length=128, blank=True)
    employment_form_id = models.CharField(max_length=32, blank=True,
db_index=True)
    employment_form_name = models.CharField(max_length=128, blank=True)
    is_internship = models.BooleanField(default=False, db_index=True)
    accept_temporary = models.BooleanField(default=False, db_index=True)
    accept_incomplete_resumes = models.BooleanField(default=False)
    accept_kids = models.BooleanField(default=False)

    url = models.URLField(max_length=500, blank=True)
    published_at = models.DateTimeField()
    raw_json = models.JSONField(default=dict)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    # Вакансии, созданные на сайте (работодатель заполняет форму на этом
    сайте)
    created_by = models.ForeignKey(
        'auth.User',
        null=True, blank=True,
        on_delete=models.SET_NULL,
        related_name='created_vacancies',
    )
    is_active = models.BooleanField(default=True, db_index=True)
    # Софт-удаление модератором: вакансия скрыта от всех, но
    # быть восстановлено администратором через модератора панели отчета.
    is_moderator_deleted = models.BooleanField(default=False,
db_index=True)

    # Extended employer-form fields
    employee_type = models.CharField('Тип сотрудника', max_length=16,
blank=True) # permanent / temporary
    contract_labor = models.BooleanField('Трудовой договор',
default=False)
    contract_gpc = models.BooleanField('Договор ГПХ', default=False)
    work_schedule = models.CharField('График', max_length=32,
blank=True) # 5/2, 6/1, По выходным и т.д.
    hours_per_day = models.CharField('Часов в день', max_length=8,
blank=True)
    has_night_shifts = models.BooleanField('Ночные смены',
default=False)
    salary_gross = models.BooleanField('До вычета налогов',
default=True)
    salary_period = models.CharField('Период зарплаты',
max_length=16, blank=True) # month / project
    payment_frequency = models.CharField('Частота выплат', max_length=16,
blank=True)
    work_address = models.CharField('Адрес работы', max_length=500,
blank=True)
    hide_address = models.BooleanField('Скрыть адрес',

```

```

default=False)
    address_comment = models.CharField('Комментарий к адресу',
max_length=255, blank=True)
    # Site-created vacancy location & contact
    lat = models.FloatField('Широта', null=True,
blank=True)
    lon = models.FloatField('Долгота', null=True,
blank=True)
    contact_phone = models.CharField('Контактный телефон',
max_length=30, blank=True)
    metro_station_name = models.CharField('Станция метро',
max_length=128, blank=True)
    metro_line_color = models.CharField('Цвет линии метро',
max_length=16, blank=True)
    metro_line_name = models.CharField('Линия метро',
max_length=128, blank=True)
    metro_city_id = models.CharField('ID города метро',
max_length=10, blank=True)
    benefits_text = models.TextField('Льготы и преимущества',
blank=True)
    requirements_text = models.TextField('Требования', blank=True)
    working_conditions_text = models.TextField('Условия работы',
blank=True)

    objects = VacancyQuerySet.as_manager()

    class Meta:
        unique_together = ("external_id",)
        ordering = ("-published_at",)

    def __str__(self):
        return self.title

    @property
    def employer_logo_url(self):
        """Return a URL for the employer logo.
        Priority: linked Employer.logo_url → created_by manager's
company_logo → raw_json payload.
        """
        if self.employer:
            logo = getattr(self.employer, 'logo_url', None)
            if logo:
                return logo
            logo = getattr(self.employer, 'employer_logo_url', None)
            if logo:
                return logo
        # site-created vacancy: check manager's uploaded company_logo
        if self.created_by_id:
            try:
                from accounts.models import Manager
                mgr =
Manager.objects.filter(user_id=self.created_by_id).first()
                if mgr and mgr.company_logo:
                    return mgr.company_logo.url
                # fall back to manager's personal avatar so the employer
card
placeholder
                # always has a recognisable image instead of a letter
                if mgr and mgr.avatar:
                    return mgr.avatar.url
                # also check applicant record (avatar is stored there as
primary)
                from accounts.models import Applicant

```

```

        appl =
Applicant.objects.filter(user_id=self.created_by_id).first()
        if appl and appl.avatar:
            return appl.avatar.url
        except Exception:
            pass
        # fallback to legacy raw_json stored on Vacancy
        emp = self.raw_json.get('employer') if isinstance(self.raw_json,
dict) else None
        if not isinstance(emp, dict):
            return None
        logos = emp.get('logo_urls') or emp.get('logo')
        if isinstance(logos, dict):
            return logos.get('original') or logos.get('240') or
logos.get('90') or None
        if isinstance(logos, str):
            return logos
        # Absolute fallback: local inline SVG avatar (no external network
dependency).
        name = (self.company or self.title or "Company").strip() or
"Company"
        letters = "".join([w[:1] for w in name.split()[:2]]).upper() or
name[:1].upper()
        svg = (
            "<svg xmlns='http://www.w3.org/2000/svg' width='256'
height='256' viewBox='0 0 256 256'>"
            "<defs><linearGradient id='g' x1='0' y1='0' x2='1' y2='1'>"
            "<stop offset='0%' stop-color='#C2692A'>"
            "<stop offset='100%' stop-color='#7D3F1C'>"
            "</linearGradient></defs>"
            "<rect width='256' height='256' rx='36' fill='url(#g)'>"
            f"<text x='128' y='145' text-anchor='middle' font-
family='Inter,Arial,sans-serif' "
            "font-size='92' font-weight='700' fill='#ffffff'>"
            f"{letters}</text></svg>"
        )
        return "data:image/svg+xml;utf8," + quote(svg, safe=":/#?&=,+-.
_~!$'()*@")

    @property
    def work_formats_display(self):
        labels = []
        if self.is_remote:
            labels.append(self.WorkFormat.REMOTE.label)
        if self.is_hybrid:
            labels.append(self.WorkFormat.HYBRID.label)
        if self.is_onsite:
            labels.append(self.WorkFormat.ONSITE.label)
        return ", ".join(labels) if labels else
self.get_work_format_display()

    @property
    def work_formats_list(self):
        """Return work formats as a list of labels for templates."""
        display = self.work_formats_display or ""
        return [s.strip() for s in display.split(",") if s.strip()]

    @property
    def is_hh(self):
        """True for any imported external vacancy, not site-created."""
        return self.created_by_id is None and bool(self.url)

class Employer(models.Model):

```

```

    """Normalized employer entity storing IDs, logos and ratings from
multiple sources."""
    hh_id = models.CharField(max_length=64, unique=True, null=True,
blank=True)
    name = models.CharField(max_length=255)
    logo_url = models.URLField(blank=True)
    # External rating fields (legacy-compatible).
    hh_rating = models.FloatField(blank=True, null=True)
    rating_raw = models.CharField(max_length=128, blank=True)
    rating_updated_at = models.DateTimeField(blank=True, null=True)

    # Additional external rating fields (legacy-compatible).
    dreamjob_rating = models.FloatField(blank=True, null=True)
    dreamjob_rating_raw = models.CharField(max_length=128, blank=True)
    dreamjob_rating_updated_at = models.DateTimeField(blank=True,
null=True)
    raw = models.JSONField(default=dict, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ("name",)

    def __str__(self):
        return self.name

    @property
    def employer_logo_url(self):
        """Return employer logo using explicit `logo_url` if present,
otherwise try to extract from stored `raw` payload.
"""
        # explicit stored URL has priority
        if self.logo_url:
            return self.logo_url
        # fall back to raw payload saved on Employer
        emp = self.raw if isinstance(self.raw, dict) else None
        if not isinstance(emp, dict):
            return None
        # Some providers expose logos under 'logo_urls' or 'logo'
        logos = emp.get('logo_urls') or emp.get('logo')
        if isinstance(logos, dict):
            return logos.get('original') or logos.get('240') or
logos.get('90') or None
        if isinstance(logos, str):
            return logos
        return None

class HhArea(models.Model):
    area_id = models.CharField(max_length=32, unique=True)
    name = models.CharField(max_length=255)
    parent_id = models.CharField(max_length=32, blank=True)
    raw_json = models.JSONField(default=dict)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ("name",)

    def __str__(self):
        return self.name

class HhDictionaryItem(models.Model):
    dictionary = models.CharField(max_length=64)
    item_id = models.CharField(max_length=64)

```

```

name = models.CharField(max_length=255)
parent_id = models.CharField(max_length=64, blank=True)
raw_json = models.JSONField(default=dict)
updated_at = models.DateTimeField(auto_now=True)

class Meta:
    unique_together = ("dictionary", "item_id")
    ordering = ("dictionary", "name")

def __str__(self):
    return f"{self.dictionary}: {self.name}"

class Review(models.Model):
    """Simple persistent review model for testing UI.

    This model is intentionally minimal: it attaches to a Vacancy and
    stores
    an author name and free-text content with a timestamp.
    """
    vacancy = models.ForeignKey('Vacancy', on_delete=models.CASCADE,
related_name='reviews')
    author = models.CharField(max_length=128, blank=True)
    text = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ('-created_at',)

    def __str__(self):
        return f"Review @{self.vacancy.external_id} by {self.author or
'anon'}"

class Bookmark(models.Model):
    """Applicant bookmarks / saved vacancies."""
    user = models.ForeignKey('auth.User', on_delete=models.CASCADE,
related_name='bookmarks')
    vacancy = models.ForeignKey('Vacancy', on_delete=models.CASCADE,
related_name='bookmarked_by')
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ('user', 'vacancy')
        ordering = ('-created_at',)
        verbose_name = 'Закладка'
        verbose_name_plural = 'Закладки'

    def __str__(self):
        return f"{self.user_id} → {self.vacancy_id}"

class VacancyView(models.Model):
    """Track when a logged-in user views a vacancy detail page."""
    user = models.ForeignKey('auth.User', on_delete=models.CASCADE,
related_name='vacancy_views')
    vacancy = models.ForeignKey('Vacancy', on_delete=models.CASCADE,
related_name='views')
    viewed_at = models.DateTimeField(auto_now=True) # updated on each
revisit

    class Meta:
        unique_together = ('user', 'vacancy')
        ordering = ('-viewed_at',)
        verbose_name = 'Просмотр вакансии'
        verbose_name_plural = 'Просмотры вакансий'

```

```

def __str__(self):
    return f"{self.user_id} viewed {self.vacancy_id}"

class VacancyReport(models.Model):
    """A user complaint about a site-created vacancy."""
    REASON_SCAM = 'scam'
    REASON_SPAM = 'spam'
    REASON_MISLEADING = 'misleading'
    REASON_SUSPICIOUS = 'suspicious_conditions'
    REASON_OTHER = 'other'
    REASON_CHOICES = [
        (REASON_SCAM, 'Подозрение на мошенничество'),
        (REASON_SPAM, 'Спам или реклама'),
        (REASON_MISLEADING, 'Некорректное описание вакансии'),
        (REASON_SUSPICIOUS, 'Подозрительные условия работы'),
        (REASON_OTHER, 'Другое'),
    ]

    SELF_STATUS_NEW = 'new'
    SELF_STATUS_IN_WORK = 'in_work'
    SELF_STATUS_DONE = 'done'
    SELF_STATUS_CHOICES = [
        (SELF_STATUS_NEW, 'Новая'),
        (SELF_STATUS_IN_WORK, 'В работе'),
        (SELF_STATUS_DONE, 'Обработано'),
    ]

    vacancy = models.ForeignKey('Vacancy', on_delete=models.CASCADE,
related_name='reports', verbose_name='Вакансия')
    user = models.ForeignKey('auth.User', on_delete=models.CASCADE,
related_name='vacancy_reports', verbose_name='Пользователь')
    reason_code = models.CharField('Причина', max_length=64,
choices=REASON_CHOICES)
    reason_text = models.TextField('Комментарий', blank=True)
    self_status = models.CharField('Статус', max_length=16,
choices=SELF_STATUS_CHOICES, default=SELF_STATUS_NEW)
    moderator_note = models.TextField('Заметка модератора', blank=True)
    reviewed_by = models.ForeignKey(
        'auth.User',
        null=True,
        blank=True,
        on_delete=models.SET_NULL,
        related_name='reviewed_vacancy_reports',
    )
    reviewed_at = models.DateTimeField('Проверено', null=True,
blank=True)
    created_at = models.DateTimeField('Создано', auto_now_add=True)
    updated_at = models.DateTimeField('Обновлено', auto_now=True)

    class Meta:
        ordering = ('-created_at',)
        constraints = [
            models.UniqueConstraint(fields=['user', 'vacancy'],
name='uniq_user_vacancy_report'),
        ]
        verbose_name = 'Жалоба на вакансию'
        verbose_name_plural = 'Жалобы на вакансии'

    def __str__(self):
        return f"Report #{self.pk}: user={self.user_id}
vacancy={self.vacancy_id}"

```

```

class VacancyModerationState(models.Model):
    """Moderator's personal review state for a vacancy (card-level, not
    per report)."""
    STATUS_NEW = 'new'
    STATUS_IN_WORK = 'in_work'
    STATUS_WAITING = 'waiting'
    STATUS_CHOICES = [
        (STATUS_NEW, 'Новая'),
        (STATUS_IN_WORK, 'В работе'),
        (STATUS_WAITING, 'Ожидание'),
    ]

    vacancy = models.ForeignKey('Vacancy', on_delete=models.CASCADE,
    related_name='moderation_states')
    moderator = models.ForeignKey('auth.User', on_delete=models.CASCADE,
    related_name='vacancy_moderation_states')
    status = models.CharField(max_length=16, choices=STATUS_CHOICES,
    default=STATUS_NEW)
    note = models.TextField(blank=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ('-updated_at',)
        constraints = [
            models.UniqueConstraint(fields=['vacancy', 'moderator'],
    name='uniq_vacancy_moderator_state'),
        ]
        verbose_name = 'Состояние модерации вакансии'
        verbose_name_plural = 'Состояния модерации вакансий'

    def __str__(self):
        return f"VacancyState #{self.pk}: vacancy={self.vacancy_id}
    moderator={self.moderator_id}"

    def _moderator_deletion_photo_path(instance, filename):
        return f"moderator_reports/{instance.report_id}/{filename}"

class ModeratorDeletionReport(models.Model):
    """A record of a vacancy soft-deletion performed by a moderator.

    Snapshot fields preserve essential vacancy information so the report
    stays
    readable (and the PDF stays accurate) even if the underlying Vacancy
    row
    is later hard-deleted elsewhere.
    """
    vacancy = models.ForeignKey(
        'Vacancy',
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='moderator_deletions',
    )
    moderator = models.ForeignKey(
        'auth.User',
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='moderator_deletion_reports',
    )
    manager = models.ForeignKey(
        'auth.User',
        on_delete=models.SET_NULL,
        null=True, blank=True,

```



```

        related_name='deleted_vacancy_reports',
        help_text='Менеджер, создавший удалённую вакансию',
    )
    reason = models.TextField('Причина удаления')

    # Snapshot fields captured at the moment of deletion.
    vacancy_title = models.CharField(max_length=255, blank=True)
    vacancy_company = models.CharField(max_length=255, blank=True)
    vacancy_description = models.TextField(blank=True)
    vacancy_external_id = models.CharField(max_length=64, blank=True)
    manager_full_name = models.CharField(max_length=255, blank=True)
    manager_email = models.CharField(max_length=255, blank=True)
    moderator_full_name = models.CharField(max_length=255, blank=True)
    moderator_email = models.CharField(max_length=255, blank=True)
    reports_count = models.PositiveIntegerField(default=0)
    dominant_reason_code = models.CharField(max_length=64, blank=True)
    dominant_reason_label = models.CharField(max_length=128, blank=True)

    # Restoration bookkeeping.
    is_restored = models.BooleanField(default=False, db_index=True)
    restored_by = models.ForeignKey(
        'auth.User',
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='restored_vacancy_reports',
    )
    restored_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ('-created_at',)
        verbose_name = 'Отчёт об удалении вакансии'
        verbose_name_plural = 'Отчёты об удалении вакансий'

    def __str__(self):
        return f"DeletionReport #{self.pk}: '{self.vacancy_title}' by {self.moderator_full_name}"

class ModeratorDeletionPhoto(models.Model):
    """Photographic evidence attached to a moderator deletion report."""
    report = models.ForeignKey(
        ModeratorDeletionReport,
        on_delete=models.CASCADE,
        related_name='photos',
    )
    image = models.ImageField(upload_to=_moderator_deletion_photo_path)
    order = models.PositiveSmallIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ('order', 'id')
        verbose_name = 'Фото-доказательство модератора'
        verbose_name_plural = 'Фото-доказательства модератора'

    def __str__(self):
        return f"Photo #{self.pk} for report {self.report_id}"

```

30. vacancies/new_test.py

```

"""
Tests for the vacancies app.

Coverage:

```

```

- vacancy_branded_frame_view (iframe HTML, X-Frame-Options, 404)
- vacancy_description_api      (data return, pending trigger, 404)
- backfill_descriptions_task  (dedup via cache, limit, skip in-flight)
- fetch_vacancy_description   (saves fields, no-hh-id, not-found)
- VacancyDetailView           (200, branded branch, plain branch)
"""

```

```

import json
from io import BytesIO
from unittest.mock import MagicMock, patch
from datetime import datetime, timezone

from django.test import TestCase, Client
from django.urls import reverse
from django.core.cache import cache

from vacancies.models import Vacancy, Employer

# — helpers

```

```

def make_vacancy(
    external_id='test-1',
    title='Test Vacancy',
    description='',
    branded_description='',
    raw_json=None,
):
    """Create a minimal Vacancy row suitable for tests."""
    return Vacancy.objects.create(
        external_id=external_id,
        title=title,
        company='Test Corp',
        country='Россия',
        url='https://hh.ru/vacancy/123',
        published_at=datetime(2026, 1, 1, tzinfo=timezone.utc),
        description=description,
        branded_description=branded_description,
        raw_json=raw_json or {'id': '123456'},
    )

```

```

# — vacancy_branded_frame_view

```

```

class BrandedFrameViewTests(TestCase):

    def setUp(self):
        self.client = Client()
        self.vacancy = make_vacancy(
            branded_description='<h1>Hello Employer</h1>',
        )

    def _url(self, pk=None):
        return reverse('vacancy-branded-frame', args=[pk or self.vacancy.pk])

    def test_returns_200_with_content(self):
        resp = self.client.get(self._url())
        self.assertEqual(resp.status_code, 200)
        self.assertIn(b'Hello Employer', resp.content)

    def test_content_type_is_html(self):
        resp = self.client.get(self._url())

```

```

        self.assertIn('text/html', resp['Content-Type'])

    def test_xframe_options_absent(self):
        """@xframe_options_exempt must remove the deny header."""
        resp = self.client.get(self._url())
        self.assertNotIn('X-Frame-Options', resp)

    def test_postmessage_script_present(self):
        resp = self.client.get(self._url())
        self.assertIn(b'postMessage', resp.content)
        self.assertIn(b'branded-height', resp.content)

    def test_falls_back_to_description(self):
        v = make_vacancy(
            external_id='plain-1',
            description='<p>Plain text desc</p>',
            branded_description='',
        )
        resp = self.client.get(reverse('vacancy-branded-frame',
args=[v.pk]))
        self.assertIn(b'Plain text desc', resp.content)

    def test_404_for_missing_vacancy(self):
        resp = self.client.get(reverse('vacancy-branded-frame',
args=[99999]))
        self.assertEqual(resp.status_code, 404)

    def test_background_color_set(self):
        """iframe body must have light background so dark employer text
is readable."""
        resp = self.client.get(self._url())
        self.assertIn(b'#faf8f5', resp.content)

# — vacancy_description_api

```

```

class DescriptionApiTests(TestCase):

    def setUp(self):
        self.client = Client()

    def _url(self, pk):
        return reverse('vacancy-description-api', args=[pk])

    def test_returns_description_when_filled(self):
        v = make_vacancy(
            external_id='api-1',
            description='<p>Some desc</p>',
            branded_description='<b>brand</b>',
        )
        resp = self.client.get(self._url(v.pk))
        self.assertEqual(resp.status_code, 200)
        data = resp.json()
        self.assertFalse(data['pending'])
        self.assertIn('Some desc', data['description'])
        self.assertIn('brand', data['branded'])

    def test_returns_pending_when_empty(self):
        v = make_vacancy(external_id='api-2')
        with
patch('vacancies.tasks.fetch_vacancy_description.apply_async') as
mock_task:
            resp = self.client.get(self._url(v.pk))

```

```

        self.assertEqual(resp.status_code, 200)
        data = resp.json()
        self.assertTrue(data['pending'])
        mock_task.assert_called_once_with(args=[v.id], priority=9)

    def test_404_for_missing_vacancy(self):
        resp = self.client.get(self._url(99999))
        self.assertEqual(resp.status_code, 404)

    def test_returns_unavailable_when_sentinel(self):
        v = make_vacancy(external_id='api-unav',
description='__unavailable__')
        resp = self.client.get(self._url(v.pk))
        data = resp.json()
        self.assertFalse(data['pending'])
        self.assertTrue(data['unavailable'])
        self.assertEqual(data['description'], '')

    def test_only_description_filled_not_pending(self):
        v = make_vacancy(external_id='api-3', description='<p>desc
only</p>')
        resp = self.client.get(self._url(v.pk))
        data = resp.json()
        self.assertFalse(data['pending'])

```

— backfill_descriptions_task

```

class BackfillTaskTests(TestCase):

    def setUp(self):
        cache.clear()

    def tearDown(self):
        cache.clear()

    def test_queues_vacancies_without_description(self):
        v1 = make_vacancy(external_id='bf-1')
        v2 = make_vacancy(external_id='bf-2')
        with
patch('vacancies.tasks.fetch_vacancy_description.apply_async') as
mock_task:
            from vacancies.tasks import backfill_descriptions_task
            result = backfill_descriptions_task()
            self.assertIn('queued:2', result)
            self.assertEqual(mock_task.call_count, 2)

    def test_skips_filled_vacancies(self):
        make_vacancy(external_id='bf-filled', description='<p>already
here</p>')
        with
patch('vacancies.tasks.fetch_vacancy_description.apply_async') as
mock_task:
            from vacancies.tasks import backfill_descriptions_task
            backfill_descriptions_task()
            mock_task.assert_not_called()

    def test_respects_limit(self):
        for i in range(10):
            make_vacancy(external_id=f'bf-limit-{i}')
        with
patch('vacancies.tasks.fetch_vacancy_description.apply_async') as
mock_task:

```

```

        from vacancies.tasks import backfill_descriptions_task
        backfill_descriptions_task(limit=3)
        self.assertEqual(mock_task.call_count, 3)

    def test_deduplication_skips_inflight(self):
        """Second call must not re-queue a vacancy already in the
        cache."""
        v = make_vacancy(external_id='bf-dedup')
        with
        patch('vacancies.tasks.fetch_vacancy_description.apply_async') as
        mock_task:
            from vacancies.tasks import backfill_descriptions_task
            backfill_descriptions_task()    # first run - queues v
            backfill_descriptions_task()    # second run - v still empty
        but in cache
            # Should only have been dispatched once
            self.assertEqual(mock_task.call_count, 1)

# — fetch_vacancy_description task


---



def _make_hh_response(description='<p>desc</p>', branded=''):
    """Build a fake urlopen response with HH JSON payload."""
    payload = json.dumps({
        'id': '123456',
        'description': description,
        'branded_description': branded,
        'key_skills': [{'name': 'Python'}, {'name': 'Django'}],
    }).encode()
    mock_resp = MagicMock()
    mock_resp.read.return_value = payload
    mock_resp.__enter__ = lambda s: s
    mock_resp.__exit__ = MagicMock(return_value=False)
    return mock_resp

class FetchVacancyDescriptionTaskTests(TestCase):

    def test_saves_description_and_branded(self):
        v = make_vacancy(external_id='fvd-1')
        with patch('vacancies.tasks.urlopen',
        return_value=_make_hh_response(
            description='<p>Full desc</p>',
            branded='<div class="brand">Brand</div>',
        )):
            from vacancies.tasks import fetch_vacancy_description
            result = fetch_vacancy_description(v.id)

            v.refresh_from_db()
            self.assertIn('Full desc', v.description)
            self.assertIn('Brand', v.branded_description)
            self.assertIn('ok:desc=', result)

    def test_saves_key_skills(self):
        v = make_vacancy(external_id='fvd-2')
        with patch('vacancies.tasks.urlopen',
        return_value=_make_hh_response()):
            from vacancies.tasks import fetch_vacancy_description
            fetch_vacancy_description(v.id)
            v.refresh_from_db()
            self.assertIn('Python', v.key_skills_text)
            self.assertIn('Django', v.key_skills_text)

    def test_returns_not_found_for_missing_vacancy(self):

```

```

        from vacancies.tasks import fetch_vacancy_description
        result = fetch_vacancy_description(99999)
        self.assertEqual(result, 'not_found')

    def test_marks_unavailable_on_403(self):
        v = make_vacancy(external_id='fvd-403')
        from urllib.error import HTTPError
        http_err = HTTPError(url='', code=403, msg='Forbidden',
hdrs=None, fp=None)
        with patch('vacancies.tasks.urlopen', side_effect=http_err):
            from vacancies.tasks import fetch_vacancy_description
            result = fetch_vacancy_description(v.id)
            self.assertEqual(result, 'skipped:403')
        v.refresh_from_db()
        self.assertEqual(v.description, '__unavailable__')

    def test_returns_no_hh_id_when_raw_json_empty(self):
        v = make_vacancy(external_id='fvd-3', raw_json={})
        # Force external_id to empty so hh_id resolution yields nothing
        Vacancy.objects.filter(pk=v.pk).update(external_id='',
raw_json={})
        from vacancies.tasks import fetch_vacancy_description
        result = fetch_vacancy_description(v.id)
        self.assertEqual(result, 'no_hh_id')

```

— VacancyDetailView

```

class VacancyDetailViewTests(TestCase):

    def setUp(self):
        self.client = Client()

    def test_detail_page_200(self):
        v = make_vacancy(external_id='detail-1',
description='<p>hello</p>')
        resp = self.client.get(reverse('vacancy-detail', args=[v.pk]))
        self.assertEqual(resp.status_code, 200)

    def test_detail_page_404_for_missing(self):
        resp = self.client.get(reverse('vacancy-detail', args=[99999]))
        self.assertEqual(resp.status_code, 404)

    def test_branded_iframe_rendered_when_branded_present(self):
        v = make_vacancy(
            external_id='detail-branded',
            branded_description='<div>brand</div>',
        )
        resp = self.client.get(reverse('vacancy-detail', args=[v.pk]))
        self.assertEqual(resp.status_code, 200)
        self.assertContains(resp, 'branded-frame')
        # SSR iframe src should point to the branded frame URL
        self.assertContains(resp, f'/{v.pk}/branded/')

    def test_plain_description_rendered_when_no_branded(self):
        v = make_vacancy(
            external_id='detail-plain',
            description='<p>Plain description</p>',
        )
        resp = self.client.get(reverse('vacancy-detail', args=[v.pk]))
        self.assertContains(resp, 'vd-description')

    def test_skeleton_shown_when_both_empty(self):

```

```

        v = make_vacancy(external_id='detail-empty')
        resp = self.client.get(reverse('vacancy-detail', args=[v.pk]))
        self.assertContains(resp, 'desc-skeleton')

```

31. vacancies/rating.py

```

from typing import Any, Optional, Tuple

```

```

def extract_rating_candidate(payload: Any) -> Optional[str]:
    """Return first candidate rating found in payload dict."""
    if not isinstance(payload, dict):
        return None
    for k in ('hh_rating', 'rating', 'rating_avg', 'rating_value',
'rating_raw'):
        v = payload.get(k)
        if v:
            return v
    hh = payload.get('hh')
    if isinstance(hh, dict):
        for k in ('rating', 'ratingValue', 'rating_avg', 'rating_value',
'hh_rating', 'rating_raw'):
            v = hh.get(k)
            if v:
                return v
    return None

def parse_rating(value: Any) -> Optional[float]:
    """Parse numeric rating (0..5) from various input types."""
    if value is None:
        return None
    try:
        v = float(value) if isinstance(value, (int, float)) else
float(str(value).strip().replace(',', '.'))
    except Exception:
        return None
    return v if 0 <= v <= 5 else None

def _positive(value) -> Optional[float]:
    """Parse rating, treating 0 as missing."""
    v = parse_rating(value)
    return v if v and v > 0 else None

def compute_vacancy_rating(vacancy) -> Tuple[str, Optional[float],
Optional[str]]:
    """Compute display rating for a vacancy.

    Returns (display_string, numeric_value_or_None,
raw_candidate_or_None).
    """
    emp = getattr(vacancy, 'employer', None)
    if not emp:
        return '', None, None

    hh = _positive(getattr(emp, 'hh_rating', None))
    dj = _positive(getattr(emp, 'dreamjob_rating', None))

    if hh and dj:
        avg = round((hh + dj) / 2, 2)
        raw = f"hh:{getattr(emp, 'rating_raw', '')} or

```

```

''}|dj:{getattr(emp, 'dreamjob_rating_raw', '') or ''}"
    return f"{avg:.1f}", avg, raw[:128]

    if hh:
        return f"{hh:.1f}", hh, str(getattr(emp, 'hh_rating', ''))[:128]
    if dj:
        return f"{dj:.1f}", dj, str(getattr(emp, 'dreamjob_rating',
''))[:128]

    # Fallback: employer.raw payload
    candidate = extract_rating_candidate(getattr(emp, 'raw', None) or {})
    parsed = parse_rating(candidate)
    if parsed is not None:
        return f"{parsed:.1f}", parsed, str(candidate)[:128] if candidate
    else None

    return '', None, None

```

32. vacancies/tasks.py

```

import json
import time
from datetime import timedelta
from urllib.request import Request, urlopen
from urllib.error import HTTPError
import requests

from celery import shared_task
from celery.exceptions import Retry
from celery.exceptions import MaxRetriesExceededError
from django.core.management import call_command
from django.db import close_old_connections
from django.conf import settings as dj_settings
from django.core.cache import cache
import logging
from vacancies.hh_client import hh_openapi_headers

logger = logging.getLogger(__name__)
INGEST_GLOBAL_LOCK_KEY = 'lock:vacancy_ingest_global'
INGEST_LOCK_TTL_SEC = 20 * 60

def _close_connections():
    """Safely close stale DB connections in worker process."""
    try:
        close_old_connections()
    except Exception:
        pass

def _run_command(name, **kwargs):
    payload = {k: v for k, v in kwargs.items() if v is not None and v !=
''}
    call_command(name, **payload)

def _acquire_lock(key, timeout=INGEST_LOCK_TTL_SEC):
    return bool(cache.add(key, 1, timeout=timeout))

@shared_task(bind=True, max_retries=2, default_retry_delay=30)
def fetch_hh_task(self, pages=5, per_page=100, text='', texts='',
    use_default_texts=False, area=None):
    """Fetch vacancies via the `fetch_hh` management command.

```



```

Ratings are fetched lazily on demand (when user opens a vacancy).
"""
    _close_connections()
    hh_lock_key = 'lock:fetch_hh_task'
    if not _acquire_lock(hh_lock_key):
        return 'skipped:already_running'
    has_global_lock = False
    try:
        if not _acquire_lock(INGEST_GLOBAL_LOCK_KEY):
            # Another heavy ingest is writing to SQLite right now.
            # Retry shortly instead of dropping this cycle.
            raise self.retry(countdown=20)
        has_global_lock = True
        kwargs = {'pages': pages, 'per_page': per_page, 'text': text,
'texts': texts, 'area': area}
        if use_default_texts:
            kwargs['use_default_texts'] = True
        _run_command('fetch_hh', **kwargs)
    except Retry:
        raise
    except MaxRetriesExceededError:
        logger.warning('Импорт НН пропущен: очередь занята слишком
долго')
        return 'skipped:ingest_busy_timeout'
    except Exception as exc:
        logger.exception('Ошибка задачи импорта НН')
        raise self.retry(exc=exc)
    finally:
        if has_global_lock:
            cache.delete(INGEST_GLOBAL_LOCK_KEY)
            cache.delete(hh_lock_key)
            _close_connections()
    return 'ok'

@shared_task(bind=True, max_retries=2, default_retry_delay=30)
def fetch_trudvsem_task(self, limit=200, offset=None, text=''):
    _close_connections()
    lock_key = 'lock:fetch_trudvsem_task'
    if not _acquire_lock(lock_key, timeout=15 * 60):
        return 'skipped:already_running'
    has_global_lock = False
    try:
        if not _acquire_lock(INGEST_GLOBAL_LOCK_KEY):
            # Another heavy ingest is writing to SQLite right now.
            # Retry shortly instead of dropping this cycle.
            raise self.retry(countdown=20)
        has_global_lock = True
        kwargs = {
            'limit': max(1, int(limit or 200)),
            'offset': max(0, int(offset)) if offset is not None else
None,
            'text': text,
        }
        _run_command('fetch_trudvsem', **kwargs)
    except Retry:
        raise
    except MaxRetriesExceededError:
        logger.warning('Импорт Работа России пропущен: очередь занята
слишком долго')
        return 'skipped:ingest_busy_timeout'
    except Exception as exc:

```

```

        logger.exception('Ошибка задачи импорта Работа России')
        raise self.retry(exc=exc)
    finally:
        if has_global_lock:
            cache.delete(INGEST_GLOBAL_LOCK_KEY)
        cache.delete(lock_key)
        _close_connections()
    return 'ok'

@shared_task(bind=True, max_retries=3, default_retry_delay=60,
ignore_result=False)
def fetch_vacancy_description(self, vacancy_id):
    """Fetch full description for a single vacancy from the HH detail
    endpoint.

    Saves to dedicated `description` and `branded_description` fields so
    the
    list view never has to touch these large text blobs.
    Called automatically by fetch_hh_task for newly created vacancies.
    """
    from .models import Vacancy # local import avoids circular import at
    module load

    _close_connections()
    try:
        vacancy = Vacancy.objects.get(id=vacancy_id)
    except Vacancy.DoesNotExist:
        logger.warning('Описание HH: вакансия id=%s не найдена',
vacancy_id)
        return 'not_found'

    hh_id = (vacancy.raw_json or {}).get('id') or vacancy.external_id
    if not hh_id:
        logger.warning('Описание HH: у вакансии id=%s нет HH id',
vacancy_id)
        return 'no_hh_id'

    url = f'https://api.hh.ru/vacancies/{hh_id}'
    try:
        req = Request(url, headers=hh_openapi_headers())
        with urlopen(req, timeout=15) as resp:
            data = json.loads(resp.read().decode('utf-8'))
    except HTTPError as exc:
        if exc.code == 429:
            raise self.retry(exc=exc, countdown=60)
        if exc.code in (403, 404, 410):
            # Vacancy is archived, restricted, or deleted – no point
retrying.
            # Write a sentinel so backfill won't keep picking it up.
            logger.info('Описание HH: вакансия id=%s HTTP %s, помечена
недоступной', vacancy_id, exc.code)
            Vacancy.objects.filter(id=vacancy_id).update(
                description='__unavailable__'
            )
            return f'skipped:{exc.code}'
        raise self.retry(exc=exc)
    except Exception as exc:
        raise self.retry(exc=exc)

    vacancy.description = data.get('description', '') or ''
    vacancy.branded_description = data.get('branded_description', '') or
    ''

```

```

        vacancy.key_skills_text = ', '.join(
            s.get('name', '') for s in (data.get('key_skills') or []) if
            isinstance(s, dict)
        )
        vacancy.raw_json = {**(vacancy.raw_json or {}), **data}
        vacancy.save(update_fields=['description', 'branded_description',
                                     'key_skills_text', 'raw_json'])

    _close_connections()
    desc_len = len(vacancy.description)
    branded_len = len(vacancy.branded_description)
    logger.info('Описание HH: вакансия id=%s, desc=%d, branded=%d',
                vacancy_id, desc_len, branded_len)
    return f'ok:desc={desc_len},branded={branded_len}'

@shared_task
def backfill_descriptions_task(limit=50):
    """Periodically queue description fetches for vacancies that still
    lack them.

    Uses Django's cache to avoid re-queuing vacancies that are already
    in-flight.
    Cache key expires after 10 minutes – long enough for a task to
    complete.
    """
    from .models import Vacancy
    from django.core.cache import cache

    _close_connections()
    ids = list(
        Vacancy.objects.filter(description='')
        .values_list('id', flat=True)
        .order_by('published_at')[:limit]
    )
    queued = 0
    for i, vid in enumerate(ids):
        cache_key = f'desc_inflight_{vid}'
        if cache.get(cache_key):
            continue # already dispatched, skip
        cache.set(cache_key, 1, timeout=600) # 10-minute TTL
        fetch_vacancy_description.apply_async(args=[vid], countdown=i *
        0.5, priority=1)
        queued += 1
    logger.info('Добор описаний HH: в очереди=%d, пропущено(in-
    flight)=%d',
                queued, len(ids) - queued)
    return f'queued:{queued}'

@shared_task(bind=False, ignore_result=False)
def backup_database_task():
    """Create a rotating SQLite backup.

    Scheduled daily at 00:00 via Celery Beat. Stores files under
    settings.BACKUP_DIR with the pattern
    ``db_backup_YYYYMMDD_HHMMSS.sqlite3``.
    Once more than DB_BACKUP_MAX_COUNT (default 3) files exist, the
    oldest is
    deleted so the count stays at the limit.
    """
    import sqlite3 as _sqlite3
    from contextlib import closing as _closing

```

```

from pathlib import Path as _Path
from datetime import datetime as _dt
from django.conf import settings as _cfg

db_path = _Path(_cfg.DATABASES['default']['NAME'])
backup_dir = _Path(getattr(_cfg, 'BACKUP_DIR', db_path.parent /
'backups'))
backup_dir.mkdir(parents=True, exist_ok=True)

timestamp = _dt.now().strftime('%Y%m%d_%H%M%S')
backup_path = backup_dir / f'db_backup_{timestamp}.sqlite3'

# sqlite3.Connection.backup() uses SQLite's online backup API - safe
in WAL mode.
with _closing(_sqlite3.connect(str(db_path))) as src:
    with _closing(_sqlite3.connect(str(backup_path))) as dst:
        src.backup(dst)

# Rotation: sorted ascending (oldest first); delete until <=
max_count.
max_count = getattr(_cfg, 'DB_BACKUP_MAX_COUNT', 3)
existing = sorted(backup_dir.glob('db_backup_*.sqlite3'))
while len(existing) > max_count:
    existing[0].unlink()
    logger.info('Бэкап БД: удален старый файл %s', existing[0].name)
    existing = existing[1:]

logger.info('Бэкап БД: создан %s (всего файлов: %d)',
backup_path.name, len(existing))
return str(backup_path)

@shared_task(bind=True, max_retries=2, default_retry_delay=30)
def fetch_employer_rating(self, employer_id):
    """Background scraping of employer rating from HH and DreamJob.

    Called when the rating API finds no cached value. Scrapes both
    sources
    and persists the results so subsequent requests return instantly.
    """
    from .models import Employer
    from .management.commands.fetch_employer_details import (
        extract_rating_from_html, fetch_employer_page, _parse_rating,
    )
    from .dreamjob import (
        fetch_dreamjob_page,
        extract_rating_from_html as dj_extract,
        search_employer_links_by_name,
    )
    from .rating import _positive
    from django.utils import timezone

    _close_connections()
    try:
        emp = Employer.objects.get(id=employer_id)
    except Employer.DoesNotExist:
        return 'not_found'

    hh_val = _positive(emp.hh_rating)
    dj_val = _positive(emp.dreamjob_rating)

    # — Scrape HH employer page —————
    dj_id = None

```

```

if not hh_val:
    urls = []
    raw_emp = emp.raw if isinstance(emp.raw, dict) else {}
    alt = raw_emp.get('alternate_url') or raw_emp.get('url')
    if alt:
        urls.append(alt)
    if emp.hh_id:
        urls.append(f'https://hh.ru/employer/{emp.hh_id}')
    for u in urls:
        try:
            candidate, dj_id =
extract_rating_from_html(fetch_employer_page(u))
            parsed = _parse_rating(candidate)
            if parsed:
                hh_val = parsed
                break
        except Exception:
            pass

# — Scrape DreamJob —————
if not dj_val:
    dj_candidate = None
    if dj_id:
        try:
            dj_candidate = dj_extract(fetch_dreamjob_page(
                f'https://dreamjob.ru/employers/{dj_id}'))
        except Exception:
            pass
    if not dj_candidate:
        raw_emp = emp.raw if isinstance(emp.raw, dict) else {}
        for k in ('alternate_url', 'url', 'site'):
            v = raw_emp.get(k)
            if isinstance(v, str) and 'dreamjob.ru' in v:
                try:
                    dj_candidate = dj_extract(fetch_dreamjob_page(v))
                    if dj_candidate:
                        break
                except Exception:
                    pass
    if not dj_candidate and emp.name:
        try:
            for dj_u in search_employer_links_by_name(emp.name):
                try:
                    dj_candidate =
dj_extract(fetch_dreamjob_page(dj_u))
                    if dj_candidate:
                        break
                except Exception:
                    pass
        except Exception:
            pass
    parsed_dj = _parse_rating(dj_candidate) if dj_candidate else None
    if parsed_dj:
        dj_val = parsed_dj

# — Persist —————
now = timezone.now()
update_fields = []
if hh_val and hh_val != _positive(emp.hh_rating):
    emp.hh_rating = hh_val
    emp.rating_updated_at = now
    update_fields += ['hh_rating', 'rating_updated_at']
if dj_val and dj_val != _positive(emp.dreamjob_rating):

```

```

        emp.dreamjob_rating = dj_val
        emp.dreamjob_rating_updated_at = now
        update_fields += ['dreamjob_rating',
'dreamjob_rating_updated_at']
        if update_fields:
            emp.save(update_fields=update_fields)

    _close_connections()
    return f'hh={hh_val},dj={dj_val}'

@shared_task(bind=False, ignore_result=False)
def check_hh_vacancy_status_task(batch_size=50):
    """Soft-delete HH vacancies that are no longer available.

    Two-stage approach:
    1. TTL: instantly deactivate HH vacancies older than 35 days
       (HH max lifetime is 30 days; 5-day buffer for extensions).
    2. API check: for remaining active HH vacancies, request the HH API
       in small batches and deactivate any that return 404 / 410.
    """
    from .models import Vacancy
    from django.utils import timezone as tz

    _close_connections()

    # Stage 1 – TTL deactivation (no API needed)
    cutoff_ttl = tz.now() - timedelta(days=getattr(dj_settings,
'HH_STALE_TTL_DAYS', 35))
    expired = Vacancy.objects.filter(
        created_by__isnull=True,
        is_active=True,
    ).exclude(external_id__startswith='trudvsem-').filter(
        published_at__lt=cutoff_ttl,
    ).update(is_active=False)

    # Stage 2 – API check for newer HH vacancies (oldest first)
    candidates = list(
        Vacancy.objects.filter(
            created_by__isnull=True,
            is_active=True,
        ).exclude(external_id__startswith='trudvsem-')
        .order_by('published_at').values_list('id',
'external_id')[:batch_size]
    )

    deactivated = 0
    for vid, ext_id in candidates:
        url = f'https://api.hh.ru/vacancies/{ext_id}'
        try:
            req = Request(url, headers=hh_openapi_headers())
            with urlopen(req, timeout=10) as resp:
                resp.read() # 200 OK – vacancy is still live
        except HTTPError as exc:
            if exc.code in (404, 410):
                Vacancy.objects.filter(id=vid).update(is_active=False)
                deactivated += 1
                logger.info('HH статус: деактивирована %s (HTTP %s)',
ext_id, exc.code)
            elif exc.code == 429:
                logger.warning('HH статус: лимит запросов, ранняя
остановка проверки')
                break

```

```

        except Exception as exc:
            logger.debug('HH статус: пропуск %s, причина=%s', ext_id,
exc)
            time.sleep(0.3) # stay within HH rate limits

        _close_connections()
        logger.info(
            'HH статус: ttl_expired=%d, api_deactivated=%d',
            expired, deactivated,
        )
        return f'ttl_expired:{expired},api_deactivated:{deactivated}'

@shared_task(bind=False, ignore_result=False)
def check_trudvsem_vacancy_status_task(batch_size=100):
    """Hard-delete Trudvsem vacancies that are stale or gone on
source."""
    from .models import Vacancy
    from django.utils import timezone as tz

    _close_connections()
    ttl_days = max(1, int(getattr(dj_settings,
'FALLBACK_TRUDVSEM_TTL_DAYS', 5)))
    cutoff = tz.now() - timedelta(days=ttl_days)

    # Stage 1: hard-delete stale rows not refreshed for TTL period.
    stale_deleted, _ = Vacancy.objects.filter(
        created_by__isnull=True,
        external_id__startswith='trudvsem-',
        updated_at__lt=cutoff,
    ).delete()

    # Stage 2: source availability probe for recent rows.
    candidates = list(
        Vacancy.objects.filter(
            created_by__isnull=True,
            external_id__startswith='trudvsem-',
        )
        .order_by('updated_at')
        .values_list('id', 'url')[:batch_size]
    )

    source_deleted = 0
    for vid, url in candidates:
        target = (url or '').strip()
        if not target:
            continue
        try:
            resp = requests.get(target, timeout=10, allow_redirects=True)
            if resp.status_code == 404:
                Vacancy.objects.filter(id=vid).delete()
                source_deleted += 1
        except requests.RequestException:
            # Network issues should not delete records.
            continue
        time.sleep(0.1)

    _close_connections()
    logger.info(
        'Работа России статус: stale_deleted=%d, source_deleted=%d',
        stale_deleted, source_deleted,
    )
    return

```

```
f'stale_deleted:{stale_deleted},source_deleted:{source_deleted}'
```

33. vacancies/templatetags/__init__.py

34. vacancies/templatetags/vacancy_extras.py

```
from django import template
from django.utils import timezone

register = template.Library()

@register.filter
def ru_timesince(value):
    """Return a human-readable Russian time-ago string for a datetime."""
    if not value:
        return ''
    now = timezone.now()
    try:
        diff = now - value
    except TypeError:
        return ''

    total_seconds = int(diff.total_seconds())
    if total_seconds < 0:
        return 'только что'

    minutes = total_seconds // 60
    hours = minutes // 60
    days = hours // 24
    weeks = days // 7
    months = days // 30
    years = days // 365

    def plural(n, one, few, many):
        n = abs(n) % 100
        n1 = n % 10
        if 11 <= n <= 19:
            return many
        if n1 == 1:
            return one
        if 2 <= n1 <= 4:
            return few
        return many

    if total_seconds < 60:
        return 'только что'
    if minutes < 60:
        return f'{minutes} {plural(minutes, "минуту", "минуты", "минут")}'
    назад'
    if hours < 24:
        return f'{hours} {plural(hours, "час", "часа", "часов")} назад'
    if days < 7:
        return f'{days} {plural(days, "день", "дня", "дней")} назад'
    if weeks < 5:
        return f'{weeks} {plural(weeks, "неделю", "недели", "недель")}'
    назад'
    if months < 12:
        return f'{months} {plural(months, "месяц", "месяца", "месяцев")}'
    назад'
    return f'{years} {plural(years, "год", "года", "лет")} назад'
```



```

@register.filter
def salary_fmt(value):
    """Format a salary integer as '100 000' with thin-space thousands
    separator."""
    try:
        n = int(value)
    except (TypeError, ValueError):
        return ''
    # Format with non-breaking thin space (U+202F) as thousands separator
    formatted = f'{n:,}'.replace(',', ' '\u202f')
    return formatted

@register.simple_tag
def salary_range(salary_from, salary_to, currency=''):
    """
    Return formatted salary string.
    • If both values are equal: show only once.
    • If only one is set: prefix with 'от' / 'до'.
    • Thousands-formatted with narrow no-break space.
    """
    def fmt(v):
        try:
            n = int(v)
            return f'{n:,}'.replace(',', ' '\u202f')
        except (TypeError, ValueError):
            return ''

    cur = f' {currency}' if currency else ''

    if salary_from and salary_to:
        f = int(salary_from)
        t = int(salary_to)
        if f == t:
            return f'{fmt(f)}{cur}'
        return f'{fmt(f)} – {fmt(t)}{cur}'
    if salary_from:
        return f'от {fmt(salary_from)}{cur}'
    if salary_to:
        return f'до {fmt(salary_to)}{cur}'
    return ''

```

35. vacancies/tests_reports.py

```

from datetime import datetime, timezone
from io import BytesIO

from django.contrib.auth.models import User
from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase
from django.urls import reverse

from accounts.models import Moderator, Applicant, Administrator
from vacancies.models import (
    Vacancy, VacancyReport, VacancyModerationState,
    ModeratorDeletionReport, ModeratorDeletionPhoto,
)

def _make_png_bytes():
    """1x1 transparent PNG – valid image content for ImageField tests."""
    try:
        from PIL import Image

```

```

        buf = BytesIO()
        Image.new('RGBA', (1, 1), (0, 0, 0, 0)).save(buf, format='PNG')
        return buf.getvalue()
    except Exception:
        # Minimal valid PNG header (1x1 transparent).
        return (

b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x00\x01\x00\x00\x00\x01'

b'\x08\x06\x00\x00\x00\x1f\x15\xc4\x89\x00\x00\x00\rIDATx\x9cc\x00\x01'
        b'\x00\x00\x05\x00\x01\r\n-
\xb4\x00\x00\x00\x00IEND\xaeB`\x82'
        )

def _make_vacancy(external_id='site-vac', *, hh=False):
    url = 'https://hh.ru/vacancy/123' if hh else 'http://localhost/site-
vacancy'
    return Vacancy.objects.create(
        external_id=external_id,
        title='Vacancy',
        company='Company',
        country='Россия',
        url=url,
        published_at=datetime(2026, 1, 1, tzinfo=timezone.utc),
        raw_json={},
    )

class VacancyReportApiTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user('user1',
'user1@example.com', 'test-pass-123')
        Applicant.objects.create(user=self.user, telegram='@user1')
        self.moderator_user = User.objects.create_user('mod1',
'mod1@example.com', 'test-pass-123')
        Moderator.objects.create(user=self.moderator_user)
        self.admin_user = User.objects.create_user(
            'admin1', 'admin1@example.com', 'test-pass-123',
            is_superuser=True, is_staff=True
        )
        Administrator.objects.create(user=self.admin_user)

    def test_user_can_report_only_once(self):
        vac = _make_vacancy('site-report-1')
        self.client.force_login(self.user)

        url = reverse('vacancy-report', args=[vac.pk])
        first = self.client.post(
            url,
            data='{"reason_code":"scam","reason_text":"Suspicious"}',
            content_type='application/json',
        )
        second = self.client.post(
            url,
            data='{"reason_code":"spam","reason_text":"Spam"}',
            content_type='application/json',
        )

        self.assertEqual(first.status_code, 200)
        self.assertEqual(second.status_code, 200)
        self.assertEqual(VacancyReport.objects.filter(user=self.user,
vacancy=vac).count(), 1)

```

```

self.assertTrue(second.json().get('already_reported'))

def test_user_cannot_report_hh_vacancy(self):
    vac = _make_vacancy('hh-report-1', hh=True)
    self.client.force_login(self.user)
    resp = self.client.post(
        reverse('vacancy-report', args=[vac.pk]),
        data='{"reason_code":"scam","reason_text":"Suspicious"}',
        content_type='application/json',
    )
    self.assertEqual(resp.status_code, 400)
    self.assertEqual(resp.json().get('error'),
'hh_vacancy_not_reportable')

def test_other_reason_requires_text(self):
    vac = _make_vacancy('site-report-2')
    self.client.force_login(self.user)
    resp = self.client.post(
        reverse('vacancy-report', args=[vac.pk]),
        data='{"reason_code":"other","reason_text":""}',
        content_type='application/json',
    )
    self.assertEqual(resp.status_code, 400)
    self.assertEqual(resp.json().get('error'),
'reason_text_required')

def test_moderator_cannot_report_vacancy(self):
    vac = _make_vacancy('site-report-4')
    self.client.force_login(self.moderator_user)
    resp = self.client.post(
        reverse('vacancy-report', args=[vac.pk]),
        data='{"reason_code":"scam","reason_text":"Suspicious"}',
        content_type='application/json',
    )
    self.assertEqual(resp.status_code, 403)
    self.assertEqual(resp.json().get('error'), 'forbidden_role')

def test_admin_cannot_report_vacancy(self):
    vac = _make_vacancy('site-report-5')
    self.client.force_login(self.admin_user)
    resp = self.client.post(
        reverse('vacancy-report', args=[vac.pk]),
        data='{"reason_code":"scam","reason_text":"Suspicious"}',
        content_type='application/json',
    )
    self.assertEqual(resp.status_code, 403)
    self.assertEqual(resp.json().get('error'), 'forbidden_role')

def test_moderator_updates_self_status(self):
    vac = _make_vacancy('site-report-3')
    report = VacancyReport.objects.create(
        vacancy=vac,
        user=self.user,
        reason_code=VacancyReport.REASON_MISLEADING,
        reason_text='Wrong description',
    )
    self.client.force_login(self.moderator_user)
    resp = self.client.post(
        reverse('report-self-status', args=[report.pk]),
        data='{"self_status":"in_work","moderator_note":"Checking
now"}',
        content_type='application/json',
    )

```

```

        self.assertEqual(resp.status_code, 200)
        report.refresh_from_db()
        self.assertEqual(report.self_status,
VacancyReport.SELF_STATUS_IN_WORK)
        self.assertEqual(report.reviewed_by, self.moderator_user)

    def test_moderator_updates_vacancy_card_state(self):
        vac = _make_vacancy('site-report-6')
        self.client.force_login(self.moderator_user)
        resp = self.client.post(
            reverse('vacancy-moderation-state', args=[vac.pk]),
            data='{"status":"waiting","note":"Жду ответ от менеджера"}',
            content_type='application/json',
        )
        self.assertEqual(resp.status_code, 200)
        state = VacancyModerationState.objects.get(vacancy=vac,
moderator=self.moderator_user)
        self.assertEqual(state.status,
VacancyModerationState.STATUS_WAITING)
        self.assertEqual(state.note, 'Жду ответ от менеджера')

    def test_non_moderator_cannot_update_vacancy_card_state(self):
        vac = _make_vacancy('site-report-7')
        self.client.force_login(self.user)
        resp = self.client.post(
            reverse('vacancy-moderation-state', args=[vac.pk]),
            data='{"status":"in_work","note":"try"}',
            content_type='application/json',
        )
        self.assertEqual(resp.status_code, 403)

class ModeratorVacancyDeletionTests(TestCase):
    def setUp(self):
        self.applicant_user = User.objects.create_user('applicant',
'a@example.com', 'pw-test-123')
        Applicant.objects.create(user=self.applicant_user, telegram='@a')
        self.moderator_user = User.objects.create_user('modx',
'm@example.com', 'pw-test-123')
        Moderator.objects.create(user=self.moderator_user)
        self.admin_user = User.objects.create_user(
            'adminx', 'x@example.com', 'pw-test-123', is_superuser=True,
is_staff=True,
        )
        Administrator.objects.create(user=self.admin_user)

    def test_moderator_deletes_vacancy_and_creates_report(self):
        vac = _make_vacancy('site-del-1')
        VacancyReport.objects.create(
            vacancy=vac, user=self.applicant_user,
            reason_code=VacancyReport.REASON_SCAM, reason_text='fraud',
        )
        self.client.force_login(self.moderator_user)
        photo = SimpleUploadedFile('evidence.png', _make_png_bytes(),
content_type='image/png')
        resp = self.client.post(
            reverse('moderator-vacancy-delete', args=[vac.pk]),
            data={'reason': 'Это явный скам.', 'photos': [photo]},
        )
        self.assertEqual(resp.status_code, 200, resp.content)
        body = resp.json()
        self.assertTrue(body['ok'])
        vac.refresh_from_db()

```

```

        self.assertTrue(vac.is_moderator_deleted)
        self.assertFalse(vac.is_active)
        report =
ModeratorDeletionReport.objects.get(pk=body['report_id'])
        self.assertEqual(report.moderator, self.moderator_user)
        self.assertEqual(report.reports_count, 1)
        self.assertEqual(report.dominant_reason_code, 'scam')
        self.assertEqual(report.photos.count(), 1)

    def test_non_moderator_cannot_delete_vacancy(self):
        vac = _make_vacancy('site-del-2')
        self.client.force_login(self.applicant_user)
        resp = self.client.post(
            reverse('moderator-vacancy-delete', args=[vac.pk]),
            data={'reason': 'test'},
        )
        self.assertEqual(resp.status_code, 403)

    def test_deletion_requires_reason(self):
        vac = _make_vacancy('site-del-3')
        self.client.force_login(self.moderator_user)
        resp = self.client.post(
            reverse('moderator-vacancy-delete', args=[vac.pk]),
            data={'reason': ''},
        )
        self.assertEqual(resp.status_code, 400)
        self.assertEqual(resp.json().get('error'), 'reason_required')

    def test_deleted_vacancy_hidden_from_public(self):
        vac = _make_vacancy('site-del-4')
        vac.is_moderator_deleted = True
        vac.is_active = False
        vac.save(update_fields=['is_moderator_deleted', 'is_active'])
        self.client.force_login(self.applicant_user)
        resp = self.client.get(reverse('vacancy-detail',
args=[vac.external_id]))
        self.assertEqual(resp.status_code, 404)

    def test_admin_can_restore_vacancy(self):
        vac = _make_vacancy('site-del-5')
        self.client.force_login(self.moderator_user)
        self.client.post(
            reverse('moderator-vacancy-delete', args=[vac.pk]),
            data={'reason': 'Мошенничество'},
        )
        report = ModeratorDeletionReport.objects.get(vacancy=vac)
        self.client.logout()
        self.client.force_login(self.admin_user)
        resp = self.client.post(
            reverse('moderator-report-restore', args=[report.pk]),
        )
        self.assertEqual(resp.status_code, 200, resp.content)
        report.refresh_from_db()
        vac.refresh_from_db()
        self.assertTrue(report.is_restored)
        self.assertEqual(report.restored_by, self.admin_user)
        self.assertFalse(vac.is_moderator_deleted)
        self.assertTrue(vac.is_active)

    def test_non_admin_cannot_restore_vacancy(self):
        vac = _make_vacancy('site-del-6')
        self.client.force_login(self.moderator_user)
        self.client.post(

```

```

        reverse('moderator-vacancy-delete', args=[vac.pk]),
        data={'reason': 'reason'},
    )
    report = ModeratorDeletionReport.objects.get(vacancy=vac)
    self.client.logout()
    self.client.force_login(self.moderator_user)
    resp = self.client.post(
        reverse('moderator-report-restore', args=[report.pk]),
    )
    self.assertEqual(resp.status_code, 403)

def test_admin_pdf_download(self):
    vac = _make_vacancy('site-del-7')
    self.client.force_login(self.moderator_user)
    self.client.post(
        reverse('moderator-vacancy-delete', args=[vac.pk]),
        data={'reason': 'Мошенничество'},
    )
    report = ModeratorDeletionReport.objects.get(vacancy=vac)
    self.client.logout()
    self.client.force_login(self.admin_user)
    resp = self.client.get(
        reverse('accounts:admin_moderator_report_pdf',
args=[report.pk]),
    )
    self.assertEqual(resp.status_code, 200)
    self.assertEqual(resp['Content-Type'], 'application/pdf')
    self.assertTrue(resp.content.startswith(b'%PDF'))

def test_admin_reports_page_lists_report(self):
    vac = _make_vacancy('site-del-8')
    self.client.force_login(self.moderator_user)
    self.client.post(
        reverse('moderator-vacancy-delete', args=[vac.pk]),
        data={'reason': 'Мошенничество'},
    )
    self.client.logout()
    self.client.force_login(self.admin_user)
    resp =
self.client.get(reverse('accounts:admin_moderator_reports'))
    self.assertEqual(resp.status_code, 200)
    self.assertContains(resp, 'Отчёты модераторов')
    self.assertContains(resp, vac.title)

```

36. vacancies/urls.py

```

from django.urls import path, re_path

from vacancies.views import (
    VacancyListView, VacancyDetailView,
    employer_rating_api, vacancy_description_api,
    vacancy_branded_frame_view,
    vacancy_create, vacancy_edit, vacancy_delete, my_vacancies,
    api_vacancy_toggle_active,
    api_vacancy_report, api_report_self_status_update,
    api_vacancy_moderation_update,
    api_moderator_delete_vacancy, api_moderator_report_restore,
)

urlpatterns = [
    path("", VacancyListView.as_view(), name="vacancy-list"),
    path("create/", vacancy_create, name="vacancy-create"),
    path("my/", my_vacancies, name="my-vacancies"),

```

```

        re_path(r"^(?P<pk>[\w-]+)/$", VacancyDetailView.as_view(),
name="vacancy-detail"),
        path("<int:pk>/edit/", vacancy_edit, name="vacancy-edit"),
        path("<int:pk>/delete/", vacancy_delete, name="vacancy-delete"),
        path("<int:pk>/toggle-active/", api_vacancy_toggle_active,
name="vacancy-toggle-active"),
        path("<int:pk>/report/", api_vacancy_report, name="vacancy-report"),
        path("api/reports/<int:pk>/self-status/",
api_report_self_status_update, name="report-self-status"),
        path("api/reports/vacancy/<int:vacancy_id>/state/",
api_vacancy_moderation_update, name="vacancy-moderation-state"),
        path("api/moderator/vacancy/<int:vacancy_id>/delete/",
api_moderator_delete_vacancy, name="moderator-vacancy-delete"),
        path("api/admin/moderator-report/<int:report_id>/restore/",
api_moderator_report_restore, name="moderator-report-restore"),
        path("<int:pk>/branded/", vacancy_branded_frame_view, name="vacancy-
branded-frame"),
        path("api/employer-rating/<str:hh_id>/", employer_rating_api,
name="employer-rating-api"),
        path("api/vacancy-description/<int:pk>/", vacancy_description_api,
name="vacancy-description-api"),
]

```

37. vacancies/views.py

```

import uuid
import json
from datetime import timedelta
from urllib.parse import urlencode

from django.conf import settings
from django.contrib.auth.decorators import login_required
from django.db.models import Q, Case, When, IntegerField
from django.http import JsonResponse
from django.shortcuts import render, redirect, get_object_or_404
from django.utils import timezone
from django.views.generic import ListView, DetailView

from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny, IsAuthenticated
from drf_yasg.utils import swagger_auto_schema
from drf_yasg import openapi
from vacancies.hh_client import hh_openapi_headers

from vacancies.models import (
    HhDictionaryItem,
    Vacancy, Employer, VacancyReport, VacancyModerationState,
    ModeratorDeletionRepo
rt, ModeratorDeletionPhoto,
)
from .rating import compute_vacancy_rating
from accounts.models import FilterPreset

class VacancyListView(ListView):
    model = Vacancy
    template_name = "vacancies/vacancy_list.html"
    context_object_name = "vacancies"
    paginate_by = 20

    def dispatch(self, request, *args, **kwargs):

```

```

# Admins go to their
own panel
        if
request.user.is_authenticated and request.user.is_superuser and
hasattr(request.user, 'admin_profile'):
        from
django.shortcuts import redirect
        return
redirect('accounts:admin_panel')
# Managers see their
vacancies
        if
request.user.is_authenticated and hasattr(request.user, 'manager'):
        from
django.shortcuts import redirect
        return
redirect('my-vacancies')
# Moderators see
moderation workspace
        if
request.user.is_authenticated and hasattr(request.user,
'moderator_profile'):
        from
django.shortcuts import redirect
        return
redirect('accounts:moderator_analytics')
return
super().dispatch(request, *args, **kwargs)

def setup(self,
request, *args, **kwargs):

super().setup(request
self._filters =

def
GET =
formats_allowed =
return {
    'q':
        'q_scope':
'exclude_words':
        'region':
'selected_experiences
        'employer':
        'skills':
'only_with_salary':
GET.get('only_with_salary') == '1',

```



```

GET.get('salary_min', '').strip(),
GET.get('salary_max', '').strip(),
GET.get('sort', ''),

[v for v in GET.getlist('format') if v in formats_allowed],
GET.getlist('schedule'),

':      GET.getlist('employment'),

forms': GET.getlist('employment_form'),

GET.getlist('label'),

GET.get('published_since', ''),
GET.get('per_page', '20'),
GET.get('metro', '').strip(),

GET.get('metro_city_id', '').strip(),

GET.get('with_address') == '1',
GET.get('accept_kids') == '1',

quencies': GET.getlist('payment_frequency'),

pes':      GET.getlist('employee_type'),

pes':      GET.getlist('contract'),

ods':      GET.getlist('salary_period'),

les':      GET.getlist('work_schedule'),

rns':      GET.getlist('shift_pattern'),

ay':      GET.getlist('hours_per_day'),

GET.get('night_shifts') == '1',

GET.get('with_contact_phone') == '1',

```

```

'salary_min':
'salary_max':
'sort':

'selected_formats':
'selected_schedules':
'selected_employments
'selected_employment_
'selected_labels':
'published_since':
'per_page':
'metro':

'metro_city_id':

'with_address':
'accept_kids':

'selected_payment_fre
'selected_employee_ty
'selected_contract_ty
'selected_salary_peri
'selected_work_schedu
'selected_shift_patte
'selected_hours_per_d
'only_night_shifts':
'with_contact_phone':

```

```

GET.get('source', '').strip(),

GET.get('details_query', '').strip(),

get_queryset(self):
    Vacancy.objects.visible_for_ru().filter(is_active=True)

    f['q_scope']
    f['exclude_words']

    = f['selected_experiences']
    f['employer']

    f['only_with_salary']
    f['salary_min']
    f['salary_max']

    f['selected_formats']
    f['selected_schedules']
    = f['selected_employments']

    orms = f['selected_employment_forms']
    f['selected_labels']
    f['published_since']

    f['metro_city_id']
    f['with_address']
    f['accept_kids']

    uencies = f['selected_payment_frequencies']

    es = f['selected_employee_types']

    es = f['selected_contract_types']

    ds = f['selected_salary_periods']

'source':

'details_query':
    }

def
    queryset =
    f = self._filters
    query = f['q']
    q_scope =

    exclude_words =

    region = f['region']
    selected_experiences

    employer =

    skills = f['skills']
    only_with_salary =

    salary_min =

    salary_max =

    sort = f['sort']
    selected_formats =

    selected_schedules =

    selected_employments

selected_employment_f

    selected_labels =

    published_since =

    metro = f['metro']
    metro_city_id =

    with_address =

    accept_kids =

selected_payment_freq

selected_employee_typ

selected_contract_typ

selected_salary_perio

```

```

es = f['selected_work_schedules']

ns = f['selected_shift_patterns']

y = f['selected_hours_per_day']
f['only_night_shifts']
f['with_contact_phone']

f['details_query']

"company":
= queryset.filter(company__icontains=query)
== "description":
= queryset.filter(description__icontains=query)
== "all":
= queryset.filter(
Q(title__icontains=query) | Q(company__icontains=query) | Q(description__icontains=query)
)
else:
queryset

if exclude_words:
for term in
queryset

[t.strip() for t in exclude_words.split() if t.strip()]:
queryset.exclude(Q(title__icontains=term) |
Q(description__icontains=term))

queryset.filter(region__icontains=region)

queryset.filter(company__icontains=employer)

selected_experiences:
queryset.filter(experience_id__in=selected_experiences)

queryset.filter(Q(salary_from__isnull=False) |
Q(salary_to__isnull=False))

int(salary_min)

= queryset.filter(salary_from__gte=val)

selected_work_schedules

selected_shift_patterns

selected_hours_per_day

only_night_shifts =

with_contact_phone =

source = f['source']
details_query =

if query:
if q_scope ==

queryset

elif q_scope

queryset

elif q_scope

queryset

Q(title__icontains=query) | Q(company__icontains=query) | Q(description__icontains=query)
)
else:
queryset

if exclude_words:
for term in
queryset

[t.strip() for t in exclude_words.split() if t.strip()]:
queryset.exclude(Q(title__icontains=term) |
Q(description__icontains=term))

queryset.filter(region__icontains=region)

queryset.filter(company__icontains=employer)

selected_experiences:
queryset.filter(experience_id__in=selected_experiences)

queryset.filter(Q(salary_from__isnull=False) |
Q(salary_to__isnull=False))

int(salary_min)

= queryset.filter(salary_from__gte=val)

except

```

```

ValueError:
    pass
    if salary_max:
        try:
            val =
            queryset
        except
            pass
            if skills:
                for term in
                    queryset
                    Q(title__icontains=te
                        |
                    Q(company__icontains=term)
                        |
                    Q(key_skills_text__icontains=term)
                )
            if selected_formats:
                format_filter
                if "remote" in
                    format_filter |=
                    if "hybrid" in
                        format_filter |=
                        if "onsite" in
                            format_filter |=
                            queryset =
                            if
                                queryset =
                                if
                                    queryset =
                                    if
                                        queryset =
                                        if "internship" in
                                            queryset =
                                            if "temporary" in
                                                queryset =

```

```

int(salary_max)
= queryset.filter(salary_to__lte=val)
ValueError:
[t.strip() for t in skills.split(",") if t.strip()]:
= queryset.filter(
    rm)
Q(company__icontains=term)
Q(key_skills_text__icontains=term)
= Q()
selected_formats:
Q(is_remote=True)
selected_formats:
Q(is_hybrid=True)
selected_formats:
Q(is_onsite=True)
queryset.filter(format_filter)
selected_schedules:
queryset.filter(schedule_id__in=selected_schedules)
selected_employments:
queryset.filter(employment_id__in=selected_employments)
selected_employment_forms:
queryset.filter(employment_form_id__in=selected_employment_forms)
selected_labels:
queryset.filter(is_internship=True)
selected_labels:

```

```

queryset.filter(accept_temporary=True)
selected_labels:
queryset.filter(accept_incomplete_resumes=True)
timezone.now()
published_since == "day":
= queryset.filter(published_at__gte=now - timedelta(days=1))
published_since == "3days":
= queryset.filter(published_at__gte=now - timedelta(days=3))
published_since == "week":
= queryset.filter(published_at__gte=now - timedelta(days=7))
published_since == "month":
= queryset.filter(published_at__gte=now - timedelta(days=30))
queryset.filter(metro_station_name__icontains=metro)
selected_payment_frequencies:
queryset.filter(payment_frequency__in=selected_payment_frequencies)
selected_employee_types:
queryset.filter(employee_type__in=selected_employee_types)
selected_contract_types:
Q()
selected_contract_types:
Q(contract_labor=True)
selected_contract_types:
Q(contract_gpc=True)
queryset.filter(contract_q)
selected_salary_periods:
queryset.filter(salary_period__in=selected_salary_periods)
selected_work_schedules:
queryset.filter(
elected_work_schedules)

```

```

if "incomplete" in
    queryset =
if published_since:
    now =
    if
        queryset
    elif
        queryset
    elif
        queryset
    elif
        queryset
    if metro:
        queryset =
    if
        queryset =
    if
        queryset =
        contract_q =
        if "labor" in
            contract_q |=
            if "gpc" in
                contract_q |=
                queryset =
            if
                queryset =
            if
                queryset =
Q(work_schedule__in=s
|

```

```

Q(schedule_name__in=selected_work_schedules)
    )
    if
selected_shift_patterns:
        pattern_q =
Q()
        for pattern in
selected_shift_patterns:
            clean =
(pattern or "").strip()
            if not
clean:
                continue
                pattern_q |=
Q(work_schedule__icontains=clean)
                pattern_q |=
Q(schedule_name__icontains=clean)
                pattern_q |=
Q(description__icontains=clean)
                pattern_q |=
Q(working_conditions_text__icontains=clean)
                pattern_q |=
Q(requirements_text__icontains=clean)
                if pattern_q:
                    queryset
= queryset.filter(pattern_q)
                if
selected_hours_per_day:
                    hours_q =
Q(hours_per_day__in=selected_hours_per_day)
                    for value in
selected_hours_per_day:
                        value =
(value or "").strip()
                        if not
value:
                            continue
                            hours_q
|= Q(hours_per_day__icontains=value)
                            hours_q
|= Q(description__icontains=f"{value} ¶ac")
                            hours_q
|= Q(working_conditions_text__icontains=f"{value} ¶ac")
                            hours_q
|= Q(requirements_text__icontains=f"{value} ¶ac")
                            queryset =
queryset.filter(hours_q)
                            if
only_night_shifts:
                                queryset =
queryset.filter(has_night_shifts=True)
                                if
with_contact_phone:
                                    queryset =
queryset.exclude(contact_phone="")
                                    if source == "hh":

```

```

queryset.filter(url__icontains="hh.ru")
"trudvsem":
queryset.filter(external_id__startswith='trudvsem-')
"external":
queryset.filter(created_by__isnull=True)
"local":
queryset.filter(created_by__isnull=False)

queryset.filter(

icontains=details_query)
Q(working_conditions_text__icontains=details_query)
Q(benefits_text__icontains=details_query)
Q(description__icontains=details_query)

queryset.exclude(work_address="")

queryset.filter(accept_kids=True)

queryset.annotate(

has_salary=Case(
    When(Q(salary_from__i
        snull=False) | Q(salary_to__isnull=False), then=1),
        default=0,
        output_field=IntegerF
            )
        ).order_by("-
    elif sort ==
        queryset =

Q(salary_from__isnull

).order_by("salary_fr
    else:
        queryset =

return

```

```

queryset.select_related('employer')

get_paginate_by(self, queryset):

int(self.request.GET.get("per_page", ""))

(20, 50, 100):

per_page

TypeError):

self.paginate_by

get_context_data(self, **kwargs):

super().get_context_data(**kwargs)

f['q_scope']

f['exclude_words']

= f['selected_experiences']

f['employer']

f['only_with_salary']

f['salary_min']

f['salary_max']

f['selected_formats']

f['selected_schedules']

= f['selected_employments']

orms = f['selected_employment_forms']

f['selected_labels']

f['published_since']

f['per_page']

f['metro_city_id']

f['with_address']

f['accept_kids']

```

```

def

    try:

        per_page =

            if per_page in

                return

    except (ValueError,

            pass

    return

def

    context =

    f = self._filters
    query = f['q']
    q_scope =

    exclude_words =

    region = f['region']
    selected_experiences

    employer =

    skills = f['skills']
    only_with_salary =

    salary_min =

    salary_max =

    sort = f['sort']
    selected_formats =

    selected_schedules =

    selected_employments

selected_employment_f

    selected_labels =

    published_since =

    per_page =

    metro = f['metro']
    metro_city_id =

    with_address =

    accept_kids =

```



```

uencies = f['selected_payment_frequencies']

es = f['selected_employee_types']

es = f['selected_contract_types']

ds = f['selected_salary_periods']

es = f['selected_work_schedules']

ns = f['selected_shift_patterns']

y = f['selected_hours_per_day']
f['only_night_shifts']
f['with_contact_phone']

f['details_query']

query))
q_scope != "title":

pe", q_scope))

de_words", exclude_words))

n", region))

yer", employer))

s", skills))

with_salary", "1"))

y_min", salary_min))

selected_payment_freq

selected_employee_type

selected_contract_type

selected_salary_perio

selected_work_schedul

selected_shift_patter

selected_hours_per_da

only_night_shifts =

with_contact_phone =

source = f['source']
details_query =

params = []
if query:

params.append(("q",

if q_scope and

params.append(("q_sco

if exclude_words:

params.append(("exclu

if region:

params.append(("regio

if employer:

params.append(("emplo

if skills:

params.append(("skill

if only_with_salary:

params.append(("only_

if salary_min:

params.append(("salar

if salary_max:

```

```

y_max", salary_max))

, sort))

shed_since", published_since))
per_page not in ("", "20"):

age", per_page))
selected_experiences:

ience", exp))
selected_formats:

t", fmt))
selected_schedules:

ule", sch))
selected_employments:

yment", emp))
selected_employment_forms:

yment_form", ef))
selected_labels:

", lbl))

", metro))

_city_id", metro_city_id))

address", "1"))

t_kids", "1"))

```

```

params.append(("salar

if sort:

params.append(("sort"

if published_since:

params.append(("publi

if per_page and

params.append(("per_p

for exp in

params.append(("exper

for fmt in

params.append(("forma

for sch in

params.append(("sched

for emp in

params.append(("emplo

for ef in

params.append(("emplo

for lbl in

params.append(("label

if metro:

params.append(("metro

if metro_city_id:

params.append(("metro

if with_address:

params.append(("with_

if accept_kids:

params.append(("accep

for pf in

```

```
selected_payment_frequencies:
```

```
nt_frequency", pf))
```

```
selected_employee_types:
```

```
yee_type", et))
```

```
selected_contract_types:
```

```
act", ct))
```

```
selected_salary_periods:
```

```
y_period", sp))
```

```
selected_work_schedules:
```

```
schedule", ws))
```

```
selected_shift_patterns:
```

```
_pattern", sp))
```

```
selected_hours_per_day:
```

```
_per_day", hpd))
```

```
only_night_shifts:
```

```
_shifts", "1"))
```

```
with_contact_phone:
```

```
contact_phone", "1"))
```

```
e", source))
```

```
ls_query", details_query))
```

```
q_scope != "title")
```

```
exclude_words
```

```
selected_experiences
```

```
selected_employments
```

```
params.append(("payme
```

```
for et in
```

```
params.append(("emplo
```

```
for ct in
```

```
params.append(("contr
```

```
for sp in
```

```
params.append(("salar
```

```
for ws in
```

```
params.append(("work_
```

```
for sp in
```

```
params.append(("shift
```

```
for hpd in
```

```
params.append(("hours
```

```
if
```

```
params.append(("night
```

```
if
```

```
params.append(("with_
```

```
if source:
```

```
params.append(("sourc
```

```
if details_query:
```

```
params.append(("detai
```

```
has_advanced = bool(  
    (q_scope and
```

```
or
```

```
or
```

```
or
```

```
or
```

```

selected_employment_forms
selected_schedules
selected_formats
only_with_salary
metro_city_id
with_address
selected_payment_frequencies
selected_labels
selected_employee_types
selected_contract_types
selected_salary_periods
selected_work_schedules
selected_shift_patterns
selected_hours_per_day
only_night_shifts
with_contact_phone
details_query

)

context.update(f)

context["has_advanced
try:
context["region_optio
except Exception:
context["region_optio
try:
context["experience_o
except Exception:
context["experience_o
try:

```

```

ions"] = self._schedule_options()

ions"] = []

ptions"] = self._employment_options()

ptions"] = []

orm_options"] = self._employment_form_options()

orm_options"] = []

e_options"] = self._employee_type_options()

e_options"] = []

d_options"] = self._salary_period_options()

d_options"] = []

e_options"] = self._work_schedule_options()

e_options"] = []

n_options"] = self._shift_pattern_options()

y_options"] = self._hours_per_day_options()

y_options"] = []

uency_options"] = self._payment_frequency_options()
context["schedule_opt
    except Exception:
context["schedule_opt
    try:
context["employment_o
    except Exception:
context["employment_o
    try:
context["employment_f
    except Exception:
context["employment_f
    try:
context["employee_typ
    except Exception:
context["employee_typ
    try:
context["salary_perio
    except Exception:
context["salary_perio
    try:
context["work_schedul
    except Exception:
context["work_schedul

context["shift_patter
    try:
context["hours_per_da
    except Exception:
context["hours_per_da
    try:
context["payment_freq
    except Exception:

```

```

        uency_options"] = []

        y"] = urlencode(params)

        self.request.user.is_authenticated:

        "] = list(

        filter(user=self.request.user).values("id", "name", "filters")
        )
        else:

        context["user_presets

        return context

    def

        return list(

        Vacancy.objects.exclu

        .values_list("region"

        .distinct()

        .order_by("region")
        )

    def

    _dict_options(self, dictionary, id_field, name_field, exclude_ids=()):
        dict_rows = list(

        HhDictionaryItem.obje

        .exclude(item_id="")

        .exclude(item_id__in=

        .values_list("item_id

        .distinct().order_by(

        )
        vac_rows = list(

        Vacancy.objects.exclu

        .exclude(**{name_fiel

        d: ""})

```

```

, name_field)

name_field)

self._merge_id_name_options(dict_rows, vac_rows)

_experience_options(self):
self._dict_options("experience", "experience_id", "experience_name")

_schedule_options(self):
self._dict_options("schedule", "schedule_id", "schedule_name")

_employment_options(self):
self._dict_options("employment", "employment_id", "employment_name",
exclude_ids=("probation",))

_employment_form_options(self):
self._dict_options("employment_form", "employment_form_id",
"employment_form_name")

_merge_id_name_options(self, *iterables):

iterables:
name in rows:
= (item_id or "").strip()
(name or "").strip()
item_id and name and item_id not in merged:

name
sorted(merged.items(), key=lambda row: row[1].lower())

_field_value_options(self, field_name):

de(**{field_name: ""})

me, flat=True)

```

```

.values_list(id_field

.distinct().order_by(
)
return

def
return

def
return

def
return

def
merged = {}
for rows in

    for item_id,

        item_id

        name =

        if

merged[item_id] =

return

def
return list(

Vacancy.objects.exclu

.values_list(field_na

.distinct()

.order_by(field_name)
)

```

```

def
_employee_type_options(self):
    "Постоянная работа",
    "Временная работа",

    self._field_value_options("employee_type"):
        mapping.get(value, value.replace("_", " ").strip().capitalize())
        label))

_salary_period_options(self):
    месяц",
    проект",

    self._field_value_options("salary_period")
    ["month", "project"]

    mapping.get(value, value.replace("_", " ").strip().capitalize())
    label))

_work_schedule_options(self):
    = self._field_value_options("work_schedule")
    = list(

    de(schedule_name="")

    e_name", flat=True)

    ame")

    work_schedule_values + schedule_name_values:
        def
            mapping = {
                "permanent":
                "temporary":
            }
            result = []
            for value in
                label =
            result.append((value,
                return result

            def
                mapping = {
                    "month": "3a
                    "project": "3a
                }
                values =
                if not values:
                    values =
                result = []
                for value in values:
                    label =
                result.append((value,
                    return result

            def
                work_schedule_values
                schedule_name_values

                Vacancy.objects.exclu

                .values_list("schedul

                .distinct()

                .order_by("schedule_n

                )
                merged = []
                seen = set()
                for value in
                    clean = (value

```



```

or "").strip()
clean not in seen:

self._normalize_option_label(clean))

_normalize_option_label(self, value):
    "").strip()

    in clean if ch.isalpha()]

    sum(1 for ch in letters if ch.isupper()) / len(letters)
    >= 0.7:
    clean.lower().capitalize()

_hours_per_day_options(self):
    "3", "4", "5", "6", "7", "8", "9", "10", "11", "12"]
    set(defaults)

    field_value_options("hours_per_day")

sorted(values, key=lambda v: int(v) if str(v).isdigit() else 999):
or "").strip()

clean.isdigit():
int(clean)

10 == 1 and n != 11:

10 in (2, 3, 4) and n not in (12, 13, 14):

clean

```

```

if clean and
seen.add(clean)
merged.append((clean,
return merged

def
    clean = (value or
    if not clean:
        return clean
    letters = [ch for ch
    if letters:
        upper_ratio =
        if upper_ratio
            return
    return clean

def
    defaults = ["2",
    values =

    values.update(self._f

    result = []
    for value in
        clean = (value
        if not clean:
            continue
        if
            n =
            if n %

    label = f"{n} час"
        elif n %

    label = f"{n} часа"
        else:

    label = f"{n} часов"
        else:
            label =

```

```

label))

    _shift_pattern_options(self):

        "1/1"),
        "2/2"),
        "2/3"),
        "3/3"),
        "5/2"),
        "6/1"),
        "7/7"),
        "15/15"),

    _payment_frequency_options(self):

        "Ежедневно",
        в неделю",
        "Два раза в месяц",
        "Раз в месяц",
        проект",

        self._field_value_options("payment_frequency")

        ["daily", "weekly", "biweekly", "monthly", "project"]

        mapping.get(value, value.replace("_", " ").strip().capitalize())

        label))

class VacancyDetailView(DetailView):

    "vacancies/vacancy_detail.html"

    "vacancy"

    queryset=None):

result.append((clean,

    return result

def

    return [

        ("1/1",

            ("2/2",

                ("2/3",

                    ("3/3",

                        ("5/2",

                            ("6/1",

                                ("7/7",

                                    ("15/15",

                                        ]

def

    mapping = {

        "daily":

            "weekly": "Раз

            "biweekly":

            "monthly":

            "project": "За

    }

    values =

    if not values:

        values =

        result = []

        for value in values:

            label =

            result.append((value,

                return result

    model = Vacancy

    template_name =

    context_object_name =

    def get_object(self,

```

```

self.kwargs.get('pk')
Vacancy.objects.select_related('employer')
vacancies from everyone except moderators/admins so
applicants/managers don't get stuck on a stale detail page.
getattr(self.request, 'user', None)
user.is_authenticated and (
hasattr(user, 'moderator_profile')

qs.exclude(is_moderator_deleted=True)
get_object_or_404(qs, external_id=external_id)

_resolve_metro_coords(station_id):
lon) for a metro station from the cached HH JSON."""
_json, os as _os
_os.path.join(settings.BASE_DIR, 'tools', 'metro_hh.json')
open(metro_path, encoding='utf-8') as f:
_json.load(f)
(FileNotFoundError, ValueError):
str(station_id)

city.get('lines', []):
in line.get('stations', []):
str(st.get('id')) == sid:

str(st.get('lat', '')), str(st.get('lng', ''))

_is_hh_source(vacancy):
and vacancy.url and 'hh.ru' in vacancy.url)

external_id =
qs =
# Hide soft-deleted
# that
user =
is_priv = bool(
    user and

user.is_superuser or
)
)
if not is_priv:
    qs =

return

@staticmethod
def
    """Look up (lat,
import json as
metro_path =
try:
    with
        cities =

except
    return '', ''
sid =

for city in cities:
    for line in
        for st
            if

return
    return '', ''

@staticmethod
def
    return bool(vacancy

@staticmethod
def

```

```

_is_trudvsem_source(vacancy):
    if not vacancy:
        return False
    raw =
vacancy.raw_json if isinstance(vacancy.raw_json, dict) else {}
    return
str(vacancy.external_id or '').startswith('trudvsem-') or
raw.get('source') == 'trudvsem'

    @staticmethod
    def
        _trudvsem_address(raw_json):
            addresses =
            (raw_json or {}).get('addresses') if isinstance(raw_json, dict) else None
            if not
isinstance(addresses, dict):
                return '',
            None, None
            addresses.get('address')
            addr_list =
            if not
                return '',
            None, None
            first = addr_list[0]
            location =
            try:
                lat =
                lon =
            except Exception:
                lat, lon =
            return location,

    @staticmethod
    def
        _trudvsem_description_html(raw_json):
            if not
                return ''
            parts = []
            for title, key in (
                ('Должностные
обязанности', 'duty'),
                ('Требования',
'requirement'),
                ('Дополнительные
требования', 'requirements'),
            ):
                value =
                if value:
                    parts.append(
                        f"<section class='tv-
block'><h3>{title}</h3><p>{value}</p></section>"

```

```

    )
    return "<div
class='tv-desc'>" + ''.join(parts) + "</div>" if parts else ''

def
    ctx =
    vac = self.object
    raw_json =
    is_hh_source =
    is_trudvsem_source =
    ctx['is_hh_source']

    ctx['is_trudvsem_sour
ce'] = is_trudvsem_source

    ctx['is_external_sour
ce'] = bool(vac.created_by_id is None and vac.url)
    ctx['source_label']
= 'HeadHunter' if is_hh_source else ('Работа России' if
is_trudvsem_source else ('На сайте' if vac.created_by_id else 'Внешний
источник'))

    ctx['trudvsem_descrip
tion_html'] = self._trudvsem_description_html(raw_json) if
is_trudvsem_source else ''

    # Pre-compute logo
URL once (property can hit DB multiple times)

    ctx['employer_logo_ur
l'] = vac.employer_logo_url

    # — Inline
description fetch (sync, short timeout) —————
    # Skip remote fetch
for non-HH (site-created) vacancies entirely —
    # they legitimately
have no HH description and would only hang the
    # request for a 404
round-trip.
    _is_hh_vac =
is_hh_source
    if (not
vac.description and not vac.branded_description and _is_hh_vac):
        hh_id =
(vac.raw_json or {}).get('id') or vac.external_id
        if hh_id:
            try:

import json as _json

from urllib.request

    _url =
f'https://api.hh.ru/vacancies/{hh_id}'

```

```

headers=hh_openapi_headers()

timeout=3) as _resp:

    _json.loads(_resp.read().decode('utf-8'))

    _data.get('description', '') or ''

    on = _data.get('branded_description', '') or ''

    ', '.join(

        s.get('name', '') for
        s in (_data.get('key_skills') or []) if isinstance(s, dict)

    )

    vac.raw_json =

    vac.save(update_field

    except

    #

    Timeout or network error – fall back to Celery async

    try:

    from .tasks import

    fetch_vacancy_descrip

    except Exception:

    pass

    # — Server-side

    from .rating import

    emp = getattr(vac,

    hh_val =

    dj_val =

    _positive(getattr(emp, 'dreamjob_rating', None)) if emp else None

    # If no cached

    if emp and not

    try:

    from

rating (skip AJAX when DB already has it) —————

_positive

'employer', None)

_positive(getattr(emp, 'hh_rating', None)) if emp else None

_positive(getattr(emp, 'dreamjob_rating', None)) if emp else None

rating, try a quick synchronous scrape (≤2 s)

hh_val and not dj_val:

```

```

.management.commands.fetch_employer_details import (
    extract_rating_from_html
    )
    from
.dreamjob import (
    fetch_dreamjob_page,
    extract_rating_from_html
    )
    from
django.utils import timezone as _tz
socket as _socket
import

_socket.setdefaulttimeout()

_socket.setdefaulttimeout(2)

HH employer page
not hh_val and emp.hh_id:

extract_rating_from_html(

'https://hh.ru/employer/{emp.hh_id}'))

_parse_rating(_candidate)

_parsed

= hh_val

= _tz.now()

s=['hh_rating', 'rating_updated_at'])

dj_id found on HH page

_dj_id:

extract_rating_from_html(
    _candidate, _dj_id =
    fetch_employer_page(f
    _parsed =
    if _parsed:
        hh_val =
        emp.hh_rating
    emp.rating_updated_at
    emp.save(update_fields
    # Try DreamJob via
    if not dj_val and

```

```

try:
    _dj_cand

= dj_extract(fetch_dreamjob_page(

f'https://dreamjob.ru

/employers/{_dj_id}'))

_parse_rating(_dj_cand)

_dj_parsed =

if

_dj_parsed:

dj_val = _dj_parsed

emp.dreamjob_rating =

dj_val

emp.dreamjob_rating_u

pdated_at = _tz.now()

emp.save(update_fields

s=['dreamjob_rating', 'dreamjob_rating_updated_at'])

except

Exception:

pass

except Exception:

pass

finally:

_socket.setdefaulttim

except

pass

# If sync

if not hh_val

try:

from .tasks import

fetch_employer_rating

fetch_employer_rating

except

pass

if hh_val and

```



```

dj_val:

g'] = f'{round((hh_val + dj_val) / 2, 2):.1f}'
g'] = f'{hh_val:.1f}'
g'] = f'{dj_val:.1f}'
g'] = ''

branded_srcdoc (inline iframe content, saves HTTP trip) -
vac.branded_description:
vac.branded_description

= (

html><html><head><meta charset="utf-8">'
name="viewport" content="width=device-width,initial-scale=1">'
gin:0;padding:0;overflow:hidden;background:#faf8f5}'
family:Arial,sans-serif}img{max-width:100%;height:auto;display:block}'
width:100%}</style>'

_p=window.parent&&window.parent.postMessage?window.parent.postMessage.bin
d(window.parent):null;'

_nh(){{if(!_p)return;var
h=document.body.scrollHeight;try{{_p({{type:"branded-
height",pk:{vac.pk},height:h}},"*")}}catch(e){{}}}}</script>'

.addEventListener("load",function(){_nh();setTimeout(_nh,300);setTimeout(
_nh,800);setTimeout(_nh,2000)});'

ResizeObserver!=="undefined"){new
ResizeObserver(function(){_nh()}).observe(document.body)}</script>'

)

```

```

# --- geo data ---
lat = lon = None
geocode_query = None
raw_json =
vac.raw_json if isinstance(vac.raw_json, dict) else {}
trud_addr, trud_lat,
trud_lon = self._trudvsem_address(raw_json) if is_trudvsem_source else
(' ', None, None)

# 1) Try to get
addr =
if isinstance(addr,
dict):
    try:
        _lat =
        _lng =
        if _lat
and _lng:
lat = float(_lat)
lon = float(_lng)
    except
pass
(TypeError, ValueError):
# 2) Build geocode
query as fallback for client-side geocoding
# build a user-
facing display address (prefer address.raw/value/display)
display_address = ''
# prefer a complete
raw string when available
dict) and addr.get('raw'):
display_address =
str(addr.get('raw')).strip()
# otherwise assemble
from structured parts: city, street, building, house/block, metro
if not
display_address and isinstance(addr, dict):
    parts = []
    city =
    if city:
        parts.append(str(city
).strip())
    street =
    if street and
str(street).strip():
        parts.append(str(stre
et).strip())

```

```

building/house/block if present
('building','house','block'):
    addr.get(key)
    and str(val).strip():

        .strip())
to 3 nearby metro station names
addr.get('metro')
isinstance(metro, dict) and metro.get('station_name'):

o.get('station_name')).strip())
addr.get('metro_stations') or []
isinstance(mstations, list):
0
in mstations:
isinstance(st, dict) and st.get('station_name'):

et('station_name')).strip())

vacancy.region if still empty
and vac.region:

on)
parts preserving order

parts:
p not in seen:

'.join(out)

# include
for key in
    val =
    if val

parts.append(str(val))
# include up
metro =
if
parts.append(str(metr
mstations =
if
count =
for st
if
parts.append(str(st.g
count += 1
if count >= 3:
    break
    # fallback to
    if not parts

parts.append(vac.regi
# join unique
seen = set()
out = []
for p in
    if p and

out.append(p)
seen.add(p)
if out:

display_address = ',

```

```

fall back to area+region
is_trudvsem_source and trud_addr:

trud_addr
display_address:
raw_json.get('area')

isinstance(area, dict) and area.get('name'):

('name'))
isinstance(area, str) and area:

and vac.region not in parts:

on)

'.join(parts)

display_address for client-side geocoding

= display_address

vacancies raw_json is empty – fall back to model fields
trud_lat is not None:

trud_lon is not None:

vac.lat:

vac.lon:

the explicit street address entered by the manager –
over the region-only fallback derived from raw_json

vac.work_address

= display_address

# if still empty,
if
display_address =
elif not
area =
parts = []
if
parts.append(area.get
elif
parts.append(area)
if vac.region
parts.append(vac.regi
if parts:
display_address = ',
# prefer
if display_address:
geocode_query

# For site-created
if lat is None and
lat = trud_lat
if lon is None and
lon = trud_lon
if not lat and
lat = vac.lat
if not lon and
lon = vac.lon
# work_address is
# always prefer it
if vac.work_address:
display_address =
geocode_query

# — Server-side

```

```

geocode via 2GIS (skip client-side fetch) -----
geocode_query:

json as _json
urllib.request import Request, urlopen
urllib.parse import quote as _quote
= (

2gis.com/3.0/items/geocode?q='
_quote(geocode_query)
'&fields=items.point&key=' + settings.DGIS_API_KEY

Request(_geo_url, headers={'User-Agent': 'job-aggregator-diploma/1.0'})
urlopen(_req, timeout=2) as _resp:

_json.loads(_resp.read().decode('utf-8'))
(_gdata.get('result') or {}).get('items') or []
_items and _items[0].get('point'):

float(_items[0]['point']['lat'])

float(_items[0]['point']['lon'])
Exception:
Fall back to client-side geocoding

f'{lat:.6f}' if lat is not None else None
f'{lon:.6f}' if lon is not None else None
= geocode_query or ''

] = display_address
= settings.DGIS_API_KEY

for pre-filling the route panel -----
self.request.user
''

if not lat and
    try:
        import
        from
        from
        _geo_url

'https://catalog.api.
+
+
)
_req =
with

_gdata =
_items =
if

lat =

lon =

except
    pass #

ctx['map_lat'] =
ctx['map_lon'] =
ctx['geocode_query']

ctx['display_address']
ctx['dgis_api_key']

# — User location
user =
user_location_str =

user_metro_lat = ''
user_metro_lon = ''
if

```

```

user.is_authenticated and hasattr(user, 'applicant'):
    try:
        applicant =
user.applicant
        if
applicant.location_type == 'metro' and applicant.metro_station_name:
        city = applicant.city
or ''
        name =
applicant.metro_station_name
        user_location_str =
('ct. metpo ' + name + ', ' + city).strip(', ') if city else 'ct. metpo '
+ name
        station_id =
applicant.metro_station_id or ''
        if
station_id:
        user_metro_lat,
user_metro_lon = self._resolve_metro_coords(station_id)
        elif
applicant.location_type == 'address' and applicant.address:
        user_location_str =
applicant.address
        except
Exception:
        pass
        ctx['user_location_str
r'] = user_location_str
        ctx['user_metro_lat']
= user_metro_lat
        ctx['user_metro_lon']
= user_metro_lon
        # — Application
status for logged-in users —————
        is_applicant =
user.is_authenticated and hasattr(user, 'applicant')
        ctx['is_applicant']
= is_applicant
        ctx['user_application
'] = None
        if is_applicant:
            from
accounts.models import Application
        ctx['user_application
'] = Application.objects.filter(
vacancy=vac,
applicant=user
        ).first()
        user_can_report = (

```

```

vac.is_hh)
    user.is_authenticated
    and (not
    and
    (hasattr(user, 'applicant') or hasattr(user, 'manager'))
    )
    ctx['can_report_vacan
cy'] = user_can_report
    ctx['has_reported_vac
ancy'] = False
    ctx['report_reason_ch
oices'] = VacancyReport.REASON_CHOICES
    if user_can_report:
    ctx['has_reported_vac
ancy'] = VacancyReport.objects.filter(
        user=user,
        vacancy=vac,
        ).exists()
    ctx['user_is_creator'
] = user.is_authenticated and vac.created_by_id == user.pk
    if
user.is_authenticated and hasattr(user, 'manager'):
    ctx['application_coun
t'] = vac.applications.count()
    ctx['is_owner'] =
    else:
    ctx['application_coun
t'] = 0
    ctx['is_owner'] =
False
    # Track view and
    if
    from
vacancies.models import VacancyView, Bookmark
    vv, created =
VacancyView.objects.get_or_create(user=user, vacancy=vac)
    if not
created:
    vv.save(update_fields
=['viewed_at']) # auto_now refreshes timestamp
    ctx['is_bookmarked']
= Bookmark.objects.filter(user=user, vacancy=vac).exists()
    else:
    ctx['is_bookmarked']

```

```

= False

return ctx

@swagger_auto_schema(method='get', operation_summary="Рейтинг
работодателя", tags=['vacancies'])
@api_view(['GET'])
@permission_classes([AllowAny])
def employer_rating_api(request, hh_id):

    """Lazy-load employer
    rating by hh_id.

    Returns cached DB
    dispatches a Celery
    {rating: null,
    pending: true} so the client can poll again shortly.
    """
    from .rating import

    try:
        emp =
    except
        return

    hh_val =
    dj_val =

    if hh_val and dj_val:
        avg = round((hh_val
        + dj_val) / 2, 2)
        resp =
        JsonResponse({'rating': f'{avg:.1f}', 'source': 'hh+dreamjob',
        'hh': f'{hh_val:.1f}', 'dj': f'{dj_val:.1f}'})
        resp['Cache-
        return resp
        if hh_val:
            resp =
            resp['Cache-
            return resp
            if dj_val:
                resp =
                resp['Cache-
                return resp

    # No cached rating -
    from
    cache_key =

    _positive
    Employer.objects.get(hh_id=str(hh_id))
    Employer.DoesNotExist:
    JsonResponse({'rating': None, 'source': None})
    _positive(emp.hh_rating)
    _positive(emp.dreamjob_rating)

    dispatch background scraping, tell client to retry
    django.core.cache import cache

```



```

f'rating_inflight_{emp.pk}'
cache.get(cache_key):
1, timeout=300)

import fetch_employer_rating

.delay(emp.pk)

may be unavailable in local dev; keep API non-failing.

if not
    cache.set(cache_key,
    try:
        from .tasks
        fetch_employer_rating
    except Exception:
        # Redis/Celery
        pass

    return
JsonResponse({'rating': None, 'source': None, 'pending': True})

from django.http import HttpResponse
from django.views.decorators.clickjacking import xframe_options_exempt

@xframe_options_exempt
def vacancy_branded_frame_view(request, pk):
    """Serves
    branded_description as a self-contained HTML page for iframe embedding.

    Using an iframe is
    the only way to fully isolate the employer's <style> blocks
    from the parent page,
    preventing hh.ru template rules from leaking globally.
    """
    try:
        v =
        Vacancy.objects.only('branded_description', 'description').get(pk=pk)
    except
        Vacancy.DoesNotExist:
            return
            HttpResponse(status=404)

            content =
            v.branded_description or (v.description if v.description !=
            '__unavailable__' else '') or ''

            # The iframe posts
            its scrollHeight to the parent so it can be resized.
            html = f'''<!doctype
html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<style>
    html, body {{ margin: 0; padding: 0; overflow: hidden; background:
    #faf8f5; }}
    body {{ font-family: Arial, sans-serif; background: #faf8f5; }}
    img {{ max-width: 100%; height: auto; display: block; }}
    video {{ max-width: 100%; }}
</style>
<script>
    /* Save postMessage reference BEFORE employer scripts can override it
    */

```

```

var _pmRef = window.parent && window.parent.postMessage
            ? window.parent.postMessage.bind(window.parent)
            : null;
function notifyHeight() {{
    if (!_pmRef) return;
    document.body.style.overflow = 'hidden';
    var h = document.body.scrollHeight ||
document.documentElement.scrollHeight;
    try {{ _pmRef({{ type: "branded-height", pk: {pk}, height: h }},
    "**"); }} catch(e) {{}}
    }}
</script>
</head>
<body>
{content}
</script>
// Fire immediately and on full load
notifyHeight();
window.addEventListener("load", function() {{
    notifyHeight();
    setTimeout(notifyHeight, 300);
    setTimeout(notifyHeight, 800);
    setTimeout(notifyHeight, 2000);
}});
// ResizeObserver catches layout shifts from lazy images, deferred
scripts, etc.
if (typeof ResizeObserver !== 'undefined') {{
    var ro = new ResizeObserver(function() {{ notifyHeight(); }});
    ro.observe(document.body);
}}
// MutationObserver fallback
if (typeof MutationObserver !== 'undefined') {{
    var mo = new MutationObserver(function() {{ notifyHeight(); }});
    mo.observe(document.body, {{ childList: true, subtree: true,
attributes: true }});
}}
</script>
</body>
</html>'''

return
HttpResponse(html, content_type='text/html; charset=utf-8')

@swagger_auto_schema(method='get', operation_summary="Получить описание
вакансии", tags=['api'])
@api_view(['GET'])
@permission_classes([AllowAny])
def vacancy_description_api(request, pk):

    try:
        v =
Vacancy.objects.only('id', 'external_id', 'description',
'branded_description', 'url').get(pk=pk)

    except
Vacancy.DoesNotExist:
        return
JsonResponse({'error': 'not found'}, status=404)
if not v.description
and not v.branded_description:
    # Non-HH vacancies
    (site-created) may legitimately have no description;
    # do not keep
    frontend in pending polling loop for them.
    is_hh = 'hh.ru' in

```

```

(v.url or '')
str(v.external_id or '').startswith('site-'):
    JsonResponse({'pending': False, 'description': '', 'branded': ''})
    # Fire Celery task
    try:
        from .tasks
    fetch_vacancy_descrip
    except Exception:
        pass # Celery
    return
JsonResponse({'pending': True, 'description': '', 'branded': ''})

    if v.description ==
    # HH returned
    return
JsonResponse({'pending': False, 'unavailable': True,
'description': '', 'branded': ''})

    return JsonResponse({
        'pending': False,
        'description':
        'branded':
    })

v.description,
v.branded_description,

# ----- Employer vacancy management
-----

EXPERIENCE_CHOICES = [
    ('noExperience', 'Нет
опыта'),
    ('between1And3', '1-3
года'),
    ('between3And6', '3-6
лет'),
    ('moreThan6', 'От 6
лет'),
]

PAYMENT_FREQ_CHOICES = [
    ('daily',
    'Ежедневно'),
    ('weekly', 'Раз
в неделю'),
    ('biweekly', 'Два
раза в месяц'),
    ('monthly', 'Раз
в месяц'),
    ('project', 'За
проект'),
]

```

```

SCHEDULE_CHOICES = [
    'Полный день'),
    'Сменный'),
    'Гибкий'),
    'Удалённая работа'),
    'вахтовый'),
]

EMPLOYMENT_CHOICES = [
    'Полная занятость'),
    'Частичная занятость'),
    'Вахта'),
    'Проектная / временная'),
    'Волонтерство'),
    'Стажировка'),
]

CURRENCY_CHOICES = [
    ('RUR', '₽ Рубль'),
    ('USD', '$ Доллар'),
    ('EUR', '€ Евро'),
]

def _safe_coord(val):
    try:
        f = float(val or 0)
        return f if f != 0.0
    except (ValueError,
            TypeError):
        return None

def _parse_vacancy_post(post):
    """Extract and
    validate vacancy fields from POST. Returns (data_dict, errors_dict)."""
    errors = {}
    title =
    company =
    region =
    if not title:
    if not company:
    if not region:
    post.get('title', '').strip()
    post.get('company', '').strip()
    post.get('region', '').strip()
    errors['title'] = 'Введите название вакансии'
    errors['company'] = 'Введите название компании'
    errors['region'] = 'Укажите город или регион'

```

```

salary_to = None

post.get('salary_from', '').strip()

post.get('salary_to', '').strip()

= int(sf_raw)

errors['salary_from'] = 'Некорректное число'

int(st_raw)

errors['salary_to'] = 'Некорректное число'

None and salary_to is not None and salary_from > salary_to:
    errors['salary_to']
= 'Максимальная зарплата должна быть не меньше минимальной'

post.get('experience_id', '').strip()

post.get('schedule_id', '').strip()

post.get('employment_id', '').strip()

post.get('salary_currency', 'RUR').strip()

fields — reject values not in known sets
for c in EXPERIENCE_CHOICES}
for c in SCHEDULE_CHOICES}
for c in EMPLOYMENT_CHOICES}
for c in CURRENCY_CHOICES}
for c in PAYMENT_FREQ_CHOICES}
{'month', 'hour', 'project', ''}

experience_id not in _valid_exp:

schedule_id not in _valid_sch:

employment_id not in _valid_emp:

not in _valid_cur:
'RUR'

'is_remote' in post

'is_hybrid' in post

salary_from =

sf_raw =

st_raw =

if sf_raw:
    try:    salary_from

    except ValueError:

if st_raw:
    try:    salary_to =

    except ValueError:

if salary_from is not
    errors['salary_to']

experience_id  =

schedule_id    =

employment_id  =

salary_currency =

# Validate choice
_valid_exp = {c[0]
_valid_sch = {c[0]
_valid_emp = {c[0]
_valid_cur = {c[0]
_valid_per = {c[0]
_valid_sp  =

if experience_id and

    experience_id = ''
if schedule_id and

    schedule_id = ''
if employment_id and

    employment_id = ''
if salary_currency

    salary_currency =

is_remote =

is_hybrid =

is_onsite =

```

```

'is_onsite' in post
not is_hybrid and not is_onsite:
    default

Vacancy.WorkFormat.HYBRID
not is_onsite:
Vacancy.WorkFormat.REMOTE

Vacancy.WorkFormat.ONSITE

company=company, region=region,

'description', '').strip(),

get('key_skills', '').strip(),

nce_id,

EXPERIENCE_CHOICES).get(experience_id, ''),

id,

CHEDULE_CHOICES).get(schedule_id, ''),

ent_id,

EMPLOYMENT_CHOICES).get(employment_id, ''),

om,

y_currency,

at,

umes='accept_incomplete_resumes' in post,

ept_temporary' in post,

if not is_remote and
    is_onsite = True #

if is_hybrid:
    work_format =

elif is_remote and
    work_format =

else:
    work_format =

data = dict(
    title=title,

description=post.get(

key_skills_text=post.

experience_id=experie

experience_name=dict(

schedule_id=schedule_

schedule_name=dict(SC

employment_id=employm

employment_name=dict(

salary_from=salary_fr

    salary_to=salary_to,

salary_currency=salar

work_format=work_form

    is_remote=is_remote,
    is_hybrid=is_hybrid,
    is_onsite=is_onsite,

accept_incomplete_res

accept_temporary='acc

```

```

ernship' in post,
form fields

t('employee_type', '').strip(),

act_labor' in post,

t_gpc' in post,

t('work_schedule', '').strip(),

t('hours_per_day', '').strip(),

_night_shifts' in post,

('salary_gross', 'true') == 'true',

t('salary_period', 'month').strip() if post.get('salary_period',
'month').strip() in _valid_sp else 'month',

t.get('payment_frequency', '').strip() if post.get('payment_frequency',
'').strip() in _valid_per | {''} else '',

('address', post.get('work_address', '')).strip(),

dress' in post,

get('address_comment', '').strip(),

get('lat')),

get('lon')),

t('contact_phone', '').strip(),

contact_phone if provided
data.get('contact_phone', '')

_re.sub(r'\D', '', _cp)

is_internship='is_int
# Extended employer-
employee_type=post.ge
contract_labor='contr
contract_gpc='contrac
work_schedule=post.ge
hours_per_day=post.ge
has_night_shifts='has
salary_gross=post.get
salary_period=post.ge
payment_frequency=pos
t.get('payment_frequency', '').strip() if post.get('payment_frequency',
'').strip() in _valid_per | {''} else '',
work_address=post.get
hide_address='hide_ad
address_comment=post.
# Location & contact
lat=_safe_coord(post.
lon=_safe_coord(post.
contact_phone=post.ge
)
# Validate
_cp =
if _cp:
import re as _re
_cp_d =
if

```

```

_cp_d.startswith('8'): _cp_d = '7' + _cp_d[1:]
_re.match(r'^7[0-9]{10}$', _cp_d):

'] = 'Введите номер в формате +7 (XXX) XXX-XX-XX'

st.get('metro_station_name', '').strip(),

.get('metro_line_color', '').strip(),

get('metro_line_name', '').strip(),

t('benefits_text', '').strip(),

t.get('requirements_text', '').strip(),

xt=post.get('working_conditions_text', '').strip(),

t('metro_city_id', '').strip(),

def _form_ctx(extra=None):
the create/edit form."""
import settings as _s

PERIENCE_CHOICES,

DULE_CHOICES,

PLOYMENT_CHOICES,

ENCY_CHOICES,

PAYMENT_FREQ_CHOICES,

_s, 'DGIS_API_KEY', ''),

@login_required

```

```

if not
errors['contact_phone
data.update(dict(
metro_station_name=po
metro_line_color=post
metro_line_name=post.
benefits_text=post.ge
requirements_text=pos
working_conditions_te
metro_city_id=post.ge
))
return data, errors

"""Base context for
from django.conf
ctx = dict(
experience_choices=EX
schedule_choices=SCHE
employment_choices=EM
currency_choices=CURR
payment_freq_choices=
dgis_api_key=getattr(
)
if extra:
ctx.update(extra)
return ctx

```



```

def vacancy_create(request):
    if not
    hasattr(request.user, 'manager'):
        return
    redirect('vacancy-list')

    errors = {}
    form_data =
    {'salary_currency': 'RUR', 'company': request.user.manager.company}

    if request.method ==
    'POST':
        form_data =
        request.POST
        data, errors =
        _parse_vacancy_post(request.POST)
        if not errors:
            ext_id =
            f"site-{uuid.uuid4().hex}"
            vac =
            Vacancy.objects.create(
                external_id=ext_id,
                country='Россия',
                published_at=timezone
                .now(),
                created_by=request.us
                er,
                url='',
                **data,
            )
            vac.url =
            request.build_absolute_uri(f'/{vac.external_id}/')

            vac.save(update_field
            s=['url'])
            return
            redirect('vacancy-detail', pk=vac.external_id)

            return
            render(request, 'vacancies/vacancy_create.html', _form_ctx({
                'form_data':
                form_data,
                'errors': errors,
                'editing': False,
                'manager':
                request.user.manager,
            }))

@login_required
def vacancy_edit(request, pk):
    vac =
    get_object_or_404(Vacancy, pk=pk)
    if not
    hasattr(request.user, 'manager') or vac.created_by != request.user:
        return
    redirect('vacancy-detail', pk=vac.external_id)

    errors = {}

```

existing vacancy	# Pre-fill from
company=vac.company, region=vac.region,	form_data = dict(title=vac.title,
description, key_skills=vac.key_skills_text,	description=vac.descr
experience_id,	experience_id=vac.exp
schedule_id,	schedule_id=vac.sched
employment_id,	employment_id=vac.emp
salary_from or '', salary_to=vac.salary_to or '',	salary_from=vac.salar
salary_currency or 'RUR',	salary_currency=vac.s
vac.is_remote else '',	is_remote='on' if
vac.is_hybrid else '',	is_hybrid='on' if
vac.is_onsite else '',	is_onsite='on' if
accept_incomplete_resumes='on' if vac.accept_incomplete_resumes else '',	accept_incomplete_res
if vac.accept_temporary else '',	accept_temporary='on'
if vac.is_internship else '',	is_internship='on'
employee_type,	employee_type=vac.emp
if vac.contract_labor else '',	contract_labor='on'
vac.contract_gpc else '',	contract_gpc='on' if
work_schedule,	work_schedule=vac.wor
hours_per_day,	hours_per_day=vac.hou
if vac.has_night_shifts else '',	has_night_shifts='on'
if vac.salary_gross else 'false',	salary_gross='true'
salary_period or 'month',	salary_period=vac.sal
payment_frequency,	payment_frequency=vac
	work_address=vac.work

```

        _address,
        vac.hide_address else '',

        address_comment,
        new sections (template uses name="address", not work_address)

        ess,
        tact_phone,
        c.metro_station_name,
        ro_city_id,
        metro_line_color or '#e42313',
        etro_line_name,
        efits_text,
        .requirements_text,
        xt=vac.working_conditions_text,

        'POST':
        request.POST
        _parse_vacancy_post(request.POST)
        hide_address: no toggle exists in the form UI
        = vac.hide_address

        data.items():

        redirect('vacancy-detail', pk=vac.external_id)

        render(request, 'vacancies/vacancy_create.html', _form_ctx({
            form_data,

            hide_address='on' if
            address_comment=vac.a
            # Form fields for
            address=vac.work_addr
            lat=vac.lat or '',
            lon=vac.lon or '',
            contact_phone=vac.con
            metro_station_name=va
            metro_city_id=vac.met
            metro_line_color=vac.
            metro_line_name=vac.m
            benefits_text=vac.ben
            requirements_text=vac
            working_conditions_te
        )

        if request.method ==
            form_data =
            data, errors =
            # Preserve
            data['hide_address']

            if not errors:
                for k, v in

                setattr(vac, k, v)
                vac.save()
                return

        return
        'form_data':
        'errors': errors,

```

```

        'editing': True,
        'vacancy': vac,
        'manager':
request.user.manager,
    )))

@login_required
def vacancy_delete(request, pk):
    get_object_or_404(Vacancy, pk=pk)
    hasattr(request.user, 'manager') and vac.created_by == request.user:
        if request.method ==
            'POST':
                vac.delete()
                return
    redirect('my-vacancies')
    return
    redirect('vacancy-detail', pk=vac.external_id)

@swagger_auto_schema(method='post', operation_summary="Пожаловаться на
вакансию", tags=['vacancies'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_vacancy_report(request, pk):
    """Create a complaint
    for a site-created vacancy (one report per user)."""
    if request.method !=
        'POST':
            return
    JsonResponse({'error': 'method_not_allowed'}, status=405)
    if (not
        hasattr(request.user, 'applicant')) and (not hasattr(request.user,
        'manager')):
        return
    JsonResponse({'error': 'forbidden_role'}, status=403)
    vac =
    get_object_or_404(Vacancy, pk=pk)
    if vac.is_hh:
        return
    JsonResponse({'error': 'hh_vacancy_not_reportable'}, status=400)
    try:
        data =
        json.loads(request.body or b'{}')
    except Exception:
        data = {}
    reason_code =
    reason_text =
    allowed_codes = {c[0]
    for c in VacancyReport.REASON_CHOICES}
    if reason_code not in
        allowed_codes:
            return
    JsonResponse({'error': 'invalid_reason_code'}, status=400)
    if reason_code ==

```

```

VacancyReport.REASON_OTHER and not reason_text:
    return
JsonResponse({'error': 'reason_text_required'}, status=400)

report, created =
    VacancyReport.objects.get_or_create(
        user=request.user,
        vacancy=vac,
        defaults={
            'reason_code':
            'reason_text':
        },
    )
    if not created:
        return
JsonResponse({'ok': True, 'already_reported': True, 'id': report.pk})
    return
JsonResponse({'ok': True, 'already_reported': False, 'id': report.pk})

@swagger_auto_schema(methods=['post'], operation_summary="Обновить статус
жалобы (модератор)", tags=['moderation'])
@swagger_auto_schema(methods=['patch'], operation_summary="Обновить
статус жалобы (модератор, PATCH)", tags=['moderation'])
@api_view(['POST', 'PATCH'])
@permission_classes([IsAuthenticated])
@login_required
def api_report_self_status_update(request, pk):
    """Moderator-only
    endpoint: update personal processing fields for a report."""
    if request.method not
    in ('POST', 'PATCH'):
        return
    JsonResponse({'error': 'method_not_allowed'}, status=405)
    if not
    hasattr(request.user, 'moderator_profile'):
        return
    JsonResponse({'error': 'forbidden'}, status=403)

    report =
    get_object_or_404(VacancyReport, pk=pk)
    try:
        data =
        json.loads(request.body or b'{}')
    except Exception:
        data = {}

    new_status =
    note =
    if new_status not in
    {s[0] for s in VacancyReport.SELF_STATUS_CHOICES}:
        return
    JsonResponse({'error': 'invalid_status'}, status=400)

    new_status
    report.self_status =
    report.moderator_note
    = note
    report.reviewed_by =

```

```

request.user

report.reviewed_at =
timezone.now()
report.save(update_fi
elds=['self_status', 'moderator_note', 'reviewed_by', 'reviewed_at',
'updated_at'])
return
JsonResponse({'ok': True})

@swagger_auto_schema(methods=['post'], operation_summary="Обновить статус
модерации вакансии", tags=['moderation'])
@swagger_auto_schema(methods=['patch'], operation_summary="Обновить
статус модерации вакансии (PATCH)", tags=['moderation'])
@api_view(['POST', 'PATCH'])
@permission_classes([IsAuthenticated])
@login_required
def api_vacancy_moderation_update(request, vacancy_id):
    """Moderator-only
    endpoint: update card-level vacancy moderation status and note."""
    if request.method not
in ('POST', 'PATCH'):
        return
    JsonResponse({'error': 'method_not_allowed'}, status=405)
    if not
hasattr(request.user, 'moderator_profile'):
        return
    JsonResponse({'error': 'forbidden'}, status=403)

    vac =
get_object_or_404(Vacancy, pk=vacancy_id)
    try:
        data =
json.loads(request.body or b'{}')
    except Exception:
        data = {}

    new_status =
note =
allowed_status =
[s[0] for s in VacancyModerationState.STATUS_CHOICES]
    if new_status not in
allowed_status:
        return
    JsonResponse({'error': 'invalid_status'}, status=400)

    state, _ =
VacancyModerationState.objects.get_or_create(
        vacancy=vac,
        moderator=request.use
r,
        defaults={'status':
VacancyModerationState.STATUS_NEW, 'note': ''},
    )
    state.status =
state.note = note
state.save(update_fie
lds=['status', 'note', 'updated_at'])
return

```

```

JsonResponse({'ok': True})

# -----
# Moderator – vacancy soft-deletion with PDF report
# -----

MAX_DELETION_PHOTOS = 8
MAX_PHOTO_BYTES = 8 * 1024 * 1024 # 8 MB per photo

def _compute_dominant_report_reason(vacancy):
    """Return (code,
    label) for the most common report reason on the vacancy.

    If no reports exist,
    returns ('', ''). If several reasons tie on top,
    returns ('',
    'Неизвестно') so the PDF reflects the ambiguity.

    """
    reason_counts = {}
    reports =
    list(vacancy.reports.all())

    for r in reports:
        reason_counts[r.reason_code] = reason_counts.get(r.reason_code, 0) + 1

    if not reason_counts:
        return ('', '')
    sorted_reasons =
    sorted(reason_counts.items(), key=lambda x: x[1], reverse=True)
    if
    len(sorted_reasons) > 1 and sorted_reasons[0][1] == sorted_reasons[1][1]:
        return ('',
        'Неизвестно')

    code =
    sorted_reasons[0][0]
    label =
    dict(VacancyReport.REASON_CHOICES).get(code, code)
    return (code, label)

@swagger_auto_schema(method='post', operation_summary="Мягкое удаление
вакансии модератором", tags=['moderation'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_moderator_delete_vacancy(request, vacancy_id):
    """Moderator-only:
    soft-delete a vacancy and record a deletion report.

    Accepts

    - reason
    - photos
    - photos_b64[]
    (one or more data:image/...;base64,... strings from clipboard paste,
    optional)

    """
    from
    django.core.files.base import ContentFile

```

```

import base64

if request.method !=
    'POST':
        return
    JsonResponse({'error': 'method_not_allowed', 'detail': 'Неверный метод
запроса.'}, status=405)
    if not
        hasattr(request.user, 'moderator_profile'):
            return
        JsonResponse({'error': 'forbidden', 'detail': 'Нет прав для выполнения
этого действия.'}, status=403)

    vac =
    get_object_or_404(Vacancy, pk=vacancy_id)
    if
        vac.is_moderator_deleted:
            return
        JsonResponse({'error': 'already_deleted', 'detail': 'Вакансия уже была
удалена ранее.'}, status=400)

    reason =
    (request.POST.get('reason') or '').strip()
    if not reason:
        return
        JsonResponse({'error': 'reason_required', 'detail': 'Необходимо указать
причину удаления.'}, status=400)
    if len(reason) >
        4000:
            return
            JsonResponse({'error': 'reason_too_long', 'detail': 'Причина слишком
длинная (максимум 4000 символов).'}, status=400)

    photo_files = []
    for f in
        request.FILES.getlist('photos'):
            if not
                str(f.content_type or '').startswith('image/'):
                    return
                    JsonResponse({'error': 'invalid_photo_type', 'detail': 'Один из файлов не
является изображением.'}, status=400)
            if f.size and f.size
                > MAX_PHOTO_BYTES:
                    return
                    JsonResponse({'error': 'photo_too_large', 'detail': 'Один из файлов
превышает допустимый размер (8 МБ).'}, status=400)

            photo_files.append(('
upload', f, None, None))

        b64_list =
        request.POST.getlist('photos_b64[]') or
        request.POST.getlist('photos_b64')
        for idx, b64_payload
            in enumerate(b64_list):
                if not
                    isinstance(b64_payload, str) or not b64_payload:
                        continue
                    raw = base64_payload
                    ext = 'png'
                    if
                        raw.startswith('data:'):
                            try:

```



```

raw = raw.split(',', 1)
'image/jpeg' in head:
'image/gif' in head:
'image/webp' in head:
ValueError:
JsonResponse({'error': 'invalid_photo_payload', 'detail': 'Некорректный
формат изображения из буфера обмена.'}, status=400)
base64.b64decode(raw, validate=False)
JsonResponse({'error': 'invalid_photo_payload', 'detail': 'Некорректный
формат изображения из буфера обмена.'}, status=400)
MAX_PHOTO_BYTES:
JsonResponse({'error': 'photo_too_large', 'detail': 'Одно из вставленных
изображений превышает допустимый размер (8 МБ).'}, status=400)
paste', None, raw_bytes, ext))
MAX_DELETION_PHOTOS:
JsonResponse({'error': 'too_many_photos', 'detail': f'Можно прикрепить не
более {MAX_DELETION_PHOTOS} фото.'}, status=400)
data.
vac.created_by
(manager_user.get_full_name() or '').strip() or manager_user.username
manager_user.email or ''
dominant_label = _compute_dominant_report_reason(vac)
vac.reports.count()
(request.user.get_full_name() or '').strip() or request.user.username
ModeratorDeletionReport.objects.create(

```

```

vacancy=vac,

moderator=request.use

r,

manager=manager_user,
reason=reason,

vacancy_title=vac.tit

le or '',

vacancy_company=vac.c

company or '',

vacancy_description=(
vac.description or vac.branded_description or '')[:8000],

vacancy_external_id=v

ac.external_id or '',

manager_full_name=man

ager_full,

manager_email=manager

_email,

moderator_full_name=m

oderator_full,

moderator_email=reque

st.user.email or '',

reports_count=reports

_count,

dominant_reason_code=

dominant_reason_label

)

for order, item in

kind, upload,

if kind == 'upload':

ModeratorDeletionPhot

else:

fname =

ModeratorDeletionPhot

report=report,

image=ContentFile(raw

_bytes, name=fname),

order=order,

```

```

vacancy.
ted = True
s=['is_moderator_deleted', 'is_active', 'updated_at'])

return JsonResponse({
    'ok': True,
    'report_id':
        report.pk,
        len(photo_files),
        'photos':
    })

@swagger_auto_schema(method='post', operation_summary="Восстановить
вакансию после удаления модератором", tags=['moderation'])
@api_view(['POST'])
@permission_classes([IsAuthenticated])
@login_required
def api_moderator_report_restore(request, report_id):
    """Admin-only:
    restore a vacancy that was soft-deleted by a moderator."""
    if request.method !=
'POST':
        return
JsonResponse({'error': 'method_not_allowed', 'detail': 'Неверный метод
запроса.'}, status=405)
    if not
(request.user.is_superuser and hasattr(request.user, 'admin_profile')):
        return
JsonResponse({'error': 'forbidden', 'detail': 'Нет прав для выполнения
этого действия.'}, status=403)
    report =
get_object_or_404(ModeratorDeletionReport, pk=report_id)
    if
report.is_restored:
        return
JsonResponse({'error': 'already_restored', 'detail': 'Вакансия уже была
восстановлена ранее.'}, status=400)
    vac = report.vacancy
    if not vac:
        return
JsonResponse({'error': 'vacancy_missing', 'detail': 'Вакансия не найдена
или была удалена из базы.'}, status=400)
    vac.is_moderator_dele
ted = False
vac.is_active = True
vac.save(update_field
s=['is_moderator_deleted', 'is_active', 'updated_at'])
report.is_restored =
report.restored_by =
request.user
report.restored_at =
timezone.now()

```

```

        report.save(update_fields=
elds=['is_restored', 'restored_by', 'restored_at'])

        return

    JsonResponse({'ok': True})

@swagger_auto_schema(method='patch',
operation_summary="Включить/выключить активность вакансии",
tags=['vacancies'])
@api_view(['PATCH'])
@permission_classes([IsAuthenticated])
@login_required
def api_vacancy_toggle_active(request, pk):
    """Toggle the is_active flag for a manager's own vacancy."""
    vac = get_object_or_404(Vacancy, pk=pk)
    if not hasattr(request.user, 'manager') or vac.created_by !=
request.user:
        return JsonResponse({'error': 'forbidden'}, status=403)
    vac.is_active = not vac.is_active
    vac.save(update_fields=['is_active'])
    return JsonResponse({'ok': True, 'is_active': vac.is_active})

@login_required
def my_vacancies(request):
    if not
        return

    vacancies = (
        Vacancy.objects

        .filter(created_by=re
quest.user)

        .exclude(is_moderator

        .order_by('-
created_at')

    )
    return

    render(request, 'vacancies/my_vacancies.html', {
        'vacancies':
        'manager':
    })

def custom_404(request, exception=None):
    return render(request, '404.html', status=404)

def custom_403(request, exception=None):
    return

    render(request, '403.html', status=403)

```

38. config/__init__.py

39. config/asgi.py

"""

ASGI config for config project.

It exposes the ASGI callable as a module-level variable named
`application`.

For more information on this file, see
<https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/>
"""

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

application = get_asgi_application()
```

40. config/celery.py

```
import os
from celery import Celery
from celery.schedules import crontab
from celery.signals import heartbeat_sent, worker_ready, worker_shutdown
from django.conf import settings as django_settings

from .service_status import mark_celery_alive, mark_celery_stopped

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

app = Celery('job_aggregator')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

@worker_ready.connect
def _mark_worker_ready(**kwargs):
    mark_celery_alive()
    if not getattr(django_settings, 'VACANCY_FETCH_ON_STARTUP', True):
        return
    # Warm start: kick off both sources immediately, without waiting
    # first beat interval.
    try:
        from vacancies.tasks import fetch_hh_task, fetch_trudvsem_task
        fetch_hh_task.delay(
            pages=getattr(django_settings, 'HH_STARTUP_FETCH_PAGES', 1),
            per_page=getattr(django_settings,
                'HH_STARTUP_FETCH_PER_PAGE', 10),
        )
        fetch_trudvsem_task.delay(
            limit=getattr(django_settings,
                'FALLBACK_TRUDVSEM_STARTUP_LIMIT', 10),
        )
    except Exception:
        pass

@heartbeat_sent.connect
def _mark_worker_heartbeat(**kwargs):
    mark_celery_alive()

@worker_shutdown.connect
def _mark_worker_shutdown(**kwargs):
```

```

    mark_celery_stopped()

# — Periodic schedule


---


# Pipeline order:  vacancies → ratings → regions/experience
# (dictionaries)
#
# To change the interval, edit SYNC_INTERVAL_SECONDS below.
# Current: 180 seconds (3 minutes) — for testing.
# Production recommendation: 600 (10 min) or 3600 (1 hour).

SYNC_INTERVAL_SECONDS = getattr(django_settings, 'HH_SYNC_INTERVAL_SEC',
                                  600)
HH_FETCH_PAGES = getattr(django_settings, 'HH_FETCH_PAGES', 3)
HH_FETCH_PER_PAGE = getattr(django_settings, 'HH_FETCH_PER_PAGE', 50)
TRUDVSEM_INTERVAL_SECONDS = getattr(
    django_settings,
    'FALLBACK_TRUDVSEM_SYNC_INTERVAL_SEC',
    SYNC_INTERVAL_SECONDS,
)
TRUDVSEM_LIMIT = getattr(django_settings, 'FALLBACK_TRUDVSEM_LIMIT', 200)

app.conf.beat_schedule = {
    'fetch-vacancies': {
        'task': 'vacancies.tasks.fetch_hh_task',
        'schedule': SYNC_INTERVAL_SECONDS,
        'kwargs': {'pages': HH_FETCH_PAGES, 'per_page':
HH_FETCH_PER_PAGE},
    },
    # After each vacancy fetch wave, fill in any missing descriptions.
    # Offset by 30 s so it runs after fetch_hh_task finishes.
    'backfill-descriptions': {
        'task': 'vacancies.tasks.backfill_descriptions_task',
        'schedule': SYNC_INTERVAL_SECONDS,
        'kwargs': {'limit': getattr(django_settings, 'HH_BACKFILL_LIMIT',
80)},
    },
    # Soft-delete HH vacancies that are no longer available: TTL + API
    # check.
    # Runs every 6 hours; checks 50 vacancies per run via the HH API.
    'check-hh-vacancy-status': {
        'task': 'vacancies.tasks.check_hh_vacancy_status_task',
        'schedule': getattr(django_settings,
'HH_STALE_CHECK_INTERVAL_SEC', 21600),
        'kwargs': {'batch_size': getattr(django_settings,
'HH_STALE_CHECK_BATCH', 50)},
    },
    'check-trudvsem-vacancy-status': {
        'task': 'vacancies.tasks.check_trudvsem_vacancy_status_task',
        'schedule': getattr(django_settings,
'FALLBACK_TRUDVSEM_STATUS_CHECK_INTERVAL_SEC', 3600),
        'kwargs': {'batch_size': getattr(django_settings,
'FALLBACK_TRUDVSEM_STATUS_CHECK_BATCH', 100)},
    },
    'fetch-trudvsem-fallback': {
        'task': 'vacancies.tasks.fetch_trudvsem_task',
        'schedule': TRUDVSEM_INTERVAL_SECONDS,
        'kwargs': {'limit': TRUDVSEM_LIMIT},
    },
    # Rotating database backup daily at 00:00 (keeps last
    # DB_BACKUP_MAX_COUNT copies).
    'backup-database': {
        'task': 'vacancies.tasks.backup_database_task',

```

```

        'schedule': crontab(hour=0, minute=0), # every day at 00:00
    },
    # Send interview reminders (1 day before, 1 hour before, 5 min
before).
    'notify-interview-reminders': {
        'task': 'accounts.tasks.notify_interview_reminders_task',
        'schedule': 60, # every 1 minute – needed for the 5-min window
    },
    # Send calendar-note reminders close to the exact note time.
    'notify-calendar-note-reminders': {
        'task': 'accounts.tasks.notify_calendar_note_reminders_task',
        'schedule': 60, # every 1 minute
    },
}

```

41. config/openapi_generator.py

```

"""Генератор OpenAPI: добавляет русские описания и примеры к операциям
(ReDoc / Swagger)."""

from drf_yasg.generators import OpenAPISchemaGenerator

from .openapi_ru_docs import RU_OPERATION_DESCRIPTIONS,
RU_TAG_DESCRIPTIONS

class RussianOpenAPISchemaGenerator(OpenAPISchemaGenerator):
    """Дополняет операции текстом из `openapi_ru_docs` (ключ: путь в
схеме + HTTP-метод)."""

    def get_operation(self, view, path, prefix, method, components,
request):
        operation = super().get_operation(view, path, prefix, method,
components, request)
        if operation is None:
            return None

        suffix = path[len(prefix) :]
        if not suffix.startswith("/"):
            suffix = "/" + suffix

        extra = RU_OPERATION_DESCRIPTIONS.get((suffix, method.lower()))
        if not extra:
            return operation

        current = (operation.get("description") or "").strip()
        operation["description"] = (extra.rstrip() + ("\n\n" + current if
current
else "")).strip()
        return operation

    def get_schema(self, request=None, public=False):
        schema = super().get_schema(request, public)
        if RU_TAG_DESCRIPTIONS:
            schema["tags"] = RU_TAG_DESCRIPTIONS
        return schema

```

42. config/openapi_ru_docs.py

```

# Русские описания и примеры для ReDoc / Swagger (дополняют
operation_summary).
import json

def _j(data) -> str:
    return "`json\n" + json.dumps(data, ensure_ascii=False, indent=2) +

```

```
"\n```\n"
```

```
def _d(
    text: str,
    *,
    req_json=None,
    resp_json=None,
    note: str | None = None,
) -> str:
    parts = [text.strip()]
    if req_json is not None:
        parts.append("**Пример тела запроса (JSON):**\n\n" +
            _j(req_json))
    if resp_json is not None:
        parts.append("**Пример ответа:**\n\n" + _j(resp_json))
    if note:
        parts.append(note.strip())
    return "\n\n".join(parts)
```

```
RU_TAG_DESCRIPTIONS = [
    {
        "name": "accounts",
        "description": "Аккаунты: регистрация, вход, профиль, файлы,
отклики, закладки, чаты, тема UI, Telegram/e-mail.",
    },
    {
        "name": "presets",
        "description": "Пользовательские пресеты фильтров поиска
вакансий.",
    },
    {
        "name": "documents",
        "description": "Документы для верификации в профиле (паспорт,
СНИЛС и др.).",
    },
    {
        "name": "admin",
        "description": "Резервное копирование БД и модерация обратной
связи (только администратор).",
    },
    {
        "name": "calendar",
        "description": "Календарь: события, заметки, экспорт месяца,
календарь менеджера.",
    },
    {
        "name": "interviews",
        "description": "Назначение, отмена и перенос собеседований.",
    },
    {
        "name": "vacancies",
        "description": "Вакансии: жалобы, переключение активности,
описание, рейтинг работодателя.",
    },
    {
        "name": "moderation",
        "description": "Модерация: статусы жалоб, карточка вакансии,
удаление, восстановление.",
    },
    {
        "name": "api",
```



```

        "description": "Публичные вспомогательные методы (описание
вакансии и т.п.).",
    },
]

_ENTRIES: list[tuple[str, str, str]] = []

# — accounts: auth / register —
_ENTRIES += [
    (
        "/accounts/api/register/",
        "post",
        _d(
            "Регистрация соискателя. Допустимы JSON, form-urlencoded или
multipart. "
            "Обязательны: фамилия, имя, username, телефон в формате РФ,
пол (M/F), город, дата рождения (YYYY-MM-DD), гражданство, пароль.",
            req_json={
                "last_name": "Иванов",
                "first_name": "Иван",
                "patronymic": "Иванович",
                "username": "ivan",
                "email": "ivan@example.com",
                "phone": "79161234567",
                "gender": "M",
                "city": "Москва",
                "birth_date": "1995-05-01",
                "citizenship": "RU",
                "telegram": "@ivan",
                "consent_email": True,
                "consent_telegram": False,
                "password": "Secret123",
            },
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/register-manager/",
        "post",
        _d(
            "Регистрация менеджера (работодателя). JSON с полями
username, password, имя, фамилия и др.",
            req_json={
                "username": "manager1",
                "password": "Secret123",
                "last_name": "Петров",
                "first_name": "Пётр",
                "patronymic": "Петрович",
                "email": "hr@company.ru",
                "telegram": "@company_hr",
                "company": "ООО Ромашка",
                "phone": "74951234567",
            },
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/login/",
        "post",
        _d(
            "Вход в систему. В теле JSON укажите `username` или `email`,
и `password`. "
            "Устанавливается сессионная cookie; в Swagger/ReDoc

```

```

используйте Authorize → Session.",
    req_json={"username": "ivan", "password": "Secret123"},
    resp_json={"ok": True, "redirect_to": "/"},
),
),
(
    "/accounts/api/logout/",
    "post",
    _d(
        "Выход: сессия сбрасывается.",
        resp_json={"ok": True},
    ),
),
(
    "/accounts/api/ui/theme/",
    "get",
    _d(
        "Текущая тема интерфейса для авторизованного пользователя
(`light` / `dark`) или `null` для гостя.",
        resp_json={"ok": True, "theme": "dark"},
    ),
),
(
    "/accounts/api/ui/theme/",
    "post",
    _d(
        "Сохранить тему. Только для авторизованных. Тело JSON:
`theme` = `light` или `dark`.",
        req_json={"theme": "dark"},
        resp_json={"ok": True, "theme": "dark"},
    ),
),
]

# — profile / data —
_ENTRIES += [
    (
        "/accounts/api/profile-data/",
        "get",
        _d(
            "Полные данные профиля (соискатель и/или менеджер). Требуется
вход; для администратора вернётся ошибка `admin_has_no_profile`.",
            resp_json={"ok": True, "first_name": "Иван", "last_name":
"Иванов", "email": "ivan@example.com"},
        ),
    ),
    (
        "/accounts/api/profile/update/",
        "patch",
        _d(
            "Частичное обновление профиля JSON-объектом (имя, контакты,
город и т.д. — см. реализацию на сервере).",
            req_json={"first_name": "Иван", "city": "Санкт-Петербург"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/profile/education/",
        "post",
        _d("Добавить запись об образовании.", req_json={"level": "ВО",
"institution": "МГУ", "graduation_year": 2020}),
    ),
    (

```

```

        "/accounts/api/profile/education/{id}/",
        "patch",
        _d("Обновить запись об образовании по `id`."),
req_json={"institution": "СПбГУ"}),
    ),
    (
        "/accounts/api/profile/education/{id}/",
        "delete",
        _d("Удалить запись об образовании."),
    ),
    (
        "/accounts/api/profile/extra-education/",
        "post",
        _d(
            "Добавить дополнительное образование (в т.ч.
`document_serial`)." ,
            req_json={"title": "Курс Python", "organization": "Школа X",
"year": 2024, "document_serial": "AB-123456"},
        ),
    ),
    (
        "/accounts/api/profile/extra-education/{id}/",
        "patch",
        _d("Обновить доп. образование.", req_json={"document_serial":
"CD-999"})),
    ),
    (
        "/accounts/api/profile/extra-education/{id}/",
        "delete",
        _d("Удалить доп. образование."),
    ),
    (
        "/accounts/api/profile/work/",
        "post",
        _d(
            "Добавить опыт работы.",
            req_json={"company": "ООО Тест", "position": "Разработчик",
"start_month": 1, "start_year": 2020},
        ),
    ),
    (
        "/accounts/api/profile/work/{id}/",
        "patch",
        _d("Обновить опыт работы.", req_json={"position": "Senior"})),
    ),
    (
        "/accounts/api/profile/work/{id}/",
        "delete",
        _d("Удалить запись об опыте работы."),
    ),
    (
        "/accounts/api/profile/skills/",
        "put",
        _d(
            "Полная замена списка навыков.",
            req_json={"skills": ["Python", "Django", "SQL"]},
        ),
    ),
]

# — documents —
_MP_DOC = (
    "***Формат:** `multipart/form-data`. Передавайте поля как в HTML-

```

```

форме; для файлов — поле `files` (несколько файлов)."
```

```

)
_ENTRIES += [
    (
        "/accounts/api/profile/documents/",
        "get",
        _d("Список документов текущего пользователя (маскированные
номера, ссылки на файлы)."),
    ),
    (
        "/accounts/api/profile/documents/",
        "post",
        _d(
            "Добавить документ: тип, серия/номер, дата выдачи, файлы
(JPG/PNG/WEBP/PDF, до 10 МБ каждый).",
            note=_MP_DOC
            + "\n\n**Поля формы (пример имён):** `doc_type`, `serial`,
`number`, `issued_date`, `issued_by`, `division_code`, `files`.",
        ),
    ),
    (
        "/accounts/api/profile/documents/{id}/delete/",
        "post",
        _d("Удалить документ с идентификатором `id` (только свой).",
resp_json={"ok": True}),
    ),
]

# — uploads —
_ENTRIES += [
    (
        "/accounts/api/upload-avatar/",
        "post",
        _d(
            "Загрузка аватара. `multipart/form-data`, поле **`avatar`** —
файл изображения.",
            note=_MP_DOC,
        ),
    ),
    (
        "/accounts/api/upload-company-logo/",
        "post",
        _d(
            "Логотип компании (только менеджер). Поле **`logo`**.",
            note=_MP_DOC,
        ),
    ),
    (
        "/accounts/api/upload-resume/",
        "post",
        _d(
            "Загрузка резюме Word. Поле **`resume`**, расширения `.doc` /
`.docx`.",
            note=_MP_DOC,
        ),
    ),
    (
        "/accounts/api/delete-resume/",
        "post",
        _d("Удалить загруженный файл резюме соискателя.",
resp_json={"ok": True}),
    ),
]

```

```

# — apply / bookmark / analytics —
_ENTRIES += [
    (
        "/accounts/api/apply/",
        "post",
        _d(
            "Отклик на вакансию.",
            req_json={"vacancy_id": 123, "cover_letter": "Готов обсудить
проект"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/applications/{id}/status/",
        "post",
        _d(
            "Сменить статус отклика (менеджер).",
            req_json={"status": "accepted"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/bookmark/{id}/",
        "post",
        _d(
            "Переключить закладку на вакансии; `id` — первичный ключ
вакансии в БД.",
            resp_json={"ok": True, "bookmarked": True},
        ),
    ),
    (
        "/accounts/api/analytics/",
        "get",
        _d("Персональная аналитика соискателя: закладки, просмотры,
агрегаты для графиков."),
    ),
    (
        "/accounts/api/resume/analyze/",
        "get",
        _d("Анализ заполненности резюме (подсказки для пользователя)."),
    ),
]

# — feedback / telegram / email —
_ENTRIES += [
    (
        "/accounts/api/feedback/",
        "get",
        _d("Статус лимита отправки предложений/критики (раз в неделю для
обычных ролей)."),
    ),
    (
        "/accounts/api/feedback/",
        "post",
        _d(
            "Отправить предложение или критику по сайту.",
            req_json={"kind": "suggestion", "text": "Добавьте тёмную тему
везде"},
            resp_json={"ok": True},
        ),
    ),
    (

```

```

        "/accounts/api/send-telegram-welcome/",
        "post",
        _d("Инициировать приветствие в Telegram (см. логику согласий).",
req_json={"telegram": "@user"}),
    ),
    (
        "/accounts/api/set-consent/",
        "post",
        _d("Согласие на уведомления Telegram.",
req_json={"consent_telegram": True}),
    ),
    (
        "/accounts/api/test-message/",
        "post",
        _d("Тестовое сообщение в Telegram для текущего пользователя."),
    ),
    (
        "/accounts/api/unlink-telegram/",
        "post",
        _d("Отвязать Telegram.", resp_json={"ok": True}),
    ),
    (
        "/accounts/api/set-email-consent/",
        "post",
        _d("Согласие на рассылку по e-mail.", req_json={"consent_email":
True}),
    ),
    (
        "/accounts/api/test-email/",
        "post",
        _d("Отправить тестовое письмо."),
    ),
    (
        "/accounts/api/unlink-email/",
        "post",
        _d("Отключить уведомления по почте."),
    ),
]

# — presets / metro / role / delete —
_ENTRIES += [
    (
        "/accounts/api/presets/",
        "get",
        _d("Список пресетов фильтров."),
    ),
    (
        "/accounts/api/presets/",
        "post",
        _d(
            "Создать пресет.",
            req_json={"name": "Моя подборка", "filters": {"query":
"Python"}}),
            resp_json={"ok": True, "id": 1},
        ),
    ),
    (
        "/accounts/api/presets/{id}/",
        "patch",
        _d("Обновить пресет.", req_json={"name": "Новое имя"}),
    ),
    (
        "/accounts/api/presets/{id}/",

```

```

        "delete",
        _d("Удалить пресет."),
    ),
    (
        "/accounts/api/metro-data/",
        "get",
        _d("Справочник станций метро для форм."),
    ),
    (
        "/accounts/api/switch-role/",
        "post",
        _d(
            "Переключить активную роль (если у пользователя есть и
соискатель, и менеджер).",
            req_json={"role": "applicant"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/delete/",
        "post",
        _d("Удаление своего аккаунта (подтверждение на стороне
клиента)."),
    ),
    (
        "/accounts/telegram-webhook/",
        "post",
        _d(
            "Вебхук Telegram Bot API. Вызывается только серверами
Telegram; для ручного теста нужен валидный update JSON.",
            note="**Пример:** стандартный объект `Update` из Bot API.",
        ),
    ),
]

# — chats —
_ENTRIES += [
    (
        "/accounts/api/chats/start/",
        "post",
        _d(
            "Начать чат с соискателем (менеджер).",
            req_json={"applicant_user_id": 5},
            resp_json={"ok": True, "chat_id": 1},
        ),
    ),
    (
        "/accounts/api/chats/{id}/send/",
        "post",
        _d(
            "Отправить сообщение в чат `id`.",
            req_json={"text": "Здравствуйте!"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/chats/{id}/messages/",
        "get",
        _d("Получить новые сообщения (long-poll / инкремент — см. query-
параметры в коде)."),
    ),
    (
        "/accounts/api/chats/{id}/delete/",

```

```

        "delete",
        _d("Удалить чат для текущего пользователя."),
    ),
]

# — admin —
_ENTRIES += [
    (
        "/accounts/api/admin/feedback/{feedback_id}/action/",
        "post",
        _d(
            "Действие над сообщением обратной связи: `action` = `archive`
| `reject` | `resolve` | `restore` (form или query).",
            req_json={"action": "resolve"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/admin/backup/create/",
        "post",
        _d(
            "Создать файл резервной копии SQLite в каталоге бэкапов.",
            resp_json={"ok": True, "filename": "backup_20260101.sqlite3"},
            "size_bytes": 12345},
        ),
    ),
    (
        "/accounts/api/admin/backup/restore/",
        "post",
        _d(
            "Восстановить БД из имени файла бэкапа (только безопасное имя
файла).",
            req_json={"filename": "backup_20260101.sqlite3"},
            resp_json={"ok": True},
        ),
    ),
    (
        "/accounts/api/admin/backup/delete/",
        "post",
        _d(
            "Удалить файл бэкапа по имени.",
            req_json={"filename": "backup_old.sqlite3"},
            resp_json={"ok": True},
        ),
    ),
]

# — calendar —
_CAL_NOTE = (
    "**Заметки календаря (JSON-тело):** создание — `POST` с полями `date`
(YYYY-MM-DD), `title`, `text`, `color`, `time`; "
    "обновление — `PATCH` с `id` и изменяемыми полями; удаление —
`DELETE` с `id`."
)
_ENTRIES += [
    (
        "/accounts/api/calendar/events/",
        "get",
        _d(
            "События на дату. Query: `date=YYYY-MM-DD`. Набор событий
зависит от роли (соискатель / менеджер / админ).",
        ),
    ),
]

```



```

(
    "/accounts/api/calendar/note/",
    "post",
    _d("Создать заметку.", note=_CAL_NOTE, req_json={"date": "2026-
04-27", "title": "Созвон", "text": "Обсудить оффер"}),
),
(
    "/accounts/api/calendar/note/",
    "patch",
    _d("Обновить заметку.", note=_CAL_NOTE, req_json={"id": 1,
"title": "Новый заголовок"}),
),
(
    "/accounts/api/calendar/note/",
    "delete",
    _d("Удалить заметку.", note=_CAL_NOTE, req_json={"id": 1}),
),
(
    "/accounts/api/calendar/month/",
    "get",
    _d("Все заметки и интервью за месяц. Query: `year`, `month`."),
),
(
    "/accounts/api/calendar/notes-index/",
    "get",
    _d("Плоский список заметок для поиска по календарю."),
),
(
    "/accounts/api/manager/calendar/events/",
    "get",
    _d("События календаря менеджера на дату (`date=YYYY-MM-DD`):
отклики, интервью, заметки."),
),
]

# — interviews —
_ENTRIES += [
    (
        "/accounts/api/interviews/schedule/",
        "post",
        _d(
            "Назначить собеседование.",
            req_json={
                "applicant_user_id": 3,
                "vacancy_id": 10,
                "date": "2026-05-01",
                "time": "14:00",
                "location": "Офис, ул. Примерная, 1",
                "notes": "Возьмите паспорт",
            },
            resp_json={"ok": True, "id": 1},
        ),
    ),
    (
        "/accounts/api/interviews/cancel/",
        "delete",
        _d(
            "Отменить собеседование. Тело JSON: `id` интервью.",
            req_json={"id": 1},
            resp_json={"ok": True},
        ),
    ),
    (

```

```

        "/accounts/api/interviews/reschedule/",
        "patch",
        _d(
            "Перенос собеседования (менеджер).",
            req_json={"id": 1, "date": "2026-05-02", "time": "15:30"},
            resp_json={"ok": True},
        ),
    ),
]

# — vacancies app (prefix /api/ or /{id}/) —
_ENTRIES += [
    (
        "/api/employer-rating/{hh_id}/",
        "get",
        _d("Рейтинг работодателя по идентификатору hh.ru (`hh_id`)."),
    ),
    (
        "/api/vacancy-description/{id}/",
        "get",
        _d("Текст описания вакансии (ленивая подгрузка для карточки)."),
    ),
    (
       ("/{id}/report/",
        "post",
        _d(
            "Жалоба на локальную вакансию. `id` — первичный ключ
            вакансии. Одна жалоба от пользователя на вакансию.",
            req_json={"reason_code": "spam", "reason_text":
            "Подозрительные условия"},
            resp_json={"ok": True, "already_reported": False, "id": 1},
        ),
    ),
    (
       ("/{id}/toggle-active/",
        "patch",
        _d(
            "Переключить признак активности своей вакансии (менеджер-
            владелец).",
            resp_json={"ok": True, "is_active": False},
        ),
    ),
    (
        "/api/reports/{id}/self-status/",
        "post",
        _d(
            "Модератор: обновить личный статус обработки жалобы
            (`self_status`, заметка и т.д.).",
            req_json={"self_status": "in_work", "moderator_note":
            "Проверяю"},
        ),
    ),
    (
        "/api/reports/{id}/self-status/",
        "patch",
        _d("То же, что POST — частичное обновление полей жалобы
            модератором.", req_json={"self_status": "waiting"}),
    ),
    (
        "/api/reports/vacancy/{vacancy_id}/state/",
        "post",
        _d(
            "Модератор: статус модерации карточки вакансии.",

```

```

        req_json={"status": "in_review", "note": "Нужна проверка
контактов"},
    ),
    (
        "/api/reports/vacancy/{vacancy_id}/state/",
        "patch",
        _d("Частичное обновление статуса модерации вакансии.",
req_json={"status": "approved"}),
    ),
    (
        "/api/moderator/vacancy/{vacancy_id}/delete/",
        "post",
        _d(
            "Мягкое удаление вакансии модератором. Обычно `multipart`:
текст причины и фото-доказательства.",
            note=_MP_DOC + "\n\nПоле **`reason`** обязательно; файлы – по
логике представления `api_moderator_delete_vacancy`.",
        ),
    ),
    (
        "/api/admin/moderator-report/{report_id}/restore/",
        "post",
        _d(
            "Администратор: восстановить вакансию после удаления
модератором (`report_id` – отчёт об удалении).",
            resp_json={"ok": True},
        ),
    ),
]

RU_OPERATION_DESCRIPTIONS: dict[tuple[str, str], str] = {}
for path, method, doc in _ENTRIES:
    key = (path, method)
    if key in RU_OPERATION_DESCRIPTIONS:
        raise ValueError(f"Duplicate OpenAPI doc key: {key}")
    RU_OPERATION_DESCRIPTIONS[key] = (
        doc
        + "\n\n---\n**Общее:** для POST/PATCH/DELETE с сессией в браузере
может потребоваться заголовок `X-CSRFToken` "
        + "(в Swagger он подставляется после входа при использовании схемы
Session).")
    )

```

43. config/service_status.py

```

from __future__ import annotations

import time
from pathlib import Path

from django.conf import settings

HEARTBEAT_FILE_NAME = 'celery_worker_heartbeat.touch'
DEFAULT_HEARTBEAT_TTL = 90

def get_heartbeat_path() -> Path:
    base_dir = Path(getattr(settings, 'BASE_DIR', Path.cwd()))
    return base_dir / 'logs' / HEARTBEAT_FILE_NAME

def mark_celery_alive() -> None:
    heartbeat_path = get_heartbeat_path()
    heartbeat_path.parent.mkdir(parents=True, exist_ok=True)

```

```

        heartbeat_path.touch()

def mark_celery_stopped() -> None:
    heartbeat_path = get_heartbeat_path()
    try:
        heartbeat_path.unlink()
    except FileNotFoundError:
        pass

def is_celery_online() -> bool:
    heartbeat_path = get_heartbeat_path()
    if not heartbeat_path.exists():
        return False
    ttl = int(getattr(settings, 'CELERY_STATUS_TTL_SECONDS',
DEFAULT_HEARTBEAT_TTL))
    age_seconds = time.time() - heartbeat_path.stat().st_mtime
    return age_seconds <= ttl

def service_status_context(request):
    online = is_celery_online()
    return {
        'service_center_online': online,
        'service_center_status_text': 'online' if online else 'offline',
    }

```

44. config/settings.py

```

"""
Django settings for config project.

Generated by 'django-admin startproject' using Django 6.0.2.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

from pathlib import Path
import os
from dotenv import load_dotenv

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# Load .env from project root (if present); silently ignored in
production.
load_dotenv(BASE_DIR / '.env')

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
5anve2ez#k4xki7oaqx%2cc=%@(rw^i(t)qxsf4_9o@hje9g'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Base URL used in outgoing notifications (email, Telegram).

```

```

SITE_URL = os.environ.get('SITE_URL', 'http://localhost:8000')
HH_API_USER_AGENT = os.environ.get('HH_API_USER_AGENT', f'job-aggregator-
diploma/1.0 ({SITE_URL})')
HH_API_TOKEN = os.environ.get('HH_API_TOKEN', '').strip()
HH_API_CLIENT_ID = os.environ.get('HH_API_CLIENT_ID', '').strip()
HH_API_CLIENT_SECRET = os.environ.get('HH_API_CLIENT_SECRET', '').strip()
HH_SYNC_INTERVAL_SEC = int(os.environ.get('HH_SYNC_INTERVAL_SEC', '300'))
HH_FETCH_PAGES = int(os.environ.get('HH_FETCH_PAGES', '5'))
HH_FETCH_PER_PAGE = int(os.environ.get('HH_FETCH_PER_PAGE', '100'))
HH_STARTUP_FETCH_PAGES = int(os.environ.get('HH_STARTUP_FETCH_PAGES',
'1'))
HH_STARTUP_FETCH_PER_PAGE =
int(os.environ.get('HH_STARTUP_FETCH_PER_PAGE', '10'))
VACANCY_FETCH_ON_STARTUP = os.environ.get('VACANCY_FETCH_ON_STARTUP',
'false').lower() in (
    '1', 'true', 'yes', 'on',
)
HH_BACKFILL_LIMIT = int(os.environ.get('HH_BACKFILL_LIMIT', '80'))
HH_STALE_CHECK_INTERVAL_SEC =
int(os.environ.get('HH_STALE_CHECK_INTERVAL_SEC', '21600'))
HH_STALE_CHECK_BATCH = int(os.environ.get('HH_STALE_CHECK_BATCH', '50'))

# — Media files (user uploads)


---


MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'drf_yasg',
    'vacancies',
    'accounts',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

SWAGGER_SETTINGS = {
    'SECURITY_DEFINITIONS': {
        'Session': {
            'type': 'apiKey',
            'name': 'sessionid',
            'in': 'cookie',
        }
    },
    'USE_SESSION_AUTH': True,
    'LOGIN_URL': '/accounts/login/',
    'LOGOUT_URL': '/accounts/logout/',
    'DEFAULT_INFO': 'config.urls.api_info',
}

```

```

        'DEFAULT_GENERATOR_CLASS':
'config.openapi_generator.RussianOpenAPISchemaGenerator',
    }

# Документация ReDoc (тот же OpenAPI, что и у Swagger)
REDOC_SETTINGS = {
    'LAZY_RENDERING': False,
    'HIDE_HOSTNAME': False,
    'EXPAND_RESPONSES': 'all',
    'PATH_IN_MIDDLE': False,
    'NATIVE_SCROLLBARS': False,
    'REQUIRED_PROPS_FIRST': True,
    'HIDE_DOWNLOAD_BUTTON': False,
    'FETCH_SCHEMA_WITH_QUERY': True,
}

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'config.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
                'config.service_status.service_status_context',
            ],
        },
    },
]

WSGI_APPLICATION = 'config.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
        'OPTIONS': {
            'timeout': 30,
            # init_command убираем — это MySQL-опция, не SQLite
        },
    }
}

# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-
# validators

```

```

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/

LANGUAGE_CODE = 'ru-ru'

TIME_ZONE = 'Europe/Moscow'

USE_I18N = True

USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = 'static/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# reload trigger

# Celery configuration
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'
CELERY_TIMEZONE = TIME_ZONE
CELERY_WORKER_POOL = 'threads'      # solo breaks on Windows with >1
concurrent task
CELERY_WORKER_CONCURRENCY = 4      # 4 threads – good for I/O-bound HTTP
fetches
# Enable Redis priority queues (0=lowest ... 9=highest).
# User-triggered description fetches use priority=9; backfill uses
priority=1.
CELERY_TASK_QUEUE_MAX_PRIORITY = 10
CELERY_TASK_DEFAULT_PRIORITY = 5

FALLBACK_TRUDVSEM_ENABLED = os.environ.get('FALLBACK_TRUDVSEM_ENABLED',
'true').lower() in (
    '1', 'true', 'yes', 'on',
)
FALLBACK_TRUDVSEM_LIMIT = int(os.environ.get('FALLBACK_TRUDVSEM_LIMIT',
'200'))

```

```

FALLBACK_TRUDVSEM_STARTUP_LIMIT =
int(os.environ.get('FALLBACK_TRUDVSEM_STARTUP_LIMIT', '10'))
FALLBACK_TRUDVSEM_SYNC_INTERVAL_SEC = int(
    os.environ.get('FALLBACK_TRUDVSEM_SYNC_INTERVAL_SEC',
str(HH_SYNC_INTERVAL_SEC))
)
FALLBACK_TRUDVSEM_TTL_DAYS =
int(os.environ.get('FALLBACK_TRUDVSEM_TTL_DAYS', '5'))
FALLBACK_TRUDVSEM_STATUS_CHECK_INTERVAL_SEC = int(
    os.environ.get('FALLBACK_TRUDVSEM_STATUS_CHECK_INTERVAL_SEC', '3600')
)
FALLBACK_TRUDVSEM_STATUS_CHECK_BATCH = int(
    os.environ.get('FALLBACK_TRUDVSEM_STATUS_CHECK_BATCH', '100')
)
FALLBACK_TRUDVSEM_RECENT_MINUTES =
int(os.environ.get('FALLBACK_TRUDVSEM_RECENT_MINUTES', '5'))
FALLBACK_TRUDVSEM_MAX_PAGES_PER_RUN =
int(os.environ.get('FALLBACK_TRUDVSEM_MAX_PAGES_PER_RUN', '1000'))
FALLBACK_TRUDVSEM_TEXTS = os.environ.get(
    'FALLBACK_TRUDVSEM_TEXTS',
    'продавец, кассир, водитель, бухгалтер, менеджер, администратор, python, ана
литик, разработчик',
)

# 2GIS API key (can be overridden via environment variable)
DGIS_API_KEY = os.environ.get('DGIS_API_KEY', '0cd687b3-98c4-463e-9f30-
cf4bd2dc4c4e')

# — Database backups


---


BACKUP_DIR = BASE_DIR / 'backups'
DB_BACKUP_MAX_COUNT = 3 # keep at most this many backups; oldest is
deleted when exceeded
CELERY_STATUS_TTL_SECONDS = 90

# Telegram bot token used to send welcome messages to users who consent.
# For production, override via environment variable `TELEGRAM_BOT_TOKEN`.
TELEGRAM_BOT_TOKEN = os.environ.get('TELEGRAM_BOT_TOKEN',
'8503249417:AAGWJdHqYCpki4TDZEhbegecGYVbVIXLHQY')
# Celery beat schedule can be set in config/celery.py; default fetch
every 10 minutes

# — Email settings


---


# Configure credentials in .env (see .env.example). Port 587 + STARTTLS
works
# for Mail.ru, Gmail, Yandex and most other providers.
EMAIL_BACKEND = os.environ.get(
    'EMAIL_BACKEND',
    'django.core.mail.backends.smtp.EmailBackend',
)
EMAIL_HOST = os.environ.get('EMAIL_HOST', 'smtp.mail.ru')
EMAIL_PORT = int(os.environ.get('EMAIL_PORT', 587))
EMAIL_USE_TLS = os.environ.get('EMAIL_USE_TLS', 'True') != 'False'
EMAIL_USE_SSL = False # TLS (STARTTLS) and SSL are mutually exclusive
EMAIL_HOST_USER = os.environ.get('EMAIL_HOST_USER', '')
EMAIL_HOST_PASSWORD = os.environ.get('EMAIL_HOST_PASSWORD', '')
DEFAULT_FROM_EMAIL = os.environ.get('DEFAULT_FROM_EMAIL', 'JobFlex
<noreply@jobflex.ru>')

```

45. config/urls.py

```

"""

```


URL configuration for config project.

The `urlpatterns` list routes URLs to views. For more information please see:

<https://docs.djangoproject.com/en/6.0/topics/http/urls/>

Examples:

Function views

1. Add an import: `from my_app import views`
2. Add a URL to urlpatterns: `path('', views.home, name='home')`

Class-based views

1. Add an import: `from other_app.views import Home`
2. Add a URL to urlpatterns: `path('', Home.as_view(), name='home')`

Including another URLconf

1. Import the `include()` function: `from django.urls import include, path`

2. Add a URL to urlpatterns: `path('blog/', include('blog.urls'))`

"""

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import include, path, re_path
from django.views.generic.base import RedirectView
from django.template.tags.static import static as static_url
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework import permissions
from vacancies.views import custom_403, custom_404
```

```
handler404 = custom_404
```

```
handler403 = custom_403
```

```
class IsAdminUserWithProfile(permissions.BasePermission):
```

```
    """
```

```
    Grants access only to users who are both superusers
    and have an Administrator profile in the database.
```

```
    """
```

```
    def has_permission(self, request, view):
```

```
        return (
```

```
            request.user.is_authenticated
            and request.user.is_superuser
            and hasattr(request.user, 'admin_profile')
```

```
        )
```

```
api_info = openapi.Info(
```

```
    title="JobFlex — API агрегатора вакансий",
```

```
    default_version='v1',
```

```
    description=(
```

```
        "Документация REST API платформы JobFlex (агрегация вакансий, профили, модерация).\n\n"
```

```
        "## Как пользоваться из Swagger UI и ReDoc\n\n"
```

```
        "1. **Сессия.** Большинство методов требуют авторизации через cookie `sessionid`. "
```

```
        "Сначала выполните `POST /accounts/api/login/` (или войдите через сайт), "
```

```
        "затем в интерфейсе нажмите **Authorize** и выберите схему **Session**.\n\n"
```

```
        "2. **CSRF.** Для изменяющих запросов (POST, PUT, PATCH, DELETE) из браузера нужен заголовок "
```

```
        "`X-CSRFToken` с тем же значением, что и cookie `csrftoken`. После входа через Swagger/ReDoc "
```

```
        "заголовок обычно подставляется автоматически.\n\n"
```

```
        "3. **Примеры.** У каждой операции в описании приведены примеры тел запросов и ответов в Markdown "
```

```

        "(ReDoc и Swagger отображают их под заголовком операции).\n\n"
        "## Разделы\n\n"
        "- **accounts** – регистрация, вход, профиль, файлы, отклики,
чаты, календарь, собеседования.\n"
        "- **vacancies / moderation** – жалобы, модерация, мягкое
удаление вакансий.\n"
        "- **api** – вспомогательные публичные методы (описание вакансии,
рейтинг работодателя).\n\n"
        "△ **Доступ к этой странице документации** (Swagger и ReDoc) разрешён
только администраторам "
        "с профилем в системе."
    ),
    contact=openapi.Contact(name="Поддержка JobFlex",
email="admin@jobapp.local"),
)

schema_view = get_schema_view(
    api_info,
    public=False,
    permission_classes=[IsAdminUserWithProfile],
)

urlpatterns = [
    path('favicon.ico',
RedirectView.as_view(url=static_url('logo/logo.png'), permanent=True)),
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),
    # Swagger / ReDoc – must come before the vacancies catch-all
    re_path(r'^swagger(?:P<format>\.json|\.yaml)$',
schema_view.without_ui(cache_timeout=0), name='schema-json'),
    path('swagger/', schema_view.with_ui('swagger', cache_timeout=0),
name='schema-swagger-ui'),
    path('redoc/', schema_view.with_ui('redoc', cache_timeout=0),
name='schema-redoc'),
    path('403/', custom_403), # dev preview
    path('404/', custom_404), # dev preview
    path('', include('vacancies.urls')),
]
# Serve uploaded media files during development
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

# Catch-all – must be last; renders custom 404 even when DEBUG=True
urlpatterns += [re_path(r'^.*$', custom_404)]

```

46. config/wsgi.py

```

"""
WSGI config for config project.

It exposes the WSGI callable as a module-level variable named
`application`.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/
"""

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

```

```
application = get_wsgi_application()
```

47. tools/check_hh_page.py

```
import urllib.request

url = 'https://hh.ru/employer/5490091'
req = urllib.request.Request(url, headers={'User-Agent': 'job-aggregator-diploma/1.0'})
with urllib.request.urlopen(req, timeout=15) as r:
    html = r.read().decode('utf-8')

print('Length:', len(html))
for token in ['Рейтинг', 'рейтинг', 'rating', 'ratingValue', 'data-rating', 'rating-average', 'employer-rating']:
    idx = html.find(token)
    print(token, '->', idx)
    if idx != -1:
        print('--- snippet ---')
        print(html[max(0, idx-120):idx+120])
        print('--- end ---\n')
```

48. tools/dump_addresses.py

```
import os, sys, json
sys.path.insert(0, '.')
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
import django
django.setup()
from vacancies.models import Vacancy
qs=Vacancy.objects.exclude(raw_json={})[:50]
for v in qs:
    r=v.raw_json if isinstance(v.raw_json,dict) else {}
    addr=r.get('address')
    print('---',v.pk,v.region)
    print('address type:', type(addr).__name__)
    print('address repr:', repr(addr)[:400])
    if isinstance(addr,dict):
        for k in
('raw','value','display','lat','lng','city','street','building','house'):
            if k in addr:
                print(' ',k,':',addr.get(k))
    area=r.get('area')
    if area:
        print(' area:', repr(area)[:200])
    print()
```

49. tools/fetch_metro.py

```
"""
Fetch metro station data from HH.ru API and save as tools/metro_hh.json.
```

Output structure:

```
[
  {
    "city_id": "1",
    "city_name": "Москва",
    "lines": [
      {
        "id": 1,
        "name": "Сокольническая",
        "hex_color": "E42313",
        "stations": [
          {"id": "s1", "name": "Бульвар Рокоссовского", "lat": ...,
```

```

        "lng": ...},
        ...
    ]
    },
    ...
]
}
}
]
"""

import json
import time
from urllib.request import Request, urlopen

HEADERS = {
    'User-Agent': 'job-aggregator-diploma/1.0 (+http://localhost:8000)',
    'Accept': 'application/json',
    'Accept-Language': 'ru-RU,ru;q=0.9',
}
BASE = 'https://api.hh.ru'

def fetch(url):
    req = Request(url, headers=HEADERS)
    with urlopen(req, timeout=15) as r:
        return json.loads(r.read().decode('utf-8'))

def main():
    print('Fetching city list from /metro ...')
    cities = fetch(f'{BASE}/metro')

    result = []
    for city in cities:
        city_id = city['id']
        city_name = city['name']
        print(f' Fetching {city_name} (id={city_id}) ...')
        try:
            detail = fetch(f'{BASE}/metro/{city_id}')
            lines = []
            for line in detail.get('lines', []):
                stations = []
                for s in line.get('stations', []):
                    stations.append({
                        'id': s['id'],
                        'name': s['name'],
                        'lat': s.get('lat'),
                        'lng': s.get('lng'),
                    })
                lines.append({
                    'id': line['id'],
                    'name': line['name'],
                    'hex_color': line.get('hex_color', 'aaaaaa'),
                    'stations': stations,
                })
            result.append({
                'city_id': city_id,
                'city_name': city_name,
                'lines': lines,
            })
            time.sleep(0.3) # be polite
        except Exception as e:
            print(f' ERROR: {e}')

```

```

    out_path = __file__.replace('fetch_metro.py', 'metro_hh.json')
    with open(out_path, 'w', encoding='utf-8') as f:
        json.dump(result, f, ensure_ascii=False, indent=2)
    print(f'Saved {len(result)} cities → {out_path}')

if __name__ == '__main__':
    main()
50. tools/inspect_hh_employer_page.py
import urllib.request
import re
import sys

url = 'https://hh.ru/employer/600589'
req = urllib.request.Request(url, headers={'User-Agent': 'job-aggregator-
diploma/1.0'})
try:
    with urllib.request.urlopen(req, timeout=20) as r:
        html = r.read().decode('utf-8', errors='replace')
except Exception as e:
    print('FETCH-ERR', e)
    sys.exit(1)

print('LEN', len(html))
markers = ['aggregateRating', 'ratingValue', '"rating"', 'рейтинг',
'class="rating"', 'data-qa']
for m in markers:
    if m in html:
        print('HAS', m)

print('\n---SNIPPET START---\n')
print(html[:2000])
print('\n---SNIPPET END---\n')

m = re.search(r'(\рейтинг|rating)\.{0,80}?([0-5][\.\,][0-9])', html, re.I |
re.S)
if m:
    print('\nFOUND NEAR WORD:', m.group(0))
else:
    m2 = re.search(r'([0-5][\.\,][0-9])', html)
    if m2:
        print('\nFOUND NUMBER:', m2.group(1))
    else:
        print('\nNO VISIBLE RATING PATTERN FOUND')

```

51. tools/seed_applicants.py

```

import django, os, sys
sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup()

from django.contrib.auth.models import User
from accounts.models import Applicant

USERS = [
    {
        'username': 'ivanov_a',
        'first_name': 'Алексей',
        'last_name': 'Иванов',
        'email': 'ivanov@example.com',
        'applicant': {
            'patronymic': 'Сергеевич',

```

```

        'phone': '+7 900 111-22-33',
        'city': 'Москва',
        'gender': 'M',
        'skills': ['Python', 'Django', 'PostgreSQL', 'REST API',
'Docker'],
        'location_type': 'metro',
        'metro_station_name': 'Курская',
        'metro_line_name': 'Кольцевая',
        'metro_line_color': 'b44f22',
    }
},
{
    'username': 'petrova_m',
    'first_name': 'Мария',
    'last_name': 'Петрова',
    'email': 'petrova@example.com',
    'applicant': {
        'patronymic': 'Игоревна',
        'phone': '+7 912 333-44-55',
        'city': 'Санкт-Петербург',
        'gender': 'F',
        'skills': ['JavaScript', 'React', 'TypeScript', 'CSS',
'Node.js', 'Figma'],
        'location_type': 'address',
        'address': 'Невский проспект, 28',
    }
},
{
    'username': 'sidorov_k',
    'first_name': 'Кирилл',
    'last_name': 'Сидоров',
    'email': 'sidorov@example.com',
    'applicant': {
        'patronymic': 'Олегович',
        'phone': '+7 926 555-66-77',
        'city': 'Москва',
        'gender': 'M',
        'skills': ['Java', 'Spring Boot', 'Kafka', 'Kubernetes',
'SQL', 'Redis', 'Microservices'],
        'location_type': 'metro',
        'metro_station_name': 'Белорусская',
        'metro_line_name': 'Замоскворецкая',
        'metro_line_color': '00853e',
    }
},
]

for u_data in USERS:
    a_data = u_data.pop('applicant')
    u, created = User.objects.get_or_create(username=u_data['username'],
defaults=u_data)
    if created:
        u.set_password('testpass123')
        u.save()
    a, _ = Applicant.objects.get_or_create(user=u)
    for k, v in a_data.items():
        setattr(a, k, v)
    a.save()
    status = 'created' if created else 'updated'
    print(status + ': ' + u.get_full_name() + ' (' + u.username + ')')

print('Done.')
```

52. tools/seed_filters.py

```
"""
Seed HhArea and HhDictionaryItem from local JSON files.
Run with: python manage.py shell < tools/seed_filters.py
Or: python manage.py runscript tools/seed_filters (with django-
extensions)
"""
import json, os, django, sys

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
BASE = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, BASE)
django.setup()

from django.db import transaction
from vacancies.models import HhArea, HhDictionaryItem

# ---- Areas ----
with open(os.path.join(BASE, "tools", "hh_areas.json"), encoding="utf-8")
as f:
    areas_payload = json.load(f)

def iter_area_nodes(nodes, parent_id=""):
    if not isinstance(nodes, list):
        return
    for node in nodes:
        if not isinstance(node, dict):
            continue
        item_id = node.get("id")
        if not item_id:
            continue
        flat = dict(node)
        flat["_parent_id"] = str(node.get("parent_id") or parent_id or
    "")

        yield flat
        yield from iter_area_nodes(node.get("areas") or [],
parent_id=str(item_id))

area_objs = [
    HhArea(
        area_id=str(a.get("id") or ""),
        name=str(a.get("name") or ""),
        parent_id=str(a.get("_parent_id") or ""),
        raw_json=a,
    )
    for a in iter_area_nodes(areas_payload)
    if a.get("id")
]

with transaction.atomic():
    HhArea.objects.all().delete()
    HhArea.objects.bulk_create(area_objs, batch_size=500)
print(f"Areas inserted: {HhArea.objects.count()}")

# ---- Dictionaries ----
with open(os.path.join(BASE, "tools", "hh_dictionaries.json"),
encoding="utf-8") as f:
    dicts_payload = json.load(f)
```

```

dict_objs = []
for dict_name, items in dicts_payload.items():
    if not isinstance(items, list):
        continue
    for item in items:
        if not isinstance(item, dict):
            continue
        item_id = str(item.get("id") or "")
        name = str(item.get("name") or item.get("text") or "")
        if not item_id or not name:
            continue
        dict_objs.append(HhDictionaryItem(
            dictionary=dict_name,
            item_id=item_id,
            name=name,
            raw_json=item,
        ))

with transaction.atomic():
    HhDictionaryItem.objects.all().delete()
    HhDictionaryItem.objects.bulk_create(dict_objs, batch_size=500)
print(f"Dictionary items inserted: {HhDictionaryItem.objects.count()}")

# ---- Verify ----
print(f"Regions (parent=113):
{HhArea.objects.filter(parent_id='113').count()}")
for d in ["experience", "schedule", "employment", "employment_form"]:
    items =
list(HhDictionaryItem.objects.filter(dictionary=d).values_list("item_id",
"name"))
print(f" [{d}]: {items}")

```

53. tools/strip_swagger.py

```

"""
Strips @swagger_auto_schema decorators down to just (method/methods,
operation_summary, tags).
Keeps all other code intact. Run from the project root.
Usage: python tools/strip_swagger.py
"""

import re
from pathlib import Path

def find_matching_paren(text: str, open_pos: int) -> int:
    """Return index AFTER the matching ')' given position of '('."""
    depth = 1
    i = open_pos + 1
    in_str = False
    str_char = ''
    while i < len(text) and depth > 0:
        ch = text[i]
        if in_str:
            if ch == '\\\\':
                i += 2
                continue
            if ch == str_char:
                in_str = False
        else:
            if ch in ('"', "'"):
                in_str = True
                str_char = ch
                # Handle triple-quotes
                if text[i:i+3] in ('"""', "''"):

```



```

        triple = text[i:i+3]
        i += 3
        end = text.find(triple, i)
        if end == -1:
            break
        i = end + 3
        continue
    elif ch == '(':
        depth += 1
    elif ch == ')':
        depth -= 1
    i += 1
return i

def extract_str_value(text: str, key: str):
    """Extract a single-quoted or double-quoted string value for a
    key."""
    pattern = rf'(?<!\w){re.escape(key)}\s*=\s*(?:u?["\'])((^"\')*)["\']'
    m = re.search(pattern, text)
    if m:
        return m.group(1)

    # Try triple-quoted
    pattern3 =
rf'(?<!\w){re.escape(key)}\s*=\s*(?:u?["\']{{3}})(.*?){["\']{{3}}}'
    m3 = re.search(pattern3, text, re.DOTALL)
    if m3:
        # Flatten multi-line / concatenated strings
        raw = m3.group(1)
        raw = re.sub(r'\s*["\'][\s\n]+["\']?\s*', ' ', raw)
        return raw.strip()

    # Try parenthesized concatenated string: ( "part1" "part2" ... )
    paren_pattern =
rf'(?<!\w){re.escape(key)}\s*=\s*(\s*(?:["\'](^"\')*["\'][, \s]*))+\s*)'
    mp = re.search(paren_pattern, text, re.DOTALL)
    if mp:
        raw = mp.group(1)
        parts = re.findall(r'["\'](^"\')*["\']', raw)
        return ' '.join(p.strip() for p in parts).strip()

    return None

def extract_list_value(text: str, key: str):
    """Extract a list literal value like tags=['accounts']."""
    pattern = rf'(?<!\w){re.escape(key)}\s*=\s*(\[([^\]]*\s*))'
    m = re.search(pattern, text)
    if m:
        # Compact spaces inside the list
        inner = re.sub(r'\s+', ' ', m.group(1))
        return inner
    return None

def strip_one_decorator(dec_text: str) -> str:
    """Convert a full @swagger_auto_schema(...) block to minimal form."""
    # Find method= or methods= value
    method_m = re.search(r'\bmethod\s*=\s*(["\'](^"\')*["\']', dec_text)
    methods_m = re.search(r'\bmethods\s*=\s*(["\'](^"\')*["\']', dec_text)

    summary = extract_str_value(dec_text, 'operation_summary')
    tags = extract_list_value(dec_text, 'tags')

    parts = []

```

```

if method_m:
    parts.append(f'method={method_m.group(1)}')
elif methods_m:
    val = re.sub(r'\s+', ' ', methods_m.group(1))
    parts.append(f'methods={val}')

if summary:
    escaped = summary.replace('"', '\\"')
    parts.append(f'operation_summary="{escaped}"')

if tags:
    parts.append(f'tags={tags}')

return '@swagger_auto_schema(' + ', '.join(parts) + ')'

def strip_swagger_in_file(filepath: str):
    path = Path(filepath)
    text = path.read_text(encoding='utf-8')
    original_lines = text.count('\n')

    result_parts = []
    pos = 0

    while True:
        idx = text.find('@swagger_auto_schema', pos)
        if idx == -1:
            result_parts.append(text[pos:])
            break

        # Include text before decorator
        result_parts.append(text[pos:idx])

        # Find the opening '('
        paren_pos = text.index('(', idx)

        # Find the matching ')'
        end_pos = find_matching_paren(text, paren_pos)

        dec_text = text[idx:end_pos]
        mini = strip_one_decorator(dec_text)
        result_parts.append(mini)

        pos = end_pos

    new_text = ''.join(result_parts)

    # Remove unused _preset_schema variable (only present in
    accounts/views.py)
    new_text = re.sub(
        r'\n_preset_schema\s*=\s*openapi\.Schema\([^\)]*(?:\([^\)]*\)[^\)]*)'
        *')\s*\n',
        '\n',
        new_text,
        flags=re.DOTALL,
    )

    new_lines = new_text.count('\n')
    path.write_text(new_text, encoding='utf-8')
    print(f'{filepath}: {original_lines} -> {new_lines} lines (saved {original_lines - new_lines})')

if __name__ == '__main__':
    base = Path(__file__).parent.parent

```

```

strip_swagger_in_file(str(base / 'accounts' / 'views.py'))
strip_swagger_in_file(str(base / 'vacancies' / 'views.py'))
print('Done.')

```

54. tools/update_employer_rating.py

```

import os
import django
from django.utils import timezone

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup()

from vacancies.models import Employer

hh_id = '600589'
new_rating = 3.5

emp = Employer.objects.filter(hh_id=hh_id).first()
if not emp:
    print('Employer not found:', hh_id)
else:
    emp.hh_rating = new_rating
    emp.rating_raw = str(new_rating)
    emp.rating_updated_at = timezone.now()
    emp.save(update_fields=['hh_rating', 'rating_raw',
'rating_updated_at'])
    print('Updated employer', emp.id, 'hh_id', emp.hh_id, '->',
emp.hh_rating)

```

55. get_schema.py

```

import sqlite3

conn = sqlite3.connect('db.sqlite3')
cur = conn.cursor()

cur.execute("""
    SELECT sql FROM sqlite_master
    WHERE type='table'
    AND sql IS NOT NULL
    AND (name LIKE 'accounts_%' OR name LIKE 'vacancies_%')
    ORDER BY name
""")

with open('schema.sql', 'w', encoding='utf-8') as f:
    for row in cur.fetchall():
        f.write(row[0] + ';\n\n')

conn.close()
print('Done — schema.sql создан')

```

56. locustfile.py

```

import json
import random
import re
import uuid

from faker import Faker
from locust import HttpUser, SequentialTaskSet, TaskSet, between, task

fake = Faker("ru_RU")
Faker.seed(0)

```

```

SEARCH_TERMS = [
    "python", "django", "менеджер", "java", "разработчик",
    "аналитик", "дизайнер", "маркетолог", "бухгалтер", "водитель",
    "продавец", "инженер", "тестировщик", "DevOps", "data scientist",
]

REGIONS = [
    "Москва", "Санкт-Петербург", "Екатеринбург",
    "Новосибирск", "Казань", "Нижний Новгород",
]

EXPERIENCE_IDS = ["noExperience", "between1And3", "between3And6",
"moreThan6"]

CITIZENSHIP_CODES = ["RU", "BY", "KZ", "KG", "AM", "TJ", "UZ"]

SORT_OPTIONS = ["", "salary", "salary_asc"]

WORK_FORMATS = ["remote", "hybrid", "onsite"]

SALARY_LEVELS = [30_000, 50_000, 80_000, 100_000, 150_000, 200_000]

_discovered_vacancy_ids: list[int] = []

def _csrf(client) -> str:
    return client.cookies.get("csrftoken", "")

def _json_headers(client) -> dict:
    return {
        "Content-Type": "application/json",
        "X-CSRFToken": _csrf(client),
    }

def _extract_vacancy_ids(html: str) -> list[int]:
    ids = re.findall(r"/vacancies/(\d+)/", html or "")
    return [int(i) for i in set(ids)]

def _random_phone() -> str:
    digits = "".join(str(random.randint(0, 9)) for _ in range(9))
    return f"79{digits}"

def _make_applicant_payload() -> dict:
    uid = uuid.uuid4().hex[:10]
    first_name = fake.first_name()
    last_name = fake.last_name()
    username = f"u_{uid}"

    return {
        "last_name": last_name,
        "first_name": first_name,
        "patronymic": fake.middle_name(),
        "username": username,
        "email": f"{uid}@locust.invalid",
        "telegram": f"@{username}",
        "phone": _random_phone(),
        "gender": random.choice(["M", "F"]),
        "city": random.choice(REGIONS),
        "birth_date": fake.date_of_birth(minimum_age=18,
maximum_age=50).isoformat(),
        "citizenship": random.choice(CITIZENSHIP_CODES),
        "password": "LocustPass1!",
        "consent_email": False,
        "consent_telegram": False,
    }

```

```

    }

class AnonymousBrowseTasks(TaskSet):
    @task(10)
    def browse_homepage(self):
        resp = self.client.get("/", name="[GET] Главная / все вакансии")
        ids = _extract_vacancy_ids(resp.text)
        if ids:
            _discovered_vacancy_ids.extend(ids)
            del _discovered_vacancy_ids[300:]

    @task(10)
    def search_vacancies_by_keyword(self):
        term = random.choice(SEARCH_TERMS)
        self.client.get(f"/?q={term}", name="[GET] Поиск по ключевому
слову")

    @task(8)
    def filter_by_salary(self):
        salary_min = random.choice(SALARY_LEVELS)
        self.client.get(
            f"/?only_with_salary=1&salary_min={salary_min}",
            name="[GET] Фильтр: с зарплатой ≥ N",
        )

    @task(7)
    def filter_by_experience_and_format(self):
        exp = random.choice(EXPERIENCE_IDS)
        fmt = random.choice(WORK_FORMATS)
        self.client.get(
            f"/?experience={exp}&format={fmt}",
            name="[GET] Фильтр: опыт + формат работы",
        )

    @task(5)
    def search_with_combined_filters_and_sort(self):
        term = random.choice(SEARCH_TERMS)
        exp = random.choice(EXPERIENCE_IDS)
        sort = random.choice(SORT_OPTIONS)
        sal_min = random.choice(SALARY_LEVELS)
        self.client.get(
            f"/?q={term}&experience={exp}&only_with_salary=1"
            f"&salary_min={sal_min}&sort={sort}",
            name="[GET] Комбинированный поиск с сортировкой",
        )

    @task(2)
    def register_new_applicant(self):
        self.client.get("/accounts/register/", name="[GET] Страница
регистрации")

        payload = _make_applicant_payload()
        self.client.post(
            "/accounts/api/register/",
            data=json.dumps(payload),
            headers=_json_headers(self.client),
            name="[POST] Регистрация нового соискателя",
        )

class RegistrationAndLoginFlow(SequentialTaskSet):

    def on_start(self):
        self._creds = _make_applicant_payload()

```

```

        self._target_vacancy_id = None

    @task
    def s01_homepage(self):
        resp = self.client.get("/", name="[SEQ][GET] 01 Главная
страница")
        ids = _extract_vacancy_ids(resp.text)
        if ids:
            self._target_vacancy_id = random.choice(ids)
            _discovered_vacancy_ids.extend(ids)
            del _discovered_vacancy_ids[300:]

    @task
    def s02_search_vacancies(self):
        term = random.choice(SEARCH_TERMS)
        resp = self.client.get(f"/?q={term}", name="[SEQ][GET] 02 Поиск
вакансий")
        ids = _extract_vacancy_ids(resp.text)
        if ids and not self._target_vacancy_id:
            self._target_vacancy_id = random.choice(ids)

    @task
    def s03_filter_vacancies(self):
        exp = random.choice(EXPERIENCE_IDS)
        self.client.get(
            f"/?experience={exp}&only_with_salary=1",
            name="[SEQ][GET] 03 Фильтрация вакансий",
        )

    @task
    def s04_view_vacancy_detail(self):
        pk = self._target_vacancy_id or (
            random.choice(_discovered_vacancy_ids) if
_discovered_vacancy_ids else None
        )
        if pk:
            self.client.get(f"/vacancies/{pk}/", name="[SEQ][GET] 04
Страница вакансии")
        else:
            self.client.get("/", name="[SEQ][GET] 04 Главная (нет ID)")

    @task
    def s05_view_terms(self):
        self.client.get("/accounts/terms/", name="[SEQ][GET] 05 Условия
использования")

    @task
    def s06_registration_page(self):
        self.client.get("/accounts/register/", name="[SEQ][GET] 06
Страница регистрации")

    @task
    def s07_submit_registration(self):
        resp = self.client.post(
            "/accounts/api/register/",
            data=json.dumps(self._creds),
            headers=_json_headers(self.client),
            name="[SEQ][POST] 07 Регистрация пользователя",
        )
        if resp.status_code != 200:
            self._creds = _make_applicant_payload()
            self.interrupt()

```

```

@task
def s08_login_page(self):
    self.client.get("/accounts/login/", name="[SEQ][GET] 08 Страница
входа")

@task
def s09_submit_login(self):
    resp = self.client.post(
        "/accounts/api/login/",
        data=json.dumps({
            "username": self._creds["username"],
            "password": self._creds["password"],
        }),
        headers=_json_headers(self.client),
        name="[SEQ][POST] 09 Вход в систему",
    )
    if resp.status_code != 200:
        self.interrupt()

@task
def s10_homepage_authenticated(self):
    resp = self.client.get("/", name="[SEQ][GET] 10 Главная (авт.)")
    ids = _extract_vacancy_ids(resp.text)
    if ids:
        self._target_vacancy_id = random.choice(ids)
        _discovered_vacancy_ids.extend(ids)

@task
def s11_search_authenticated(self):
    term = random.choice(SEARCH_TERMS)
    self.client.get(f"?q={term}", name="[SEQ][GET] 11 Поиск (авт.)")

@task
def s12_vacancy_detail_authenticated(self):
    pk = self._target_vacancy_id or (
        random.choice(_discovered_vacancy_ids) if
_discovered_vacancy_ids else None
    )
    if pk:
        self.client.get(f"/vacancies/{pk}/", name="[SEQ][GET] 12
Вакансия (авт.)")

@task
def s13_profile_page(self):
    self.client.get("/accounts/profile/", name="[SEQ][GET] 13 Профиль
соискателя")

@task
def s14_profile_data_api(self):
    self.client.get(
        "/accounts/api/profile-data/",
        name="[SEQ][GET] 14 Данные профиля (API)",
    )

@task
def s15_applicant_analytics(self):
    self.client.get(
        "/accounts/api/analytics/",
        name="[SEQ][GET] 15 Аналитика соискателя (API)",
    )

@task
def s16_bookmark_vacancy(self):

```

```

        pk = self._target_vacancy_id or (
            random.choice(_discovered_vacancy_ids) if
            _discovered_vacancy_ids else None
        )
        if pk:
            self.client.post(
                f"/accounts/api/bookmark/{pk}/",
                headers={"X-CSRFToken": _csrf(self.client)},
                name="[SEQ][POST] 16 Закладка вакансии",
            )

    @task
    def s17_update_profile(self):
        self.client.patch(
            "/accounts/api/profile/update/",
            data=json.dumps({
                "city": random.choice(REGIONS),
                "about_me": fake.text(max_nb_chars=150),
                "desired_position": fake.job()[0:255],
                "salary_expectation_from": random.choice(SALARY_LEVELS),
            }),
            headers=_json_headers(self.client),
            name="[SEQ][PATCH] 17 Обновить профиль",
        )

    @task
    def s18_view_updated_profile(self):
        self.client.get("/accounts/profile/", name="[SEQ][GET] 18 Профиль
        (после обновления)")

    @task
    def s19_browse_before_logout(self):
        self.client.get("/?format=remote", name="[SEQ][GET] 19 Удалённые
        вакансии (перед выходом)")

    @task
    def s20_logout(self):
        self.client.post(
            "/accounts/api/logout/",
            data=json.dumps({}),
            headers=_json_headers(self.client),
            name="[SEQ][POST] 20 Выход из системы",
        )
        self.interrupt()

class AnonymousVisitor(HttpUser):
    tasks = [AnonymousBrowseTasks]
    wait_time = between(1, 3)
    weight = 3

class NewApplicant(HttpUser):
    tasks = [RegistrationAndLoginFlow]
    wait_time = between(2, 5)
    weight = 1

```

57. manage.py

```

#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():

```



```

"""Run administrative tasks."""
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
try:
    from django.core.management import execute_from_command_line
except ImportError as exc:
    raise ImportError(
        "Couldn't import Django. Are you sure it's installed and "
        "available on your PYTHONPATH environment variable? Did you "
        "forget to activate a virtual environment?"
    ) from exc
execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()

```

58. run_all.py

```

import asyncio
import sys
import signal
import re
from pathlib import Path

def _venv_python() -> str:
    script_dir = Path(__file__).resolve().parent
    candidates = [
        script_dir / ".venv" / "Scripts" / "python.exe",
        script_dir / ".venv" / "bin" / "python",
    ]
    for p in candidates:
        if p.exists():
            return str(p)
    return sys.executable

PYTHON = _venv_python()

COMMANDS = [
    ("django", [PYTHON, "manage.py", "runserver", "0.0.0.0:8000"]),
    ("celery", [PYTHON, "-m", "celery", "-A", "config", "worker", "--loglevel=info", "--pool=threads", "--concurrency=4"]),
    ("celery-beat", [PYTHON, "-m", "celery", "-A", "config", "beat", "--loglevel=info"]),
    ("poll_telegram", [PYTHON, "manage.py", "poll_telegram_updates"]),
]

ANSI = {
    "reset": "\033[0m",
    "dim": "\033[2m",
    "red": "\033[31m",
    "green": "\033[32m",
    "yellow": "\033[33m",
    "blue": "\033[34m",
    "magenta": "\033[35m",
    "cyan": "\033[36m",
}

SERVICE_COLOR = {
    "django": ANSI["green"],
    "celery": ANSI["cyan"],
    "celery-beat": ANSI["blue"],
    "poll_telegram": ANSI["magenta"],
}

```

```

def _decode_escaped_unicode(text: str) -> str:
    if "\\u" not in text:
        return text
    try:
        return bytes(text, "utf-8").decode("unicode_escape")
    except Exception:
        return text

def _beautify_line(text: str) -> str:
    text = _decode_escaped_unicode(text)

    m = re.search(r"fetch_vacancy_description: vacancy (\d+)
.*?desc=(\d+) branded=(\d+)", text)
    if m:
        vid, desc_len, branded_len = m.groups()
        return f"Описание HH: id={vid}, desc={desc_len},
branded={branded_len}"

    m = re.search(r"Task
vacancies\.tasks\.fetch_vacancy_description\[([^\]]+)\]
succeeded.*'ok:desc=(\d+),branded=(\d+)'", text)
    if m:
        desc_len, branded_len = m.groups()
        return f"Описание HH: задача выполнена (desc={desc_len},
branded={branded_len})"

    if "Task accounts.tasks.notify_interview_reminders_task" in text and
"succeeded" in text:
        return "Напоминания о собеседованиях: задача выполнена"
    if "Task accounts.tasks.notify_calendar_note_reminders_task" in text
and "succeeded" in text:
        return "Напоминания календаря: задача выполнена"

    return text

def _format_prefix(name: str) -> str:
    is_err = name.endswith("-ERR")
    base = name[:-4] if is_err else name
    color = ANSI["red"] if is_err else SERVICE_COLOR.get(base,
ANSI["dim"])
    return f"{color}[{name}]{ANSI['reset']}"

async def stream_output(name, stream, forward):
    while True:
        line = await stream.readline()
        if not line:
            break
        try:
            text = line.decode(errors="replace").rstrip()
        except Exception:
            text = str(line)
        text = _beautify_line(text)
        print(f"{_format_prefix(name)} {text}")
        if forward:
            sys.stdout.flush()

async def run_process(name, cmd):
    proc = await asyncio.create_subprocess_exec(
        *cmd, stdout=asyncio.subprocess.PIPE,
stderr=asyncio.subprocess.PIPE
    )

    tasks = [

```

```

        asyncio.create_task(stream_output(name, proc.stdout, True)),
        asyncio.create_task(stream_output(name + "-ERR", proc.stderr,
True))),
    ]

    await asyncio.wait(tasks)
    return await proc.wait()

async def main():
    loop = asyncio.get_running_loop()

    stop_event = asyncio.Event()

    def _signal_handler():
        stop_event.set()

    for s in (signal.SIGINT, signal.SIGTERM):
        try:
            loop.add_signal_handler(s, _signal_handler)
        except NotImplementedError:
            pass

    tasks = {}
    for name, cmd in COMMANDS:
        tasks[name] = asyncio.create_task(run_process(name, cmd))

    done, pending = await asyncio.wait(tasks.values(),
return_when=asyncio.FIRST_COMPLETED)

    if pending:
        for t in pending:
            t.cancel()

    await asyncio.sleep(0.3)

    if stop_event.is_set():
        for t in tasks.values():
            if not t.done():
                t.cancel()

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("Interrupted, exiting.")

```